Sorbonne University

EDITE de Paris Doctoral School (ED 130)

High-Level Programming Models for Resource-Constrained Microcontrollers

---

By Steven Varoumas

Doctoral thesis in Computer Science

Supervised by Tristan Crolard

Presented and publicly defended in Paris on 5 November 2019

This manuscript has been translated from French into English with the assistance of AI tools. The text may contain errors or imprecisions. In addition, some figures and captions remain in French, and some formatting errors may be present.

Contents

Introduction

Over the course of an ordinary day, any reader of this thesis will come into contact with numerous microcontrollers. These small electronic devices, typically measuring from a few millimeters to a few centimeters, are embedded in many everyday objects and quietly govern many aspects of our lives. Whether the reader travels to work by public transport or by car, hundreds of microcontrollers may accompany the journey, carrying out automatic tasks such as controlling a vehicle's braking system or operating the doors on metro platforms. Their presence is not limited to transport: in the workplace, for example, coffee machines, air-conditioning systems, and computer peripherals (screens, mice, keyboards, ...) may contain dozens of microcontrollers responsible for the proper operation of these systems. Household appliances are no exception: washing machines, refrigerators, microwave ovens, and many other small or large devices also contain them. In fact, almost every room in a house is concerned: alarm clocks, radiators, thermostats, lighting systems, water heaters, and even some toys may contain microcontrollers. Some may even be found on one's person, in a mobile phone[1] kept in one's pocket, or inside certain medical devices. Through their omnipresence, microcontrollers therefore contribute every day to many tasks in modern life.

Seen from the inside, a microcontroller (sometimes abbreviated as μC) is a programmable printed circuit whose various elements are equivalent to the components of a very simplified, yet complete, computer. A microcontroller has an arithmetic and logic unit, a set of memories, generally composed of RAM for processing a program's dynamic data and non-volatile memories intended to store the program or certain data, as well as several input/output devices, called *pins*: small metallic legs capable of carrying an electrical current in order to communicate with the environment (shown in Figure 1). The environment of a microcontroller is typically an electronic assembly made up of multiple components that make it possible to interact with the real world: sensors, which communicate properties of the assembly's physical environment to the microcontroller (temperature values, brightness, the state of a push button, ...), and actuators, which influence the state of the physical system when they are triggered (lighting an LED, rotating a motor, emitting sound or heat, ...). A microcontroller is therefore often the central computing unit of an electronic assembly (or of part of an assembly, in the case of more complex networks of circuits): it coordinates the reactions of the hardware to the state of the surrounding physical environment. Such devices are therefore frequently used to program embedded systems, and govern the automation of the tasks specific to these systems (for example, regulating the temperature of a room).

The physical resources of microcontrollers are often limited, comparable to those of a personal computer from the 1980s. Thus, a microcontroller's clock frequency rarely exceeds a few tens of megahertz, its RAM is limited to a few kilobytes, and its flash memory, intended for program storage, reaches at most a few megabytes. These resources may seem highly anachronistic in a world where even a basic

[1] Although recent mobile phones contain more powerful and more complex systems on chip, it is not uncommon for microcontrollers to handle auxiliary tasks such as controlling touch screens or certain sensors.

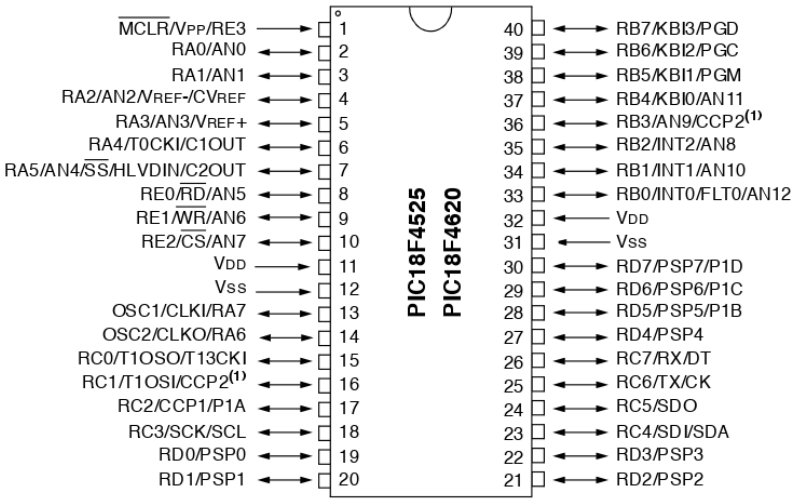


FIGURE 1: A PIC microcontroller and its input/output pins (numbered from 1 to 40)

recent system on chip has a computation speed one hundred times higher and memory resources one hundred thousand times larger. Nevertheless, microcontrollers continue to be used very frequently in industry: their low energy consumption and low purchase cost (from a few tenths of a euro to a few tens of euros) contribute to their omnipresence in many everyday objects. In addition, a growing number of hobbyist users and tinkerers are programming microcontrollers, for example to develop small assemblies for home automation (automatic plant watering, controlling room lighting, ...) or leisure applications (small game consoles, drones, LED walls, etc.). These uses, stimulated by the emergence of new categories of connected objects, make microcontrollers components that are still frequently used for developing embedded solutions, whether professional or amateur. Thus, driven by the Internet of Things (*IoT*), for which some predictions estimate the number of new devices produced between 2017 and 2025 at one *trillion*[2] [Spa17], it has been estimated that the global microcontroller market would have an annual growth rate of about 12% between 2016 and 2023 [⚓5][3].

Just as the resources of microcontrollers resemble relatively old classic computer hardware, development processes for microcontrollers have also changed little over time: microcontroller programming is traditionally carried out in languages that can be considered low level, or even very low level. It is not unusual, even today, for programs intended to run on microcontrollers to be written directly in assembly language. Microcontroller programming is therefore a difficult activity, since it forces the developer to be familiar with the often inexpressive instruction set of the assembly language of the target microcontroller. The user must also know the hardware that will execute the program precisely, since the level of hardware abstraction offered by such languages is very limited, or even non-existent. This extreme specialization of programs intended for microcontrollers, in addition to being difficult for a novice programmer more familiar with higher-level languages such as Python or Java, restricts a program's ability to be easily ported to a different architecture. As a result, even the slightest change in microcontroller model for the developed application may require a complete rewrite of the program code.

Testing and debugging a microcontroller program is a complex and equally restrictive process. Debugging a program intended for a personal computer is simplified by software debuggers that make

[2] A trillion = 10^{12}.

[3] The web references in this document are all preceded by a symbol representing an anchor ⚓.

it possible to simulate program execution step by step while monitoring the contents of the memory being used, and running a test suite on a PC can be done with a single mouse click. By contrast, development environments for microcontroller programming do not systematically offer the possibility of simulating program execution, and such simulation can be difficult because the program is strongly tied to its physical execution environment. As a result, testing a program intended for a microcontroller commonly amounts to running it on the actual assembly for which it is intended. For an amateur developer, debugging a microcontroller program very often consists in compiling it, transferring the generated executable to the microcontroller, and simply testing whether the program behaves as expected within the real electronic assembly. This process may then be repeated many times until the program *seems* to work correctly. In addition to being extremely time-consuming, this tedious and unsafe practice can damage the electronic hardware being used, for example through wear on the hardware resources of the microcontroller (whose non-volatile memory has a limited total number of writes), or even through the destruction of components peripheral to the microcontroller (or even its internal components), if it interacts with them incorrectly.

Within some communities of developers familiar with microcontroller programming, the mere use of a subset of the C language to program embedded systems may be considered high-level programming. Compared with assembly languages, C does indeed provide a significant layer of hardware abstraction, making development and debugging simpler, and it offers various guarantees about written programs (including some simple static checks on program typing) that make these programs safer and less prone to bugs. The use of relatively low-level languages such as C therefore often represents a boundary rarely crossed by embedded systems developers, who are attached to controlling precisely the resource consumption of their programs and their interaction with the hardware. Yet many higher-level languages have welcome advantages in the context of microcontroller programming. In addition to the increased expressiveness provided by languages implementing high-level programming paradigms, such as object-oriented programming or functional programming, the guarantees offered by the use of such languages (such as static or dynamic type checks) are a clear advantage for programming embedded systems whose malfunction may have disastrous consequences. Moreover, the hardware abstractions offered by these languages do not necessarily prevent a careful use of these resources: since the memory capacity of microcontrollers is small, it is precisely useful in some cases to benefit from a runtime environment capable of reusing these resources dynamically according to the program's needs. For these reasons, several works have sought to enable the development of embedded programs in higher-level programming languages such as Java, Python, or Scheme. Other solutions propose specialised languages for microcontroller programming that are themselves inspired by higher-level programming models, such as functional reactive programming [HG16], or graphical programming [Kat10, MS17].

Furthermore, the general-purpose languages classically used are not necessarily suited to the characteristics inherent in programming applications for microcontrollers. Since an embedded system generally has to react quickly to various stimuli (such as pressing a button or a change in the value computed by a sensor), microcontroller programming often exhibits concurrent behaviour. Providing a lightweight concurrent programming model suited to such applications is therefore a significant advantage for simplifying the development of such programs. Consequently, because of the resource limits of a microcontroller and its intrinsic particularities (such as the absence of an operating system), the concurrency model adopted for programming a microcontroller cannot systematically be modeled on the concurrency

models usually used to program applications for computers or mobile phones (such as *thread*-based systems). However, less common concurrent programming models are well suited to microcontroller programming. In particular, the synchronous programming model seems especially appropriate to us, because it requires very few hardware resources and because of the simplicity of its approach based on the *synchronous hypothesis*, which makes it possible to consider that the various components of a program all execute *instantaneously*, concurrently. This paradigm has the advantage of freeing the developer from concerns related to synchronization between the various components of a program, which contributes to raising the level of abstraction in the software development process.

Moreover, programs intended for embedded systems, which are sometimes critical, often need to satisfy strict physical or behavioral constraints. Some microcontroller applications (such as drones, transport systems, or certain robots) are *real-time systems* for which the reaction to external stimuli must occur within a certain time interval. This imposed delay generally corresponds to safety constraints, and being able to guarantee the system's reaction speed is an essential condition for the program to behave as expected. Other constraints may concern the logical behaviour of programs, for which certain invariants expressing program correctness can be verified.

We draw part of our inspiration from development methods used in civil avionics, where industrial tools derived from synchronous languages, such as SCADE [CPP17], enable DO-178C certification of aircraft (and their embedded software). The use of formal methods (abstract interpretation, *model checking*, ...) to verify certain program properties is also encouraged. For example, it is now possible to replace tests with proofs in the DO-178 standard[4]. These robust analyses are clearly valuable for programming critical embedded systems. Our approach, which combines high-level programming models with the synchronous dataflow programming model, reuses similar analyses to verify the correctness of program behaviour as well as certain properties related, for example, to the computation time of a synchronous instant.

The ambition of this thesis is to propose a solution that helps bring together two worlds that may seem far apart. On one side is the world of microcontroller programming, rich in relatively simple applications and in hobbyist and industrial users, but limited by hardware resources and generally supported by development processes that offer fairly weak software safety and guarantees. On the other side is the world of higher-level programming models, which offers abstractions that provide greater expressive power and greater safety, but may produce programs that consume more resources. The ambition of our approach rests on the ability to offer microcontroller developers modern and powerful development techniques, making it possible to bring additional guarantees to the programs they build, while taking into account the limited capacities of the hardware considered. Our solutions will therefore aim to be executable on hardware with very few resources, on the order of a few kilobytes of RAM. This desire for compatibility with resource-constrained microcontrollers stems from the fact that even today, 8-bit microcontrollers with only a few kilobytes of RAM still dominate the market [SSD+17]. In addition, offering solutions that work on such limited hardware allows us to ensure that our proposal is suitable for developing programs for a broad range of microcontroller models, without being limited to higher-end models that are better equipped with resources but less commonly used.

This manuscript describes our solution in detail. It is based on a series of abstractions whose goal is to simplify development processes and guarantee the correctness of programs for embedded systems.

[4] Additional information on this topic is available in a document named DO-333

In particular, we will formalize several aspects of our discussion and prove certain metatheoretical properties of our solution. This formalization has largely been mechanized by software means, and several proofs related to this formalism have been carried out using the Coq proof assistant [Tea19]. The sources of the programs, examples, and proofs of lemmas and theorems stated throughout this manuscript are available online [⚓1].

Manuscript Outline

- Chapter 1 presents classic methods for microcontroller programming, as well as existing work aimed at increasing the expressiveness and safety of microcontroller programming. We discuss the state of the art of the synchronous programming model, as well as classic techniques for providing guarantees about synchronous programs in the domain of critical embedded systems.
- Chapter 2 presents our implementation of the OCaml virtual machine, named OMicroB. This virtual machine, intended to run on microcontrollers with severe memory limitations, provides, through its portability and through the specific features of the OCaml language and its runtime library, a first level of abstraction for more expressive and safer microcontroller programming.
- In Chapter 3, we propose OCaLustre, a synchronous dataflow extension of the OCaml language. Intended for microcontroller programming and inspired by the Lustre language, this language extension provides a simple and resource-efficient programming model for developing the concurrent behaviour of an embedded system. After an overview of the language features, we describe in this chapter the aspects of its formal specification.
- In Chapter 4, we describe the main compilation steps for OCaLustre programs. This compilation process transforms synchronous code into a sequential OCaml program, fully compatible with any compiler for the language, while checking several static guarantees related to typing and scheduling of the software components of OCaLustre programs.
- In Chapter 5, we present methods for formalizing and verifying various properties derived from the language specification. In particular, we describe methods for checking the consistency of an OCaLustre program with two type systems whose rules define the correct semantics of a program.
- Chapter 6 describes a method for computing the worst-case execution time of an OCaLustre program. This method takes advantage of the portability of our approach by relying on analysis of the *bytecode* file generated after compiling such a program. We present the correctness proof of this process before describing the software prototype that implements it.
- Chapter 7 provides a performance analysis of the different software solutions presented in this manuscript. Using several example programs, each implementing features of the OCaml language, we describe the memory footprint and speed of the OMicroB virtual machine. We then detail the small memory footprint of the OCaLustre synchronous extension, as well as its speed performance.
- Chapter 8 presents several concrete applications, highlighting the advantages associated with our various levels of abstraction and with the verification of guarantees about the programs produced. In particular, they demonstrate how easy it is to describe the interactions between a microcontroller and its environment, and confirm the suitability of the chosen model for the limited resources of the devices we target.
- We finally conclude this thesis by recalling the various approaches proposed in our work, before discussing several extensions to our solutions intended to further increase the abstraction level and safety of the models considered.

1 Preliminaries

The different increases in abstraction proposed in this thesis rest on technical and technological choices motivated by practical considerations. This chapter provides an overview of state-of-the-art solutions concerning each of the abstraction levels considered. As we describe these various works, we justify the choices made in this thesis to follow one direction rather than another. We first discuss the physical composition of a microcontroller, as well as the classic programming methods used for embedded applications. We then motivate our desire to abstract the hardware through the virtual machine of a high-level language, rich in expressiveness and with a small runtime memory footprint. Our aim of providing a concurrent programming model that is both simple and lightweight is then supported by the choice of a synchronous dataflow programming model, suited to the nature of microcontroller programs. Finally, we discuss different ways to verify and guarantee certain properties of the programs built from these abstractions.

1.1 Physical and Software Characteristics of Microcontrollers

In order to convey the specificity of the issues and techniques related to microcontroller programming, this section presents a general view of the main hardware aspects of such generic integrated circuits. We briefly describe the structure of a generic microcontroller before discussing their technical characteristics, and in particular their memory limitations. We also discuss classic microcontroller programming methods, the development environments associated with them, and the limitations of these development processes.

1.1.1 Composition and Resources of a Microcontroller

A microcontroller is composed of several elements that allow it to perform the computations required to execute the programs dedicated to it, and to interact with the electronic circuit that forms its environment [BV97]. The main components of a microcontroller are the following:

- A central processing unit, or *CPU*, whose role is to perform the arithmetic and logical computations required for program execution. The computation speed of a resource-constrained microcontroller is generally measured in tens of MIPS[1].
- Random-access memory, or *RAM*, which contains the dynamic, volatile data generated during program execution. In the context of this thesis, RAM is certainly the most limited resource: on the order of a few kilobytes (KB) on the microcontrollers we consider, it forces the developer to control the memory consumption of the program very carefully.
- Non-volatile flash memory, intended to store the program code. Although flash memory is technically writable during program execution, it is customary to consider it read-only memory (or *ROM*)

[1] Millions of instructions per second.

because of the desire to separate data and programs on the one hand, and because of its physical limitations on the other. Flash memory is limited in its maximum number of writes. For example, the flash memory of an ATmega microcontroller can support (according to the manufacturer's specification) at most about 10,000 write cycles, whereas its RAM is virtually wear-free. Flash memory is generally ten to one hundred times larger than RAM. The flash memory of the microcontrollers we consider is a few tens or even a few hundreds of kilobytes: the ATmega328P microcontroller, for example, has 32 KB of flash memory.

- Counters (or *timers*) that make it possible to measure durations, in order to synchronize the various electronic components of an assembly with one another. These counters are incremented at regular time intervals. An internal clock signal, appearing whenever a program instruction is executed, determines the increment rate of the counters.

- An interrupt mechanism that triggers, as soon as a particular internal or external stimulus appears, the immediate execution of a specialised routine to handle it. External interrupts may come from the appearance (or variation) of an electrical signal on a pin (for example when a push button is pressed), while internal interrupts are caused, for example, by the *overflow* of a counter or by the pulses of an internal electronic oscillator.

- Input/output ports make it possible to communicate with the electronic components connected to the microcontroller. This communication is performed by exchanging electrical signals of variable voltage (for example 0 or 5 volts), representing the transmission of binary signals between the microcontroller and its environment. These ports correspond to a set of metal pins visible from outside the microcontroller, onto which wires or printed circuits are soldered to connect them to external electronic components. Each pin of a port generally has to be configured (typically by modifying the value of a bit in the register corresponding to the port in question) at the beginning of the program in order to declare it as an output or input interface.

- Some input/output ports also support the reading of analog values through *analog-to-digital converters* (or *ADCs*). These converters can translate, through temporal sampling, the variation of voltage values at the terminals of a pin into an analog value. In the same way, the translation of an internal digital signal into an external analog signal is made possible by the presence of a *digital-to-analog converter*, or *DAC*.

Figure 1.1 is a simplified schematic representation of the relationships between the main elements of a microcontroller. Interactions between them are mainly carried out through value transfers on one (or more) bidirectional buses, making it possible, for example, to communicate program instructions from flash memory to the CPU, or to write data produced by a computation to RAM.

Several microcontroller families currently exist on the market. Among them, AVR microcontrollers are widely used by amateur programmers because they are present in Arduino/Genuino boards [⚓15]. These boards, which contain a microcontroller and electronic components pre-connected to it (a USB port, a reset button, a power connector, etc.), simplify the development process for embedded programs. The Arduino Uno board [Bad14], for example, contains an ATmega328 AVR microcontroller with 2 kilobytes of RAM and 32 kilobytes of flash memory. Other microcontroller families, with different architectures, are also available, such as STM32 microcontrollers based on an ARM architecture and often equipped

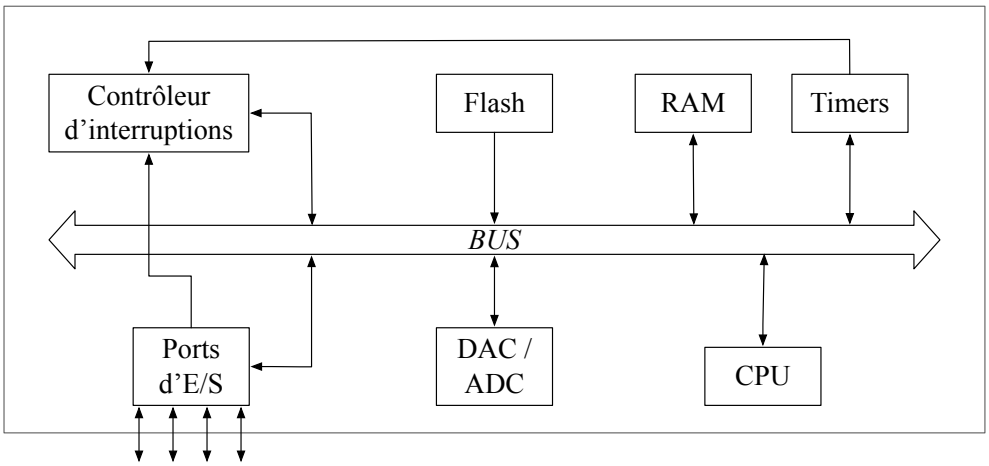


FIGURE 1.1: Simplified internal structure of a microcontroller

with more substantial resources. PIC microcontrollers, less well known among hobbyist audiences, are very commonly used by industry because of their low cost and efficiency.

As an example, the table in Figure 1.2 shows the physical characteristics of a set of microcontrollers from several families, some with extremely limited resources (less than one kilobyte of RAM) and others approaching the physical capabilities of PCs from the 1990s. In our use context, we are interested in microcontrollers closer to the lower end of the range, with fairly little flash memory (less than 100 kilobytes) and strongly limited RAM (less than 8 kilobytes), such as the PIC 18F4620.

Model	Architecture	Flash memory (KB)	RAM (B)	CPU speed (MIPS)	Operating voltage
AT89C51	Intel 8051 - 8 bits	4	128	12	4 to 6 V
ATmega328P	AVR - 8 bits	32	2048	20	1.8 to 5.5 V
ATmega2560	AVR - 8 bits	256	8192	16	1.8 to 5.5 V
PIC 18F4620	PIC - 8 bits	64	4096	10	2 to 5.5 V
PIC 24FJ128GA006	PIC - 16 bits	128	8192	16	2 to 3.6 V
STM32 L051C8	ARM - 32 bits	64	8192	32	1.65 to 3.6 V
STM32 F091VCT6	ARM - 32 bits	256	32768	48	2 to 3.6 V

FIGURE 1.2: Some microcontrollers and their characteristics

A microcontroller is therefore structurally very close to a simplified representation of a personal computer, but has far fewer resources. Its applications are, however, very different from those intended for classic PCs, or even single-board computers (such as the *Raspberry Pi*): it generally has neither an operating system, nor a file system, nor classic peripherals such as a keyboard or mouse. A microcontroller is typically dedicated to controlling an automated task, very often in an *embedded* context where it is connected only to an electronic circuit specific to that task, and where an application is executed "on bare metal': without any software intermediary between the hardware and the program.

1.1.2 Classic Microcontroller Programming Models

Low-Level Programming Languages

Building an application for a microcontroller is a fairly complex task that requires the developer to know the hardware being used well. Indeed, in an embedded system, the proximity between programs and hardware has traditionally led to the use of low-level programming languages, which offer little or no abstraction from the hardware for application development. Applications for embedded systems are often written in assembly language, with the goal of controlling hardware configuration and resource consumption precisely. Consequently, the choice of microcontroller has a significant influence on the development process, since the different available microcontroller families do not share the same assembly instruction set. A program written for a PIC microcontroller is therefore not suitable for execution on an AVR microcontroller, and vice versa. Physical differences between microcontrollers of the same family can also limit the portability of a program.

Relatively more portable subsets of the C language are also commonly used for programming embedded applications. The greater expressiveness of C compared with assembly is appreciated by embedded-application developers, while its fine control over memory resources is considered essential for developing applications intended to run on resource-constrained devices. Even so, programs traditionally written for microcontrollers remain fairly limited in hardware abstraction and portability, and offer only few static checks for certain program properties, such as the correctness of a program's typing.

For example, Figure 1.3 shows the source code of a C program intended for an ATmega328P AVR microcontroller. This program emits an electrical pulse at regular intervals on pin PB5 (i.e. pin number 5 of port B of the microcontroller), in order to blink a light-emitting diode (also called an *LED*) to which it is connected[2].

```
#ifndef F_CPU
#define F_CPU 20000000UL // 20 MHz clock speed
#endif

#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
  DDRB = DDRB | 0b00001000; // Configure PB5 as an output
  while(1)
  {
    PORTB = PORTB | 0b00001000; // Turn PB5 on
    _delay_ms(500); // wait half a second
    PORTB = PORTB ^ 0b00001000; // Turn PB5 off
    _delay_ms(500); // wait half a second
  }
}
```

Figure 1.3: A C program for an ATmega328P microcontroller

It is important here to note the fairly poor readability of such a program: interactions with input/output ports are performed by binary operations that modify, through masks, the bits of registers representing

[2]The operator ^ corresponds in C to the "exclusive or' between two bits

the program ports (`DDRB` configures the pins of port B as inputs or outputs, and `PORTB` modifies their values). In a more complex program, even a small error in this type of operation can be quite difficult for the developer to find, and can cause malfunctions that may damage the electronic assembly.

Some software libraries, such as the Arduino library, provide the programmer with C primitives that slightly abstract this type of operation (for example, by using a `digitalWrite()` function taking the name of a pin and the signal to emit as parameters), but they remain fairly limited and are comparable to simple macros that make programs a little easier to read and write without bringing additional safety to the programs produced.

The portability of such a program is also strongly limited: for example, some AVR microcontrollers do not use the same port names, and porting this program to a microcontroller from another family leads to deep changes related to the chosen microcontroller (for example, port-register configuration differs between AVR and PIC microcontrollers: a bit set to 1 in the configuration register of a port represents a pin configured as an output on an AVR microcontroller, whereas it represents a pin configured as an input on a PIC).

Development Environments

The development environments used to create and debug microcontroller programs are themselves complex and not very modular. A few proprietary solutions, generally single-platform, are provided by manufacturers. For example, the integrated development environment (*IDE*) MPLAB [⚓10] makes it possible to program, simulate, and transfer programs onto PIC microcontrollers. *AtmelStudio* [⚓8], for its part, is an IDE for AVR microcontrollers offering more or less the same functionality as MPLAB. Finally, let us mention the *Arduino IDE*[⚓7], a cross-platform application that makes it possible to write programs for Arduino development boards, compile them, and transfer the generated executables to the board. Figure 1.4 shows the very simple user interface of this software.

Most of these consumer-facing tools are fairly modest in functionality: they can generate an executable for the microcontroller, but do not necessarily make it possible to test the resulting program properly, since the behaviour of such a program is intrinsically tied to the surrounding electronic assembly. More powerful software solutions, such as the Proteus software suite [⚓17], therefore make it possible to design electronic assemblies with computer assistance and to simulate microcontroller programs in executable form. Unfortunately, these software solutions are very expensive and quite complex to approach, and are therefore difficult for non-specialist developers to use. It is therefore not uncommon for programmers of small embedded applications to prefer an *in-situ* debugging method, consisting in directly using the physical microcontroller to test their program: with a serial connection to a computer, they analyse the program's responses to several stimuli. This debugging method is then a long and painful task, and several errors in the program code, which could have been detected before transfer to the physical assembly, can slow down the development cycle of a program.

Programmers wishing to turn toward *open source* solutions can use a handful of free C compilers, most based on GCC, that are available on several platforms. For example, the avr-gcc compiler [⚓25], together with the avr-libc standard library and the avr-binutils binary tools, forms a complete compilation chain for AVR microcontrollers. The generated executable program can then be sent using the avrdude tool (*AVR Downloader UploaDEr*), provided with the compiler. On the PIC side, ongoing work within the *sdcc* compiler [⚓27] enables the translation of C source code for devices in the PIC16 and PIC18 families. The

FIGURE 1.4: The Arduino integrated development environment (IDE)

usbpicprog tool [⚓23] is, for its part, an *open source chip programmer* (i.e. a device that transfers a program from a computer to the flash memory of a microcontroller) for these microcontrollers.

Microcontrollers are programmable integrated circuits with fairly limited resources and development environments. Microcontroller programming methods generally follow this resource scarcity, and are therefore traditionally based on the use of low-level languages. As a result, the use of these languages may discourage inexperienced developers who are not very familiar with such technologies. Development in assembly languages or in C leads to programs that are rather long to write and that may, moreover, be sources of errors that are difficult to detect.

It therefore seems relevant to us to explore different programming models, inspired by the abstractions used for developing applications for personal computers, with the goal of providing microcontroller programming with tools that are easier to grasp. In particular, we are interested in the use of so-called *high-level* languages for programming embedded applications.

1.2 Hardware Abstraction and High-Level Languages

In the more common context of programming for personal computers (or for equivalent modern devices such as smartphones or tablets), it is now very rare for programs to be written directly in assembly languages. In the same way, use of the C language is increasingly limited to specific domains, such as the development of system applications that by nature are meant to manipulate memory directly. It is now

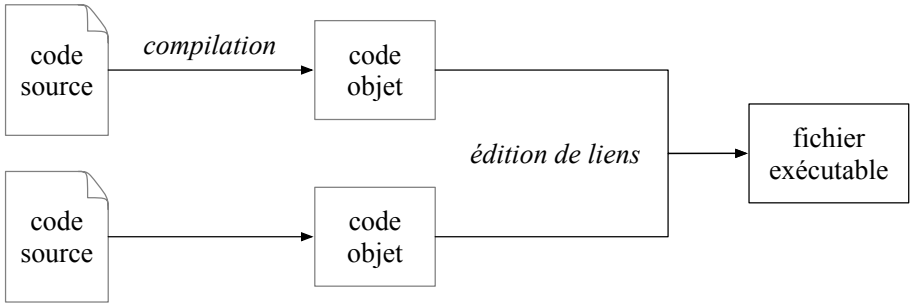


FIGURE 1.5: Compilation to a native program

commonplace to program applications with which the user interacts directly (mobile-phone programs, web applications, ...) in high-level languages, which make it easier to program complex applications.

High-level programming languages are languages whose expressive richness abstracts away from the computation model of the machine on which programs are executed. They offer rich control, implementing varied programming paradigms that can manipulate, for example, functional values, objects, exceptions, or continuations. These languages natively provide rich data structures, making it easier to build complex applications. Several high-level programming languages use a notion of static typing to improve program safety, and rely on runtime environments that provide automatic memory management for a program. The debugging process for applications written in these languages is simplified by the use of symbolic debuggers, which manipulate the complex data structures of the language. Finally, these high-level languages include mechanisms for interoperability with low-level languages in order to interface with hardware.

It therefore seems natural, in order to bring these same advantages to microcontroller programming, to consider using such languages for embedded-system programming, instead of the classic assembly/C language pair. In this section, we also discuss the compilation of these high-level languages to non-native code, interpretable in a *virtual machine* (or *abstract machine*) positioned between the program and the hardware. Below, we describe the different experiments and systems from the state of the art that enable high-level programming on microcontrollers using this *virtual-machine approach*.

1.2.1 The Virtual-Machine Approach

In a highly simplified view, the classic compilation process for a program consists in translating its source code into native code that can be "understood' by the hardware on which it will be executed. Figure 1.5 describes such a mechanism: source files are translated into the native language of the machine for which the program is intended. In the case of low-level programming languages, this translation is relatively easy, given the proximity between the instructions of the language and those of the hardware. Assembly language is merely a human-readable version of the processor's binary instructions, and compiling a C program is fairly direct (as long as one does not seek to optimise the produced code), since the language's imperative control operators are very close to those of the hardware.

Compiling programs that implement features of high-level programming languages (functional programming, object-oriented programming, ...) is a more complex process. Since physical processors can natively handle only simple imperative operations, several transformations are applied during compilation to such a program in order to convert high-level features into purely imperative native code. Compilers for high-level languages therefore manipulate many intermediate program representations,

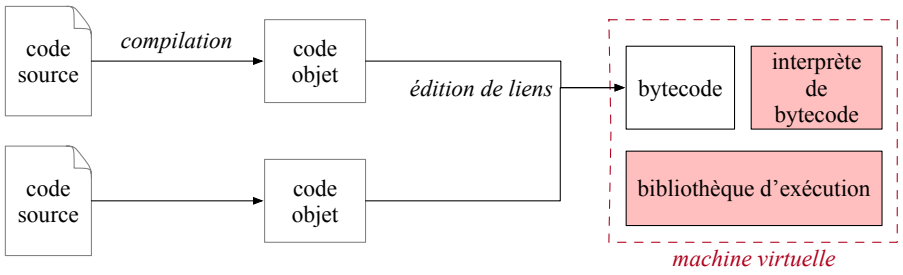


FIGURE 1.6: Virtual-machine approach: compilation to *bytecode* and interpretation

and these representations are the same whether the final program is intended to run on a machine with architecture A, or on hardware with a completely different architecture B.

The virtual-machine approach consists in following the program compilation process up to the production of a common, non-native representation of programs: *bytecode*. It is then left to a *virtual machine*, that is, a bytecode *interpreter* associated with a *runtime environment*, to execute the program in this intermediate form [DS00]. This approach enables the *portability* of the applications produced, since any program translated to the language bytecode can be executed by any device, provided that it has an interpreter adapted to this bytecode. The use of virtual machines for executing rich programs was popularized by the Java language and its JVM (*Java Virtual Machine*), whose motto *"Write once, run anywhere'* illustrated the ability of the same Java bytecode to be executed on many different platforms.

Figure 1.6 shows how the source code of a program is compiled in order to be interpreted by a virtual machine.

The *bytecode* of a high-level language is composed of instructions that are "richer' than native machine code. Thus, a *bytecode* instruction typically corresponds to a sequence of instructions in the machine language. Thanks to this *factorization*, a program translated to bytecode can therefore be more compact than its equivalent in machine language. Of course, bytecode is not executable without its runtime environment, and the size of that environment must therefore also be taken into account in order to make an honest comparison. Nevertheless, since this size is fixed, the total memory footprint of a bytecode program and its virtual machine can, in the case of substantial programs, be smaller than that of a program compiled to the machine language. Such compaction of the total program size is an obvious advantage in our use context, where memory resources must be used sparingly.

In addition, the use of a bytecode common to all execution platforms also makes it possible to factorize several analyses on programs, such as the estimation of memory resources [AGG07], computation time [SP06], vulnerability detection [LL05], or even plagiarism detection [JWC08]. Even though some analyses may lose precision because of this factorization [LF08], it remains valuable for our use case, given the variety of existing microcontroller architectures and models.

1.2.2 Microcontroller Programming in High-Level Languages

A large number of projects aiming to run high-level languages on microcontrollers have been carried out in recent years. In this section, we study a sample of the various projects that enable the development of programs implementing varied programming paradigms, such as object-oriented programming or functional programming. Some of these solutions are based on native compilation models, which consist

in generating machine code from the source code of a program, while others take advantage of the virtual-machine approach and of the common bytecode representation.

High-Level Languages and Native Compilation Models for Microcontrollers

Using a high-level language does not systematically imply using a virtual machine and a bytecode interpreter. Several solutions that aim to execute programs written in high-level languages rely on a model of compilation to native code, which can be executed directly on the target hardware without any software intermediary. Among these solutions, it is possible to write C++ programs, a multiparadigm language whose proximity to C makes it possible to preserve good performance, using for example the *avr-g++* compiler for AVR microcontrollers, or the *MPLAB XC32++* compiler, which supports C++ compilation but only for 32-bit PIC microcontrollers. The *IAR Embedded Workbench* compiler, for its part, can compile C++ programs for microcontrollers from various families, such as MSP microcontrollers, AVR microcontrollers, and STM32 microcontrollers. Nevertheless, C++ does not benefit from all the abstractions provided by higher-level languages, such as automatic memory management for programs, or a safer type system.

Several works have also aimed at executing programs written in the ADA language on microcontrollers; ADA is an object-oriented programming language often used in critical embedded systems (automotive systems, aeronautics, etc.) [Reg12]. However, some of them cannot offer all the advantages of the language because of their large memory footprint [RP19]. Thus, the AVR-Ada compiler project [⚓24] is compatible with resource-constrained 8-bit AVR microcontrollers, but does not allow, for example, use of the *Ravenscar* profile intended for programming safe real-time systems. The *GNAT GPL* compiler developed by Adacore, for its part, benefits from all the *profiles* available in the language, but can target only ARM system-on-chip microcontrollers whose resources are much greater.

Other high-level languages, such as Rust, are compiled to a common intermediate representation before being translated to the language specific to the machine. This representation, the *bitcode* of the *LLVM* compiler infrastructure (historically meaning *Low-Level Virtual Machine*), is equivalent to a generic low-level bytecode that can be translated to machine code specific to each target [Lop09]. This *bitcode* could also be interpreted, even though this approach is fairly rare. A university project aiming to build a *bitcode* interpreter for MSP430 microcontrollers thus constituted an original approach [Cam16], despite the slowness observed in the prototype. Other work is under way to compile Rust programs to AVR microcontrollers using the LLVM *backend* for AVR [⚓26]. Another project targets the execution of programs written in a subset of *Go* on any target supported by LLVM [⚓28]. Unfortunately, microcontroller support in the LLVM project is very limited: outside the work on AVR, only certain microcontrollers based on an ARM architecture are supported, and support for PIC microcontrollers, for example, was abandoned in 2011.

High-Level Languages and Their Virtual Machines on Microcontrollers

Solutions based on a virtual-machine approach seem to us the most likely to easily provide a higher-level programming model for resource-constrained microcontrollers. They increase the portability of the programs produced, thanks to the generic nature of the bytecode generated during compilation, which abstracts away from the hardware on which the virtual machine is executed. To illustrate the advantages

of this approach, in this section we describe a sample that seems representative of different projects implementing virtual machines for microcontrollers from various families.

Java on AVR and MSP: Because of its popularity, the richness of its applications, and its philosophy oriented toward program portability, Java is an obvious candidate for using a high-level language on a microcontroller. Several projects and experiments have therefore ported the Java virtual machine, called the *JVM* (*Java Virtual Machine* [LYBB14]). These projects allow embedded-application developers to benefit from the richness of Java, which offers an object-oriented, imperative, and more recently functional programming model, as well as static typing with nominal subtyping, but also from the very rich class library provided by the language (the Java *API*), which makes it easy to represent advanced data structures.

For example, the industrial solution microEJ [⚓16] enables the execution of Java programs on development boards containing microcontrollers mostly based on an ARM architecture. The Darjeeling system [BCL08], for its part, consists of a multi-threaded Java virtual machine that enables the execution of Java bytecode on AVR128 and MSP430 microcontrollers, which contain between 1 and 8 kilobytes of RAM and between 16 and 128 kilobytes of flash memory. Other solutions, such as HaikuVM [⚓13] or NanoVM [⚓14], enable the execution of Java bytecode on microcontrollers used in Arduino development boards.

The richness of the Java language can, however, sometimes be incompatible with these solutions designed to run on resource-constrained machines. Java manipulates fairly large data, which themselves depend on many objects. A program written in standard Java style, and therefore using objects and complex data types, may require a large amount of memory that is not necessarily available on such devices. The example of *Java Card* [Gut97], a system that enables Java programs to run on smart cards, illustrates this difficulty of using the entire Java language on resource-constrained hardware: version 2 of Java Card can run on hardware with only 2 KB of RAM and 64 KB of ROM, but then contains no garbage collector, does not handle multitasking, and is limited to fairly simple data types (for example, there is no support for multidimensional arrays or collection-manipulation classes). Version 3 of Java Card addresses these shortcomings, but at the cost of much higher resource consumption: it requires 24 KB of RAM and at least 128 KB of ROM to run. In the same way, the TinyVM project [HPK+09], intended to build a low-memory-footprint Java virtual machine for RCX microcontrollers (included in *Lego Mindstorm* programmable bricks), originally targeted sensor-network development, but eventually became part of the LeJOS project [LHS10] (also dedicated to running Java programs on programmable bricks), which is richer in functionality but more resource-hungry.

Python on STM32: MicroPython [Bel17] is an implementation of the Python 3 language intended mainly to run on a board named *pyboard*, which contains an STM32F405RG microcontroller with a Cortex-M4 processor clocked at 168 MHz, 1024 kilobytes of flash memory, and 192 kilobytes of RAM. MicroPython makes it possible to execute Python 3 programs, benefiting from the advantages and particularities of the language, such as its clear syntax that is easy for beginners to approach, its implementation of object-oriented programming, and certain high-level features (such as list comprehensions) that make it possible to develop complex programs concisely. MicroPython is based on *CPython*, the reference implementation of the language, which provides a bytecode interpreter for translated Python programs. An interesting capability of MicroPython is that it can communicate, from a computer, with an interaction loop (*REPL* -

Read-Eval-Print-Loop) running on the microcontroller [VVF18]. This communication makes it possible to perform small tests quickly and to debug programs easily.

MicroPython is intended for microcontrollers with relatively rich resources, beyond the capabilities of the microcontrollers considered in this thesis. The lower bound on the technical capabilities of microcontrollers able to run the MicroPython interpreter is indeed 16 KB of RAM and 256 KB of program memory. Other work has also built a Python interpreter for execution on microcontrollers [BSR12], but this work also targets devices with fairly substantial resources: Cortex-M3 processors clocked at at least 50 MHz, equipped with at least 64 KB of RAM and 512 kilobytes of flash memory. Finally, the *python-on-a-chip* [⚓12] projects, followed by *PyMite* [Hal03], now abandoned, made it possible to execute Python programs on microcontrollers with fewer resources (about fifty kilobytes of flash memory and less than 8 kilobytes of RAM were recommended for executing a program), but at the cost of limited functionality: only a subset of Python 2.5 was supported, and no standard library was provided. The body of work concerning Python also illustrates the difficulty of porting all the features of a multiparadigm language to hardware with very limited resources.

Scheme on PIC: Scheme is a functional programming language derived from Lisp. Scheme has strong expressive power thanks, for example, to its *macro* system, as well as to the explicit manipulation of continuations in programs through the *call-with-current-continuation* instruction (*call/cc*). There are multiple Scheme implementations targeting personal computers. They are based on specifications (the "*Revised Reports on the Algorithmic Language Scheme*", or *RnRS*, where *n* corresponds to the revision number of the report) that define the features of the language. Not all of these implementations (for example *Bigloo* [⚓18]) rely on a virtual machine, but some of them, for example *Guile* [⚓22] (an implementation of Scheme respecting the R6RS standard), rely on translating Scheme code to bytecode made up of 175 different instructions. An interpreter, written in C, executes this bytecode. A few virtual machines capable of executing subsets of Scheme on resource-constrained microcontrollers have been built. Among them, BIT [DF05] and PICBIT [FD03] enable execution of the R4RS standard on microcontrollers with less than 8 kilobytes of RAM and 64 kilobytes of program memory, while the PICOBIT system [SF09] allows Scheme programs written in subsets of R5RS to run on PIC18 devices that may be limited to 1 kilobyte of RAM and 6 kilobytes of ROM. Because of its lightness, Scheme thus seems better suited to resource-constrained devices.

OCaml on PIC: OCaml is a language that combines many programming paradigms. Since its creation, it has implemented object-oriented, functional, modular, and imperative programming features. This variety of models makes it easier to develop complex programs expressively. Its strong static typing, combined with a type-inference mechanism, allows it to ensure at compile time that there are no inconsistencies in the use of typed values in a program. The standard virtual machine of the OCaml language, called *ZAM* and derived from the *ZINC* virtual machine [Ler90], is a stack machine based on a uniform representation of data. The bytecode associated with the OCaml VM consists of 148 instructions, which provide classic computation and branching operators, as well as instructions dedicated to manipulating functional values and applying them. The OCaPIC project [VWC15] is an implementation of the standard OCaml virtual machine for programming PIC18-family microcontrollers. OCaPIC thus enables the execution of the entire OCaml language on resource-constrained microcontrollers (4 kilobytes of RAM and 64 kilobytes of flash memory). This ability to execute the entire OCaml language on microcontrollers

with such limited resources is noteworthy, and demonstrates both the lightness of the OCaml virtual machine and the power of the optimisations provided by this implementation. OCaPIC also provides several tools intended to improve the development of OCaml programs for microcontrollers, including in particular *ocamlclean*, a tool that statically eliminates unused closure allocations in a program, as well as a set of simulators that simplify debugging of the programs produced.

Figure 1.7 is a Venn diagram representing the various technologies discussed in this section.

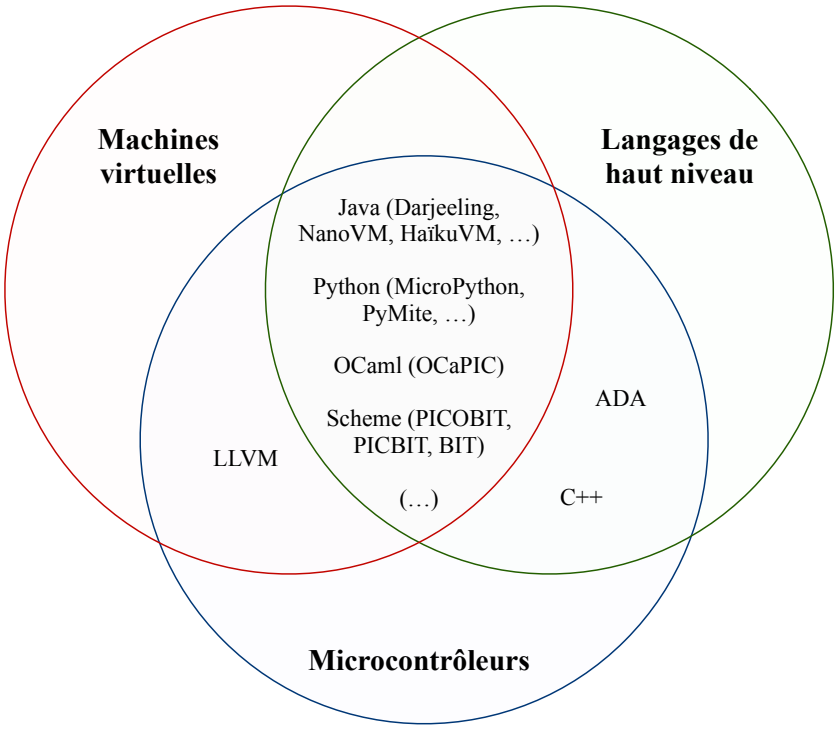


FIGURE 1.7: Microcontrollers, virtual machines, and high-level languages

The table in Figure 1.8 summarizes the main characteristics of the approaches presented. Virtual-machine-based approaches have several interesting advantages for microcontroller programming. They provide features from high-level languages, with which modern application developers are familiar, while potentially reducing the memory footprint of programs thanks to the factorization inherent in representing programs as bytecode. Nevertheless, some of these virtual machines are sometimes poorly suited to use on resource-constrained microcontrollers, which offer only a few kilobytes of RAM and less than one hundred kilobytes of flash memory. Memory limitations can restrict the use of all aspects of a program. Moreover, most of these solutions are sometimes not very portable: they often target a specific range of microcontrollers, and it can be difficult to adapt them to a different architecture.

In the context of our work, we consider the approach adopted by OCaPIC to be the most suitable for microcontroller programming. The OCaml virtual machine is light enough to support the execution of all language features on microcontrollers with fairly limited memory resources, and the language itself has advantageous aspects, such as better program safety thanks to static typing, as well as an automatic memory-management system. Nevertheless, like most of the solutions discussed in this section, OCaPIC is fairly limited in its portability, since it is available only for PIC18 microcontrollers. In the following

Language	Implementation	VM / Interpreter?	Microcontroller (family)
Java	JVM	yes	-
	Darjeeling	yes	AVR, MSP
	NanoVM	yes	AVR
	HaikuVM	yes	AVR
	MicroEJ	yes	ARM
Python	CPython	yes	-
	MicroPython	yes	STM32
Scheme	Guile	yes	-
	BIT	yes	Motorola 68HC11
	PICBIT	yes	PIC18
	PICOBIT	yes	PIC18
OCaml	ZAM	yes	-
	OCaPIC	yes	PIC18
ADA	GNAT	no	STM32
	AVR-Ada	no	AVR
C++	avr-g++	no	AVR
	IAR	no	AVR, MSP, STM32
	MPLAB XC32++	no	PIC32, SAM
C, Go, Rust, ...	LLVM	generally no (intermediate representation)	AVR, MSP

FIGURE 1.8: Implementation of programming languages on microcontrollers

chapter, we will therefore propose a *generic* OCaml virtual machine intended to run on many different targets, while preserving a small memory footprint and a satisfactory computation speed for a rich language.

1.3 Synchronous Programming

The role of a microcontroller is to be the *conductor* of an electronic assembly: it must react to stimuli coming from the electronic components to which it is connected (sensors, buttons, ...) in order to emit signals to certain components of the circuit (LCD screen, actuators, ...). For example, inside a computer keyboard, a microcontroller reacts within an imperceptible delay to the various key presses made by the user. Microcontroller programs often require the hardware to react quickly to input signals, regardless of the order in which they appear: pressing certain keys on the keyboard must not, for example, mask the simultaneous pressing of another. Thus, microcontroller programming is inherently concurrent: the software components of an embedded program that handle each signal from the system environment must generally give the impression of reacting *at the same time*.

Embedded systems, sometimes critical ones, therefore exhibit concurrent behaviours subject to more or less strict timing constraints. Such systems are therefore often described as *real-time systems*.

1.3.1 Real-Time Systems

A real-time system is a computer system in which the reaction and computation times of the different components of a program (called *tasks*) are subject to constraints that must be respected. These tasks

concurrently handle the appearance of the stimuli to which the program is supposed to react. These stimuli may appear, during program execution, periodically or sporadically.

The construction of real-time systems introduces the notion of *scheduling*, which consists in defining the order in which the different tasks of the program will be executed (generally cyclically), while taking into account the constraints associated with them (such as their frequency, priority, or deadline).

In the case of periodic tasks, program scheduling can be performed statically, before program execution: it is then possible to use software solutions, such as the Cheddar tool [SLNM04], to automatically establish (if one exists) an execution order for the different tasks of a program that is compatible with their timing constraints. Figure 1.9 shows an example of scheduling performed by Cheddar for three different periodic tasks. In a system containing only periodic tasks, and without preemption, program execution may simply correspond to successive calls to the functions representing each task, for example by traversing a table representing the scheduling sequence.

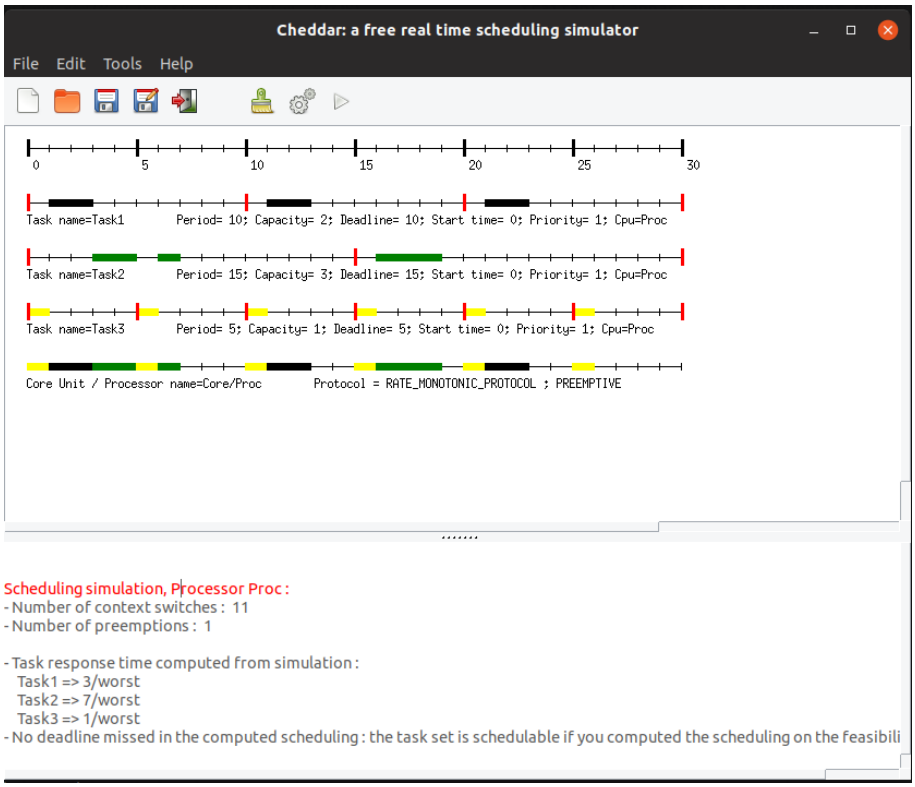


FIGURE 1.9: Static scheduling of three periodic tasks with Cheddar

For sporadic tasks, which handle the appearance of one-off events, scheduling cannot be performed "*offline*" (during program design), since the instants at which these events appear are not predictable. The scheduling of the different tasks of a program is then performed dynamically, as the events to which the program must react appear. The schedulability of the system can nevertheless be guaranteed statically as soon as the minimum delay between two appearances of sporadic events of the same type is known.

A real-time program generally corresponds to a system combining periodic stimuli and sporadic events. Real-time systems therefore use software *schedulers*, included in *real-time operating systems* (*RTOS*), such as FreeRTOS [GPPT16], which execute alongside the program and dynamically hand control to tasks, for example according to their priority level and the appearance of events.

Static or dynamic scheduling processes, as well as the use of real-time operating systems, are rather constraining and sometimes difficult to set up on certain devices: the memory resources required to execute the software components responsible for scheduling and creating concurrent tasks are not negligible, and they are sometimes unavailable on the microcontrollers we consider (which have only a few kilobytes of memory). Moreover, even when the schedulability of a program has been established, errors that are sometimes difficult to predict may still occur because of concurrent access to resources shared by several tasks of a program, through the use of synchronization primitives (mutual exclusion, synchronization barriers, etc.). This is the case, for example, for priority-inversion phenomena, in which a lower-priority task may monopolize a shared resource and never yield control to a higher-priority task. Such phenomena can have unfortunate consequences: the software of *PathFinder*, the NASA space probe sent to Mars in 1997, encountered a priority-inversion problem a few days after landing, which caused several untimely software restarts [Jon97][3].

In the context of this thesis, we adopt a concurrency model that is simpler to approach and capable of guaranteeing that this type of unexpected behaviour cannot occur. This model, *synchronous programming*, has proved itself for programming critical embedded systems (aircraft, nuclear power plants, ...), and in our view represents a suitable solution for concurrent microcontroller programming [VVC16]. The synchronous programming model makes it possible to represent the concurrent aspects of a program without requiring software machinery responsible for managing the program tasks. Indeed, synchronous programming can be considered a programming model for real-time systems with *static* and *deterministic* scheduling, which does not impose the use of an embedded software scheduler. This does not prevent its use in contexts where schedulers are used, for example after compiling synchronous components into periodic tasks executed on real-time platforms [PFB+11]. However, the lightweight nature induced by the approach of building programs that can execute on hardware without any operating system makes it a particularly compatible paradigm for programming resource-constrained microcontrollers.

1.3.2 Synchronous Hypothesis

The simplicity and expressive power of the synchronous programming paradigm rest on an abstraction principle called the *synchronous hypothesis*, which states that the time taken by the different components of a program to compute output values from input values is considered to be zero. This hypothesis is comparable to the abstractions used in circuit design (this comparison is taken from [BB91]): the time taken by an electrical signal to pass through a set of logic gates is generally ignored when designing electrical circuits. Similarly, in Newtonian mechanics, the propagation speed of a gravitational field (i.e. the speed of light) is not considered, and interactions between bodies are viewed as instantaneous. In a synchronous program, the inputs of the program are therefore assumed to be instantaneous with its outputs, and all program instructions are considered to be carried out within the same logical instant, called a *synchronous instant*. Of course, for this hypothesis to be valid, one must either be able to guarantee that the delay between two inputs is sufficient, or consider using a buffer so that none are lost. The abstraction introduced by the synchronous hypothesis is represented in Figure 1.10. It illustrates the fact that the real execution time of the code needed to process inputs and create outputs (at the top of the figure) is no longer considered by the abstraction: the outputs o_n are produced "*at the same time*" as the arrival of the inputs i_n (at the bottom of the figure).

[3] Sending a patch from Earth nevertheless made it possible to solve the problem.

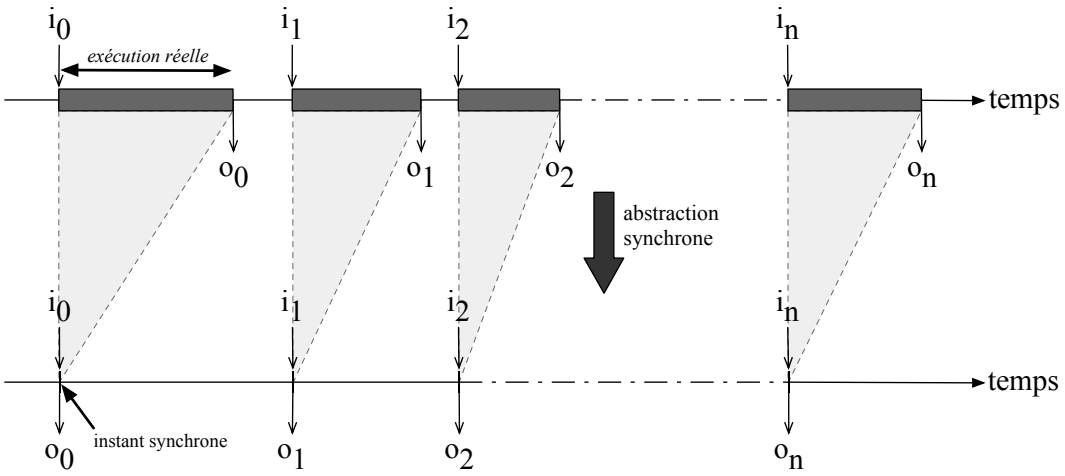


FIGURE 1.10: The synchronous hypothesis:
The computation time of a program's output values (o_n) from the value of its inputs (i_n) is considered to be zero.

Executing a synchronous program is a cyclic task that consists, at each program instant, in reading the program inputs (typically values coming from the system environment) and computing output values that will be emitted at the end of the instant. The synchronous hypothesis is satisfied as long as, at each instant, the interval between two program inputs is greater than the time needed to compute the output values. The program is thus synchronized with its inputs, and its reaction time can be considered instantaneous.

The synchronous hypothesis therefore simplifies reasoning about the concurrent aspects of a program by removing, from the developer's point of view, the temporal synchronization considerations between the different software components of an application.

1.3.3 Esterel: An Imperative Synchronous Programming Language

The synchronous programming model emerged in the early 1980s, and one of the first languages implementing this model was then created by a team of researchers from the Ecole des Mines and INRIA Sophia-Antipolis. This language, named *Esterel* [BC84], is based on an imperative style in which the various components of a program communicate by broadcasting potentially valued *signals*, which allow information to be exchanged between the concurrent components of a program. Esterel, whose *control-flow* semantics is driven by the appearance of events during program execution, was initially designed to program industrial robots with a high level of abstraction.

An Esterel program is composed of several *modules*, each responsible for a task to be carried out. The body of a module is code with imperative aspects, containing classic operators such as the sequence operator (`;`) or the conditional operator (`if`), to which synchronous operators are added, such as a parallelization operator (`||`) as well as operators for emitting (`emit`) or waiting for (`await`) signals.

A common example of Esterel programming is shown in Figure 1.11. This example defines a module named `ABRO`, whose role is to wait, in parallel, for the presence of signal `A` and signal `B` before emitting a signal `O`. The presence of signal `R` resets the behaviour of the program.

```
module ABRO:

% Interface
input A, B, R;
output O;

% Corps
loop
   [ await A || await B ];
   emit O
each R
end module
```

FIGURE 1.11: Example Esterel program: the ABRO module

Beyond the programming of industrial embedded systems, Esterel's event-based model is now used to program several types of applications. For example, the *ReactiveML* programming language [MP05] is a synchronous reactive extension of OCaml that reuses many aspects of Esterel. This language, also based on a signal emission/reception model, is intended for programming rich applications that benefit from the concurrent features offered by the synchronous programming model. ReactiveML programs cover a wide range of uses, including musical applications, games, physical-system simulations, and more classic algorithmic applications.

For example, the ReactiveML program in Figure 1.12, taken from the official ReactiveML website [⚓21], performs a breadth-first traversal of a binary tree by concurrently executing the traversal of the left subtree and the right subtree of the tree considered.

```
(* Definition of binary trees. *)
type 'a tree =
  | Empty
  | Node of 'a * 'a tree * 'a tree

(* Breadth first traveral. *)
let rec process iter_breadth f t =
  match t with
  | Empty -> ()
  | Node (x, l, r) ->
      f x;
      pause;
      run (iter_breadth f l) || run (iter_breadth f r)
```

FIGURE 1.12: Example ReactiveML program

Synchronous reactive programming is also well suited to applications in expanding domains, such as web-application programming. For example, the *Pendulum* language [SC16b] combines the algorithmic aspects of OCaml with an Esterel-inspired synchronous reactive model for developing rich multimedia applications for the web, relying on the *Js_of_OCaml* engine [VB14], which translates the bytecode of an OCaml program into a JavaScript application.

1.3.4 Lustre and Signal: Declarative Synchronous Programming Languages

At a time very close to the development of Esterel, other synchronous programming languages were developed, though inspired by different models. In particular, *Lustre* [CPHP87] also appeared in the early 1980s, developed by researchers at the Grenoble laboratory VERIMAG. Unlike Esterel, this synchronous programming language is not based on reaction to punctual events, but on a *dataflow* programming model, making it possible to represent the evolution of values over time. Lustre was initially developed to provide a programming language usable by control engineers accustomed to the declarative formalism of a dataflow model [Hal05].

Following the model of the Lucid language [AW77][4], all values manipulated by a Lustre program are data streams: sequences of values that may vary over the execution of a program. Each stream therefore has, by default, a value at every instant of the program, and this value may change from one synchronous instant to the next. Thus, a variable x in Lustre corresponds to the stream of all values taken by x from instant to instant:

$$x \equiv (x_0, x_1, x_2, x_3, \dots, x_i \dots)$$

A constant then corresponds to an invariant stream of values:

$$2 \equiv (2, 2, 2, 2, \dots, 2, \dots)$$

The arithmetic and logical operators of the language apply pointwise to the values taken by streams during program execution:

$$x + y \equiv (x_0 + y_0, x_1 + y_1, x_2 + y_2, x_3 + y_3, \dots, x_i + y_i, \dots)$$

In a way similar to Esterel modules, the basic software component of a Lustre program is the *node*. A node can be considered a function that associates a stream of outputs with a stream of inputs. The body of a node is a system of equations, similar to temporal functions [CH86], that declare variables whose values may change at each execution instant. Each of these streams is computed in the same synchronous instant, and Lustre's synchronous dataflow model is therefore a restricted class of Kahn process networks (*Kahn Process Networks - KPN* [Kah74]) in which communications can be performed without *buffers* [MPP10].

The declarative style of a Lustre program is quite similar to the structure of a functional program: each equation defines a variable whose value depends on the instant, and the system of equations in the body of a node is a set of variable declarations. The "imperative' considerations of program execution are absent from the semantics of the language: the reading order of equations does not reflect their computation "order'.

For example, Figure 1.13 defines a node named `BOOLOPS`, which receives two Boolean streams `A` and `B` as inputs, and computes as outputs the streams `ANDB`, `ORB`, and `XORB`, corresponding respectively to the computation of $\texttt{A} \wedge \texttt{B}$, $\texttt{A} \vee \texttt{B}$, and $\texttt{A} \oplus \texttt{B}$.

In the original version of Lustre, the available operators are the classic arithmetic and logical operators, as well as several *temporal* operators:

[4] LUSTRE was originally an acronym formed from the French phrase "LUcid Synchrone Temps Reel'.

```
node BOOLOPS (A:bool; B:bool) returns (ANDB:bool; ORB:bool; XORB:bool);
let
  XORB = ORB and (not ANDB);
  ANDB = if A then B else false;
  ORB = if A then true else B;
tel;
```

FIGURE 1.13: A Lustre node that computes the "and', "or', and "exclusive or' of its inputs

- The memory operator `pre`, which gives access to the value of a stream at the previous instant:

$$\texttt{pre}\ \ a \equiv (nil, a_0, a_1, a_2, \ldots, a_{i-1}, \ldots)$$

- The initialization operator `->`, which makes it possible to define a stream from the value of one stream for the first instant and another stream of values for the following instants:

$$a\ \ \texttt{->}\ \ b \equiv (a_0, b_1, b_2, ..., b_i, ...)$$

  It is then possible to define the sequence of positive integers in Lustre as follows:

```
n = 0 -> pre n + 1
```

– Finally, the sampling operator `when`, which makes it possible to slow down the execution of program components by conditioning the presence of streams on Boolean values (the *clocks*):

$$(a\ \ \texttt{when}\ \ x)_n \equiv \begin{cases} a_n \text{ if } x_n = \texttt{true} \\ \bot \text{ otherwise} \end{cases}$$

  The symbol $\bot$ represents the absence of a value.

  Each Lustre data stream is implicitly coupled to a clock, which by default is the fastest global clock. It can be explicitly restricted to a slower clock using the `when` operator; for example, the following program excerpt forces the stream `x` to be defined only when `b` is true:

```
x = 4 when b;
```

The table in Figure 1.14 shows a simulation of the execution of these three operators:

instant	0	1	2	3	4	5	...
c	true	false	true	true	false	true	...
x	x_0	x_1	x_2	x_3	x_4	x_5	...
y	y_0	y_1	y_2	y_3	y_4	y_5	...
`pre` y	*nil*	y_0	y_1	y_2	y_3	y_4	...
x `->` `pre` y	x_0	y_0	y_1	y_2	y_3	y_4	...
x `when` c	x_0	$\perp$	x_2	x_3	$\perp$	x_5	...

FIGURE 1.14: Lustre temporal operators

The compilation of a Lustre program, called *single-loop compilation*, consists in transforming the program into a sequential program made up of a single loop that polls the value of its input streams at the beginning of the synchronous instant, computes the value of its output streams, and emits those values at the end of the instant.

Signal

Based on a declarative model close to Lustre, Signal [GG87] is a synchronous dataflow programming language intended for programming real-time systems. Developed in the 1980s by a team at the IRISA laboratory in Rennes, it constitutes the third major synchronous language developed in France at that time.

Signal is a relational language that in some sense combines Lustre's dataflow view with Esterel's event-based view of signals. Indeed, a program in Signal defines (in an equational form comparable to Lustre) signals, viewed as data sequences whose values are not necessarily present at every execution instant of the program. To represent this absence, Signal shares the notion of clock with Lustre: the set of instants during which a signal has a value corresponds to its clock. Signal includes a notion of synchronicity, making it possible to establish that two signals are on the same clock, and are therefore *synchronous*, using the operator `^=`.

Signal defines temporal operators quite comparable to those of Lustre. For example, the `when` operator is similar to Lustre's, and the `$` operator, placed after the name of a signal, corresponds to Lustre's `pre` operator. One can thus, for example, define a counter `CPT` that increments at each instant and is reset to zero when a `RESET` event arrives, as follows:

```
(| CPT:= 0 when RESET default CPT$ init 0 + 1 |)
```

The `default` operator makes it possible to define a value for CPT when `RESET` is absent (it is one of the so-called *polychronous* operators because it applies to signals whose clocks are different), and the keyword `init` indicates the initialization value of the signal for the first execution instant.

Figure 1.15 corresponds to a Signal process (equivalent to a Lustre node) named `COUNTERS`, which receives an input signal (denoted by the symbol "`?`") named `RESET`, and returns two output signals (denoted by "`!`") called `CPT1` and `CPT2`. The first is an increasing counter initialised to 0, the second a decreasing counter initialised to 100, and both can be reset as soon as the signal `RESET` is present. The expression `CPT1 ^= CPT2` constrains these two signals to have the same clock.

```
process COUNTERS =
      ( ? event RESET;
        ! integer CPT1;
          integer CPT2;
      )
      (| CPT1:= 0 when RESET default CPT1$ init 0 + 1
       | CPT2:= 100 when RESET default CPT2$ init 100 - 1
       | CPT1 ^=CPT2
      |);
```

FIGURE 1.15: A Signal process

Derived Synchronous Languages

A few years after Lustre was built, the core of the language was reused to define a set of industrial tools named *SCADE* [CPP17], which enables graphical programming of synchronous systems through a representation of program elements as "sheets'. This software suite is now used in many critical applications, for example in the control system of Airbus A300-series aircraft. This use relies on the particularity that SCADE has a code generator, named KCG, that received DO-178C level A certification [⚓2], required for producing critical embedded code intended for aircraft flight. This certification frees the development process from the obligation to perform numerous tests to verify the correctness of the generated code, and thus represents a major advantage for the use of SCADE [PAM+09]. This *factorization* of certification means that application developers only have to demonstrate traceability from the specification to the SCADE program, and not all the way to the executed code. Similarly, other KCG certifications allow SCADE to be used for programming systems with strong safety requirements, such as the European EN 50128 certification concerning rail transport, the international IEC 61508 standard concerning programmable components of an electronic system, or the IEC 60880 standard concerning control systems for nuclear power plants.

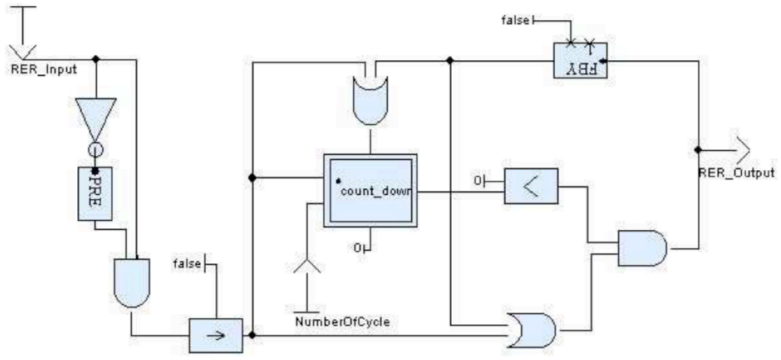


FIGURE 1.16: A SCADE sheet
(image taken from [CHP06])

Many academic synchronous languages, inspired by the dataflow-language model and in particular by Lustre (which itself continues to be enriched in version 6), have been developed since its appearance. Among these different languages, Lucid Synchrone [CP99, Pou06] is a higher-order extension of Lustre:

indeed, node parameters in Lucid Synchrone can themselves be synchronous nodes. Lucid Synchrone represents a powerful combination of "Lustre-like' synchronous languages and "ML-like' functional languages: streams may, for example, correspond to product-type values usable with a pattern-matching operator. Several constructs of this language (signals, stream-initialization analysis, state machines, clock system, . . .) were also reused in SCADE.

```
let node iter init f x = y where
   rec y = f x (init -> pre y)
```

FIGURE 1.17: A Lucid Synchrone node that iterates a function `f` over a stream of values `x`

Other synchronous languages aim to extend the Lustre model. For example, Heptagon, developed in the PARKAS team at the Ecole Normale Superieure, includes an optimised representation of arrays of values [GGPP12], and includes an extension (named BZR) that makes it possible to integrate discrete controller synthesis (DCS) into the compilation of a synchronous program [DRM11]. The hybrid language Zelus [BP13], developed by the same team, derives from Lustre and Lucid Synchrone and combines notions of discrete time and continuous time through the use of ordinary differential equations.

In the context of our work, we consider the dataflow approach adopted by these synchronous languages to be particularly well suited to the hardware and applications we target. The physical operation of the hardware is easily represented by streams: each pin of a microcontroller has a value at every moment (indicating whether or not an electric current passes through it), and each component of the application can then be represented by a synchronous node that computes, at each instant, output current values from the electrical stimuli received as inputs by the microcontroller. The schematic representation adopted by SCADE, close to the representation of an electronic circuit, also clearly illustrates this proximity between the model adopted by a language such as Lustre and the real physical model. In addition, Lustre's classic compilation model generates resource-efficient code, especially suited to the limitations of the hardware we consider.

1.4 Program Safety

Safety is the guarantee that a particular undesirable thing cannot happen [Lam77]. More precisely, program safety refers to the guarantee that an "abnormal', or even unexpected, behaviour cannot occur during execution [AS87]. This guarantee may concern several aspects of program behaviour, for example verifying that certain memory areas meant to be protected are not accessed during application execution [ACR+08], or that errors related to the illegal dereferencing of null pointers cannot occur while a program is running. In this respect, safety guarantees allow a developer to ensure that the program cannot reach states that do not share the same desired property (such as the absence of *runtime errors*).

Such guarantees are valuable in the context of embedded-system programming, where undesirable program behaviour can lead to disastrous consequences [WDS+10]. A critical embedded system whose program reacts unexpectedly can cause serious accidents involving people, which it is very important to avoid. Yet the low-level programming models generally used for microcontroller programming offer few guarantees for ensuring the consistency of program behaviour. Errors that could be detected statically

may go unnoticed during compilation in "traditional' languages. For example, the weakness of checks concerning data typing in a C program makes it possible to perform incongruous, and often unintended, operations such as multiplying an integer by the value of a character.

The use of higher-level programming languages often makes it possible to statically detect many errors that may appear in a program, and thus to reject these incorrect programs at compile time (even if, of course, some programs that are nevertheless correct may also be rejected). In this manuscript, we are particularly interested in guarantees related to type safety, to the computation of the worst-case execution time of programs, and to methods for formally specifying a language and developing the associated metatheory.

1.4.1 Static Typing

Many high-level programming languages, such as Haskell, Java, or OCaml, implement a *static typing* mechanism. This mechanism consists in associating a type (that is, information concerning the nature of the values carried by a variable or object) with each identifier at program compilation time [Pie02]. It then makes it possible to check, before program execution, errors related to the use of operators or functions incompatible with the types of the data to which they are applied. Static typing therefore increases type safety by excluding incorrect programs at compile time. Consequently, using a statically typed programming language such as OCaml is an important advantage for programming embedded systems that may sometimes be critical: the additional detection of typing errors during compilation makes it possible to avoid certain inappropriate physical reactions of programs, such as powering dangerous electronic components (such as heating resistors, which can exceed 200 degrees Celsius when powered on). In the following chapter, in Section 2.3.3, we will see an example that uses advanced typing features (with GADTs - *Generalized Algebraic Data Types*) to represent low-level interactions through OCaml's expressive static type system.

In the same way, type systems specific to certain non-general-purpose programming languages, such as Lustre, make it possible to further increase program safety. Indeed, the clock-type system of Lustre and its derivatives makes explicit the conditions governing the presence of certain values, and thus statically prevents access to the value of an *absent* stream. Moreover, the increase in abstraction provided by synchronous programming languages makes it possible to verify additional guarantees during their various compilation phases, such as checking the *causality* of a program, which consists in the possibility of statically sequentializing the different concurrent components executed by the program [BCE+03].

1.4.2 Worst-Case Execution Time

The synchronous hypothesis, on which Lustre and its derivatives rely, is valid only if the time needed to compute a program's outputs is, at every execution instant, shorter than the interval between the appearance (or polling) of its inputs. In the context of hard real-time system programming, it is therefore essential, in order to meet deadlines, to verify that the time required by the program to compute output values from any input values does not exceed, in the worst case (that is, the case in which it is slowest), a maximum time bound. This worst-case execution time (*Worst-Case Execution Time – WCET*) must therefore, so that no program input is ignored during execution, be lower than the smallest period separating the appearance of those inputs, or than the minimum delay between two inputs in the case of sporadic events.

For a long time, the most common practice for deriving a program's maximum execution time relied on empirical measurements: the program was executed a number of times on the target hardware, and the largest time measurement was considered a realistic bound on the program's execution time [WEE+08]. This method nevertheless has obvious limitations: since exhaustive testing is often impossible, it is likely that, in a real execution setting, some particular set of input values will cause a program response time greater than the one estimated during the tests performed beforehand. Safer techniques now make it possible to derive the WCET of a program by computing the greatest number of machine cycles required for its execution. Most of them apply static analyses that generate a *control-flow graph* representing the execution of a program. This graph is then analysed in order to derive the set of *valid* execution paths of the program and to estimate the cost of the longest valid execution path. Different static-analysis methods exist for performing this estimate, such as the implicit path enumeration method (*Implicit Path Enumeration Technique*, or *IPET*) [LM97]. This technique consists in representing the components of the program as integer linear constraints and using them to compute the WCET, viewed as a linear expression to be maximised. This maximisation is performed using integer linear-programming solvers (*Integer Linear Programming*, or *ILP*) or constraint-programming techniques. Several tools implement this technique, for example OTAWA [BCRS10], developed at the Toulouse Institute for Research in Computer Science (IRIT), which can estimate the worst-case execution time of programs on various architectures.

Other WCET-estimation methods rely, for example, on analysing the structure of the program's source code: a tree representing the program syntax is traversed in order to group, in the control-flow graph, several nodes corresponding to syntactic components of the program (loop, conditional, etc.) into a single node, and to repeat this process until the cost of the program can be derived. For example, the Heptane tool [CP01] performs such an analysis and can estimate the WCET of programs executed on ARM and MIPS architectures.

Figure 1.18 represents the difference between the worst time derived from a sequence of measurements on a program (which may underestimate the actual worst-case execution time), the time guaranteed by WCET-estimation tools such as Heptane or OTAWA (which may, conversely, overestimate it), and the actual worst-case execution time.

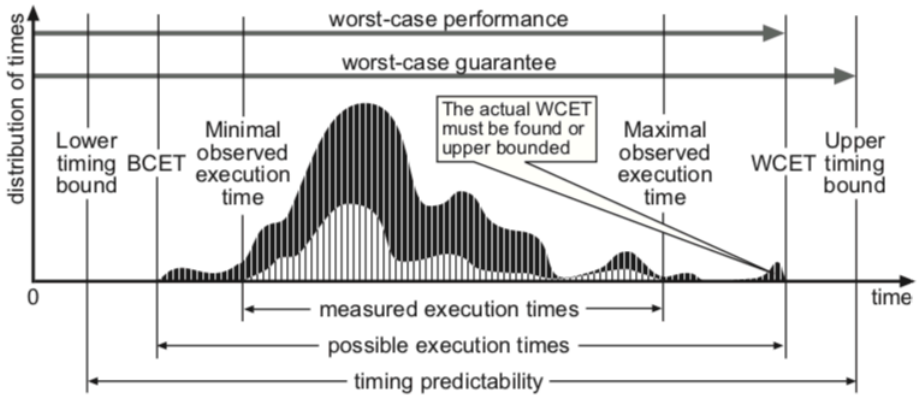


FIGURE 1.18: Measured, guaranteed, and actual worst-case execution time (image taken from [WEE+08])

Computing the WCET of a program can be made much more complex by the architecture of the processor on which the program is executed. Indeed, the different cache levels present in recent processors, as well as the modern optimisations they implement (pipeline, branch prediction, etc.), make

the computation time of an instruction variable and dependent on the execution history of the other instructions in the program. On such modern architectures, it is therefore not possible to compute the WCET of a program by considering the costs of its instructions independently, and an abstract model of the processor behaviour must be used to estimate program WCETs. However, the microcontrollers we target here do not implement these sophisticated systems that can make WCET computation so complex: a microcontroller from the ATmega328 family, for example, has no cache system, its processor does not implement branch prediction, and it is limited to a two-stage scalar pipeline that simply loads the next instruction while the current instruction is being processed. This limitation greatly simplifies the computation of the worst-case execution time, because the execution time of each instruction can then be considered separately. In particular, in Chapter 6, we will discuss a method for computing the WCET of a microcontroller program by analysing the bytecode instructions that compose it.

WCET estimation for synchronous programs is usually performed on the sequential program generated after compilation. In the case of Lustre, this program takes the form of a single loop, without recursion or dynamic allocation. The simplicity of the generated code then makes it straightforward to bound the computation time of one loop iteration using automated software tools.

1.4.3 Formal Specifications and Metatheory

A formal specification defines, in an unambiguous language, the various rules governing the behaviour of a system [Spi89]. In the context of describing a programming language, these rules describe, for example, the appropriate type systems or the semantics of the language.

Tools for *deep specification* [ABC+17], such as the Coq [Tea19] and Isabelle/HOL [NPW02] proof assistants, as well as formal methods such as the B method [Abr96], used in many critical industrial applications (for instance in the automation of lines 1 and 14 of the Paris metro), make it possible to formally specify systems with the aim of certifying their correctness. From these formal specifications, the aforementioned tools can generate executable code in a general-purpose programming language: for example, Coq supports the extraction of OCaml functions (among other languages), and Isabelle/HOL supports the generation of Haskell code and code in other high-level languages. Through a process of successive refinement of the specification, specification tools can therefore be used to produce executable programs certified as correct. Examples include the seL4 microkernel [KEH+09], whose implementation correctness was proved using Isabelle/HOL; the certified compiler CompCert [KLW14] (proved with Coq), which supports the compilation of C programs for many processor architectures (ARM, RISC-V, x86 . . .); the CakeML project [KMNO14], which includes a specified and verified compiler (using the HOL4 proof assistant [SN08]) for a functional programming language; and a Coq formalization of the JavaScript language named JSCert [BCF+14], together with its JSRef interpreter, which has been proved to respect the formal specification.

Furthermore, Lustre was designed from the outset with formally defined semantics, reflecting its creators' intention to apply the language to systems that require a certain level of safety to be guaranteed. Much work has therefore focused on formalization and on the development of certified-correct compilers for producing programs that respect Lustre semantics [BBD+17, BBP18, Aug13]. This work consists in formalizing a Lustre-like language and certifying, with Coq, a code generator compatible with CompCert, in order to provide a certified-correct compilation chain from the Lustre program down to machine code.

Chapter Conclusion

The main purpose of this thesis is to bring together all the aspects presented in this chapter. Indeed, because of the advantages offered by high-level languages, we want to execute programs written in such a language on microcontrollers whose hardware resources are limited, or even extremely limited. The use of OCaml therefore appears to us to be a relevant approach, both because of the richness of the language and because of the lightness of its model and the safety it provides through its strict static typing, which makes it possible to verify that no type-related error will occur during program execution. Thus, beginning in the next chapter, we will describe a portable implementation of the OCaml virtual machine, named OMicroB. This virtual machine is suited to execution on various microcontrollers, sometimes equipped with less than 4 kilobytes of RAM. Since microcontroller applications are inherently concurrent, because of their multiple interactions with their environment, we will then propose OCaLustre, a synchronous extension of the OCaml language, compatible with the aforementioned virtual machine. This extension will bridge the advantages of using a high-level general-purpose language and a lightweight concurrency model adapted to microcontroller resources. Our solutions will benefit from the guarantees offered by both worlds, and static analyses can also be performed directly on the bytecode of an OCaml program, thereby providing a higher level of abstraction for developing applications that benefit from the portability of this approach. On this subject, we will describe an analysis for computing the worst-case execution time of a synchronous instant of an OCaLustre program, making it possible to ensure that activations do not overlap and thus to validate the synchronous hypothesis. Moreover, although proving the correctness of the complete compilation chain of a programming language is beyond the scope of this thesis, we will discuss many formal aspects of the description of OCaLustre in this manuscript, for which we will establish a formal specification. Some proofs of metatheoretical properties carried out with Coq will also be proposed.

2 OMicroB: A Generic OCaml Virtual Machine

The work initiated by the OCaPIC virtual machine appears to be a promising and suitable solution for programming microcontrollers with limited resources. Indeed, using the OCaml language gives microcontroller programming techniques a modern development language that implements various programming paradigms, such as functional programming and object-oriented programming, without neglecting the more classic imperative programming model. Safer than the traditional assembly/C pair, this high-level programming language enables early detection of certain program errors thanks to its static type system. In addition, OCaml provides tools that simplify and strengthen program development, for example through its automatic memory-management system (*garbage collection*) and its type-inference system, which frees the developer from many superfluous type annotations while preserving the safety of static typing. Nevertheless, OCaPIC is not suited to the variety of microcontroller models available on the market: the decision to develop this tool largely in the assembly language of PIC18 microcontrollers yields significant performance (in terms of program execution speed), but greatly limits the portability of this virtual machine, since it can only be executed by hardware in that family. We consider that, given the multiplicity of microcontroller families and architectures used by industry and hobbyists (AVR, PIC16, PIC32, ARM microcontrollers, ...), a *generic and portable* solution that can be executed on many different targets would be an important advantage for the adoption of high-level programming methods in microcontroller programming. We therefore propose OMicroB, a *generic* virtual machine intended to execute OCaml programs on varied, resource-constrained hardware. This virtual machine, developed in collaboration with Benoit Vaugon (the designer of OCaPIC), deliberately takes advantage of the ubiquity of the C language in traditional microcontroller programming, which almost systematically guarantees the existence of C compilers for each target device. Thus, a large part of the OMicroB virtual machine is written in C, which we use as a *portable assembler* allowing OMicroB to be ported more easily to sometimes very different machines. Because of the strong constraints on the memory sizes of the devices considered, we aim to provide a finely controlled implementation that can run on resource-constrained hardware. Particular effort is therefore devoted to reducing the memory footprint of programs through various optimisations. This chapter first recalls the main syntactic and semantic aspects of the OCaml programming language, before describing in detail the implementation of the standard OCaml virtual machine, and then describing our implementation of a virtual machine intended to run on varied hardware with limited resources.

2.1 The OCaml Language

This section is intended to familiarize readers who are not accustomed to OCaml with the language's essential syntax and features. We present the main aspects of the language that will be used in the

examples in this manuscript. Readers already familiar with OCaml may skip directly to the next section. We provide here a non-exhaustive overview of the main high-level programming features that make it possible to build rich applications in an expressive style while offering increased safety, thanks to OCaml's type system and automatic memory-management mechanism.

Functional Values: The core of the OCaml language is both functional and imperative; it allows functions defined in the language to be manipulated as values. For example, the function `compose` computes the application to a variable x of the composition $(f \circ g)$:

```
let compose f g x = f (g x)
```

This notation is an alternative to a more explicit notation, which defines a function using the **fun** keyword. The definition of the `compose` function can therefore also be written as follows:

```
let compose = (fun f g x -> f (g x))
```

This same function can also be represented in *curried* form. *Currying* consists in transforming a function with n arguments into a function that receives a single argument and returns a chain of $n-1$ functions, each of which in turn receives only one argument, and so on until the value computed by the body of the original function is returned [Rey98]. Thus, the following definition of the `compose` function is semantically identical to the two previous definitions:

```
let compose = (fun f -> (fun g -> fun x -> f (g x)))
```

OCaml supports *lambda expressions*, anonymous functions that can be used in the body of other functions. For example, using the `compose` function, the following function uses an anonymous function (defined with the same **fun** keyword) to multiply its parameter x by two, then displays the result (using the `print_int` primitive):

```
let double_print x = compose print_int (fun x -> x * 2)
```

It should be noted that the functional values manipulated in programs, also called *closures*, may contain their own environment. Thus, the following `return_closure` function computes, from x, a closure that contains the value of x in its environment and awaits a parameter y in order to compute the value $x + y$.

```
let return_closure x = (fun y -> x + y)
```

The type of a function is an *arrow type* of the form *parameter type* $\rightarrow$ *output-value type*. Since `return_closure` is a function that takes an integer as a parameter and returns a function that takes

another integer as a parameter and returns an integer (because the + operator applies only to integers and produces an integer[1]), its type is:

$$int \rightarrow (int \rightarrow int)$$

The parenthesization of function types is right-associative, which allows this notation to be simplified as follows:

$$int \rightarrow int \rightarrow int$$

Tuples: A function can manipulate tuples of values. For example, the `sum_triple` function computes the sum of the three elements contained in the triple passed to it as a parameter:

```
let sum_triple (x,y,z) = x+y+z
```

The `plus_minus` function, for its part, returns a pair corresponding to the sum and the difference of the values passed to it as parameters:

```
let plus_minus x y = (x+y,x-y)
```

Static Typing, Polymorphism, and Compile-Time Type Inference: Defining variables and functions in OCaml generally does not require the developer to annotate each definition with its type. The types of values are inferred by the language compiler, which statically verifies, during a *typing* phase, that the programs produced are consistent with the type inferred at compilation time.

For example, the type of the `sum_triple` function, as inferred by the compiler, is therefore $(int * int * int) \rightarrow int$ [2], because the three elements of the input triple are added using the integer-addition operator "+". Likewise, the compiler infers the type $int \rightarrow int \rightarrow (int * int)$ for the `plus_minus` function. Any use of these functions with values incompatible with their types (for example, passing a triple of floating-point values as a parameter to `sum_triple`, or using the result of `plus_minus` as a simple integer) will be detected and rejected during program compilation.

The language's strict static typing therefore increases program safety by ensuring that no error related to an inconsistency in the typing of a program's values can appear during execution, and type inference spares the developer from having to annotate values with their most general type. The notion of *most general type* comes from the fact that OCaml implements a type system with parametric polymorphism, which makes it possible, for example, to represent functions applicable to arguments of various types. For example, the following `identity` function simply returns its parameter x:

```
let identity x = x
```

The type of this function, as inferred by OCaml, is therefore $\alpha \rightarrow \alpha$ (also written `'a -> 'a`), meaning that this function receives an argument of any type (α) and returns an argument of the same type.

Likewise, for the `compose` function presented at the beginning of the section, the compiler infers the following type:

$$(\alpha \rightarrow \beta) \rightarrow (\gamma \rightarrow \alpha) \rightarrow \gamma \rightarrow \beta$$

[1] The "`+.`" operator is used to add two floating-point values.

[2] The $*$ symbol in the type of a tuple separates the different types of its elements.

Sum Types and Pattern Matching: An OCaml program can define new data types in the form of *sum types* (which generalize enumerated types). Defining such a type amounts to assigning it a name, together with various constructors.

For example, the enumerated type `suit` represents the suit of a playing card:

```
type suit = Spade | Heart | Diamond | Club
```

It is also possible to define parameterized constructors, which in particular make it possible to represent recursive types. For example, the `suit_list` type represents a list of suits as either the empty list (`Nil`) or a list built (with the `Cons` constructor) from a suit (the head of the list) and a list of suits (its tail).

```
type suit_list = Nil | Cons of suit * suit_list

(* the list [Heart; Club; Club]: *)
let l_ex = Cons(Heart,Cons(Club,Cons(Club,Nil)))
```

To manipulate the values of a sum type, *pattern matching* can then be performed on the constructors of the relevant type using the *match*/*with* instruction. For example, the recursive `length` function computes the length of a list of suits:

```
let rec length l =
  match l with
  | Nil -> 0
  | Cons (_,l') -> 1 + length l'

let l = Cons(Heart,Cons(Club,Cons(Club,Nil))) in
  print_int (length l) (* displays ''3'' *)
```

It should be noted that, in a pattern, the _ character expresses "any value': here, for example, it is used with `Cons (_,l')` in the "`match`" of the `length` function to match a non-empty list whose first element has no importance for the semantics of the function in question and therefore does not need to be named.

A type can also be defined by exploiting the mechanism of parametric polymorphism. For example, the type representing a list whose elements have any type can be defined as follows:

```
type 'a list = Nil | Cons of 'a * 'a list

(* the list [Heart; Diamond; Diamond] of type "suit list"  *)
let suit_list = Cons(Heart,Cons(Diamond,Cons(Diamond,Nil)))
(* the list [1;2;3;4] of type "int list"  *)
let integer_list = Cons(1,Cons(2,Cons(3,Cons(4,Nil))))
(* the list [2.33; 4.55; 6.77] of type "float list"  *)
let float_list = Cons(2.33,Cons(4.55,Cons(6.77,Nil)))
```

Thanks to such a definition, it is therefore possible to create, among other things, lists of integers, floating-point values, values of a sum type, functions (all of the same type), or even lists of lists. It should nevertheless be noted that the generated lists must be homogeneous: the `'a list` type defines the structure of a list containing values of some type, but all elements of the *same* list must have the *same* type.

Moreover, this `'a list` type of polymorphic lists is predefined by the OCaml standard library. The `Nil` constructor is written `[]` there, and the `Cons` constructor corresponds to the infix operator `::`. The list `Cons(1,Cons(2,Cons(3,Nil)))` can therefore be written `1::2::3::[]`, or even in the form `[1;2;3]` thanks to *syntactic sugar* provided by the language compiler.

Record Types: Values composed of several distinct elements can be represented using *record* types (or *product types*). A record type is defined by giving the name and type of each element contained in the created value. The following example defines a record type to represent a point in a plane (with two coordinates: x and y), as well as a function for computing the distance between two points `p1` and `p2`:

```
type point = { x: float; y: float }

let distance p1 p2 =
  let x_diff = p1.x -. p2.x in
  let y_diff = p1.y -. p2.y in
  sqrt (x_diff *. x_diff +. y_diff *. y_diff)
```

Mutability and Imperative Programming: The contents of the fields of a record type can be declared modifiable during program execution using the **`mutable`** keyword. The value of a mutable field is modified with the <- operator.

For example, the following code declares a point whose fields are mutable, as well as a function that translates a point p by a vector with coordinates (u, v) by modifying *in place* the values contained in the x and y fields of p (the infix operator `;` makes it possible to perform a sequence of actions):

```
type point = { mutable x: float; mutable y: float }

let translate p (u,v) =
  p.x <- p.x +. u;
  p.y <- p.y +. v
```

Generally, as in several functional languages (such as Haskell or Scheme), variables are immutable in OCaml. Indeed, once a variable is declared, its value cannot be modified during program execution. Nevertheless, a particular data type, the *reference*, makes it possible to represent variables whose value can be modified during program execution, and thus to write programs with imperative traits. A reference is defined with the **`ref`** keyword, its contents are accessed with the `!` operator, and they can be modified with the `:=` operator.

The following program defines a counter `counter`, used to perform 10 iterations of a loop in which the value contained in `counter` is displayed and then incremented.

```
let loop =
   let counter = ref 0 in
   while (!counter < 10) do
     print_int!counter;
     counter:= !counter + 1
   done
```

In reality, the type of a reference corresponds to that of a record containing a single field `content`, declared as *mutable*:

```
type 'a ref = { mutable content: 'a }
```

The other basic mutable data structures in the language are arrays[3]. For example, the following program uses a "*for*' loop (as found in many programming languages) to add 2 to each element of the array *t* (defined with the syntax `[| ... |]`) by accessing cell *i* of array *t* with the notation `t.(i)`:

```
let t = [| 1; 2; 3; 4; 5 |] in
for i = 0 to 4 do
  t.(i) <- t.(i) + 2
done
```

Exceptions: As in many programming languages, OCaml implements an exception mechanism, which can affect a program's control flow. For example, the following program defines a `Division_by_zero` exception that is raised in the `divide` function if its second parameter is equal to `0.0`. This exception is then caught, using the **`try _ with`** instruction, in the calling function `call_divide`, which then returns the predefined value `max_float`:

```
exception Division_by_zero

let divide x y =
  if y = 0.0 then raise Division_by_zero;
  x /. y

let call_divide x y =
  try divide x y
  with Division_by_zero -> max_float
```

Modules: OCaml is a *modular* language, which makes it possible to group together a set of type and value declarations (variables, functions) in distinct modules. For example, the following *Card* module defines a set of types and functions for manipulating playing cards.

[3]Historically, character strings were also mutable, but this behaviour is now deprecated. Byte sequences represented by the *Bytes* module can be used instead.

```
module Card = struct

type suit = Spade | Heart | Diamond | Club
type value = Number of int | Jack | Queen | King | Ace

(* A card = a value and a suit: *)
type card = { v: value; s: suit }

(* card values for belote: *)
let points card trump =
    match card.v with
    | Number 10 -> 10
    | Number 9 -> if card.s = trump then 14 else 0
    | Number _ -> 0
    | Jack -> if card.s = trump then 20 else 2
    | Queen -> 3
    | King -> 4
    | Ace -> 11

(* display function: *)
let print_card card =
   let string_suit =
      match card.s with
        | Heart -> "Heart"
        | Spade -> "Spade"
        | Club -> "Club"
        | Diamond -> "Diamond"
   in
   let string_value =
     match card.v with
     | Number i -> string_of_int i
     | Jack -> "Jack"
     | Queen -> "Queen"
     | King -> "King"
     | Ace -> "Ace"
   in
   print_string ((string_value)^"␣of␣"^(string_suit))
end
```

Every `foo.ml` file implicitly corresponds to the declaration of a `Foo` module, whose interface can be defined in the `foo.mli` file. This modular programming model makes it possible to compile the distinct components of a program separately.

Objects: Finally, OCaml is also an object-oriented programming language. The object type system implemented in the language supports multiple inheritance as well as class polymorphism. For example,

the following OCaml code (taken from the official OCaml website [⚓29]) defines a class representing a polymorphic stack structure.

```
class ['a] stack =
    object (self)
      val mutable list = ( []: 'a list )  (* instance variable *)
      method push x =
        list <- x:: list
      method pop =
        let result = List.hd list in
        list <- List.tl list;
        result
      method peek =
        List.hd list
      method size =
        List.length list
    end
```

Because parametric polymorphism is used (denoted by the class parameter `'a`), the way an instance of this class is used restricts the type of the values contained in the stack it represents. For example, the following program manipulates a stack of playing cards:

```
open Card

let () =
  let s = new stack in
  s#push {v = Jack; s = Spade}; (* s is therefore a "card stack" *)
  s#push {v = Queen; s = Heart};
  let c = s#pop in
  print_card c; (* displays "Queen of Heart" *)
  print_int s#size (* displays 1 *)
```

Automatic Memory Management: As in Java, OCaml implements a *garbage collector*. At runtime, this mechanism automatically frees the memory associated with values that are no longer used by the program. Using OCaml therefore spares the developer from having to consider the dynamic deallocation of the various values being manipulated. Moreover, this automation makes programs less prone to bugs caused by programmer errors in memory manipulation, which are often quite difficult to detect.

Advanced Constructs: Finally, OCaml implements various other high-level programming features, such as GADTs (*Generalized Algebraic Data Types*), *polymorphic variants*, and *parameterised modules* (also called *functors*). These advanced constructs are fully compatible with the generic virtual machine described in this chapter. We will discuss some examples that take advantage of such constructs as the

presentation progresses (for example, in Section 2.3.3 we will present an example using GADTs to increase the type safety of primitives that implement communication between a microcontroller and its environment).

2.2 The ZAM: The Reference OCaml Virtual Machine

OMicroB is a virtual machine derived from the standard OCaml virtual machine. As such, it has many points in common with it. In this section, we therefore present the main technical aspects of the reference OCaml virtual machine, from the format of the *bytecode* it can interpret, through the details of the representation of the OCaml values it manipulates, to a short description of its runtime library.

2.2.1 Overview of the ZAM

Developed at Inria since 1996 in several research projects (*Cristal*, then *Gallium*), this virtual machine is a stack machine with a functional and imperative core. It can be viewed as a version of Krivine's abstract machine [Kri07] that implements a strict-application model rather than call by name. Also called the ZAM (*Zinc Abstract Machine*), in reference to the ZINC project [Ler90] from which it originated[4], the reference OCaml virtual machine is one of the two compilation targets for the OCaml language: an OCaml program can be compiled to a *bytecode* file (using the *ocamlc* compiler), which can be interpreted by the ZAM (more precisely by its reference implementation, called *ocamlrun*), or to a native executable using the *ocamlopt* compiler. In this thesis, we focus on the virtual-machine approach to compiling OCaml, because of the opportunities for portability that it offers, as well as the relative lightness and simplicity of this compilation model for the OCaml language.

2.2.2 OCaml Bytecode

In version 4.06, the standard OCaml virtual machine contains a set of 148 different bytecode instructions to which any OCaml program can be compiled. These bytecode instructions include instructions that act on the control flow of programs (such as the conditional-branch instruction *BRANCHIF*), instructions that perform arithmetic or Boolean computations (such as the *MULINT* instruction, which multiplies integer values), instructions that dynamically allocate values (such as the *CLOSURE* instruction, which stores a functional value as a closure in the virtual-machine heap), and instructions that manipulate the virtual-machine stack (such as the *PUSH* and *POP* instructions, which push and pop values). Most bytecode instructions are shortcuts corresponding to combinations of several atomic instructions (for example, the *PUSHACC1* instruction has the same behaviour as the *PUSH* instruction followed by the *ACC1* instruction, which retrieves the second element of the stack). For the sake of brevity, the full set of OCaml bytecode instructions and their descriptions is not given in this manuscript, but the main instructions will nevertheless be presented[5].

OCaml bytecode is generated from an OCaml source file by the *ocamlc* compiler. The file generated by *ocamlc* is an executable program containing several distinct sections, required for the initialisation and interpretation of the OCaml program by the virtual machine:

[4] This project consisted in a lightweight implementation of the ML language.

[5] Readers interested in the semantics of each bytecode instruction may refer to the documentation of the Cadmium project [⚓6].

- A CODE section containing the bytecode instructions corresponding to the compiled program code.
- A DATA section containing a serialised representation of the program's various global variables (constants, exceptions, etc.).
- A PRIM section, which is a table containing the correspondence between the external primitives used by a program and the integers used to refer to them in the bytecode.
- An optional DLLS section containing the names of the external libraries required to execute a program (for example, the `Unix` library, which makes it possible to perform system calls).
- An optional DLPT section containing the paths of the libraries used by the program.
- An optional DBUG section, used for program debugging.

For example, Figure 2.1 shows the OCaml definition of a function `facto`, which computes the factorial of an integer, together with its application to the value 4. A representation of the bytecode contained in the file generated by *ocamlc* is reproduced beside it. This bytecode is only an excerpt from the CODE section of this file, which contains, in addition to the instructions corresponding to the `facto` function, a large number of bytecode instructions used to initialise the program. Moreover, the generated bytecode also contains the code of functions from a module imported by default by every OCaml program, named `Pervasives`, which contains definitions of common functions (for example, display functions) and basic operators (such as addition between two integers, or logical "or').

```
let rec facto x =
  match x with
  | 0 -> 1
  | _ -> x * facto (x-1)
in
facto 4
```

```
(...)
1673   BRANCH 1685

1674   ACC 0
1675   BRANCHIFNOT 1683
1676   ACC 0
1677   OFFSETINT -1
1678   PUSHOFFSETCLOSURE 0
1679   APPLY 1
1680   PUSHACC 1
1681   MULINT
1682   RETURN 1
1683   CONST 1
1684   RETURN 1

1685   CLOSUREREC 1 0 1674 []
1686   CONST 4
1687   PUSHACC 1
1688   APPLY 1
1689   POP 1
```

FIGURE 2.1: Definition and application of the factorial function, and corresponding bytecode

2.2.3 Virtual-Machine Structure

During the interpretation of a program, the OCaml virtual machine manipulates several registers required for its execution. These registers are as follows:

- An accumulator (`acc`) used to store a value and avoid overly frequent pushes and pops on the virtual-machine stack.
- A pointer (`pc`) to the next bytecode instruction to interpret.
- A pointer (`sp`) to the last cell of the stack.
- A pointer (`trapSp`) to the current exception handler.
- A counter (`extra_args`) representing the number of parameters remaining in the application of a function.
- A pointer to the environment (`env`) of the current closure.
- A pointer to the program's global-variable array (`global_data`).

The contents of these registers are modified during program interpretation according to the semantics of each bytecode instruction contained in the file generated by *ocamlc*. The virtual machine's OCaml bytecode interpreter is written in C.

Figure 2.2 describes the effect of each instruction in the bytecode program presented in Figure 2.1 on the state of the virtual-machine registers.

Position	Instruction	Description
1673	`BRANCH 1685`	Go to instruction 1685 (`pc = 1685`)
1674	`ACC 0`	Put the value at the top of the stack in the accumulator (`acc = sp[0]`)
1675	`BRANCHIFNOT 1683`	If `acc` = 0, then go to instruction 1683 (`pc = 1683`)
1676	`ACC 0`	Put the value at the top of the stack in the accumulator (`acc = sp[0]`)
1677	`OFFSETINT -1`	Decrement `acc`
1678	`PUSHOFFSETCLOSURE 0`	Push `acc`, and put in `acc` the closure located in `env`
1679	`APPLY 1`	Push the current context (`extra_args`, `pc`, `env`), put `acc` in `env`, and go to the code of the closure in `acc` (i.e. `facto`)
1680	`PUSHACC 1`	Push `acc`, and put `sp[1]` in `acc`
1681	`MULINT`	Multiply `acc` and sp[0], put the result in `acc`, and pop sp[0]
1682	`RETURN 1`	Pop one element and leave the function (`pc = pop(), env = pop(), extra_args = pop()`)
1683	`CONST 1`	Put 1 in `acc`
1684	`RETURN 1`	Leave the function (`pc = pop(), env = pop(), extra_args = pop()`)
1685	`CLOSUREREC 1 0 1674 []`	Create the closure whose code is at address 1674 (i.e. `facto`) in the accumulator and add it to the top of the stack
1686	`CONST 4`	Put constant 4 in `acc`
1687	`PUSHACC 1`	Push `acc` and put `sp[1]` in `acc`
1688	`APPLY 1`	Push the current context (`extra_args`, `pc`, `env`), put the value of `acc` in `env`, and go to the code of the closure in `acc` (i.e. `facto`)
1689	`POP 1`	Pop one value (`sp--`)

FIGURE 2.2: Interpretation of the bytecode computing the program in Figure 2.1

2.2.4 Value Representation

Because of parametric polymorphism in the OCaml language, all values manipulated by the ZAM are based on a uniform representation: OCaml values all have the same length, defined by the compiler according to the architecture of the computer on which the program is executed (typically 32 or 64 bits). Nevertheless, during program execution, OCaml's automatic memory-management system must in particular distinguish immediate values, which it must ignore, from values dynamically allocated on the virtual-machine heap, which it must visit and process. This dichotomy, in the ZAM, is very simple: immediate values correspond to all values encoded by integers (such as constructors of enumerated types, or of course integers themselves), while every other value is *boxed* on the heap. The distinction between these two categories is made by reserving the least-significant bit of each value to represent its nature: if the least-significant bit of an OCaml value is 1, then the value is an integer. If it is 0, then the value considered corresponds to a pointer, since all addresses are even: in a 32-bit representation (resp. 64-bit representation), all OCaml values have a size of 4 bytes (resp. 8 bytes), and therefore all pointer addresses are multiples of 4 (resp. 8). This representation allows the virtual-machine components to distinguish immediate values from allocated values quickly.

Figure 2.3 illustrates the binary representation of OCaml values in the ZAM, on a computer with a processor using a 32-bit architecture.

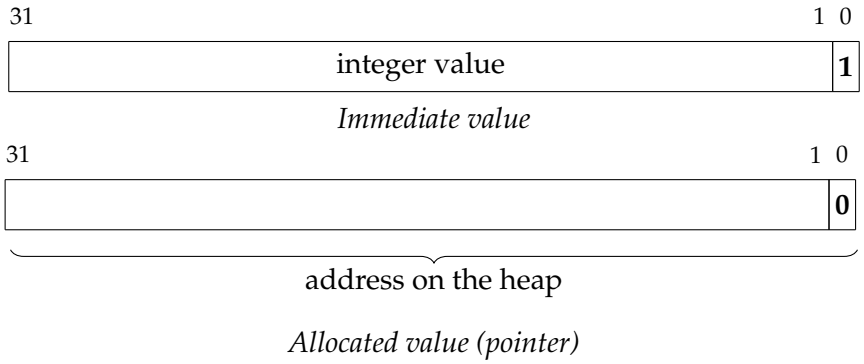


FIGURE 2.3: Representation of 32-bit OCaml values in the ZAM

Integer values can then range from -2^{30} to $2^{30} - 1$, while 2^{32} bits are available to represent pointers to the heap. The representation is equivalent on a 64-bit version of the ZAM, in which integers are represented by 63 bits while pointers also end in zero[6].

Header of Allocated Blocks: OCaml values allocated on the heap, such as closures, floating-point numbers, tuples, records, or values of recursive types (lists, trees, etc.), are boxed in blocks whose first value is a header containing several pieces of information intended for the virtual-machine interpreter and for the garbage collector. In a 32-bit configuration, this header consists of 22 bits representing the size of the allocated block (excluding the header), 2 colour bits required for the correct operation of the garbage collector, and 8 *tag* bits representing the nature of the value (closure, exception, object, etc.), which in particular indicate to the garbage collector whether it must analyse the values inside the block:

[6]More precisely, these pointers end in `000`, since all addresses are multiples of 8, just as 32-bit addresses end in `00` because they are multiples of 4.

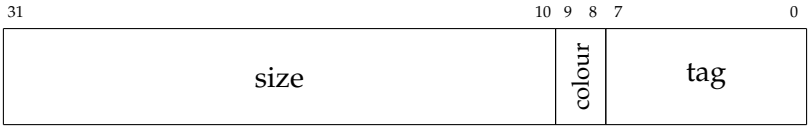


FIGURE 2.4: Header of an allocated block

The structure of a block's contents depends on the nature of the value represented: for example, in the case of a closure (tag 247), the first value is a pointer to a bytecode segment corresponding to the closure's code, and the following values correspond to the closure's environment.

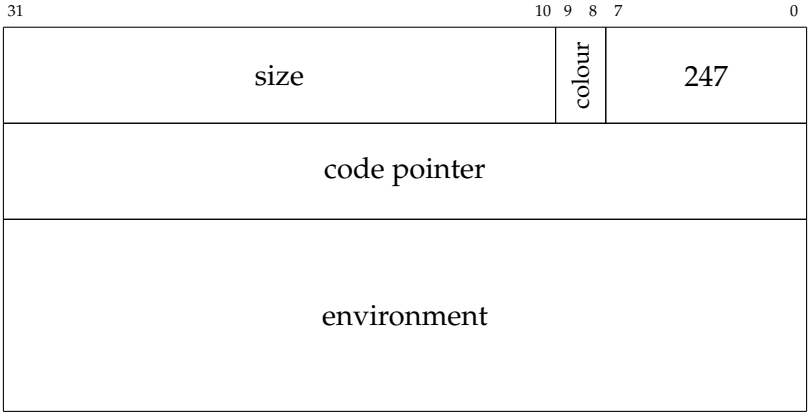
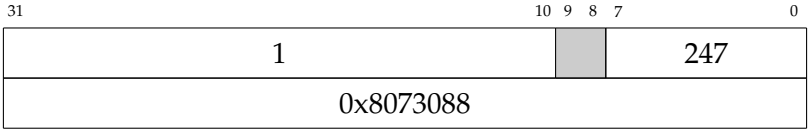
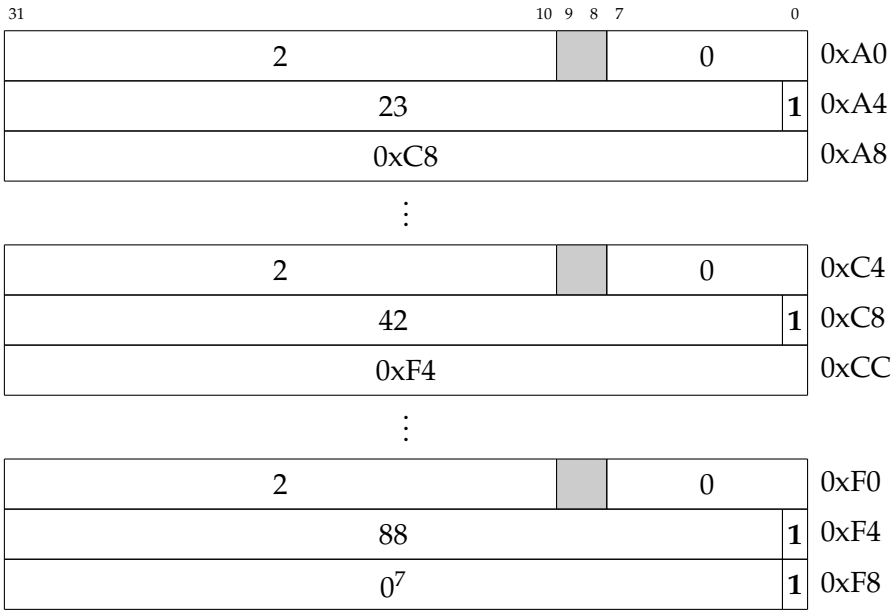


FIGURE 2.5: Representation of a closure

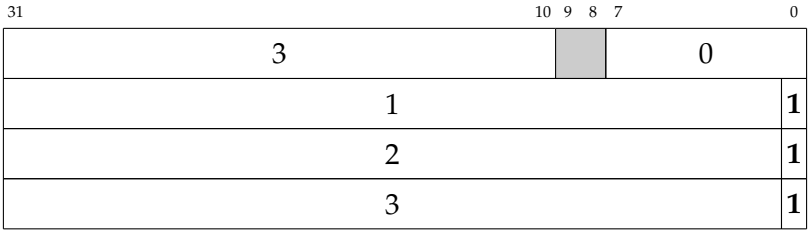
For example, the closure generated by the CLOSUREREC instruction in the example in Figure 2.1, which corresponds to the `facto` function, is reduced to a block containing a code pointer to address 0x8073088, and an empty environment:



Every data structure is allocated in a block containing each element of that structure. For example, an element of a list is represented by the value of the element followed by a pointer to the next (not necessarily contiguous) element of the list. It should be noted that a pointer to a block in fact points to the first value contained in that block, not to its header. The list `[23;42;88]` is therefore represented as follows:



Fixed-size data structures, such as arrays or tuples, are represented by blocks containing values that are contiguous in memory. For example, the triple (1,2,3) is represented by the following block:



This representation is also identical for the array [| 1; 2; 3 |] or for a record containing three fields whose values are 1, 2, and 3. Some type-related information is therefore impossible to recover after compiling the program to its bytecode (which is itself untyped), and during its execution. Static typing nevertheless ensures, for example, that an array is not used as a tuple during program execution.

2.2.5 Runtime Library

The ZAM has a rich standard library, which makes it possible to manipulate many predefined types, such as lists, mutable arrays, references, and character strings. Most modules in this library are defined directly in OCaml, with the exception of certain functions that cannot be represented by the language and are written in C, such as the generic `compare` function, which compares two elements of any OCaml data type, whether it is a basic type (int, float, etc.) or even a recursive structure, predefined or not (list, tree, etc.).

Moreover, the memory areas of values that are no longer used during program execution are eventually freed by a garbage-collection algorithm (*garbage collector*, or *GC*). The ZAM implements a hybrid incremental and generational GC that uses two heaps: a fixed-size *minor* heap in which value blocks are first allocated, and a *major* heap containing blocks that have "survived' (i.e. are still used by the program)

[7] The value 0 here represents the constant constructor `[]` of the empty list.

after cleaning the minor heap. The minor-heap cleaning phase implements a *Stop and Copy* algorithm (which consists in copying all live values from the minor heap to the major heap), while major-heap cleaning is based on an incremental *Mark and Sweep* method that traverses the heap in chunks, marking all live blocks and then freeing the others. OCaml's GC also contains a *Mark and Compact* algorithm, run regularly to compact the major heap and avoid fragmentation.

2.3 Compilation and Execution of an OCaml Program with OMicroB

In this section, we present the operation of the OMicroB virtual machine through a detailed description of the different stages that make it possible to execute a program on a microcontroller. This description also highlights the essential differences between the ZAM and this generic virtual machine, intended for programming resource-constrained devices.

Thus, in what follows we describe the various transformations applied by OMicroB to an OCaml source program in order to generate an executable program that can be written into the memory of a microcontroller. Figure 2.6 is a schematic representation of this compilation chain: the OCaml source file is compiled to a bytecode file (a), cleaned (b), then integrated into a C file (c), which is then compiled with its interpreter and runtime library (d) to an executable file for the target hardware (e).

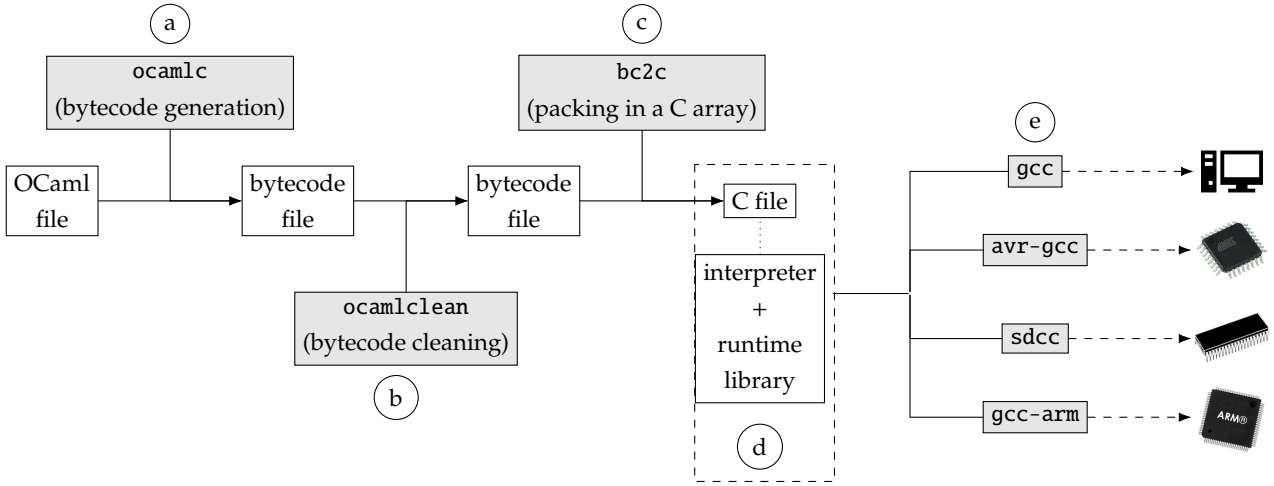


FIGURE 2.6: Compilation of an OCaml source file to a microcontroller executable

2.3.1 Representation of an OCaml Program in a C File

The preliminary steps in producing a program executable by a microcontroller from an OCaml source file compile the source program to an intermediate representation, OCaml bytecode, which can be interpreted by the virtual machine. In OMicroB, this bytecode is generated, cleaned, and then integrated into a C file containing a representation of the instructions, as well as declarations of C arrays representing, among other things, the virtual-machine stack and heap.

Generating and Cleaning a Program's Bytecode

OCaml programs intended for the OMicroB virtual machine are written in the standard syntax of the language, and are therefore fully compatible with the classic OCaml bytecode compiler, named `ocamlc`. An OCaml program intended for use with OMicroB is therefore first compiled to a standard bytecode

file using this compiler. Using the standard compiler makes it possible to reuse standard tools from the OCaml ecosystem, such as the *ocamldebug* software, to debug OCaml programs by analysing their bytecode.

The bytecode file generated by `ocamlc` may contain code unused by the final program, in particular initialisation code for closures not used in the program, induced by opening OCaml libraries in the program's source code. For example, a program using the `List` module in its source code will cause the generated bytecode to initialise all the closures present in that module. Consequently, a program's memory footprint can be significantly increased by such initialisations, and programs that are *a priori* lightweight may become incompatible with the limited memory constraints inherent in the use of microcontrollers. In order to reduce the memory consumption of OCaml programs, we use the *ocamlclean* tool [⚓9], from the OCaPIC project that preceded our work. This tool can detect unused blocks in a program by static bytecode analysis, and remove the initialisation code for the useless closures associated with them[8]. The program resulting from this cleaning step is then smaller, which makes it usable in a resource-constrained setting.

The memory gain provided by using *ocamlclean* can vary enormously from one OCaml program to another, depending on the number of external modules on which a program depends, and on its structure. As an example, consider the OCaml program at the beginning of Figure 2.7. After compiling this program to a bytecode file with the `ocamlc` bytecode compiler (for version 4.06.0 of the OCaml language), using the `-verbose` option of *ocamlclean* prints to standard output the information shown below the program source code in Figure 2.7. It follows that cleaning this program, which uses only the `map` and `iter` functions of the `List` module, divides the file size by 14.8 (from 32.34 to 2.18 kilobytes).

It should also be noted that because the `Pervasives` module (which defines certain basic operations and primitives) is implicitly loaded by every program, even a program that seems almost empty contains a substantial number of unused bytecode instructions. The *ocamlclean* tool therefore provides an almost systematic reduction in program size, including for programs that are *a priori* very frugal. For example, on a personal computer, the OCaml program reduced to the single value "`()`" (*unit*) is compiled by *ocamlc* to bytecode containing 2279 instructions, and using *ocamlclean* reduces this number to 81 instructions in total, and divides the program size by 12 (from 18.5 kB to 1.53 kB).

```
let () =
  let l = List.map (fun x -> x + 1) [1;2;3;4;5] in
  List.iter print_int l
```

```
Statistics:
* Instruction number:     5454  ->     188   (/29.01)
* CODE segment length:   23844  ->     964   (/24.73)
* Global data number:       66  ->      25   (/2.64)
* DATA segment length:     769  ->     376   (/2.05)
* Primitive number:        352  ->       9   (/39.11)
* PRIM segment length:    7065  ->     191   (/36.99)
* File length:           32340  ->    2177   (/14.86)
```

FIGURE 2.7: Dead-code elimination statistics for a program with *ocamlclean*

[8]It should be noted that *ocamlclean* "breaks' dynamic loading of programs, since code unused by the main program is removed. This limitation is not problematic in our microcontroller use case, however, because we do not use dynamic loading.

Generating a C File

The next stage of the *OMicroB* compilation chain is based on transforming the OCaml bytecode file generated by *ocamlc* into a C file. This file, generated by a tool named *bc2c*, contains a representation of the bytecode of the OCaml program, as well as the various memory areas required for its execution. Other structures defined in the generated file represent certain sets of data used by the OCaml program, such as the program's global-variable array, or a table of C primitives used by the OCaml program mainly to perform low-level interactions with its environment.

Overall Structure of the C Program: The C file generated by *bc2c* defines several elements specific to the program considered, which are manipulated by the virtual-machine interpreter during execution of this program. These elements correspond to the different sections (CODE, PRIM, DATA) of the original bytecode file, as well as to the memory used by the OCaml program, represented by C arrays. Figure 2.8 schematically represents the operation of *bc2c*. The various elements present in the generated C file are as follows:

- The bytecode of the OCaml program, in the form of an array of constants.
- The virtual-machine heap, which contains dynamically allocated values, in an array of OCaml values.
- The virtual-machine stack, also represented by an array of values.
- The array of global variables used in the program considered.
- The set of primitive functions used by the program. This is an array of pointers to C functions from the standard library, or to external functions defined by the user.
- The sizes of the generated arrays, as well as a set of constant values corresponding to bytecode-instruction numbers, are represented by macros at the beginning of the file. In particular, the sizes of the heap and stack are chosen by the developer when compiling the program.

Bytecode Representation: The standard bytecode generated by the OCaml compiler contains only 148 different instructions. The arguments of the various bytecode instructions are also statistically fairly small (for example, the argument p of the conditional-branch instruction `BRANCHIF` p, which represents a relative position in the bytecode, often corresponds to a nearby position in the program bytecode). Aligning bytecode instructions on 32-bit values, as in the standard OCaml virtual machine, would in our use case lead to fairly excessive and rather unnecessary resource consumption: memory would be used to represent "hollow' values, containing mostly 0s used only to align them on 4 bytes. Reading bytecode instructions by the interpreter would also be slow, given that the microcontrollers we consider generally have 8-bit processor architectures, and therefore require several cycles to read and manipulate larger values. The *bc2c* tool therefore represents OCaml bytecode as a constant array of bytes, each element of which corresponds either to an instruction code (or *opcode*) or to an argument of a bytecode instruction. An argument that does not fit in 8 bits is represented over several consecutive cells of the generated array. This 8-bit bytecode representation improves program execution speed, and above all

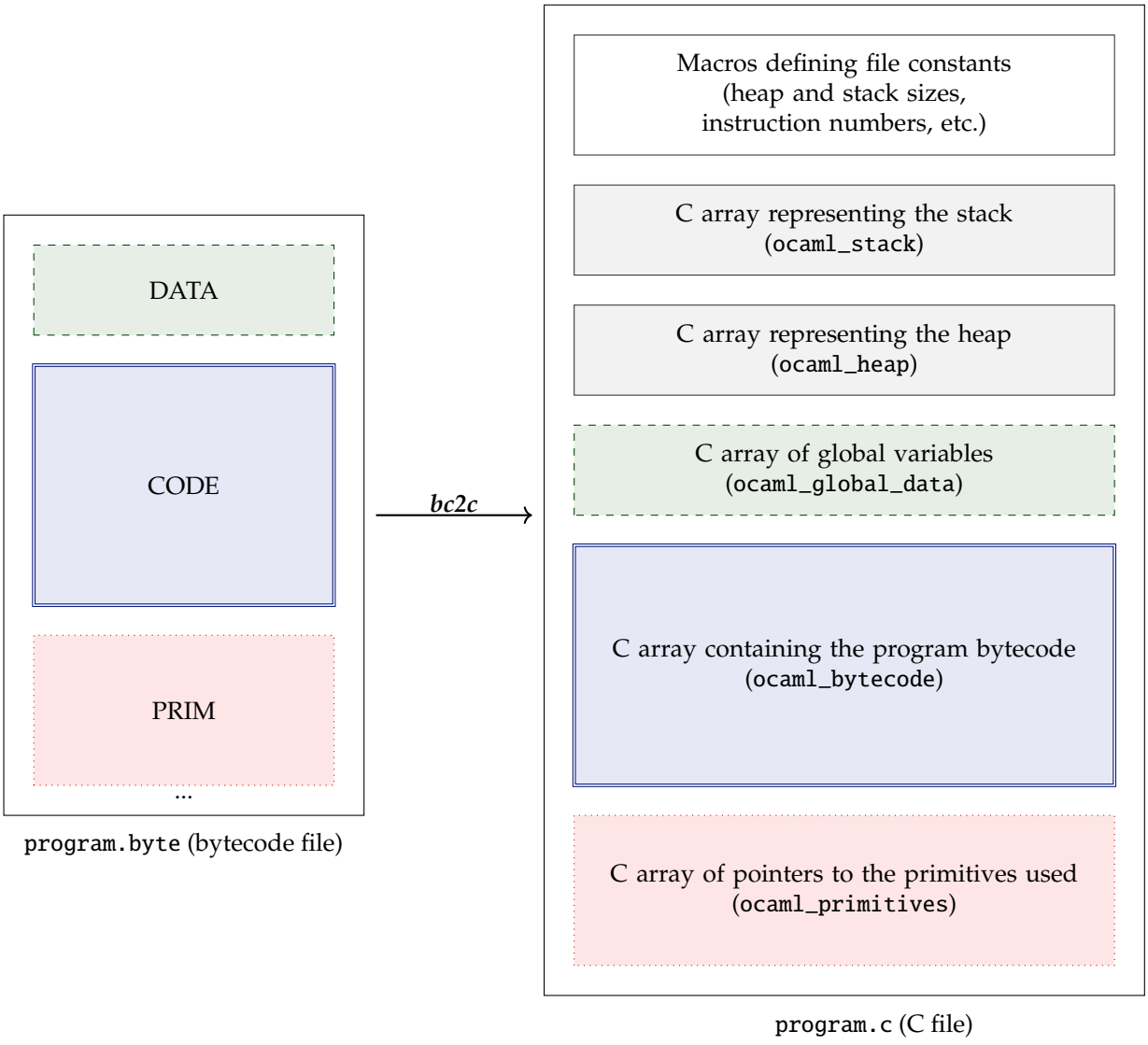


FIGURE 2.8: bc2c: inclusion of OCaml bytecode in a C file

reduces program size: from measurements carried out on a set of standard programs, we estimate that bytecode represented by an array of bytes is about 3.5 times smaller than the original bytecode.

To allow the interpreter to distinguish instructions whose arguments fit in a single byte from those whose arguments require several consecutive bytes, *specialised* versions of the relevant bytecode instructions are produced. For example, the instruction `PUSHCONSTINT` *k*, which pushes the value of the accumulator and puts in it a constant integer value *k*, is specialised in OMicroB into three versions capable of handling an integer value that can be represented on one, two, or four bytes. This instruction specialisation is indicated by adding the annotations `_1B`, `_2B`, and `_4B` to the end of the classic bytecode-instruction names (for example: `PUSHCONSTINT_1B`, `PUSHCONSTINT_2B`, and `PUSHCONSTINT_4B`).

Figure 2.9 represents the file generated by *bc2c* from the file produced by compiling the program computing the factorial of 4 presented in Figure 2.1. To save the microcontroller's RAM, the `PROGMEM` annotation tells the C compiler[9] to allocate the array representing the bytecode in flash memory, since it is only read during program execution and never modified. The `opcode_t` type is an alias of `int8_t`. The `OCAML_BYTECODE_BSIZE` macro is computed by *bc2c* and represents the total size (in bytes) of the program bytecode. It should be noted that the addresses of the various bytecode instructions are updated to account for the offsets induced by 8-bit bytecode alignment.

2.3.2 Bytecode Interpreter

At program execution, the OMicroB virtual-machine interpreter traverses the code contained in the array representing the program bytecode, and manipulates the data stored in the arrays representing the stack and the heap. This interpreter, whose semantics are identical to those of the ZAM, reads the program's bytecode instructions and modifies the program memory accordingly. OMicroB's interpreter contains the same seven registers as the ZAM (`acc`, `pc`, `trapSp`, `extra_args`, `env`, and `globaldata`), with the notable difference that the `pc` register points to the microcontroller's flash memory, since the program bytecode is stored there in order to reduce the program's RAM consumption. The dynamic elements (stack, heap, and register values) are contained in RAM. Figure 2.10 graphically represents the interaction between the various elements of the OMicroB virtual machine.

Each of the 148 different instructions (as well as their specialised versions) of OCaml bytecode is handled by OMicroB's interpreter, written in C, which manipulates these various registers and the program memory. Like the ZAM interpreter, it executes the OCaml program by modifying the contents of the virtual-machine registers as evaluation proceeds, according to the instructions encountered. We illustrate bytecode interpretation using the example program shown in Figure 2.11, which generates the functional value $\lambda y.y + 4$ and then applies it to the value 8. Figure 2.12 shows the bytecode generated from the program in Figure 2.11, together with an informal description of the semantics of each program instruction, and details of how register values evolve during interpretation of this bytecode.

In this figure, control flow is represented by arrows annotated with letters, corresponding to the different states of the virtual machine during program execution. For each bytecode instruction, we represent the state of the `pc`, `acc`, `extra_args`, and `env` registers, as well as the contents of the stack (`sp` points to its first value) after its interpretation. The `trapSp` and `global_data` registers are not manipulated during execution of this program, and are therefore not represented. Closures, allocated on the heap, are

[9] This annotation is used by the *avr-gcc* compiler.

```
#define OCAML_STACK_WOSIZE       42
#define OCAML_HEAP_WOSIZE        200
/* (...) */
#define OCAML_BYTECODE_BSIZE    28
#define OCAML_PRIMITIVE_NUMBER   0

value ocaml_stack[OCAML_STACK_WOSIZE];
value ocaml_heap[OCAML_HEAP_WOSIZE];

value ocaml_global_data[OCAML_RAM_GLOBDATA_NUMBER] = { /* ... */ };

PROGMEM opcode_t const ocaml_bytecode[OCAML_BYTECODE_BSIZE] = {
  /*  0 */  OCAML_BRANCH_1B, 17,
  /*  2 */  OCAML_ACC0,
  /*  3 */  OCAML_BRANCHIFNOT_1B, 11,
  /*  5 */  OCAML_ACC0,
  /*  6 */  OCAML_OFFSETINT_1B, (opcode_t) -1,
  /*  8 */  OCAML_PUSHOFFSETCLOSURE0,
  /*  9 */  OCAML_APPLY1,
  /* 10 */  OCAML_PUSHACC1,
  /* 11 */  OCAML_MULINT,
  /* 12 */  OCAML_RETURN, 1,
  /* 14 */  OCAML_CONST1,
  /* 15 */  OCAML_RETURN, 1,
  /* 17 */  OCAML_CLOSUREREC_1B, 1, 0, (opcode_t) -15,
  /* 21 */  OCAML_CONSTINT_1B, 4,
  /* 23 */  OCAML_PUSHACC1,
  /* 24 */  OCAML_APPLY1,
  /* 25 */  OCAML_POP, 1,
  /* 27 */  OCAML_STOP
};

PROGMEM void * const ocaml_primitives[OCAML_PRIMITIVE_NUMBER] = {};
```

FIGURE 2.9: Example of a file generated by bc2c

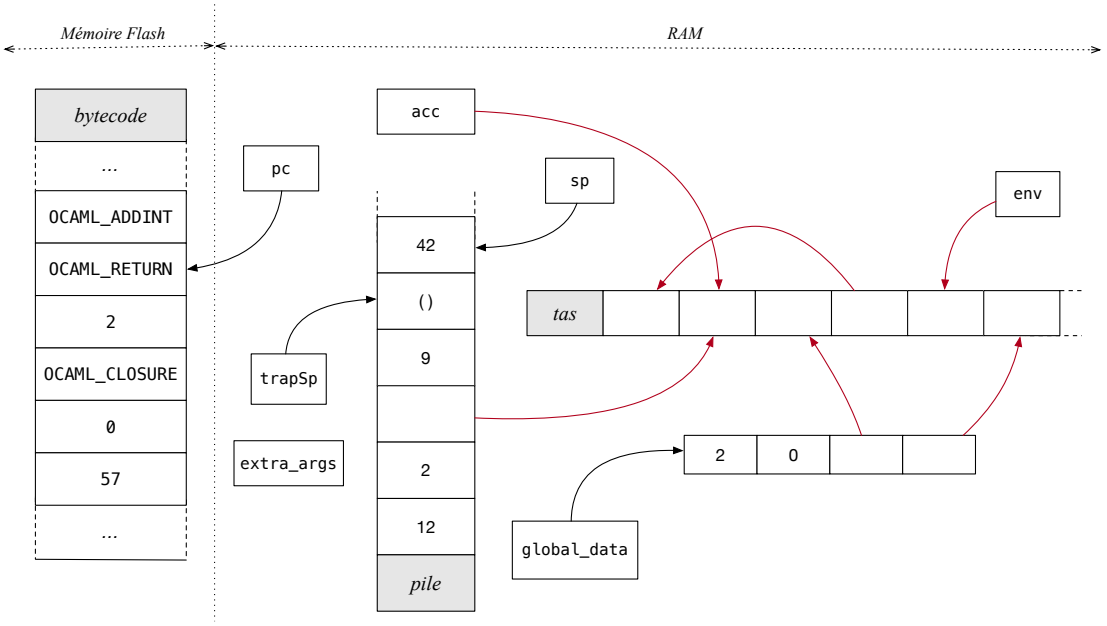
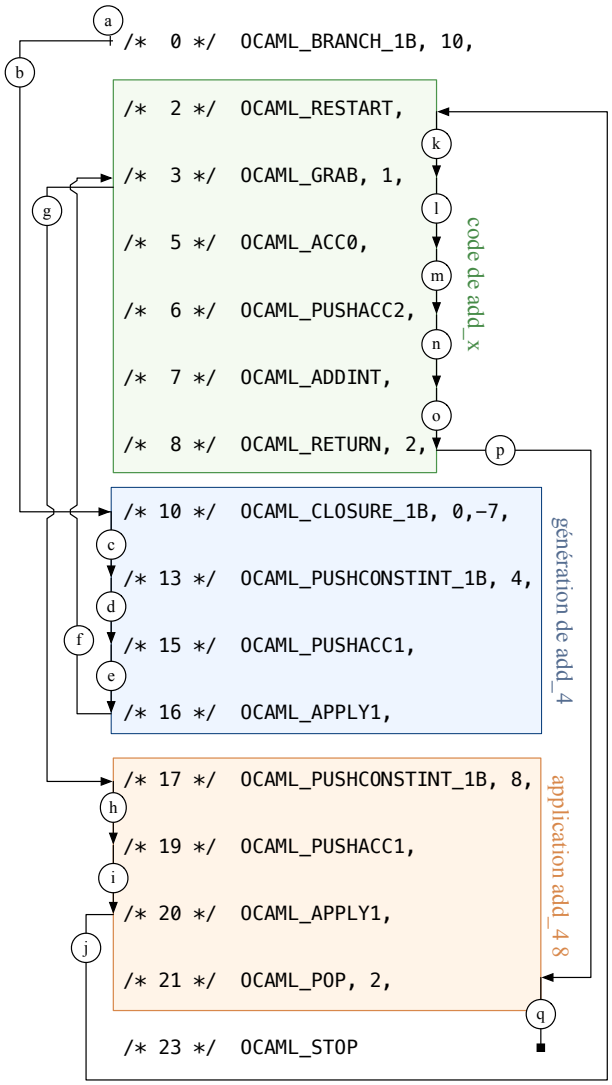


FIGURE 2.10: The OMicroB virtual machine

```
let add_x x =                  (* add_x receives an integer x and *)
  (fun y -> y + x)             (* creates a functional value       *)
in                             (* that receives y and computes y+x *)
  let add_4 = add_x 4 in       (* add_4 = (fun y -> y + 4)         *)
  add_4 8                      (* = 12                             *)
```

FIGURE 2.11: An OCaml program that manipulates a functional value



state	description
a	beginning of the program
b	go to address (0 + 10)
c	create a closure (whose code starts at address 10 − 7) in acc
d	push the contents of the accumulator (acc) and put constant 4 in acc
e	push the contents of acc and put sp[1] in acc
f	save the current context and go to the code of the closure contained in acc
g	since extra_args < 1, create a closure (whose code starts at address 2) in acc and return to the caller
h	push the contents of acc and put constant 8 in acc
i	push acc, then put sp[1] in acc
j	save the current context and go to the code of the closure contained in acc
k	push the values present in the environment (env) and put the size of env in extra_args
l	since extra_args=1, decrement extra_args
m	put the value of sp[0] in acc
n	push the contents of the accumulator and put the value of sp[2] in acc
o	pop sp[0], add it to acc, and put the result in acc
p	end of the function: pop two elements and return to the caller
q	pop two elements
	end of the program

state	pc	acc	stack	env	extra_args
a	0	()	[]	[]	0
b	10	()	[]	[]	0
c	13	{@3}	[]	[]	0
d	15	4	[{@3}]	[]	0
e	16	{@3}	[4; {@3}]	[]	0
f	3	{@3}	[4; @17; []; 0; {@3}]	[{@3}]	0
g	17	{@2; {@3}; 4}	[{@3}]	[]	0
h	19	8	[{@2; {@3}; 4}; {@2}]	[]	0
i	20	{@2; {@3}; 4}	[8; {@2; {@3}; 4}; {@3}]	[]	0
j	2	{@2; {@3}; 4}	[8; @21; []; 0; {@2; {@3}; 4}; {@3}]	[{@2; {@3}; 4}]	0
k	3	{@2; {@3}; 4}	[4; 8; @21; []; 0; {@2; {@3}; 4}; {@3}]	[{@2}]	1
l	5	{@2; {@3}; 4}	[4; 8; @21; []; 0; {@2; {@3}; 4}; {@3}]	[{@2}]	0
m	6	4	[4; 8; @14; []; 0; {@2; {@3}; 4}; {@3}]	[{@2}]	0
n	7	8	[4; 4; 8; @21; []; 0; {@2; {@3}; 4}; {@3}]	[{@2}]	0
o	8	12	[4; 8; @21; []; 0; {@2; {@3}; 4}; {@3}]	[{@2}]	0
p	21	12	[{@2; {@3}; 4}; {@3}]	[]	0
q	23	12	[]	[]	0

FIGURE 2.12: Evolution of the virtual-machine registers during execution of the program in Figure 2.11

represented in braces by a code pointer (preceded by the @ sign), potentially followed by a sequence of elements making up their environment. For example, the closure corresponding to `add_4`, whose code starts at instruction 2 and whose environment itself contains a closure (pointing to instruction 3) as well as the integer value 4, is represented as follows: `{@2; {@3}; 4}`.

Limitations of the 16-Bit Version: The bytecode instructions related to processing polymorphic variants and objects manipulate polymorphic variants and object methods as hash codes of their names. These hashes are represented on 31 bits by the standard OCaml bytecode compiler[10], and are therefore incompatible with a value representation using only 16 bits. To avoid making polymorphic variants or objects impossible to use in the 16-bit version of OMicroB, the VM is provided with a compiler *plug-in* (as a PPX syntax extension [⚓3]) that automatically translates polymorphic-variant and method names into new names whose hashed value fits in 15 bits. Indeed, for the names generated by this extension, the compiler's hash function produces hashes whose 16 most-significant bits are all zero. Generation of these names depends only on the original name, which preserves separate compilation of the different OCaml modules.

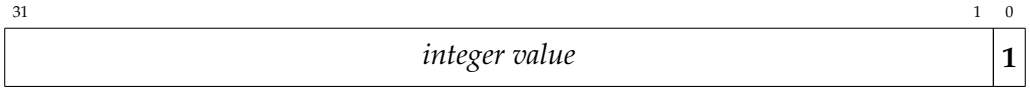
Value Representation

As in the standard OCaml virtual machine, the representation of manipulated values is uniform. The size of these values (16, 32, or 64 bits) is configurable at compile time, according to the developer's choice. We consider that the ZAM's native mechanism, which systematically allocates every non-integer value, and floating-point values in particular, is not necessarily appropriate for microcontroller programming. To avoid triggering the memory-reclamation system during execution, it can be useful to allocate, at program startup, the memory space required to represent the variables manipulated by the program, and thus avoid any dynamic allocation during its execution. We will return more precisely to the advantages of avoiding garbage-collector invocation during program execution when discussing WCET computation for a program in Chapter 5.

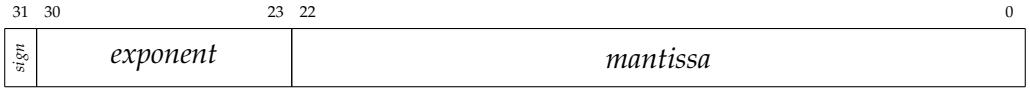
In OMicroB, immediate (non-allocated) values can therefore be of two types: values represented by integers, or floating-point values. The memory representation of values in OMicroB is therefore somewhat different from that of the ZAM. In the rest of this section, we detail OMicroB's value representation for a 32-bit configuration of the virtual machine.

[10] Regardless of the processor architecture – 32 or 64 bits – of the PC compiling the program.

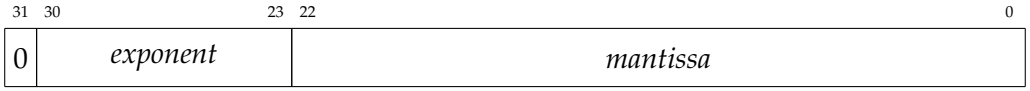
Integer Values: Integers are represented in OMicroB in the same way as in the original ZAM: they are encoded on 31 bits, and the least-significant bit of each corresponding OCaml value is set to **1**:



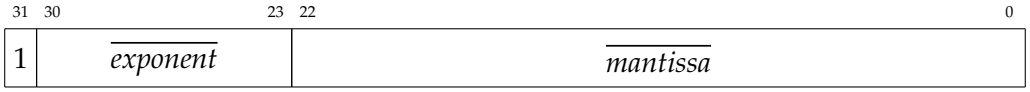
Floating-Point Values: The representation of floating-point values in OMicroB is based on the IEEE 754 standard [STD85]. This standard separates a floating-point number (32 or 64 bits) into three elements: its sign s, its exponent e, and its mantissa m. On 32 bits, a floating-point value is therefore represented by one sign bit, 8 exponent bits, and 23 mantissa bits:



In OMicroB, a *positive* floating-point number is represented naturally in this format on 32 bits:



It should be noted that the representation of an integer and that of a floating-point value may collide. Indeed, if a floating-point value has its least-significant bit set to 1, it is no longer distinguishable from an integer. This situation is not problematic for us, thanks to the safety introduced by strict static typing of OCaml programs: at no point in the program can a floating-point value be confused with an integer, or vice versa. All program operations are performed on values of compatible types: for example, there is no risk of adding a value representing an integer to a value representing a floating-point number. Nevertheless, the standard library provides a polymorphic comparison function, named `compare`, capable of comparing two variables of any type; since a floating-point value and an integer are indistinguishable during program execution, it is important that there be only one way to compare two immediate values (two integers or two floating-point values). This constraint, which in a sense allows two floating-point values to be compared by viewing them as two integers, requires the representation of *negative* floating-point values to be modified so that the ordering relation between the integer representation and the floating-point representation is identical. Negative floating-point values are therefore represented in OMicroB with the bits of their exponent and mantissa inverted:



This modification of the floating-point representation with respect to the standard induces a slight overhead for the execution of floating-point arithmetic operators: before these operators are applied, an "xor' is applied to OMicroB's negative floating-point values to invert the mantissa and exponent bits, making them compatible with standard implementations of floating-point operators.

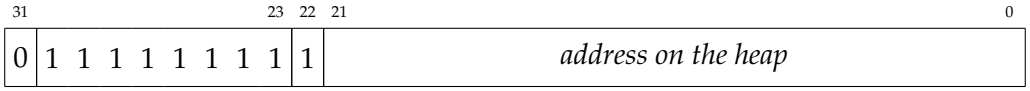
Heap Pointers: The distinction between the different categories of OCaml values is only truly useful when separating immediate values (in OMicroB: integers and floating-point values) from allocated values, represented by addresses on the heap. Indeed, the virtual machine's garbage collector needs an unambiguous dichotomy so that it can distinguish a pointer to a value on the heap (which it must

visit) from an immediate value (which it must ignore). In OMicroB, because floating-point values are represented as immediate values, it is no longer possible to use the least-significant bit of a value to determine its nature (a floating-point value may end in either 1 or 0). To represent heap pointers, OMicroB therefore uses a trick induced by a particular feature of the IEEE 754 standard. In this standard, values whose exponent bits are all equal to 1 (i.e. whose exponent is equal to 128 in a 32-bit representation) correspond to *special* values, divided into two categories:

1. If the mantissa is equal to 0, then this representation corresponds, depending on the sign bit, to the value $\pm\infty$.

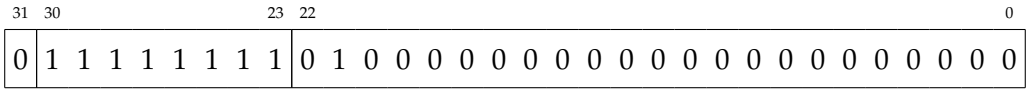
2. If the mantissa is different from 0, these values correspond to *NaN*s (*Not a Number*): values used to represent the result of certain invalid operations (for example division 0/0 or computing the square root of a negative number). These values are very numerous, since any floating-point number with a non-zero mantissa and exponent 128 is a *NaN*: in a 32-bit representation, there are therefore $2^{23}-1$ different *NaN* values. In OMicroB, we take advantage of this large space of "unused' floating-point values to represent pointers to OCaml values allocated on the heap.

Thus, we represent heap pointers by a *NaN* value beginning with the bit sequence **0111 1111 11**:



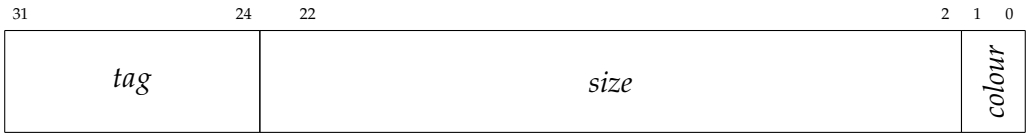
In a 32-bit representation of the virtual machine, this *NaN-boxing* system [Gud93], with marking of the least-significant bits, thus gives us a space of 2^{21} bits to represent addresses, and therefore makes it possible to allocate 2^{19} OCaml values (because all addresses are multiples of 4, since an OCaml value occupies 4 bytes). This theoretical 2-megabyte address space is largely sufficient for the hardware we target, which is limited to at most a few tens of kilobytes of RAM, and for which RAM addresses are often physically limited to 16 bits. For richer hardware, using a 64-bit configuration of the virtual machine provides a space of 2^{50} bits to represent an address, i.e. more than 2^{47} distinct OCaml values, since each one then occupies 8 bytes.

Any *NaN* value actually computed by the program is represented in OMicroB by the following single value:



OMicroB's standard library is adapted so that operations on *NaN*s are consistent with the IEEE 754 standard (which states, for example, that an equality test between two *NaN*s is always false).

Block Headers: Finally, the headers of OCaml blocks allocated on the heap are represented on 32 bits, with 8 *tag* bits, 22 size bits, and 2 colour bits:



16- and 64-Bit Representation: The details of value representation in OMicroB on 16, 32, and 64 bits are given in Appendix A. The 64-bit representation is similar to the 32-bit representation, extending integer values to 63 bits and representing floating-point values according to the IEEE 754 standard in double-precision format. The 16-bit representation is close to the IEEE 754 *binary16* representation, but only allows the use of 15 effective bits (to distinguish a floating-point value from a pointer), so we reduce the mantissa of a floating-point value from 10 bits to 9 bits.

2.3.3 Runtime Library

The standard library available with OMicroB includes many common modules from the standard virtual machine, such as the `List` module, which defines many functions manipulating lists (such as the `map` function, which applies a function to every element of a list), the `Queue` module, which represents mutable (*FIFO*) queues, and the `Hashtbl` module, which represents hash tables. In addition to these modules common to both implementations, there are modules specific to microcontroller programming, such as an `Avr` module defining low-level interaction primitives for an *ATmega* microcontroller, or a `LiquidCrystal` module[11], which communicates with a liquid-crystal display. Conversely, some modules that make little or no sense for microcontroller programming, such as the `Unix` module used to perform system calls, are not available.

Using OCaml to define the basic primitives for configuring the microcontroller and its interactions with its environment increases the safety of the programs produced. Indeed, OCaml's strict typing, combined with its support for generalized algebraic data types (*Generalized Algebraic Data Types*, or *GADT*), makes it possible to define low-level primitives that check certain typing criteria.

For example, the bits of the `SPCR` register (*SPI Control Register*), used to configure the serial port of an AVR microcontroller, are represented by the `spcr_bit` type:

```
type spcr_bit = SPR0 | SPR1 | CPHA | CPOL | MSTR | DORD | SPE | SPIE
```

Those of the `SPSR` register (*SPI Status Register*), which is used to monitor the state of serial communication (for example to check whether a transmission is complete), are represented by the `spsr_bit` type:

```
type spsr_bit = SPI2x | SPSR1 | SPSR2 | SPSR3 | SPSR4 | SPSR5 | SPSR6 | SPIF
```

The microcontroller registers are then represented by an `'a register` type parameterised by the type of the bits it contains:

```
type 'a register =
  | (* ... *)
  | SPCR: spcr_bit register
  | SPSR: spsr_bit register
  | (* ... *)
```

[11] Created by a master's student during an internship on constraint programming with OMicroB [Pes18].

The `set_bit` primitive, of type `'a register -> 'a -> unit`, sets one of the bits of a microcontroller register to 1. At compilation, any use of this function in a program then causes the type system to verify that the bit passed as the second parameter to the function is indeed a bit of the register passed as the first parameter. For example, the incorrect call `set_bit SPCR SPIF` causes an explicit error during program compilation:

```
Error: This variant expression is expected to have type Avr.spcr_bit
       The constructor SPIF does not belong to type Avr.spcr_bit
Hint: Did you mean SPIE?
```

This inconsistency between the bit name and the register name could not have been detected by simply using macros to represent bit names in a low-level language such as C. Verifying the typing correctness of a program clearly illustrates one of the advantages of using a high-level programming language, even for such low-level interactions.

In OMicroB, program memory is automatically managed by a *garbage collector* that implements a standard *stop-and-copy* algorithm. It manipulates two pools for values allocated on the heap: a *live* pool and a *copy* pool. Each time the GC runs, the values still used by the program are copied from the live pool to the copy pool, then the roles of the two pools are swapped (the live pool becomes the copy pool, and vice versa). To do this, the memory-management algorithm traverses a set of *roots* corresponding to the registers and other memory blocks that may contain pointers to values allocated on the heap (the arrows pointing to the heap in Figure 2.10 represent such pointers), copies the values to which they point into the new pool, and updates each pointer with the new location of the copied values. The GC uses the distinction introduced by OMicroB's value representation: it visits all pointer values encoded by NaN-boxing. Such an algorithm has the advantage of being fast, but has the notable drawback of reserving half of the available RAM (the copy pool) for its operation. A *Mark and Compact* garbage collector, which avoids "wasting' half of the heap (at the cost of slower execution), is also available.

2.3.4 Producing an Executable

Compilation for the Target Architecture

Compiling the executable program is the final step before transferring the program to the target hardware. It uses existing C compilers, such as *avr-gcc* or *sdcc*. The C compiler then links the C file produced by *bc2c*, the virtual-machine interpreter, and its runtime library (standard library and garbage collector). The generated file is then transferred to the microcontroller (with a suitable tool such as *avrdude*) and executed on it.

Using C as a kind of *portable assembler* makes it relatively painless to use OMicroB on many different architectures. To support a new microcontroller architecture, the virtual machine only requires the rewriting of the low-level C primitives that are essential for interacting with the hardware (such as the functions used to read the microcontroller's flash memory), and it does not require any deep modification of the interpreter code or of the other OMicroB components.

Debugging and Simulating Microcontroller Programs

Generating generic C code also makes it possible to run the virtual machine on more conventional hardware. In particular, the *gcc* compiler can be used on the computer where the program was written,

in order to compile it into an executable compatible with that computer's architecture (an x86 or x86-64 executable). This compilation mode makes it possible to simulate program execution on a computer before transferring the program to the target microcontroller, thereby simplifying the debugging process.

For this purpose, OMicroB includes tools for simulating the effects of a program by graphically representing the state of a microcontroller's input and output pins (Figure 2.13). This simulation makes it possible to check that program execution is consistent with the expected behaviour, without forcing the developer to systematically transfer the program to a real microcontroller.

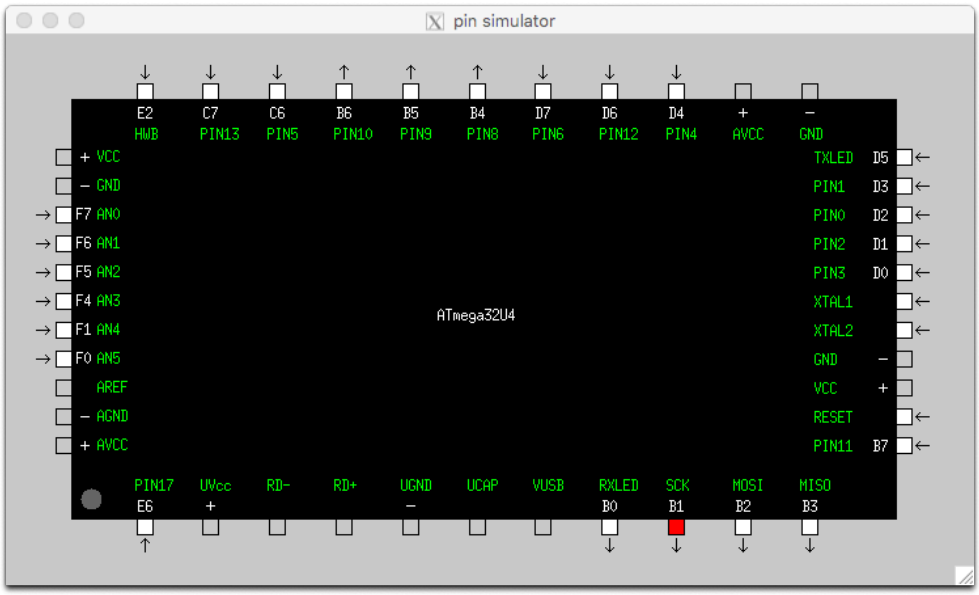
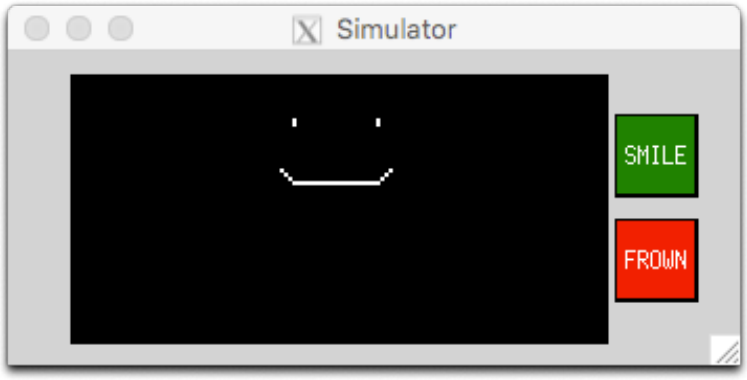


FIGURE 2.13: Simulation of the pin state of an ATmega32u4 microcontroller

Like OCaPIC's simulator, the OMicroB simulator can also represent interactions between the microcontroller and the circuit connected to it: it lets the programmer describe the components connected to the microcontroller (buttons, screens, sensors, . . .) so that the program's effect on those components can also be simulated. For example, Figure 2.14 shows the simulation of a small circuit running a program that displays a smiling face on an OLED screen (*Organic Light-Emitting Diode*) if the user presses the "SMILE' button, and a frowning face if the user presses the "FROWN' button. The different components of the circuit are described in a `circuit.txt` file used by the simulator to produce the appropriate display.

2.4 OMicroB Optimisations

Given the severe limitations of the hardware targeted by the OMicroB virtual machine, we focused our work on reducing the memory footprint of programs executed on microcontrollers. To this end, OMicroB implements several optimisations intended to limit the resource consumption of both the virtual machine and the OCaml programs produced. For example, reducing the size of bytecode-instruction *opcodes* from 32 bits to 8 bits is one such optimisation. In this section, we present two further optimisations applied to the virtual machine and to OCaml programs.



circuit.txt

```
window width=350 height=150 bgcolor=lightgray title="Simulator"
oled x=30 y=10 column_nb=128 line_nb=8 cs=PIN12 dc=PIN4 rst=PIN6
button x=310 y=100 width=40 height=40 label="SMILE" pin=PIN7 color=green
button x=310 y=50 width=40 height=40 label="FROWN" pin=PIN8 color=red
```

FIGURE 2.14: Simulation of a small circuit

2.4.1 Early Evaluation of Programs

The bytecode interpreted when an OCaml program starts corresponds to a sequence of initialisations for values needed during execution. First, the program deserialises certain constants (lists, strings, . . .) and allocates them on the heap. Program initialisation then continues by allocating the closures used in the program, creating modules, and computing global variables. This initialisation phase can be slow and can consume a non-negligible amount of memory. In particular, the stack depth required to load the modules used by the program can be substantial. For example, the program in Figure 2.15, which defines a class `pt`, needs at least 106 stack levels to load the modules responsible for using objects in OCaml.

```
class pt (x:int) (y:int) =
 object
   method get_x = x
   method get_y = y
 end

let () =
  Avr.delay 1000;
  let p = new pt 1 2 in
  p#get_x
```

FIGURE 2.15: Example of an OCaml program manipulating objects

However, execution of an OCaml program is completely deterministic during these precomputation phases, regardless of the hardware on which it runs. This determinism lasts until the program's first input/output operation, which starts communication between the program and its environment. To speed up program start-up, and above all to reduce memory consumption, it is therefore possible to

perform the different program-initialisation steps before execution on the microcontroller. Bytecode interpretation is simulated by *bc2c* on the computer where the program is compiled, up to the first input/output operation; the resulting memory state (i.e. the values contained in the heap, the stack, and the program's global-variable array) is then written directly into the C code generated by *bc2c*. Thus, when the C code embedding the OCaml bytecode is produced, the virtual machine's stack and heap have already been populated with values corresponding to the closures and values computed before the first instruction that begins interaction between the program and its environment. This early-evaluation process can be seen as a form of partial evaluation [JGS93] performed on the OCaml program's bytecode.

In the example in Figure 2.15, enabling early evaluation (which pre-executes the program up to the call to the `Avr.delay` primitive[12]) reduces the stack size needed to execute the program to only 16 levels.

2.4.2 Custom Virtual Machine

Most compiled programs do not use the entire bytecode instruction set associated with the OCaml language, but only a subset corresponding to the language features actually used in the program. For example, a program that does not use OCaml's object layer will not contain, after compilation to bytecode, the `GETPUBMET` instruction used to call an object's method. Similarly, since many arguments can fit in a single byte, specialised instructions for four-byte arguments, such as `CLOSURE_4B`, are rarely actually used. As a result, embedding in the final executable an interpreter capable of handling instructions that are absent from the program's bytecode wastes space, even though a microcontroller's memory is a scarce resource.

OMicroB avoids this unnecessary memory waste by including, when the program is compiled, a bytecode interpreter that can process only the instructions actually present in the program's bytecode. To do this, directives for the C compiler preprocessor are added to the interpreter code in order to prevent compilation of the code related to interpreting instructions not used by the program. Each opcode present in the program then corresponds to a *macro* defined in the C program generated by *bc2c*; this macro represents a constant value, chosen so that all values are contiguous. The total size of the executable transferred to the microcontroller is thereby reduced.

Figure 2.16 illustrates the structure of the custom interpreter using an excerpt from it: the code responsible for processing the `BRANCHIF_4B` instruction is compiled only if the macro corresponding to this instruction is defined in the C code generated by *bc2c*.

```
#ifdef BRANCHIF_4B
    case BRANCHIF_4B:
       /* instruction interpretation code */
    break;
#endif
```

FIGURE 2.16: Custom interpreter

Other optimisations based on the same idea of generating a specialised C program could be envisaged. We could indeed take our approach further by producing a virtual machine specific to each program: for example, by statically replacing each bytecode instruction in a program with the corresponding set of C

[12] This primitive pauses the program for the duration specified by its parameter, in milliseconds.

instructions, thereby dispensing with the bytecode interpreter altogether. This approach was followed by the *OCamlCC* tool [MV13], whose first implementation transformed OCaml bytecode into a C program by replacing each bytecode instruction with its associated low-level code through macro expansion. Such an approach gives good execution speed, but program size grows quickly because each occurrence of a given bytecode instruction duplicates the low-level code needed to execute it. Other approaches, such as CeML [Cha92] or Camlot [Cri92], consist in transforming the source code of a program written in an ML language into C code, which can then benefit from C-compiler optimisations. The code generated by these solutions is nevertheless fairly large, and their runtime libraries are also generally substantial, since they must, for example, include a generic application mechanism to build closures in the case of partial function application. These approaches are therefore better suited to hardware for which memory-consumption concerns are less important than program execution speed. OMicroB therefore seems to us to be a good compromise between the portability provided by implementing the VM in C and the reduction in program size obtained by representing programs as bytecode, which factors lower-level instruction sequences. We therefore follow this pragmatic approach, for which execution speed may be slightly lower without being particularly penalising for the applications we target.

Chapter Conclusion

Using the OMicroB virtual machine provides a generic and configurable approach for running OCaml programs on a wide variety of devices. In particular, the optimisations implemented in OMicroB make it possible to interpret OCaml bytecode on microcontrollers from ranges with few hardware resources. This virtual machine is capable of running OCaml programs on AVR microcontrollers with severely restricted RAM resources, such as the ATMega325p (2 KB of RAM), the ATmega32u4 (2.5 KB of RAM), or the ATmega2560 (8 KB of RAM). Several academic projects aiming to port OMicroB to other architectures, such as PIC32 or ARM Cortex-M0 (used by the *micro:bit* development boards, designed by the BBC to support the teaching of programming to young children [⚓20]), are under way, and the first results are very promising: the first OMicroB ports have indeed been completed without difficulty [PB19].

The OCaml program compilation model considered here, which uses the bytecode of a high-level language together with a generic interpreter, greatly increases program portability: the same OCaml program can thus be readily ported from one device to another. Moreover, this portability makes it easy to simulate the programs produced, while taking into account the memory constraints imposed by the virtual-machine configuration (such as stack size or heap size), thereby providing an accelerated and simplified debugging process.

To continue efforts aimed at reducing the RAM footprint of OCaml programs, several techniques are currently being studied. In particular, a fine-grained analysis capable of detecting immutable constant values in an OCaml program (such as strings, or *sprites* in a video-game program) would make it possible to move these values into the program's flash memory during compilation, thereby freeing the more limited RAM.

Table 2.1 lists the main differences between OMicroB and the standard OCaml virtual machine. In Chapter 7, we discuss OMicroB's performance in greater detail, both in terms of OCaml program execution speed and in terms of memory footprint.

Because of its richness and high level of expressiveness, OCaml is a powerful language for describing the algorithmic behaviour of programs, and the safety of its type system provides important guarantees when producing embedded programs. However, OCaml is not currently well suited to describing the concurrent aspects of a program, nor to developing real-time systems. Yet the embedded programs we consider have many concurrent features, and the *critical* systems they control very often correspond to real-time systems, for which execution times must be precisely controlled. For this reason, the next chapter presents *OCaLustre*, an extension of OCaml for synchronous programming, which offers a lightweight concurrency model adapted to the nature of microcontroller applications. This extension is fully compatible with OMicroB and therefore benefits from OCaml's advantages and from the virtual-machine optimisations described in this section.

	ZAM (ocamlrun)	OMicroB
Opcode size	32 bits	8 bits
Number of instructions	148	148 + 46 specialised inst. = 194
Value size OCaml	not configurable (architecture-dependent)	configurable (16, 32, 64 bits)
Address size (address space) in a 32-bit configuration	32 bits (4 GB)	21 bits (2 MB)
Floating-point representation	Boxed (heap allocation)	Immediate
GC algorithm	Hybrid (generational / incremental)	Stop and Copy or Mark and Compact

TABLE 2.1: Main differences between the standard OCaml virtual machine and OMicroB

3 OCaLustre: Synchronous Programming in OCaml

OCaLustre is a synchronous dataflow extension of the OCaml language, inspired by Lustre [CPHP87, Ber86, HCRP91]. It makes it possible to use a synchronous abstraction layer to program concurrency, while using the high-level features of the host language, OCaml, to develop the logical aspects of applications. This extension, whose memory footprint is very small, is intended to run together with the OMicroB virtual machine in order to execute lightweight and safe concurrent programs on resource-constrained microcontrollers.

In this chapter, we describe this language extension in detail. In a first section, we present the main features and principles of the language that make it possible to write synchronous programs in OCaml. In particular, this section discusses the *synchronous clock system* implemented in OCaLustre, which can condition the presence of certain values during program execution. We then present the specification of the type systems for OCaLustre programs, covering both the "standard' types that represent the values carried by program elements, and the clock types related to the notion of *clocking* in a synchronous program. Finally, we formally describe the operational semantics of the language, derived from the semantics of Lustre.

3.1 Programming in OCaLustre

In this section, we present the main concepts and general features of the OCaLustre language. The aim of this presentation is to explain to a potential OCaLustre developer the different aspects of the language that make it possible to produce a correct program. Apart from a few specific syntactic variations, these aspects should not surprise readers familiar with Lustre (or its derivatives), since OCaLustre follows the same semantic foundations.

3.1.1 Language Syntax

Like a Lustre program, an OCaLustre program is composed of nodes, which compute streams of values, but also of standard OCaml functions, which manage the algorithmic aspects of programs by drawing on the expressive power of OCaml. An OCaLustre node is defined using the keyword `let%node`, followed by its name, the names of its input parameters, and an n-tuple representing its different output streams (labelled by the keyword `return`). The body of a program is a system of equations solved at each synchronous instant of program execution, thereby assigning values to the output streams of each node in the program. The equations in the body of a node are stream definitions of the form $y = e$, where y is the name (or an n-tuple of names) of the stream being defined, and e is an expression used to evaluate the value of the stream or streams concerned.

We first present a partial version of the language, which we will enrich as the discussion progresses. In this version, an expression may correspond to a constant (including the *unit* value), a variable, a constructor of an enumerated type defined in OCaml, an application of the conditional operator (*if-then-else*), an application of an arithmetic or logical operator, or an n-tuple. Figure 3.1 describes the syntax of this partial version of the language in Backus-Naur form (*Backus-Naur Form – BNF*)[1].

⟨int⟩	::=	$[0-9]+$
⟨float⟩	::=	$[0-9]+$ `.` $[0-9]*$
⟨bool⟩	::=	**`true`** | **`false`**
⟨constant⟩	::=	`()` | ⟨int⟩ | ⟨float⟩ | ⟨bool⟩
⟨ident⟩	::=	$[a-z]+[a-zA-Z0-9]*$
⟨lidents⟩	::=	`()` | `(`⟨ident⟩$[$`,`⟨ident⟩$]*$`)` | ⟨ident⟩
⟨enum⟩	::=	$[A-Z]+[a-zA-Z0-9]*$
⟨int_op⟩	::=	`+` | `-` | `*` | `/` | **`mod`**
⟨float_op⟩	::=	`+.` | `-.` | `*.` | `/.`
⟨bool_op⟩	::=	`&&` | `||`
⟨comp_op⟩	::=	`<` | `>` | `<=` | `>=` | `=` | `<>`
⟨binop⟩	::=	⟨int_opt⟩ | ⟨float_op⟩ | ⟨bool_op⟩ | ⟨comp_op⟩
⟨unop⟩	::=	`-` | `-.` | **`not`**
⟨eqn⟩	::=	⟨lidents⟩ `=` ⟨expr⟩
⟨leqns⟩	::=	⟨eqn⟩ | ⟨eqn⟩`;`⟨leqns⟩
⟨expr⟩	::=	⟨constant⟩
		| ⟨ident⟩
		| ⟨enum⟩
		| ⟨expr⟩⟨binop⟩⟨expr⟩
		| ⟨unop⟩⟨expr⟩
		| **`if`** ⟨expr⟩ **`then`** ⟨expr⟩ **`else`** ⟨expr⟩
		| ⟨expr⟩`,`⟨expr⟩
		| `(`⟨expr⟩`)`
⟨node⟩	::=	**`let%node`** ⟨ident⟩ ⟨lidents⟩ **`~return:`**⟨lidents⟩ `=` ⟨leqns⟩

FIGURE 3.1: Partial syntax of the OCaLustre language

Thus, the example in Figure 3.2 corresponds to the definition of the node named `plus_moins`, which computes a pair corresponding to the sum and the difference of its two input parameters; the associated table represents the evolution of the values of each stream of this node during execution.

Initialised Delay

In an imperative language, it is common to assign a value to a variable on the basis of the value it already contains: for example, incrementing a variable (`x:= x + 1`) implicitly accesses the "previous' value of

[1]The representation of literals and identifiers is simplified here. In the actual implementation of the OCaLustre compiler, any literal or identifier valid in OCaml is also valid in OCaLustre.

```
let%node plus_moins (x,y) ~return:(p,m) =
  p = x + y;
  m = x - y
```

instant	0	1	2	3	4	5	6	...	*i*	...
x	3	2	6	32	8	26	67	...	7	...
y	4	12	42	9	22	7	53	...	3	...
p	7	14	48	41	30	33	120	...	10	...
m	-1	-10	-36	23	-14	19	14	...	4	...

FIGURE 3.2: Example of an OCaLustre node

the variable in order to modify its "current' value. OCaLustre is based on a programming model closer to functional programming, in which a variable cannot be modified during program execution. In fact, dataflow semantics implicitly redefines *new* variables at each instant, and each of these definitions is itself immutable. It is nevertheless sometimes necessary to access the previous value of a variable in order to assign it a current value (typically, to increment a counter). In an electronic circuit, it may be useful to access the previous value emitted by a sensor in order to compute its difference from the current value, for example to determine the temperature difference measured by a sensor between one instant and the next, and thereby infer the rate of increase.

Consequently, to allow a stream to access, at instant i, the value of a stream at instant $i-1$, OCaLustre provides an *initialised delay* operator, written $\ggg$ and similar to the `fby` operator, for "*followed-by*", used by the Lucid dataflow language [AW77] and by various synchronous languages, such as Heptagon [Gér13], Lucid Synchrone [CP99], or Lustre V6 [JRH19]. Used in the expression `0` $\ggg$ `x`, this operator means that the expression has as its value the constant to the left of $\ggg$ (i.e. `0`) at the first instant of program execution, then the *previous* value of the expression to the right of the operator (i.e. `x`) at all future instants:

$$k \ggg x \equiv (k, x_0, x_1, x_2, \ldots, x_{i-1}, \ldots)$$

Thanks to this operator, sequences of values are easy to represent. For example, the following node computes a stream `n` corresponding to the sequence of natural numbers:

```
let%node cpt () ~return:(n) =
  n = (0 ⋙ (n+1))
```

Indeed, the stream `n` has the constant value 0 at the first instant, followed by the previous value of the expression `n+1` at each subsequent instant.

In the same way, the following example computes the Fibonacci sequence:

```
let%node fibonacci () ~return:f =
  f = (0 ⋙ ((1 ⋙ f) + f))
```

The table in Figure 3.3 details the evolution over time of the different expression values in the `fibonacci` node.

instant	0	1	2	3	4	5	6	7	8	...
`f`	0	1	1	2	3	5	8	13	21	...
`(1 ⋙ f)`	1	0	1	1	2	3	5	8	13	...
`(1 ⋙ f) + f`	1	1	2	3	5	8	13	21	34	...

FIGURE 3.3: Computation of the Fibonacci sequence `f` using the initialised delay operator

Let us illustrate the use of this operator in the context of developing programs for microcontrollers. We return to the example program from Section 1.1.2, in which the microcontroller blinks an LED at regular intervals.

A very simple OCaLustre program can reproduce this behaviour. Let us define the following enumerated OCaml type and OCaLustre node:

```
type light_state = ON | OFF

let%node blinker () ~return:led =
    led = (ON ⋙ if led = ON then OFF else ON)
```

The stream `led` is therefore initialised with the value "`ON`" to switch on the *LED*, and at each following synchronous instant the lamp takes the opposite state to its previous one.

The ⋙ operator also makes it easy to represent streams whose sequence of values repeats cyclically. For example, if we define an enumerated `tictactoc` type as follows:

```
type tictactoc = Tic | Tac | Toc
```

The stream `x` then corresponds to the sequence of values (`Tic` , `Tac` , `Toc` , `Tic` , `Tac` , `Toc` ,..., `Tic` , `Tac` , `Toc` ,...) :

```
x = (Tic ⋙ (Tac ⋙ (Toc ⋙ x)))
```

The `blinker` node can therefore also be written as follows:

```
let%node blinker () ~return:led =
    led = (ON ⋙ (OFF ⋙ led))
```

It is also possible to redefine classic Lustre operators using the initialised delay operator:

- Lustre's initialisation operator `->` can be rewritten using the ⋙ operator as follows:

```
x -> y ≡ if (true ⋙ false) then x else y
```

- The delay operator **pre** could also be defined with ⋙ , provided an initial value k (of the right type) is supplied for the stream in question:

$$\texttt{pre}\ \texttt{x} \equiv k \ggg \texttt{x}$$

Our choice to use the ⋙ operator in OCaLustre, rather than a combination of these operators, frees the compilation of an OCaLustre program from certain considerations related to stream initialisation. Indeed, since for any `x` the value of the stream **pre** `x` at instant 0 is undefined (*nil*), it is usually important, when compiling a program, to check that the **pre** operator appears in a stream definition only to the right of a `->`. If this were not the case, the value of the stream at instant 0 would be unknown, and the program would have indeterminate behaviour. This issue of stream initialisation, which therefore does not arise in OCaLustre, can nevertheless be resolved statically by using a type system able to represent the position of **pre** operators in expressions [CP04].

Locally Scoped Streams

It should be noted that, in the definition of an OCaLustre node, equations do not necessarily have to correspond to input or output streams. To improve the readability of a program, represent an internal register, or factorise certain computations, it may be useful to define streams that are considered local to the node. In OCaLustre, a local stream is defined without any particular keyword; its mere absence from the node signature is enough to limit its scope.

The `fibonacci` node from the previous section can therefore be rewritten, to make it easier to read, as a `fibonacci2` node defining the local streams `sum_last` and `previous_f`:

```
let%node fibonacci2 () ~return:(f) =
  f = (0 ⋙ sum_last);
  previous_f = (1 ⋙ f);
  sum_last = previous_f + f
```

Node Application

To organise the different behaviours of a program into distinct code blocks, we introduce into the language a mechanism for *applying* nodes, as in Lustre. Calling (or applying) a node in an OCaLustre program makes it possible to factorise redundant code fragments, and thus reduce a program's memory footprint. It follows OCaml's classic function-application syntax: for example, the following node calls the `plus_moins` node defined above, with the pair `x,y` as its input parameter and the streams `a` and `b` as its outputs:

```
let%node call_pm (x,y) ~return:(a,b) =
  (a,b) = plus_moins (x,y)
```

It should be noted that, in the language semantics, each call to a node actually corresponds to a call to an *instance* of that node. Each call site of a node implicitly initialises an instance of that node and, as a result, no stream is shared between these different call sites.

For example, the following node defines *two* counters executing concurrently:

```
let%node two_cpt () ~return:(c1,c2) =
  c1 = cpt ();
  c2 = cpt ()
```

The internal counters in each call to `cpt` are distinct, and the streams `c1` and `c2` therefore increment at the same speed, since each has its own internal register updated at each synchronous instant.

External Calls to OCaml Functions

The operators built into OCaLustre are deliberately very simple: they are limited to basic arithmetic and logical operators, supplemented by a few operators referring to time. Yet the host language of this extension, OCaml, has considerable expressive richness through its implementation of several programming paradigms (imperative, functional, object-oriented) and powerful constructs (functors, generalised algebraic data types, polymorphism, . . .), which provide the developer with high-level tools for producing complex programs. To combine the synchronous aspects of a program, which govern the interaction between the different software components of an application, with its purely algorithmic aspects, which exploit the richness of the host language, OCaLustre is enriched with a **`call`** operator that makes it possible to call an OCaml function from an OCaLustre node.

For example, in the following program, the synchronous node `sqrt_cpt` calls the OCaml function `f`, which computes, *within a synchronous instant*, the square root of the stream `a`.

```
let f x = if x > 0.0 then sqrt x else 0.0

let%node sqrt_cpt () ~return:b =
  a = (0.0 ⋙ (a +. 1.0));
  b = call f a
```

It should be noted that using **`call`** can be "dangerous', because it opens OCaLustre to all the constructs of the OCaml language, which may perform operations incompatible with the semantics of the synchronous model[2], or which may never return (because of an infinite loop or because an exception is raised). It is therefore the programmer's responsibility to guard against these erroneous behaviours. For safety, in the rest of this manuscript we will assume that **`call`** is used only to execute purely functional code that terminates.

The syntax of OCaLustre expressions presented in Figure 3.1 is then extended with the initialised-delay operator, node application, and the operator for applying an n-ary OCaml function:

[2]For example, the behaviour of an OCaLustre program in which two concurrent uses of `call` modify the same mutable global variable is indeterminate.

```
⟨expr⟩ ::= (...)
         | ⟨constant⟩ ⋙ ⟨expr⟩
         | ⟨ident⟩ (⟨expr⟩)
         | call ⟨ident⟩ ⟨expr⟩[ ⟨expr⟩]*
```

3.1.2 Synchronous Clocks

Each component of an OCaLustre program executes concurrently. As a result, each equation defining the value of a stream in a node is computed at each synchronous instant, and each expression contained in these equations is also evaluated within that instant. Thus, the following equation causes both `e1` and `e2` to be executed in the same instant, the value assigned to x then being chosen according to the value of the Boolean `b`:

```
x = if b then e1 else e2
```

This semantics differs from the usual semantics of general-purpose programming languages, in which evaluation of the *if-then-else* conditional operator is commonly lazy. For example, in OCaml, evaluating the following expression causes only the *positive* branch (respectively the *negative* branch) of the conditional operator to be evaluated when `b` is true (respectively false), and the displayed message is `"true"` (respectively `"false"`):

```
if b then print_string "true" else print_string "false"
```

Conversely, in OCaLustre, every branch of a conditional is executed at each instant: for example, if we define the `count` node, which computes a counter:

```
let%node count () ~return:cpt =
  cpt = (0 ⋙ (cpt + 1))
```

Then, in the following equation, the counter increment in `count ()` is performed *at each instant*:

```
x = if b then count () else 0
```

However, in order to write non-trivial programs, it is necessary to provide a way of conditioning the execution of certain parts of a program, and thereby to recover the notion of *control flow*. From the point of view of synchronous programming, this idea amounts to *slowing down* the execution of certain branches of the program: in other words, we seek to execute expressions only at certain instants, and therefore to *subsample* the value of certain streams of an OCaLustre program over time.

Subsampling

To condition an expression `e` in OCaLustre so that it has a value only when a certain Boolean stream b is true, we use the annotation `[@when` b `]`. For example, the following equation declares a stream `x` whose

value is that of the expression `(y+1)` only when the stream `clk` evaluates to *true*, and which otherwise has no value:

```
x = (y+1) [@when clk]
```

We then say that the stream `x` is *clocked* by the clock `clk`. A clock is a Boolean stream that represents the *presence condition* of the streams it clocks: when the value of a clock is false, all the streams clocked by it are considered absent (they have no value). It is then forbidden to access the value of these streams. For example, the following OCaLustre program fragment is incorrect:

```
let%node bad_clocks () ~return:(x,y,w) =
  w = 2;
  y = 3;
  x = w [@when b] + y
```

Indeed, the stream `y`, which is not clocked by any explicit clock (we say that it is clocked by the *base clock*, i.e. the fastest clock of the node, which is an implicit parameter of each node), has its value added to that of the stream `w`, which is on the clock `b`. Computing the value of `x` when `b` is false then makes no sense, because there is "nothing' to the left of the addition[3], written $\perp$. All binary operators in OCaLustre can be applied only to streams that are both clocked by the same clock.

Moreover, streams clocked by a clock that is itself absent (because its own clock is false) are also absent. In the following example, the stream `z` therefore has no value because its clock `ck2` is absent:

```
let%node sampled_clock () ~return:(ck1,ck2,z) =
  ck1 = false;
  ck2 = true [@when ck1];
  z = 42 [@when ck2];
```

Conversely to *positive* subsampling, OCaLustre provides a *negative* subsampling annotation, `[@whennot _ ]`. It is then possible to require a value to be present only when a Boolean is *false*. Thus, in the following fragment, the streams `a`, `b`, and `c` have a value only if the stream `clk` evaluates to *false*. We say that they are clocked by the clock *not* `clk` (or $\overline{\texttt{clk}}$):

```
a = 2 [@whennot clk];
b = 3 [@whennot clk];
c = a + b
```

The only OCaLustre operator that can manipulate streams not clocked by the same clock is the **`merge`** operator: it "merges' streams clocked by complementary clocks. This operator receives, as its first parameter, a clock; then a stream positively clocked by that clock; then a stream negatively clocked by

[3]It should be noted that, in the language semantics, the *absence* of a value is not equivalent to the presence of the constant *nil*, nor of any default value.

the same clock. In the following example, the stream k has the value of the stream i when d is true, and the value of the stream j when d is false:

```
c = (cpt () < 5);
d = (true ⋙ false) [@when c];
i = 23 [@when d];
j = 45 [@whennot d];
k = merge d i j
```

The runtime values of the different streams defined in this example are given in Figure 3.4. It should also be noted here that using the **@when** operator performs *sampling* of values, and does not introduce a *delay* in the computation of these values: the expression (**true** ⋙ **false**) is evaluated at *every* execution instant (even when c is false)[4], but the corresponding value is associated with d only when c is true. We will return to the distinction between sampling and delay when we discuss conditional applications in Section 3.1.2.

instant	0	1	2	3	4	5	6	7	8	9	10	...
c	true	true	true	true	true	false	false	false	false	false	false	...
d	true	false	false	false	false	⊥	⊥	⊥	⊥	⊥	⊥	...
i	23	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	...
j	⊥	45	45	45	45	⊥	⊥	⊥	⊥	⊥	⊥	...
k	23	45	45	45	45	⊥	⊥	⊥	⊥	⊥	⊥	...

FIGURE 3.4: Evolution of the values of clocked streams

The **merge** operator can therefore be viewed as the traditional lazy *if-then-else* conditional operator of general-purpose languages. It is important to point out that the result (the stream k) is present at the same rate as the sampling clock of the streams i and j (the stream d, whether its value is *true* or *false*), and is not faster: since d is present only when c is true, k is present only under the same condition.

The syntax of OCaLustre expressions, extended with subsampling annotations and the **merge** operator, is then complete:

⟨expr⟩ ::= (...)
| **merge** ⟨ident⟩ ⟨expr⟩ ⟨expr⟩
| ⟨expr⟩ **[@when** ⟨ident⟩**]**
| ⟨expr⟩ **[@whennot** ⟨ident⟩**]**

Special Case: Constants

In OCaLustre, constants have the particularity of being clocked by any clock: their clock type can be considered polymorphic, and they are therefore compatible with any clock. This distinction stems from the desire to make OCaLustre programs easier to write and read. Indeed, in the following example

[4]This follows from Lustre's *substitution principle*, which states that if a stream x has value *expr*, then any occurrence of *expr* in the program can be replaced by x, and conversely, without changing the semantics. The expression (**true** ⋙ **false**) could therefore be extracted into a new variable, evaluated at each instant.

```
let%node ex_const (a,b) ~return:x =
  c = a;
  d = b [@when c];
  v = 12 [@whennot c];
  x = merge c d (4 [@whennot c] + v)
```

an expression such as `((4 [@whennot c])` is relatively "heavy', and is not useful given that, in this usage context (as the third parameter of a **merge** whose first parameter is c), the clock of `4` can only be $\bar{c}$. Similarly, since the variable `v` is added to a value clocked by $\bar{c}$, its clock is also $\bar{c}$.

The same node is therefore more readable when constants are stripped of their clock annotations:

```
let%node ex_const (a,b) ~return:x =
  c = a;
  d = b [@when c];
  v = 12;
  x = merge c d (4+v)
```

Conditional Application

When a stream is defined as the result of a call to a node, it is important to distinguish subsampling the call parameters from subsampling the return value. Indeed, subsampling an input value serves to *condition the execution* of the call to a node, and thus to slow down the execution frequency of the called node; by contrast, subsampling the return value of a call to a node only *limits the presence* of that value.

For example, below we give the definition of a node `cpt` that computes the sequence of natural numbers, modulo `n`:

```
let%node cpt n ~return:c =
  c = (0 ⋙ (c + 1)) mod n
```

We then define a node `call_cpt` that computes two streams: a stream `a`, corresponding to the call to `cpt` with its parameters subsampled, and a stream `b`, corresponding to the call to `cpt` with its return value subsampled:

```
let%node call_cpt ck ~return:(a,b) =
  a = cpt (10 [@when ck]);
  b = (cpt 10) [@when ck]
```

From the point of view of their clocking clocks, the streams `a` and `b` are indistinguishable, since each of these streams is present only when the stream `ck` is true. Their semantics are nevertheless different: for the stream `b`, the call `(cpt 10)` is performed at each instant (whatever the value of `ck`), and it is then the return value of this call that is subsampled by `ck`. Conversely, for the stream `a`, execution of `(cpt 10)` is performed only if `ck` is true. The counter is then incremented only when `ck` is true, and the

execution frequency of `cpt` is potentially slower than that of `call_cpt`. The table in Figure 3.5 illustrates the difference between these two semantics.

instant	0	1	2	3	4	5	6	...
`ck`	true	false	true	true	false	false	true	...
`cpt (10 [@when ck])`	0	⊥	1	2	⊥	⊥	3	...
`(cpt 10) [@when ck]`	0	⊥	2	3	⊥	⊥	6	...

FIGURE 3.5: Difference between parameter subsampling and return-value subsampling.

Subsampling a node's parameters therefore makes it possible to *slow down* the call frequency of that node, and thus to make its internal state evolve "more slowly'. Such behaviour will be called *conditional application* in this manuscript.

A classic example of conditional application is the watch example [Pla89]. In this example, the different calls to `cpt` are conditioned by clocks at increasingly slow frequencies, making it possible to simulate the computation of hours, minutes, and seconds[5]:

```
let%node watch (sec) ~return:(hour,min,h,m,s) =
  s = cpt (60 [@when sec]);
  min = (s = 60);
  m = cpt (60 [@when min]);
  hour = (m = 60);
  h = cpt (24 [@when hour])
```

Each call to `cpt` is executed at a distinct frequency:

- The stream `s` is incremented each time its clock *sec* is true.
- The stream `m` is incremented each time *min* is true (60 times more slowly than `s`).
- The stream `h` is incremented each time *hour* is true (60 times more slowly than `m`).

The notion of *conditional application* should be distinguished from that of *conditional activation*, used for example in the Scade language (where it is implemented by the `condact` operator in versions earlier than Scade 6, and then by the `activate/every` construct [Dor08]). Conditional activation corresponds to a conditional application of a node associated with a projection: if the clock of the conditional application is false, then the associated stream keeps the value computed the last time its clock was true (and if it has never yet been true, the stream has a default value explicitly provided by the programmer).

This conditional-activation mechanism can be implemented in OCaLustre by combining conditional application, the merge operator, and the delay operator. For example, the following node performs conditional activation of a call to the `cpt` node:

[5]The reason why the clocks of `h` and `m` appear in the node's output tuple will be explained when we discuss clock typing in Section 3.1.3.

```
let%node condact_cpt (c) ~return:(x) =
  x = merge c (cpt (10 [@when c])) ((0 ⋙ x) [@whennot c])
```

Determining the Application Condition

The condition governing the execution of a node call depends on the clock of its parameters. Thus, the following equation causes `cpt 60` to be executed only if `sec` is true:

```
s = cpt (60 [@when sec])
```

However, when a node has several parameters, the question arises of under which condition a call should be executed. Indeed, this condition is not limited to requiring all the call parameters to be present. For example, let us begin by defining a node `merger_swap` that "merges' two streams (`x` and `y`) on complementary clocks (`c` and *not* `c`), using the **`merge`** operator with its parameters in reverse order:

```
let%node merger_swap (x,y,c) ~return:z =
  z = merge c x y
```

And now consider the following equation:

```
m = merger_swap (x [@when c], y [@whennot c], c)
```

The parameters of the call to `merger_swap` can never all be present in the same instant, since `x` and `y` are clocked by complementary clocks (when `x` is present, `y` is absent, and vice versa). Requiring all parameters to be present in order to condition the call would be too limiting. In fact, a node is activated only if certain parameters are present: those that, in the context of the called node, are on the (fastest) base clock. Since a node's base clock is the fastest clock of the node, the presence of the parameters on this clock constitutes the guard for executing that node. For example, the single parameter `n` of the `cpt` node is on the base clock. It is therefore the presence of this parameter that conditions the call to `cpt`.

In the `merger_swap` node, it is the last parameter that is on the node's base clock. The stream `c` must therefore be present for the call `merger_swap (x,y,c)` to be performed.

This semantics, similar to that of Heptagon, is slightly more flexible than that of Lustre: in Lustre, the first parameter must be on the base clock, and the other parameters may be on slower clocks[6]. When a node is called, the presence or absence of the first parameter is then checked to condition its execution.

In the next section, we present a typing discipline that models the synchronous clock system. This synchronous clock system, whose consistency can be checked when an OCaLustre program is compiled, governs the *well-clockedness* of programs in this language.

[6]The `merger_swap` node would therefore, in Lustre, have to take *c* as its first parameter.

3.1.3 Clock Typing of Programs

In an OCaLustre program, each expression has a clock defined either explicitly (through the use of the `@when` and `@whennot` annotations) or implicitly (such as the base clock of the node, or a clock resulting from the composition of two subsampled nodes). As with the data types of OCaLustre expressions, the information concerning the clocks of each stream can be deduced at compile time. Consequently, OCaLustre's synchronous clocks can be represented by a type system that statically assigns a *clock type* to each expression in the language. This system is then associated with a set of rules representing the consistency of the *clocking* of an OCaLustre program. Thus, a *well-clocked* program is a program whose clock types respect the rules of this system, and saying that a program is well clocked amounts to saying that no access to absent values is possible during a synchronous instant, whatever the values of the streams manipulated by the program. This section presents the notion of synchronous clock typing through several examples that highlight its specificity, while the formal description of this type system is given in the next section.

A clock type is associated with a grammar, presented in Figure 3.6. This grammar makes it possible to represent the base clock of a node, or clocks corresponding to subsampling of values:

ck	::=		clock
	|	$\bullet$	base clock
	|	ck **on** x	positive subsampling
	|	ck **onnot** x	negative subsampling
	|	$ck \times ck'$	n-uplet
	|	$ck \rightarrow ck'$	function

FIGURE 3.6: Clock types in OCaLustre

- A stream that is not sampled by any clock within a node is considered to be on the node's base clock. Following the formalism adopted for Heptagon [Gér13], the base clock type is written with the following symbol: $\bullet$.
- A stream positively clocked by a stream x (itself of type ck) has clock type ck **on** x.
- A stream negatively clocked by a stream x' (itself of type ck') has clock type ck' **onnot** x'.
- An n-tuple of streams corresponds to an n-tuple of clocks.
- An instance of a node has a *functional* type of the form *input clocks* $\rightarrow$ *output clocks*.

The signature of a node is associated with a *type scheme* (written ω), that is, a type in which variables can be globally quantified (as defined by Damas and Milner in [DM82]), in the same way as in the work of Colaço and Pouzet [CP03]. As with the classic data typing described in the previous section, the clock type of a node is an arrow type whose left-hand element corresponds to the clock type of its input streams, and whose right-hand element corresponds to the clock type of its output streams. The signature is annotated with the names of the inputs $\vec{x}$ and outputs $\vec{y}$ of the corresponding node. The associated type scheme is quantified by the node's base clock:

$$\omega ::= \forall \bullet . (\vec{x} : ck) \rightarrow (\vec{y} : ck')$$

For example, the `sampler` node subsamples its first parameter with its second parameter:

```
let%node sampler (x,c) ~return:y =
  y = x [@when c]
```

Since in the body of this node there is no information indicating that the parameters `x` or `c` are subsampled by a clock slower than the base clock, these parameters are considered to be clocked by the node's base clock. The output value, by contrast, is explicitly conditioned by the presence of `c`.

The signature of `sampler` is therefore:

$$\forall \bullet . ((x : \bullet) \times (c : \bullet)) \rightarrow (y : \bullet \textbf{ on } c).$$

It should be noted that the name c in the clock type of the outputs does not mean that the parameter `c` is itself a type, but that the stream `c` is a Boolean value that subsamples the stream `y`. In what follows, we will call such variable names that appear in the type of a node "*supports*".

The **`merge`** operator combines streams whose clock types are complementary: $(ck \textbf{ on } x)$ and $(ck \textbf{ onnot } x)$. The resulting value is clocked *by the clock of* **`x`**, since the merge operator in some sense allows one to "move back up' one clock level. For example, consider the following example, which defines a node that applies the merge operator to its input parameters:

```
let%node merger (c,a,b) ~return:d =
  d = merge c a b
```

The stream `c` is on the node's base clock ($\bullet$). The streams `a` and `b` must be on two complementary clocks, clocked by `c` (the first positively and the second negatively). The `merger` node therefore has the following signature:

$$\forall \bullet . ((c : \bullet) \times (a : \bullet \textbf{ on } c) \times (b : \bullet \textbf{ onnot } c)) \rightarrow (d : \bullet)$$

In OCaLustre, if an output stream is locally subsampled by a clock defined in the body of a node, then its clock must also be returned by the node concerned. For example, the following node calls the `sampler` node and returns the stream `v`, subsampled with its clock `c`:

```
let%node sampler2 (u) ~return:(v,c) =
  v = sampler (u,c);
  c = (true ⋙ (false ⋙ c))
```

The signature of this node is then $\forall \bullet . (u : \bullet) \rightarrow ((v : \bullet \textbf{ on } c) \times (c : \bullet))$. This constraint ensures the consistency of the clocking of a program: indeed, using a stream clocked by a clock whose own clock type would be unknown (because it is defined locally to a node and therefore inaccessible from outside) leads to incomplete information about the presence conditions of this stream.

For example, suppose that the definition of the following node were allowed:

```
let%node sampler_wrong (u) ~return:(v) =
  v = sampler (u,c);
```

```
c = (true ⋙ (false ⋙ c))
```

The signature of `sampler_wrong` would therefore be $\forall \bullet . (u : \bullet) \to (v : \bullet \textbf{ on } c)$. But, since the value of c is known only locally to this node, the clocking information for v is incomplete from the point of view of any node that calls `sampler_wrong`: this is a scope-escape phenomenon for the local variable c. For example, in the following code fragment, the variable conditioning the presence of `a_sampled`, which is supposed to be passed as the first parameter of the **`merge`** operator in the definition of the stream `b`, is inaccessible because it is local to `sampler_wrong`:

```
let%node call_sampler_wrong (a) ~return:(b) =
  a_sampled = sampler_wrong (a);
  b = merge XX a_sampled 32 (* problem: XX is unknown *)
```

Node Calls and Substitution of the Base Clock

Clock typing for calls to nodes has a semantics close to the typing of function application in a programming language with a polymorphic type system: the base clock ($\bullet$) then plays the role of a *type variable*, and is therefore instantiated for each call.

In the classic type system of such a programming language (for example OCaml), if a function `f` has type scheme $\forall\alpha.\alpha \to \alpha$, then in the application `f 2` the type variable α is instantiated with the type `int`. The type of `f` in this context is therefore `int` $\to$ `int` (and the result of the application is then of type `int`).

Analogously, if we define the following node in OCaLustre:

```
let%node plus_un x ~return:y =
  y = (x + 1)
```

Then the signature of this node is $\forall \bullet . (x : \bullet) \to (y : \bullet)$, and the base clock ($\bullet$) can be substituted by a slower clock during its instantiation.

For example, consider the following equation:

```
y = plus_un (42 [@when c])
```

Since the expression `42 [@when c]` has clock type ($\bullet$ **on** c), after instantiation of the base clock, the clock type of `plus_un` in this context will be as follows:

$$(\bullet \to \bullet)\,[\bullet := \bullet \textbf{ on } c] = (\bullet \textbf{ on } c) \to (\bullet \textbf{ on } c)$$

The stream `y` will then have the clock type of the call result: ($\bullet$ **on** c).

Node Calls and Substitution of Variable Names

A particular feature of the synchronous-clock type system is that some clock types contain variable names (for example the variable c in the type ($\bullet$ *on* c)). These names, which we will call *supports*, correspond to the *formal* names of the streams used as clocks in the equations.

Of course, the supports present in node signatures may differ from the *actual* names of the arguments of a call. For example, consider the signature of the `sampler` node:

$$\forall \bullet . ((x : \bullet) \times (c : \bullet)) \rightarrow (y : \bullet \textbf{ on } c)$$

The output stream of the `sampler` node has clock type ($\bullet$ **on** c), with c as the support of its clock (corresponding to the second parameter of `sampler`); but in the following example, the name of the second argument of the call to `sampler` is not `c`, but `d`:

```
let%node call_sampler (d) ~return:w =
  w = sampler(8,d)
```

It would then be wrong to assign `w` the clock type ($\bullet$ **on** c), since the stream `c` is undefined in the body of this node. This difference between the *formal* parameter names of a node and the *actual* names of its arguments makes it necessary to substitute supports with the real names of the corresponding actual variables.

Thus, in the previous example, the type of this instance of `sampler` becomes:

$$((\bullet \times \bullet) \rightarrow (\bullet \textbf{ on } c))[c := d] = (\bullet \times \bullet) \rightarrow (\bullet \textbf{ on } d)$$

and the clock type of `w` is then ($\bullet$ **on** d). Consequently, the signature of the `call_sampler` node is:

$$\forall \bullet \ . (d : \bullet) \rightarrow (w : \bullet \textbf{ on } d)$$

Finally, consider the following example, which combines substitution of the base clock with substitution of supports:

```
let%node call_sampler_slower (c,e) ~return:w =
  d = c [@when e];
  w = sampler(8 [@when e],d)
```

The parameters of the call to `sampler` are each subsampled by `e`, so the base clock of this call has type ($\bullet$ **on** e). Moreover, the name of the second argument of this call is `d`; consequently, the type assigned to `sampler` in the equation defining `w` is:

$$((\bullet \times \bullet) \rightarrow (\bullet \textbf{ on } c))\,[c := d]\,[\bullet := \bullet \textbf{ on } e] = ((\bullet \textbf{ on } e) \times (\bullet \textbf{ on } e)) \rightarrow ((\bullet \textbf{ on } e) \textbf{ on } d)$$

The `call_sampler_slower` node then has the following signature:

$$\forall \bullet . (d : \bullet) \times (e : \bullet) \rightarrow (w : (\bullet \textbf{ on } e) \textbf{ on } d)$$

3.2 Specification of the OCaLustre Language Formalised with Ott and Coq

This section describes a formal specification for the OCaLustre language. In particular, we detail rules relating to the semantics and typing of an OCaLustre program. These different rules are expressed using

an intermediate representation of an OCaLustre program, called *normal form*. Moreover, the structure of an OCaLustre program is organised as a header containing OCaml definitions (of types or functions), followed by a list of node definitions. The OCaml header (which is unrestricted) is not formalised in this manuscript, and normal form concerns only synchronous nodes and the equations they contain.

The grammar adopted, as well as each of the inference rules described in this section, were formalised with the Ott tool [SNO+10]. This tool is intended for defining grammars and evaluation rules for programming languages, and can extract from this definition files readable by several languages and proof assistants. Thus, the formal specification of the type systems and semantics described in this section relies on the Coq proof assistant [Tea19]. The display of the various inductive rules in this section is itself produced by extracting from Ott to LaTeX.

3.2.1 Representation of a Synchronous Program in Normal Form

The various operations and analyses that we will describe formally in the rest of this manuscript rely on a structured representation of programs. This representation, the *normal form*, results from a procedure in which the OCaLustre compiler applies several static transformations intended to make the structure of programs uniform, thereby simplifying later analyses and transformations. In particular, conversion to normal form makes it possible to extract the subexpressions that induce the use of registers when compiling a program. We will describe the process of putting an OCaLustre program into normal form (or *normalisation*) precisely when we present the various steps of compiling an OCaLustre program, in Chapter 4.

Grammar

For consistency with work related to ours, the grammar associated with the normal-form representation of OCaLustre programs is inspired by the formalism introduced in [BDPR17] and by the definition of the syntax of the CoreDF language[7]. An OCaLustre program is therefore first converted from an abstract syntax tree (AST), obtained by parsing the source program, into a *normalised* AST whose components come from the following grammar:

- A synchronous OCaLustre program corresponds to a list of node definitions:

$program,\ \vec{nodes}$	::=		program
	|	$\varnothing$	empty program
	|	$node;;\vec{nodes}$	list of nodes

[7] Unlike CoreDF, our representation supports the definition of nodes with several output values.

- A node is defined by its name f, a list of variable names $\vec{x}$[8] representing the input stream or streams, a list of names $\vec{y}$ corresponding to the output streams, and a (non-empty) list of equations $\vec{eqn}$ forming its body:

node	::=		node definition
	|	**node** $f(\vec{x})$ **return** $(\vec{y}) = \vec{eqn}$	
$\vec{x}$, $\vec{y}$	::=		noms de variables
	|	()	liste vide
	|	y	unique variable
	|	$y, \vec{y}$	multiples variables
$\vec{eqn}$	::=		list of equations
	|	[*eqn*]	unique
	|	$eqn;\ \vec{eqn}$	multiples

- An equation, of the form *name(s) = expression*, corresponds to the definition of one or more stream variables. These streams may correspond to a control expression *ce*, to the use of the $\ggg$ operator (for consistency with other synchronous languages, this operator is represented as **fby** in normal form), to the application of a node f with the (non-empty) list of expressions $\vec{e}$ as parameter, or to the use of the `call` operator to apply an OCaml function:

eqn	::=		equation
	|	$y = ce$	expression
	|	$y = k$ **fby** e	fby
	|	$\vec{y} = f(\vec{e})$	application
	|	$y =$ **call** f $e_0\, e_1\, ..\, e_{n-1}$	OCaml function application
$\vec{e}$	::=		liste d'expressions
	|	[e]	unique
	|	$e, \vec{e}$	multiples

- Control expressions correspond to the use of the conditional operator **if** − **then** − **else**, the merge operator **merge**, or "simple' expressions *e*:

ce	::=		control expression
	|	e	expression
	|	**merge** $x\ ce\ ce'$	fusion
	|	**if** e **then** ce **else** ce'	alternative

[8] Throughout this manuscript, arrows placed above a syntactic category represent a list of elements of that category.

- Finally, "simple' expressions correspond to the *unit* value (represented by the standard notation "()"), constants (k), variables (x), enumerated-type constructors (X_i), application of the positive or negative subsampling operators (**when** and **whennot**), application of a unary prefix arithmetic operator ($\square$), and application of a binary infix arithmetic or logical operator ($\diamond$):

e	::=		expression
	|	()	unit
	|	k	constante
	|	x	variable
	|	X_i	constructeur de type
	|	$e \diamond e'$	binary operation
	|	$\square\, e$	unary operation
	|	e **when** x	positive sampling
	|	e **whennot** x	negative sampling
k	::=		constante
	|	*int_literal*	entier
	|	*float_literal*	floating point
	|	*bool_literal*	Boolean
$\diamond$	::=		binary operator
	|	$\diamond_{int}$	integer operator
	|	$\diamond_{float}$	floating-point operator
	|	$\diamond_{comp}$	comparison operator
	|	$\diamond_{bool}$	Boolean operator
$\square$	::=		unary operator
	|	$-$	integer opposite
	|	$-.$	floating-point opposite
	|	**not**	Boolean negation

3.2.2 Typing and Clocking

A programming language's type system is a set of rules that assign a *type* to the various constructs of a program in that language. The types most commonly considered in programming are those that represent the nature of the data being manipulated: integers, Booleans, floating-point numbers, lists of integers, functions, object instances, and so on.

However, a type system can also define rules relating to aspects of a program other than the nature of the values computed: for example, it is possible to construct a type system dedicated not to representing the nature of the values "carried' by the different variables and expressions, but to representing properties emphasised in certain kinds of applications, such as the security level of the different constructs of a program [Wal00].

The set of typing rules governing the consistency of a type system makes it possible to distinguish well-typed programs (which respect these rules) from programs whose typing is incorrect (for example, programs in which forbidden implicit type conversions are performed).

In this section, we present the typing of an OCaLustre program through two type systems that provide very different kinds of information. The first type system we describe is a classic system, which assigns data types to values or expressions in the language, representing the nature of the information being transmitted: streams of integers, streams of Booleans, streams of floating-point numbers, and so on. This type system relies on the system of the host language and therefore shares all its characteristics: in OCaLustre, as in OCaml, data is strongly typed and its types are statically inferred when programs are compiled.

The second type system we discuss in this section is the synchronous-clock type system. Unlike the previous type system, this system does not represent the nature of the values manipulated by the various components of a program, but the presence (or absence) of values. Synchronous clock typing is, like OCaLustre data typing, a strong static typing system, with stream clocks inferred during program compilation.

The advantage of implementing such systems, with this constraint of strong static typing, is that a very large number of potential bugs arising from inconsistencies introduced by the programmer when writing certain expressions of the language can be detected at compile time, before the program has even been transferred to the microcontroller. These type systems, each governing two distinct aspects of the consistency of OCaLustre programs, therefore contribute to the safety of OCaLustre programs.

3.2.3 Program Typing Rules

Like its target language OCaml, the OCaLustre language extension is a statically typed programming language that implements a type-inference mechanism at compile time. Each variable of an OCaLustre program has a type whose nature can be determined during compilation. It should be noted, however, that the OCaLustre type system is monomorphic, unlike OCaml's polymorphic system.

Saying that a stream $x \equiv (x_1, x_2, ..., x_i, ...)$ has type "*stream of t*" (written simply "*t*') means that all elements of the sequence of values to which it corresponds have type t. As with homogeneous lists in OCaml, a stream cannot "carry' values whose type changes over time. Consequently, creating a stream whose value may change type from one instant to another, such as $(0, 2, 4, \mathit{true}, 10, 4.5, 14, \dots)$, is impossible in OCaLustre.

The types manipulated in OCaLustre may be base types (int, bool, float), "arrow' types (the types of nodes), enumerated types (represented by the set of their constructors X_i), or n-tuples of types (corresponding to the type of a list of expressions):

$$t ::= \mathit{unit} \mid \mathit{int} \mid \mathit{bool} \mid \mathit{float} \mid t \rightarrow t' \mid \{X_1, \dots, X_n\} \mid t_1 \times ... \times t_n$$

Starting from the normal-form representation of OCaLustre programs, we then formally define the conditions that govern whether a program is well typed.

Let Γ be a local typing environment, that is, a function associating each variable of an OCaLustre node with its type: for example, if the stream x has type *int*, then $\Gamma(x) = \mathit{int}$. We write $\Gamma \vdash e : t$ for the predicate (called a *typing judgement*) indicating that, in the typing environment Γ, the expression e has type t.

From such judgements, we define inference rules representing the consistency of typing for OCaLustre expressions. Such rules, of the form

$$\frac{P_1 \qquad \dots \qquad P_n}{Q}$$

establish that, from the *premise* conditions P_1 to P_n, one can deduce the *judgement* Q.

Typing Expressions: Figure 3.7 represents the typing rules for OCaLustre expressions. For example, the typing rule for applying an integer arithmetic operator $\diamond_{int}$ (for example the + operator) states that this operation can be performed only between two integers, and that the result is itself an integer:

$$\frac{\Gamma \vdash e_1 : int \qquad \Gamma \vdash e_2 : int}{\Gamma \vdash e_1 \diamond_{int} e_2 : int}$$

It should be noted that, for the typing rule of comparison operators, we assume here that no comparison of values of functional type is performed in an OCaLustre program. This is left to the programmer, who must ensure that no such operation is performed[9]. Since we rely on executing the OCaml code generated by compiling OCaLustre, the comparison function is polymorphic, and using it on values of functional type raises an exception. Any exception in an OCaLustre program stops its execution.

Moreover, a judgement of the form $t = \{X_1, \dots X_n\}$ means that the type t has previously been defined (in the OCaml header) as an enumerated type composed of constructors X_1 to X_n.

Finally, we distinguish typing rules for a "simple' expression from those for a conditional expression by annotating, for rules concerning conditional expressions, the "*turnstile*" symbol ($\vdash$) as follows: $\vdash_{ce}$.

The rules relating to the typing of OCaLustre equations and nodes are given in Figure 3.8. In what follows, we review them in order to describe them precisely.

Typing Equations: Let $\mathcal{G}$ be a *global* typing environment associating each OCaml function and each OCaLustre node with its type (an "arrow' type of the form *input type* $\rightarrow$ *output type*). An equation in the body of an OCaLustre node corresponding to the application of another node $\vec{y} = f(\vec{e})$ is well typed if and only if f has a signature of type $t_1 \rightarrow t_2$, the application parameter $\vec{e}$ has type t_1, and the result $\vec{y}$ has type t_2:

$$\frac{\Gamma \vdash \vec{y} : t_2 \qquad \mathcal{G}(f) = t_1 \rightarrow t_2 \qquad \Gamma \vdash \vec{e} : t_1}{\mathcal{G}, \Gamma \vdash \vec{y} = f(\vec{e})}$$

[9] Just as partial application of functions via `call`, or division by zero, is forbidden.

$\boxed{\Gamma \vdash e : t}$

$$\frac{}{\Gamma \vdash () : unit} \qquad \frac{}{\Gamma \vdash int\_literal : int} \qquad \frac{}{\Gamma \vdash bool\_literal : bool} \qquad \frac{}{\Gamma \vdash float\_literal : float}$$

$$\frac{\Gamma(x) = t}{\Gamma \vdash x : t} \qquad \frac{t = \{X_1, ..., X_n\}}{\Gamma \vdash X_i : t}$$

$$\frac{\Gamma \vdash e_1 : int \quad \Gamma \vdash e_2 : int}{\Gamma \vdash e_1 \diamond_{int} e_2 : int} \qquad \frac{\Gamma \vdash e_1 : bool \quad \Gamma \vdash e_2 : bool}{\Gamma \vdash e_1 \diamond_{bool} e_2 : bool}$$

$$\frac{\Gamma \vdash e_1 : float \quad \Gamma \vdash e_2 : float}{\Gamma \vdash e_1 \diamond_{float} e_2 : float} \qquad \frac{\Gamma \vdash e_1 : t \quad \Gamma \vdash e_2 : t}{\Gamma \vdash e_1 \diamond_{comp} e_2 : bool}$$

$$\frac{\Gamma \vdash e : int}{\Gamma \vdash -e : int} \qquad \frac{\Gamma \vdash e : float}{\Gamma \vdash -. e : float} \qquad \frac{\Gamma \vdash e : bool}{\Gamma \vdash \textbf{not}\, e : bool}$$

$$\frac{\Gamma \vdash e : t \quad \Gamma(x) = bool}{\Gamma \vdash e\, \textbf{when}\, x : t} \qquad \frac{\Gamma \vdash e : t \quad \Gamma(x) = bool}{\Gamma \vdash e\, \textbf{whennot}\, x : t}$$

$\boxed{\Gamma \vdash \vec{e} : t}$

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash [e] : t} \qquad \frac{\Gamma \vdash e : t \quad \Gamma \vdash \vec{e} : t'}{\Gamma \vdash e, \vec{e} : t \times t'}$$

$\boxed{\Gamma \vdash_{ce} ce : t}$

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash_{ce} e : t} \qquad \frac{\Gamma \vdash e_1 : bool \quad \Gamma \vdash_{ce} ce_2 : t \quad \Gamma \vdash_{ce} ce_3 : t}{\Gamma \vdash_{ce} \textbf{if}\, e_1\, \textbf{then}\, ce_2\, \textbf{else}\, ce_3 : t}$$

$$\frac{\Gamma(x) = bool \quad \Gamma \vdash_{ce} ce_1 : t \quad \Gamma \vdash_{ce} ce_2 : t}{\Gamma \vdash_{ce} \textbf{merge}\, x\, ce_1\, ce_2 : t}$$

FIGURE 3.7: Typing rules for OCaLustre expressions

$\boxed{\Gamma \vdash \vec{y} : t}$

$$\frac{}{\Gamma \vdash () : unit} \qquad \frac{\Gamma(y) = t}{\Gamma \vdash y : t} \qquad \frac{\Gamma \vdash y : t \quad \Gamma \vdash \vec{y} : t'}{\Gamma \vdash y, \vec{y} : t \times t'}$$

$\boxed{\mathcal{G}, \Gamma \vdash eqn}$

$$\frac{\Gamma \vdash y : t \quad \Gamma \vdash_{ce} ce : t}{\mathcal{G}, \Gamma \vdash y = ce} \qquad \frac{\Gamma \vdash y : t \quad \Gamma \vdash k : t \quad \Gamma \vdash e : t}{\mathcal{G}, \Gamma \vdash y = k \,\mathbf{fby}\, e}$$

$$\frac{\Gamma \vdash \vec{y} : t_2 \quad \mathcal{G}(f) = t_1 \to t_2 \quad \Gamma \vdash \vec{e} : t_1}{\mathcal{G}, \Gamma \vdash \vec{y} = f(\vec{e})}$$

$$\frac{\Gamma \vdash y : t_n \quad \mathcal{G}(\mathtt{f}) = t_0 \to t_1 \to \ldots \to t_{n-1} \to t_n \quad \Gamma \vdash e_0 : t_0 \quad \Gamma \vdash e_1 : t_1 \quad \ldots \quad \Gamma \vdash e_{n-1} : t_{n-1}}{\mathcal{G}, \Gamma \vdash y = \mathbf{call}\, \mathtt{f}\, e_0\, e_1 \ldots e_{n-1}}$$

$\boxed{\mathcal{G}, \Gamma \vdash e\vec{q}n}$

$$\frac{\mathcal{G}, \Gamma \vdash eqn}{\mathcal{G}, \Gamma \vdash [eqn]} \qquad \frac{\mathcal{G}, \Gamma \vdash eqn \quad \mathcal{G}, \Gamma \vdash e\vec{q}n}{\mathcal{G}, \Gamma \vdash eqn;\ e\vec{q}n}$$

$\boxed{\mathcal{G} \vdash node : t}$

$$\frac{\mathcal{G}, \Gamma \vdash e\vec{q}n \quad \Gamma \vdash \vec{x} : t_1 \quad \Gamma \vdash \vec{y} : t_2}{\mathcal{G} \vdash \mathbf{node}\, f(\vec{x})\, \mathbf{return}\, (\vec{y}) = e\vec{q}n : t_1 \to t_2}$$

$\boxed{\mathcal{G} \vdash program}$

$$\frac{}{\mathcal{G} \vdash \varnothing} \qquad \frac{\mathcal{G} \vdash \mathbf{node}\, f(\vec{x})\, \mathbf{return}\, (\vec{y}) = e\vec{q}n : t_1 \to t_2 \quad (f : t_1 \to t_2) \uplus \mathcal{G} \vdash \vec{nodes}}{\mathcal{G} \vdash \mathbf{node}\, f(\vec{x})\, \mathbf{return}\, (\vec{y}) = e\vec{q}n;; \vec{nodes}}$$

FIGURE 3.8: Typing rules for equations and nodes

An equation corresponding to application of the "followed-by' operator is well typed provided that the constant to the left of the $\ggg$ operator and the expression to its right have the same type, and that the variable to the left of the equality operator also has that same type:

$$\frac{\Gamma \vdash y : t \quad \Gamma \vdash k : t \quad \Gamma \vdash e : t}{\mathcal{G}, \Gamma \vdash y = k \,\mathbf{fby}\, e}$$

Application of an OCaml function has type t_n if and only if the OCaml function has type $t_0 \to \cdots \to t_{n-1} \to t_n$ and its successive parameters have types t_0 to t_{n-1}:

$$\frac{\Gamma \vdash y : t_n \quad \mathcal{G}(\mathtt{f}) = t_0 \to t_1 \to \ldots \to t_{n-1} \to t_n \quad \Gamma \vdash e_0 : t_0 \quad \Gamma \vdash e_1 : t_1 \quad \ldots \quad \Gamma \vdash e_{n-1} : t_{n-1}}{\mathcal{G}, \Gamma \vdash y = \mathbf{call}\, \mathtt{f}\, e_0\, e_1 \ldots e_{n-1}}$$

It should be noted, however, that this rule induces a slight semantic shift: we move from *streams* to OCaml functions, which are supposed to be able to manipulate only "standard' values. For example, a

value of type "stream of `int`" in OCaLustre is regarded by the called OCaml function as a value of type `int`. This distinction is nevertheless not problematic, since at each program instant a stream has only a single value (a "stream of `int`" is therefore reduced to a simple `int`)[10]. Formally, the **`call`** operator can be viewed as a *map* function that "*lifts*" an OCaml function so that it can be applied to streams:

$$\texttt{call}\ f\ e_0\ e_1\ \ldots\ e_{n-1} \equiv (map\ f)\ e_0\ e_1\ \ldots\ e_{n-1}$$

Any other equation $y = ce$ is well typed if the associated expression ce is well typed and the variable y has the same type in Γ:

$$\frac{\Gamma \vdash y : t \qquad \Gamma \vdash_{ce} ce : t}{\mathcal{G}, \Gamma \vdash y = ce}$$

A list of equations (the body of a node) is well typed as soon as all the equations it contains are well typed:

$$\frac{\mathcal{G}, \Gamma \vdash eqn}{\mathcal{G}, \Gamma \vdash [eqn]} \qquad \frac{\mathcal{G}, \Gamma \vdash eqn \qquad \mathcal{G}, \Gamma \vdash \vec{eqn}}{\mathcal{G}, \Gamma \vdash eqn;\ \vec{eqn}}$$

Typing a Node and a Synchronous Program: The type of an OCaLustre node is similar to the type of a function in OCaml. It is an arrow type $t_1 \rightarrow t_2$, where t_1 is the type of the streams that can be passed to it as input, and t_2 is the type of the values it computes as output:

$$\frac{\mathcal{G}, \Gamma \vdash \vec{eqn} \qquad \Gamma \vdash \vec{x} : t_1 \qquad \Gamma \vdash \vec{y} : t_2}{\mathcal{G} \vdash \textbf{node}\, f(\vec{x})\, \textbf{return}\, (\vec{y}) = \vec{eqn} : t_1 \rightarrow t_2}$$

Finally, a synchronous program (i.e. a list of nodes) is well typed provided that each node it contains is well typed:

$$\frac{}{\mathcal{G} \vdash \varnothing} \qquad \frac{\mathcal{G} \vdash \textbf{node}\, f(\vec{x})\, \textbf{return}\, (\vec{y}) = \vec{eqn} : t_1 \rightarrow t_2 \qquad (f : t_1 \rightarrow t_2) \uplus \mathcal{G} \vdash \vec{nodes}}{\mathcal{G} \vdash \textbf{node}\, f(\vec{x})\, \textbf{return}\, (\vec{y}) = \vec{eqn};; \vec{nodes}}$$

The typing judgement for a synchronous program is initially invoked with the typing environment corresponding to the OCaml functions previously defined in the program header. The typing environments of OCaml functions and nodes are therefore shared, and we will assume that the namespaces of functions and nodes are distinct, so that there are no name conflicts (which would cause shadowing phenomena).

The typing rules of the OCaLustre extension are close enough to OCaml's typing rules that a well-typed OCaLustre program remains well typed after translation to OCaml, and that generating an ill-typed OCaml program indicates that the original OCaLustre program was ill typed. Therefore, thanks to the *correctness of typing with respect to translation to OCaml* (whose proof we will present in Section 5.1), the rules stated in this section do not need to be explicitly checked by the OCaLustre compiler in order to

[10] This phenomenon also occurs when an OCaLustre program is "connected' to functions that compute the inputs and outputs of a synchronous instant.

maintain program safety. Indeed, we take advantage of the target language, whose compiler performs static type checking and inference. In our situation, this analysis is therefore carried out after translating OCaLustre code into classic OCaml code. The OCaml compiler is then able to point out typing errors in programs, including when they originate in OCaLustre code.

Moreover, since OCaLustre is implemented with tools that belong to the OCaml compilation ecosystem, it is fully compatible with other powerful static analysis tools such as Merlin [BRS18], which provides many features that simplify OCaml program development (such as context-sensitive autocompletion, or "live' access to typing information) by integrating with various text editors (Vim, Emacs, Atom, ...). Such compatibility gives the OCaLustre user immediate and precise feedback about the location of errors relating, among other things, to the typing and clocking consistency of their programs, directly within their favourite text editor.

3.2.4 Program Well-Clockedness Rules

The consistency of the synchronous-clock type system governs the *well-clockedness* of an OCaLustre program. This well-clockedness is defined by a set of typing rules whose satisfaction induces the clocking safety of a program.

As with the formalisation of OCaLustre program typing, we define a *clocking judgement* as a predicate of the form $\mathcal{H}, \mathcal{C} \vdash eqn$, meaning that in a global clocking environment $\mathcal{H}$ (which associates each node name with its clock) and a local clocking environment $\mathcal{C}$ (which associates a clock with each variable name of a node), an equation eqn is well clocked. Similarly, a judgement $\mathcal{C} \vdash e : ck$ states that, in the typing environment $\mathcal{C}$, the expression e has clock type ck.

From such typing judgements, we then describe inference rules that formally establish the conditions required for programs, nodes, equations, and expressions to be well clocked. These rules are derived from a type system that considers clocks as abstract data types [CP03], implemented in the *Lucid Synchrone* language. Our solution is less expressive because it does not allow *several* clock-type variables within a node, but only a single one (the base clock $\bullet$). It nevertheless greatly simplifies the separate-compilation model for nodes, without significantly limiting the expressive power of the language. The OCaLustre type system is thus closer to the type system of the synchronous language *Heptagon*, midway between the clock system of Lustre and that of Lucid Synchrone.

The clocking rules for OCaLustre expressions are presented in Figure 3.9, and those relating to equations and nodes are given in Figure 3.10. In what follows, we describe the main rules that govern whether an OCaLustre program is well clocked.

In OCaLustre, a constant is always well clocked; it is considered well typed whatever the clock type:

$$\frac{}{\mathcal{C} \vdash k : ck}$$

The clock type of a variable comes from the typing environment:

$$\frac{\mathcal{C}(x) = ck}{\mathcal{C} \vdash x : ck}$$

$\boxed{\mathcal{C} \vdash e : ck}$

$$\frac{}{\mathcal{C} \vdash () : ck} \quad \frac{}{\mathcal{C} \vdash k : ck} \quad \frac{}{\mathcal{C} \vdash X_i : ck} \quad \frac{\mathcal{C}(x) = ck}{\mathcal{C} \vdash x : ck}$$

$$\frac{\mathcal{C} \vdash e : ck}{\mathcal{C} \vdash \square e : ck} \quad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash e' : ck}{\mathcal{C} \vdash e \diamond e' : ck}$$

$$\frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash x : ck}{\mathcal{C} \vdash e \textbf{ when } x : ck \textbf{ on } x} \quad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash x : ck}{\mathcal{C} \vdash e \textbf{ whennot } x : ck \textbf{ onnot } x}$$

$\boxed{\mathcal{C} \vdash \vec{e} : ck}$

$$\frac{\mathcal{C} \vdash e : ck}{\mathcal{C} \vdash [e] : ck} \quad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash \vec{e} : ck'}{\mathcal{C} \vdash e, \vec{e} : ck \times ck'}$$

$\boxed{\mathcal{C} \vdash_{ce} ce : ck}$

$$\frac{\mathcal{C} \vdash e : ck}{\mathcal{C} \vdash_{ce} e : ck}$$

$$\frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash_{ce} ce : ck \quad \mathcal{C} \vdash_{ce} ce' : ck}{\mathcal{C} \vdash_{ce} \textbf{if } e \textbf{ then } ce \textbf{ else } ce' : ck} \quad \frac{\mathcal{C} \vdash x : ck \quad \mathcal{C} \vdash_{ce} ce : ck \textbf{ on } x \quad \mathcal{C} \vdash_{ce} ce' : ck \textbf{ onnot } x}{\mathcal{C} \vdash_{ce} \textbf{merge } x\, ce\, ce' : ck}$$

FIGURE 3.9: Clocking rules for expressions

Using any operator requires its operands to have the same clock type. For example, the following rule, which defines well-clockedness for any arithmetic or logical operator ($\diamond$), states that in the clocking environment $\mathcal{C}$, if each operand of the operator $\diamond$ is clocked by the clock ck, then its result is itself clocked by ck:

$$\frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash e' : ck}{\mathcal{C} \vdash e \diamond e' : ck}$$

Using the **merge** operator requires its second and third parameters to be clocked positively and negatively by its first parameter. The result is on the clock of the first parameter:

$$\frac{\mathcal{C} \vdash x : ck \quad \mathcal{C} \vdash_{ce} ce : ck \textbf{ on } x \quad \mathcal{C} \vdash_{ce} ce' : ck \textbf{ onnot } x}{\mathcal{C} \vdash_{ce} \textbf{merge } x\, ce\, ce' : ck}$$

The application of a node $\vec{y} = f(\vec{e})$ is well clocked provided that the clock type $ck_1 \to ck_2$ is an *instance* of the signature ω of f (i.e. a clock type in which the substitutions needed for conditional application and support-name consistency have been performed), that the parameters $\vec{e}$ of the application have clock type ck_1, and that the result has clock type $\vec{ck_2}$:

$$\frac{\mathcal{H}(f) = \omega \quad ck'_1 \to ck'_2 = inst(\omega) \quad \mathcal{C} \vdash \vec{e} : ck'_1 \quad \mathcal{C} \vdash \vec{y} : ck'_2}{\mathcal{H}, \mathcal{C} \vdash \vec{y} = f(\vec{e})}$$

$\boxed{\mathcal{C} \vdash \vec{y} : ck}$

$$\frac{}{\mathcal{C} \vdash () : \bullet} \qquad \frac{\mathcal{C}(y) = ck}{\mathcal{C} \vdash y : ck} \qquad \frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash \vec{y} : ck'}{\mathcal{C} \vdash y, \vec{y} : ck \times ck'}$$

$\boxed{\mathcal{H}, \mathcal{C} \vdash eqn}$

$$\frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash_{ce} ce : ck}{\mathcal{H}, \mathcal{C} \vdash y = ce} \qquad \frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash e_0 : ck \quad \mathcal{C} \vdash e_1 : ck \quad \ldots \quad \mathcal{C} \vdash e_{n-1} : ck}{\mathcal{H}, \mathcal{C} \vdash y = \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1}}$$

$$\frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash e : ck}{\mathcal{H}, \mathcal{C} \vdash y = k\ \mathbf{fby}\ e} \qquad \frac{\mathcal{H}(f) = \omega \quad ck'_1 \to ck'_2 = inst(\omega) \quad \mathcal{C} \vdash \vec{e} : ck'_1 \quad \mathcal{C} \vdash \vec{y} : ck'_2}{\mathcal{H}, \mathcal{C} \vdash \vec{y} = f(\vec{e})}$$

$\boxed{\mathcal{H}, \mathcal{C} \vdash e\vec{q}n}$

$$\frac{\mathcal{H}, \mathcal{C} \vdash eqn}{\mathcal{H}, \mathcal{C} \vdash [eqn]} \qquad \frac{\mathcal{H}, \mathcal{C} \vdash eqn \quad \mathcal{H}, \mathcal{C} \vdash e\vec{q}n}{\mathcal{H}, \mathcal{C} \vdash eqn;\ e\vec{q}n}$$

$\boxed{\mathcal{H} \vdash node : \omega}$

$$\frac{\mathcal{H}, \mathcal{C} \vdash e\vec{q}n \quad \mathcal{C} \vdash \vec{x} : ck \quad \mathcal{C} \vdash \vec{y} : ck'}{\mathcal{H} \vdash \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n : \forall \bullet\, .\, (\vec{x} : ck) \to (\vec{y'} : ck')}$$

$\boxed{\mathcal{H} \vdash program}$

$$\frac{}{\varnothing \vdash \varnothing} \qquad \frac{\mathcal{H} \vdash \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n : \omega \quad (f : \omega) \uplus \mathcal{H} \vdash \vec{nodes}}{\mathcal{H} \vdash \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n;; \vec{nodes}}$$

FIGURE 3.10: Clocking rules for equations and nodes

Application of an OCaml function is well clocked if *all* its parameters have the same clock type. The clock of the result is also identical to the clock of the parameters:

$$\frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash e_0 : ck \quad \mathcal{C} \vdash e_1 : ck \quad \ldots \quad \mathcal{C} \vdash e_{n-1} : ck}{\mathcal{H}, \mathcal{C} \vdash y = \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1}}$$

A node is well clocked provided that the equations it contains, as well as its input and output parameters, are all well clocked. Its signature then corresponds to a type scheme quantified by the node's base clock. It is an "arrow' type from the node's input parameters to its output parameters, all annotated with their clock types:

$$\frac{\mathcal{H}, \mathcal{C} \vdash e\vec{q}n \quad \mathcal{C} \vdash \vec{x} : ck \quad \mathcal{C} \vdash \vec{y} : ck'}{\mathcal{H} \vdash \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n : \forall \bullet\, .\, (\vec{x} : ck) \to (\vec{y'} : ck')}$$

Finally, a program is well clocked if all its nodes are well clocked:

$$\frac{}{\varnothing \vdash \varnothing} \qquad \frac{\mathcal{H} \vdash \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n : \omega \quad (f : \omega) \uplus \mathcal{H} \vdash \vec{nodes}}{\mathcal{H} \vdash \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n;; \vec{nodes}}$$

A program that respects these various typing rules is guaranteed not to access absent stream values ($\perp$) during execution, and therefore not to exhibit unpredictable and erroneous behaviour. In Section 5.2, we detail the process that automatically checks these various rules when an OCaLustre program is compiled.

3.2.5 Operational Semantics

The operational semantics of the OCaLustre language is similar to the semantics of Lustre defined in [CPHP87]. However, that reference semantics considers node *inlining*, whereas we will see in Chapter 4 that the OCaLustre compiler follows a separate-compilation model. Although this compilation model does not accept certain synchronous programs that are valid in Lustre (we will discuss this limitation in 4.2.2), the programs it does accept do indeed respect the semantics described in this section. This semantics is presented succinctly, in the form of inference rules applied to a simplified representation of the normal form of OCaLustre programs. In this simplified representation, a program is a list of all the equations computed during one execution instant. This amounts to saying that all calls to nodes are replaced by the bodies of those nodes, and that the integration (or *inlining*) of node calls is therefore performed, as in the Lustre compilation model. Moreover, program equations are represented annotated with their respective clocks, placed below the equality symbol. In addition, each constant is also annotated with its clock, placed as a superscript.

Let σ be the *memory* associated with an OCaLustre program, that is, a function associating each variable name with a value at each execution instant. A memory σ is said to be *compatible* with a program's system of equations if, for every equation $x = e$ in the system, the value of the right-hand side of the equation is identical to $\sigma(x)$ when the clock of x is true (otherwise, $\sigma(x) = \perp$ regardless of the value of e). In other words, a memory is compatible with the program's system of equations if it is a solution of that system.

We then define the following judgements:

- $\sigma \vdash exp : v$ means that, during a synchronous instant, the expression exp reduces to the value v when evaluated in the context of the memory σ.
- $eqn \xrightarrow{\sigma} eqn'$ means that the equation eqn is compatible with the memory σ, and that it will be replaced by the expression eqn' for execution at the next instant.
- $\vec{eqn} \xrightarrow{\sigma} \vec{eqn}'$ means that the list of equations $\vec{eqn}$ is compatible with the memory σ, and that it will be replaced by the list of equations $\vec{eqn}'$ for execution at the next instant.
- Finally, $h \vdash \vec{eqn} : h'$ means that, from the history of program inputs h, the program's system of equations $\vec{eqn}$ produces the history of outputs h'.

All the rules of the language's operational semantics are described in Figure 3.11.

Here we describe the main rules of the operational semantics:

- The value of a variable is extracted from the memory σ:

$$\frac{\sigma(x) = v}{\sigma \vdash x : v}$$

$\boxed{\sigma \vdash e : v}$

$$\frac{\sigma(ck) = true}{\sigma \vdash ()^{ck} : ()} \qquad \frac{\sigma(ck) \neq true}{\sigma \vdash ()^{ck} : \bot} \qquad \frac{\sigma(ck) = true}{\sigma \vdash k^{ck} : k} \qquad \frac{\sigma(ck) \neq true}{\sigma \vdash k^{ck} : \bot}$$

$$\frac{\sigma(ck) = true}{\sigma \vdash X_i^{ck} : X_i} \qquad \frac{\sigma(ck) \neq true}{\sigma \vdash X_i^{ck} : \bot} \qquad \frac{\sigma(x) = v}{\sigma \vdash x : v} \qquad \frac{\sigma \vdash e : v}{\sigma \vdash \square\, e : \square\, v}$$

$$\frac{\sigma \vdash e_1 : v_1 \quad \sigma \vdash e_2 : v_2}{\sigma \vdash e_1 \diamond e_2 : v_1 \diamond v_2} \qquad \frac{\sigma(x) = true \quad \sigma \vdash e : v}{\sigma \vdash e\ \mathbf{when}\ x : v} \qquad \frac{\sigma(x) \neq true}{\sigma \vdash e\ \mathbf{when}\ x : \bot}$$

$$\frac{\sigma(x) = false \quad \sigma \vdash e : v}{\sigma \vdash e\ \mathbf{whennot}\ x : v} \qquad \frac{\sigma(x) \neq false}{\sigma \vdash e\ \mathbf{whennot}\ x : \bot}$$

$\boxed{\sigma \vdash ce : v}$

$$\frac{\sigma \vdash e_1 : true \quad \sigma \vdash ce_2 : v \quad \sigma \vdash ce_3 : w}{\sigma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ ce_2\ \mathbf{else}\ ce_3 : v} \qquad \frac{\sigma \vdash e_1 : false \quad \sigma \vdash ce_2 : v \quad \sigma \vdash ce_3 : w}{\sigma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ ce_2\ \mathbf{else}\ ce_3 : w}$$

$$\frac{\sigma \vdash x : true \quad \sigma \vdash ce_1 : v \quad \sigma \vdash ce_2 : \bot}{\sigma \vdash \mathbf{merge}\ x\ ce_1\ ce_2 : v} \qquad \frac{\sigma \vdash x : false \quad \sigma \vdash ce_1 : \bot \quad \sigma \vdash ce_2 : w}{\sigma \vdash \mathbf{merge}\ x\ ce_1\ ce_2 : w}$$

$\boxed{eqn \xrightarrow{\sigma} eqn'}$

$$\frac{\sigma(ck) = true \quad \sigma(y) = k \quad \sigma \vdash e : k'}{y \underset{ck}{=} k\ \mathbf{fby}\ e \xrightarrow{\sigma} y \underset{ck}{=} k'\ \mathbf{fby}\ e} \qquad \frac{\sigma(ck) \neq true \quad \sigma(y) = \bot}{y \underset{ck}{=} k\ \mathbf{fby}\ e \xrightarrow{\sigma} y \underset{ck}{=} k\ \mathbf{fby}\ e}$$

$$\frac{\sigma(ck) = true \quad \sigma(y) = w \quad \sigma \vdash e_0 : v_0 \quad \sigma \vdash e_1 : v_1 \quad \ldots \quad \sigma \vdash e_{n-1} : v_{n-1} \quad (\mathtt{f}\, v_0\, v_1 \ldots v_{n-1}) \Downarrow w}{y \underset{ck}{=} \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1} \xrightarrow{\sigma} y \underset{ck}{=} \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1}}$$

$$\frac{\sigma(ck) \neq true \quad \sigma(y) = \bot}{y \underset{ck}{=} \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1} \xrightarrow{\sigma} y \underset{ck}{=} \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1}}$$

$$\frac{\sigma(ck) = true \quad \sigma(y) = v \quad \sigma \vdash ce : v}{y \underset{ck}{=} ce \xrightarrow{\sigma} y \underset{ck}{=} ce} \qquad \frac{\sigma(ck) \neq true \quad \sigma(y) = \bot}{y \underset{ck}{=} ce \xrightarrow{\sigma} y \underset{ck}{=} ce}$$

$\boxed{\vec{eqn} \xrightarrow{\sigma} \vec{eqn}'}$

$$\frac{eqn \xrightarrow{\sigma} eqn' \quad \vec{eqn} \xrightarrow{\sigma} \vec{eqn}'}{eqn;\ \vec{eqn} \xrightarrow{\sigma} eqn';\ \vec{eqn}'}$$

$\boxed{h \vdash \vec{eqn} : h'}$

$$\frac{\vec{eqn} \xrightarrow{\sigma} \vec{eqn}' \quad h \vdash \vec{eqn}' : h'}{\sigma[input].h \vdash \vec{eqn} : \sigma[output].h'}$$

FIGURE 3.11: Operational semantics of OCaLustre

- If the expression e_1 reduces to the value v_1, and if the expression e_2 reduces to the value v_2, then applying a binary infix operator to these two expressions $e_1 \diamond e_2$ reduces to the value $v_1 \diamond v_2$:

$$\frac{\sigma \vdash e_1 : v_1 \qquad \sigma \vdash e_2 : v_2}{\sigma \vdash e_1 \diamond e_2 : v_1 \diamond v_2}$$

- The result of merging (**merge**) two streams corresponds to the value of its second (respectively third) parameter when its first parameter is true (respectively false).

$$\frac{\sigma \vdash x : true \qquad \sigma \vdash ce_1 : v \qquad \sigma \vdash ce_2 : \bot}{\sigma \vdash \mathbf{merge}\, x\, ce_1\, ce_2 : v} \qquad \frac{\sigma \vdash x : false \qquad \sigma \vdash ce_1 : \bot \qquad \sigma \vdash ce_2 : w}{\sigma \vdash \mathbf{merge}\, x\, ce_1\, ce_2 : w}$$

- A stream defined by an equation has no value ($\bot$) if its clock is false (or if that clock is itself absent):

$$\frac{\sigma(ck) \neq true \qquad \sigma(y) = \bot}{y \underset{ck}{=} ce \xrightarrow{\sigma} y \underset{ck}{=} ce}$$

- Evaluating an equation containing the **fby** operator entails a change in the form of the equation for the next instant. The value on the left of the operator then corresponds to the value computed at the previous instant:

$$\frac{\sigma(ck) = true \qquad \sigma(y) = k \qquad \sigma \vdash e : k'}{y \underset{ck}{=} k\ \mathbf{fby}\ e \xrightarrow{\sigma} y \underset{ck}{=} k'\ \mathbf{fby}\ e}$$

- Evaluating an OCaml function call (denoted by a **call**) amounts to evaluating all the call parameters (e_0 to e_n) in OCaLustre, then applying the function to the computed values. The result w of this application, computed in the semantics of OCaml[11], corresponds to the value of the resulting stream y.

$$\frac{\sigma(ck) = true \quad \sigma(y) = w \quad \sigma \vdash e_0 : v_0 \quad \sigma \vdash e_1 : v_1 \quad \ldots \quad \sigma \vdash e_{n-1} : v_{n-1} \quad (\mathtt{f}\, v_0\, v_1 \ldots v_{n-1}) \Downarrow w}{y \underset{ck}{=} \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1} \xrightarrow{\sigma} y \underset{ck}{=} \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1}}$$

- Finally, evaluating a program amounts to evaluating the equations it contains by computing a solution of the system of equations for given input values. It then produces a set of values corresponding to the program's output variables, and recursively induces the evaluation of new outputs from the inputs of the following instant.

$$\frac{\vec{eqn} \xrightarrow{\sigma} \vec{eqn}' \qquad h \vdash \vec{eqn}' : h'}{\sigma[input].h \vdash \vec{eqn} : \sigma[output].h'}$$

To illustrate these various semantic rules, Figure 3.12 gives a derivation of the evaluation of the equation $y \underset{\bullet}{=} 2$ **fby** $(y + 1)$: at the next execution instant, this equation becomes $y \underset{\bullet}{=} 3$ **fby** $(y + 1)$.

$$\frac{\sigma(\bullet) = true \qquad \sigma(y) = 2 \qquad \dfrac{\dfrac{\sigma(y) = 2}{\sigma \vdash y : 2} \qquad \dfrac{}{\sigma \vdash 1 : 1}}{\sigma \vdash (y + 1) : 3}}{y \underset{\bullet}{=} 2\ \mathbf{fby}\ (y + 1) \xrightarrow{\sigma} y \underset{\bullet}{=} 3\ \mathbf{fby}\ (y + 1)}$$

FIGURE 3.12: Derivation of the evaluation of an OCaLustre stream

A memory is *consistent* from the point of view of clocking if and only if it associates $\perp$ with every variable whose value is not computed (i.e. if its clock is false, or if that clock is itself not computed). An expected property of this semantics is that, if the program is well clocked, then a memory compatible with this program is necessarily consistent (consequently, $\perp$ is never consulted when evaluating the expressions of a well-clocked program). The proof of this property linking typing and semantics is nevertheless left to future work, which could draw on techniques similar to those used in related work on certified Lustre compilation [Aug13, BBD+17]. On the other hand, with the semantics described in this section, several solutions may exist for the system of equations of a synchronous instant. Additional static constraints are necessary to guarantee the existence of a unique solution and to allow this solution to be computed sequentially. In the next chapter, we detail the nature of these causality and schedulability properties, which underlie the single-loop compilation method.

[11] The notation $e \Downarrow w$ indicates that, in the semantics of OCaml, the expression e evaluates to the value v.

Chapter Conclusion

This chapter has provided a complete overview of the constructs and principles offered by our synchronous extension of the OCaml language. We have discussed many formal aspects, in particular typing properties inherent in the use of this programming paradigm. OCaLustre is intended to be used together with the OMicroB virtual machine presented in the previous chapter, and it is therefore essential that it be fully compatible with OMicroB's bytecode interpreter. This is also the case for Lucid Synchrone, but its great expressiveness (higher order, multiple base clocks for a single node, automata, ...) produces programs that use more resources and are not compatible with the hardware limitations we consider. We will discuss some comparison points on this topic in Chapter 7, illustrating that our solution can limit the memory footprint of synchronous programs because OCaLustre's compilation model is simplified by more limited expressiveness.

In the next chapter, we therefore present the steps that make it possible to compile an OCaLustre program into a standard sequential OCaml program, fully compatible with OMicroB and the target devices, but also with any compiler for the OCaml language.

4 Compiling OCaLustre Programs

OCaLustre is an extension of the OCaml language, and as such we use a set of software tools that make it possible to define such extensions. In particular, we use *PPX* [⚓3], a tool of the standard OCaml compiler that allows syntactic extensions to be defined through the use of specific annotations in the program source code. In our use case, we annotate what the OCaml compiler initially regards as a function definition with the keyword "`node`" in order to construct an OCaLustre node (hence the syntax of annotations of the form `let%node`).

The benefit of using PPX is that the standard OCaml compiler[1] performs the steps needed to parse the source program and build an internal compiler representation of it. The OCaLustre compiler can then hook directly into this internal representation (an OCaml abstract syntax tree, or "*AST*"), apply several transformations to it, and then hand it back to the standard compiler, which produces executable code. The OCaLustre compiler can thus be regarded as a *preprocessor*, responsible for modifying the structure of the program AST before the standard compiler resumes processing it.

In this chapter, we describe the steps required to compile an OCaLustre program, from putting it into normal form to generating a standard representation of an OCaml program. We follow a process well documented in the literature [Pla88, BCHP08], which has previously been used to build compilers for synchronous languages comparable to OCaLustre.

Figure 4.1 schematically represents the compilation chain of an OCaLustre program. It consists of four main stages:

- A first *normalisation* stage transforms the program by restructuring its components in order to simplify the static analyses applied during compilation.
- A second *scheduling* stage sorts the various equations that make up the body of an OCaLustre node so that their reading order corresponds to their declaration order when OCaml code is generated.
- The third compilation stage consists in *inferring the clock types* of OCaLustre equations. This stage automatically annotates each program equation with its clock, following the clock-typing rules defined in Section 3.2.4.
- The final compilation stage consists in *translating* the annotated OCaLustre program into a standard OCaml program, in the form of an AST understandable by the language compiler. This AST is then handled by that compiler and compiled (in the case of compilation to a bytecode file) into a file fully compatible with any OCaml virtual machine, including OMicroB.

It should be noted that our decision to follow a *separate-compilation* model for OCaLustre nodes (using a model comparable to SCADE, unlike Lustre V4 [HR01], which performs call *inlining*) is motivated by the desire to avoid duplicating code when compiling several calls to the same node. Given the limited

[1] Whether *ocamlc*, the compiler that generates OCaml bytecode, or *ocamlopt*, the compiler that generates native code.

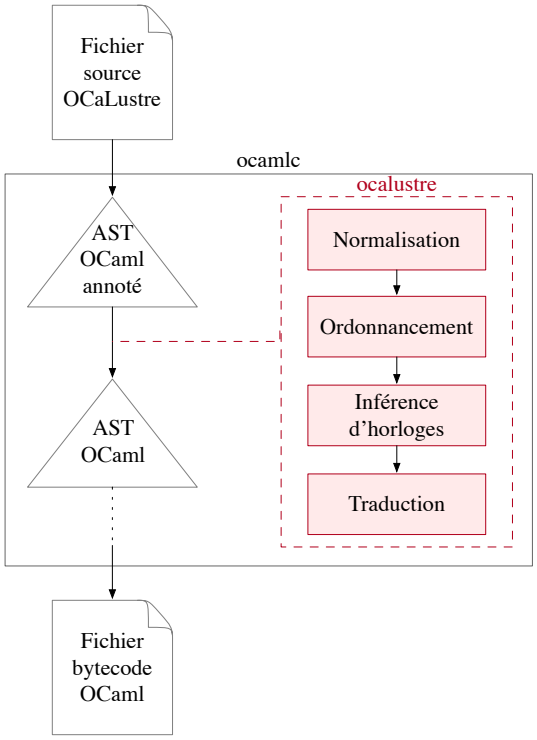


FIGURE 4.1: Compilation chain of an OCaLustre program

memory resources of a microcontroller, it is important in our use case to limit the memory footprint of programs. When describing the scheduling process, we will see that this decision has consequences for the static validation of certain programs, but this disadvantage seems acceptable to us given the importance, in our work, of limiting program resource consumption.

4.1 Conversion to Normal Form

The first compilation stage for an OCaLustre program consists in transforming the nodes obtained from parsing and lexical analysis of the source program into their representation in the *normal form* defined in Section 3.2.1.

The main role of conversion to normal form, or *normalisation*, is to extract from node equations the expressions that we call "*side-effect expressions*", in order to simulate, in the target language (OCaml), the use of OCaLustre's strict conditional operator. This transformation thus preserves the semantics of OCaLustre without having to define an "`if`" operator specifically intended for executing OCaLustre nodes. Side-effect expressions are those that, when evaluated, influence the internal state of a node. In OCaLustre, there are three of them:

1. The delay operator ⋙ implicitly entails manipulating a register in the internal state of the node in which it is used: the expression `0` ⋙ `e` is therefore a side-effect expression that induces the computation of the value of the expression `e` at instant t, as well as the storage of this value in the node's internal state so that it is accessible to the program at instant $t + 1$.

   So that this evaluation is not "frozen' if the expression is nested in the consequence or alternative of a conditional, every subexpression appearing in the definition of a stream `x` that contains an ⋙

is extracted from the expression in which it was nested and moved into the definition of a new stream, named `x_aux`. In this way, the value of `x_aux` is genuinely computed at each execution instant of the program.

For example, the following non-normalised node:

```
let%node ex_norm (b) ~return:(x) =
  x = if b then (0 ⋙ (x + 1)) else (0 ⋙ x)
```

is translated into normal form as this semantically equivalent representation:

```
node ex_norm b return x =
  x_aux1 = 0 fby (x + 1);
  x_aux2 = 0 fby x;
  x = if b then x_aux1 else x_aux2
```

Thus, the computation of `0 ⋙ (x + 1)` and `0 ⋙ x` is indeed performed at each execution instant (whatever the value of `b`), without having to distinguish OCaml's lazy `if` from OCaLustre's strict `if`.

2. In the same way, since the body of a node may contain equations using the delay operator, any call to that node may produce the same kind of side effect. The extraction mechanism is therefore also extended to applications. For example, the following node:

```
let%node call_cpt_cond (b) ~return:(x) =
  x = if b then cpt () else 0
```

becomes, after normalisation:

```
node ex_norm b return x =
  x_aux = cpt ();
  x = if b then x_aux else 0
```

3. Finally, the same operation as for node calls is applied for each use of the `call` operator, since the called function may itself produce side effects.

More generally, conversion to normal form consists in transforming an OCaLustre program from a representation corresponding to the concrete syntax of the language into an AST, defined from the normalised grammar described in Section 3.2.1, that can be manipulated by the language compiler. This process makes the internal representation of nodes, and their analysis, uniform by structuring the various kinds of expressions (control expressions, normal expressions, ...) and simplifying certain constructs. For example, the n-tuples present in the concrete syntax of the language are split into several distinct equations after normalisation. The following program:

```
let%node ex_tuples (a,b) ~return:(c,d) =
  (c,d) = (a,b)
```

is represented, in normal form, as follows:

```
node ex_tuples (a,b) return (c,d) =
  c = a;
  d = b
```

Putting an OCaLustre program into normal form corresponds to applying a function *normalize* that transforms the abstract syntax tree obtained from parsing and lexical analysis of an OCaLustre source program into an AST corresponding to the program's internal, normalised representation (ast_{norm}). The normalisation function can be decomposed into two functions: first, a *split* function that "splits' n-tuples (given in Figure 4.2), followed by the *norm* function, which actually performs normalisation by distributing expressions towards their equivalent in the normal-form grammar. The *norm* function for normalising a node is illustrated in Figure 4.3; it calls mutually recursive auxiliary functions $norm_x$, where the index x corresponds to the name of the syntactic category to which each function converts a given expression or equation. The role of these functions is to traverse the program AST produced by parsing (whose code we represent in roman type) in order to produce an AST in normalised form, collecting new equations created by reorganising the AST according to normal form. When a new equation must be created, they call a $fresh()$ function that generates a fresh variable name at each call.

$$
\begin{aligned}
split(\vec{y} = f\ e) &= [\vec{y} = f\ e] \\
split(y = e) &= [y = e] \\
split(y\,,\vec{y} = e\,,\vec{e}) &= (y = e) :: split(\vec{y} = \vec{e}) \\
split(eqn_1\,;eq\vec{n}_2) &= let\ \vec{eqn}'_1 = split(eqn_1)\ in \\
&\qquad let\ \vec{eqn}_2{}' = split(eq\vec{n}_2)\ in \\
&\qquad \vec{eqn}'_1 +\!\!+ \vec{eqn}'_2 \\
split(\texttt{let\%node}\ f\ \vec{x}\ \texttt{\sim return:}\vec{y} = e\vec{qn}) &= let\ \vec{eqn}' = split(\vec{eqn})\ in \\
&\qquad \texttt{let\%node}\ f\ \vec{x}\ \texttt{\sim return:}\vec{y} = \vec{eqn}'
\end{aligned}
$$

FIGURE 4.2: Tuple-splitting function

4.2 Scheduling

In an OCaLustre program, the equations that make up the body of a node are not constrained by any condition on the order in which they appear. Since the body of a node is a system of equations whose values are computed at each instant, nothing prevents one equation, as in an ordinary mathematical system of equations, from referring to a variable defined several lines later. The following node is therefore perfectly valid, even though the equation defining the stream `z` refers to the stream `k`, which

$$
\begin{aligned}
norm_e(e_1 \diamond e_2) &= let\ (l_1, e_1') = norm_e(e_1)\ in \\
&\quad let\ (l_2, e_2') = norm_e(e_2)\ in\ (l_1 +\!\!+ l_2, e_1' \diamond e_2') \\
norm_e(\square\ e_0) &= let\ (l, e_0') = norm_e(e_0)\ in\ (l, \square\ e_0') \\
norm_e(e_0\ \texttt{[@when x]}) &= let\ (l, e_0') = norm_e(e_0)\ in\ (l, e_0'\ \textbf{when}\ x) \\
norm_e(e_0\ \texttt{[@whennot x]}) &= let\ (l, e_0') = norm_e(e)\ in\ (l, e_0'\ \textbf{whennot}\ x) \\
norm_e(\texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3) &= let\ x = fresh()\ in \\
&\quad let\ (l, eqn) = norm_{eqn}(x = \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3)\ in\ (eqn :: l, x) \\
norm_e(\texttt{merge}\ x\ e_1\ e_2) &= let\ x = fresh()\ in \\
&\quad let\ (l, eqn) = norm_{eqn}(x = \texttt{merge}\ x\ e_1\ e_2)\ in\ (eqn :: l, x) \\
norm_e(k\ \texttt{-»}\ e) &= let\ x = fresh()\ in \\
&\quad let\ (l, eqn) = norm_{eqn}(x = k\ \texttt{-»}\ e)\ in\ (eqn :: l, x) \\
norm_e(\texttt{call f}\ e_0\ e_1\ \ldots\ e_{n-1}) &= let\ x = fresh()\ in \\
&\quad let\ (l, eqn) = norm_{eqn}(x = \texttt{call f}\ e_0\ e_1\ \ldots\ e_{n-1})\ in\ (eqn :: l, x) \\
norm_e(f\ \texttt{(}e_0\texttt{)}) &= let\ x = fresh()\ in \\
&\quad let\ (l, eqn) = norm_{eqn}(x = f\texttt{(}e_0\texttt{)})\ in\ (eqn :: l, x) \\
norm_e(e_0) &= ([], e_0) \text{ for all other forms of } e_0
\end{aligned}
$$

$$
\begin{aligned}
norm_{ce}(\texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3) &= let\ (l_1, e_1') = norm_e(e_1)\ in \\
&\quad let\ (l_2, ce_2) = norm_{ce}(e_2)\ in \\
&\quad let\ (l_3, ce_3) = norm_{ce}(e_3)\ in \\
&\quad (l_1 +\!\!+ l_2 +\!\!+ l_3, \textbf{if}\ e_1'\ \textbf{then}\ ce_2\ \textbf{else}\ ce_3) \\
norm_{ce}(\texttt{merge}\ x\ e_1\ e_2) &= let\ (l_1, e_1') = norm_e(e_1)\ in \\
&\quad let\ (l_2, e_2') = norm_e(e_2)\ in \\
&\quad (l_1 +\!\!+ l_2, \textbf{merge}\ x\ e_1'\ e_2') \\
norm_{ce}(e_0) &= norm_e(e_0) \text{ for all other forms of } e_0
\end{aligned}
$$

$$
\begin{aligned}
norm_{\vec{e}}(e_0) &= norm_e(e_0) \\
norm_{\vec{e}}(e_1, e_2) &= let\ (l_1, e_1') = norm_e(e_1)\ in \\
&\quad let\ (l_2, \vec{e_2}) = norm_{\vec{e}}(e_2)\ in\ (l_1 +\!\!+ l_2, e_1' :: \vec{e_2})
\end{aligned}
$$

$$
\begin{aligned}
norm_{eqn}(y = k\ \texttt{-»}\ e) &= let\ (l, e') = norm_e(e)\ in\ (l, y = k\ \textbf{fby}\ e') \\
norm_{eqn}(\vec{y} = f\texttt{(}e\texttt{)}) &= let\ (l, \vec{e}) = norm_{\vec{e}}(e)\ in\ (l, \vec{y} = f\ (\vec{e})) \\
norm_{eqn}(y = \texttt{call f}\ e_0\ e_1\ \ldots\ e_{n-1}) &= let\ (l_0, e_0') = norm_e(e_0)\ in \\
&\quad let\ (l_1, e_1') = norm_e(e_1)\ in \ldots \\
&\quad let\ (l_{n-1}, e_{n-1}') = norm_e(e_{n-1})\ in\ (l_0 +\!\!+ l_1 +\!\!+ \ldots +\!\!+ l_{n-1}, y = \textbf{call f}\ e_0'\ e_1'\ \ldots\ e_{n-1}') \\
norm_{eqn}(y = e) &= let\ (l, ce) = norm_{ce}(e)\ in\ (l, y = ce)
\end{aligned}
$$

$$
\begin{aligned}
norm_{e\vec{q}n}(eqn) &= let\ (l, eqn) = norm_{eqn}(eqn)\ in\ (l, [eqn]) \\
norm_{e\vec{q}n}(eqn\texttt{;}e\vec{q}n) &= let\ (l, eqn') = norm_{eqn}(eqn)\ in \\
&\quad let\ (l', e\vec{q}n') = norm_{e\vec{q}n}(e\vec{q}n)\ in\ (l +\!\!+ l', eqn' :: e\vec{q}n')
\end{aligned}
$$

$$
norm(\texttt{let\%node}\ f\ \vec{x}\ \texttt{~return:}\ \vec{y} = e\vec{q}n) = let\ (e\vec{q}n', e\vec{q}n'') = norm_{e\vec{q}n}(e\vec{q}n)\ in\ \textbf{node}\ f\ \vec{x}\ \textbf{return}\ \vec{y} = (e\vec{q}n' +\!\!+ e\vec{q}n'')
$$

FIGURE 4.3: Normalisation functions

has not yet been defined in the natural top-to-bottom reading order, and the equation defining `k` refers to `w` before it too has been defined:

```
node ordo (x,y) return (z,k) =
  z = 3 * k;
  k = if x then w else 4;
  w = y + 42
```

In order ultimately to transform OCaLustre equations into OCaml variable declarations, the OCaLustre compiler therefore performs a *scheduling* pass over the body of a node. This process reorganises the system of equations so that each stream definition appears before it is used in other equations. Thus, scheduling the previous example amounts to processing a version of `ordo` in which each stream definition appears before its use:

```
node ordo_correct (x,y) return (z,k) =
  w = y + 42;
  k = if x then w else 4;
  z = 3 * k
```

Of course, scheduling the body of a node does not change the semantics of the language in any way, since each equation is considered to be computed *in parallel* with the other equations: the reading order of the equations does not reflect their actual execution semantics, which define no sequential evaluation order.

Equation scheduling relies on building a directed acyclic graph[2] that represents dependencies between the various streams *within the same instant*. A stream y is considered to be a dependency of a stream x whenever y is referred to, at the same instant, in the definition of x. In the dependency graph, an edge from a vertex x to a vertex y represents the fact that stream x depends on y. For example, Figure 4.4 shows the dependency graph of the node `ordo`.

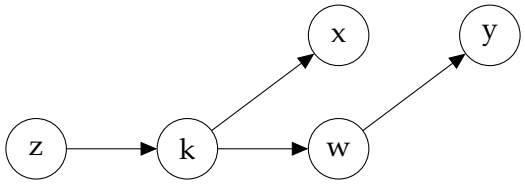


FIGURE 4.4: Dependency graph of the node `ordo`

Once this graph has been built, scheduling a node simply consists in following its reverse topological ordering, ignoring the node's input streams, to produce the sequence of equations that forms the body of the node under consideration. In our example, scheduling `ordo` therefore consists in defining `w` first, then `k`, and finally `z`.

[2]The erroneous case in which this graph is cyclic is described in the following subsection.

An equation is well scheduled if all the variables it contains, and on which it depends *instantaneously*, have previously been defined. Formally, we define the function *idents*, which returns the list of all variables occurring in an expression, or in a list of expressions:

$\boxed{idents(e)}$

$$
\begin{aligned}
&idents(k) \equiv \varnothing \\
&idents(x) \equiv [x] \\
&idents(X_i) \equiv \varnothing \\
&idents(e \diamond e') \equiv idents(e) +\!\!+ idents(e') \\
&idents(\square\, e) \equiv idents(e) \\
&idents(e\ \mathbf{when}\ x) \equiv x :: idents(e) \\
&idents(e\ \mathbf{whennot}\ x) \equiv x :: idents(e)
\end{aligned}
$$

$\boxed{idents(\vec{e})}$

$$
\begin{aligned}
&idents([e]) \equiv idents(e) \\
&idents(e, \vec{e}) \equiv idents(e) +\!\!+ idents(\vec{e})
\end{aligned}
$$

$\boxed{idents(ce)}$

$$
\begin{aligned}
&idents(e) \equiv idents(e) \\
&idents(\mathbf{merge}\ x\ ce\ ce') \equiv x :: idents(ce) +\!\!+ idents(ce') \\
&idents(\mathbf{if}\ e\ \mathbf{then}\ ce'\ \mathbf{else}\ ce'') \equiv idents(e) +\!\!+ idents(ce') +\!\!+ idents(ce'')
\end{aligned}
$$

For any list of variables $\mathcal{V}$, the following rules define the notion of *well-scheduled* equations in a context where all variables contained in $\mathcal{V}$ have already been defined. We write $\mathcal{V} \vdash_{ws} eqn$ to mean that the equation eqn is well scheduled ("*ws*" for "*well scheduled*").

$\boxed{\mathcal{V} \vdash_{ws} eqn}$

$$
\frac{y \notin \mathcal{V} \quad idents(ce) \in \mathcal{V}}{\mathcal{V} \vdash_{ws} y = ce} \qquad \frac{y \notin \mathcal{V}}{\mathcal{V} \vdash_{ws} y = k\ \mathbf{fby}\ e}
$$

$$
\frac{\vec{y} \notin \mathcal{V} \quad idents(\vec{e}) \in \mathcal{V}}{\mathcal{V} \vdash_{ws} \vec{y} = f(\vec{e})} \qquad \frac{y \notin \mathcal{V} \quad idents(e_0) \in \mathcal{V} \quad idents(e_1) \in \mathcal{V} \quad \ldots \quad idents(e_{n-1}) \in \mathcal{V}}{\mathcal{V} \vdash_{ws} y = \ \mathbf{call}\ \mathtt{f}\ e_0\ e_1 \ldots e_{n-1}}
$$

$\boxed{\mathcal{V} \vdash_{ws} e\vec{q}n}$

$$\frac{\mathcal{V} \vdash_{ws} eqn}{\mathcal{V} \vdash_{ws} [eqn]} \qquad \frac{\mathcal{V} \vdash_{ws} y = ce \quad y :: \mathcal{V} \vdash_{ws} e\vec{q}n}{\mathcal{V} \vdash_{ws} y = ce;\ e\vec{q}n}$$

$$\frac{\mathcal{V} \vdash_{ws} y = k\ \mathbf{fby}\ e \quad y :: \mathcal{V} \vdash_{ws} e\vec{q}n}{\mathcal{V} \vdash_{ws} y = k\ \mathbf{fby}\ e;\ e\vec{q}n} \qquad \frac{\mathcal{V} \vdash_{ws} \vec{y} = f(\vec{e}) \quad \vec{y} +\!\!+ \mathcal{V} \vdash_{ws} e\vec{q}n}{\mathcal{V} \vdash_{ws} \vec{y} = f(\vec{e});\ e\vec{q}n}$$

$$\frac{\mathcal{V} \vdash_{ws} y = \ \mathbf{call}\ \mathtt{f}\ e_0\, e_1 \ldots e_{n-1} \quad y :: \mathcal{V} \vdash_{ws} e\vec{q}n}{\mathcal{V} \vdash_{ws} y = \ \mathbf{call}\ \mathtt{f}\ e_0\, e_1 \ldots e_{n-1};\ e\vec{q}n}$$

A node is then well scheduled if all the equations that make up its body are themselves well scheduled in the context where the variables $\vec{x}$ representing its input parameters are defined:

$\boxed{\vdash_{ws} node}$

$$\frac{\vec{x} \vdash_{ws} e\vec{q}n}{\vdash_{ws} \mathbf{node}\, f(\vec{x})\ \mathbf{return}\ (\vec{y}) = e\vec{q}n}$$

The scheduling process for OCaLustre nodes is performed after the program has been put into normal form. Scheduling is therefore carried out on the intermediate representation of the program, and can be viewed as a function *schedule* of type $ast_{norm} \rightarrow ast_{norm}$.

For example, consider the following (normalised) program $\mathcal{P}$:

```
node f a return b =
  b = c + 1;
  c = 0 fby a

node g x return (y,z) =
  z = y + x;
  y = x
```

Then the result of applying $schedule(\mathcal{P})$ is:

```
node f a return b =
  c = 0 fby a;
  b = c + 1

node g x return (y,z) =
  y = x;
  z = y + x
```

4.2.1 Detecting Causality Loops

The scheduling step for OCaLustre nodes makes it possible to statically detect errors or inconsistencies in the definition of a node's streams. For example, the following node has indeterminate semantics:

```
node causal_loop () return (c,d) =
  c = 5 * d;
  d = 2 - c
```

Indeed, in this example the stream `c` depends on the stream `d`, and the stream `d` in turn depends on `c`. The streams `c` and `d` therefore depend on each other *within the same instant*. It is then impossible, for example, to compute the value of `c`, since it depends on the value of `d`, which itself depends on `c`: there are cyclic dependencies between these two streams.

This situation, called a *causality loop*, may appear less obviously, in a long chain of dependencies between streams, or even through several calls to auxiliary nodes. For this reason, the OCaLustre compiler automatically detects such cyclic dependencies and rejects programs that contain them. Detection is performed simply by checking that no loop exists in the graph representing dependencies between streams: the presence of a cycle in this graph stops compilation and displays an error message:

```
Error: Causality loop in node "causal_loop" with variables (d, c)
```

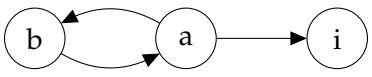
It should also be noted that an equation such as `y = 0 ⋙ (y+1)` does not mean that the stream `y` depends on itself, since use of the ⋙ operator means that the value of stream `y` at the *previous* instant is considered in the expression defining `y`; dependency relations are computed only for the same instant. Such an equation is therefore entirely compatible with the semantics of the language, and does not indicate the existence of a causality loop.

4.2.2 Limitation Due to Separate Compilation

This process for scheduling node bodies, combined with our model of separate compilation for each OCaLustre node, may cause some correct programs to be rejected. Consider, for example, the following program fragment:

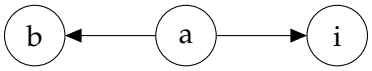
```
node shift_one x return y =
  y = 0 fby x

node call_shift_one i return b =
  a = b + i;
  b = shift_one a
```

When scheduling the node `call_shift_one`, the following dependency graph is generated:



Since this graph contains a loop, the node is rejected during compilation. Yet if we consider a version of the program in which `shift_one` has been *inlined*, that is, where the call to `shift_one` has been replaced by the equation contained in its body, then a different dependency graph is generated:

```
node call_shift_one i return b =
  a = b + i;
  b = 0 fby a
```



In reality, the stream b does not depend *instantaneously* on `a`, since `a` appears to the right of the $\ggg$ operator once `shift_one` has been inlined: the program did not contain a causality loop. The OCaLustre compiler does not know the temporal-shift information between a node's inputs and outputs. It treats external nodes as black boxes, and therefore assumes that every argument of a node call is an instantaneous dependency for its output streams.

In our setting, this limitation nevertheless seems acceptable, since the separate-compilation model avoids copying a node body for every equation that calls it, thereby reducing the final memory footprint of OCaLustre programs.

4.3 Clock Inference

Once an OCaLustre program has been normalised and scheduled, the next compilation step consists in inferring the clocks of each expression contained in the bodies of its synchronous nodes.

This inference mechanism is derived from the type-inference system for a Hindley-Milner-style system [Mil78], following the work of Colaço and Pouzet [CP03] on clock typing, while limiting polymorphism to a single type variable: the base clock ($\bullet$).

Once a clock type has been assigned to every expression in a program, and therefore once the program has been shown to respect the clock-typing semantics (3.2.4), the corresponding AST is annotated by associating each equation and each constant with its clock. These annotations are particularly necessary for the static verification of clock typing, discussed in Section 5.2.

The generated abstract syntax tree therefore has a structure slightly different from that of a normalised program. It corresponds to the normal-form grammar in which certain elements are annotated with their clock type. We write $ast_{clocked}$ for an abstract syntax tree built from this grammar, and the clock-inference process then corresponds to a function *clocks* of type $ast_{norm} \rightarrow ast_{clocked}$. In what follows, we describe only the constructs that differ from the normal-form grammar.

- Constants are annotated with their clock type, as inferred by the compiler:

$$\begin{array}{lll} e & ::= & \\ & | & k^{ck} \\ & | & (\ldots) \end{array}$$

- Equations are annotated with their clock; in particular, an application is annotated with the clock of its conditional application:

$$\begin{array}{lll} eqn & ::= & \text{equation} \\ & | \quad y \underset{ck}{=} ce & \text{expression} \\ & | \quad y \underset{ck}{=} k \textbf{ fby } e & \text{fby} \\ & | \quad \vec{y} \underset{ck}{=} f(\vec{e}) & \text{application} \\ & | \quad y \underset{ck}{=} \textbf{call } \texttt{f}\ e_0\ e_1\ ..\ e_{n-1} & \text{OCaml function application} \end{array}$$

For example, consider the following program $\mathcal{Q}$:

```
node g (a,b,c) return z =
  x = a when c;
  y = b + 2;
  z = merge c x y
```

Then the result of applying $clocks(\mathcal{Q})$ is:

```
node g (a,b,c) return z =
  x (= on c) a when c;
  y (= onnot c) b + 2 (• onnot c);
  z = merge c x y
      •
```

The OCaLustre compiler also contains a *checker* for the inferred clock types. This checker, whose correctness proof is given in Chapter 5, ensures that, for a given program, the types inferred by the OCaLustre compiler are consistent with the formal description of the type system given above. The checker is enabled using the compiler option `-check_clocks`.

4.4 Translation to OCaml

The final compilation step for an OCaLustre program consists in translating the program, in its clock-annotated intermediate representation, into an OCaml AST compatible with any OCaml compiler.

This *translation* step is performed by converting each node of an OCaLustre program separately into an OCaml function, following the approach described by Biernacki *et al.* [BCHP08] and used for compiling SCADE 6 [CPP17], as well as in other compilers for Lustre-like languages [GHKT14, MDLM18]. This separate-compilation model, distinct from the Lustre compilation model in which all components of a program are grouped within a main loop, was chosen to reduce the size of programs in which several calls are made to the same nodes, and also to allow better modularisation of the various components of a program.

Our solution follows Lustre's *single-loop* compilation model, but with separate compilation: each OCaLustre node is compiled into a distinct function, while the main node of the program, which calls the generated functions, is executed in an infinite loop that reads the program inputs, executes the main node, and writes the computed outputs.

In this section, we describe *compilation schemes* representing the translation of the AST of an OCaLustre program into an OCaml program. Translation from the OCaLustre AST to OCaml code relies on the translation function $[\![\cdot]\!]$ of type $ast_{clocked} \to ast_{ocaml}$, which traverses the OCaLustre program and returns an abstract syntax tree corresponding to an OCaml program that can be handled by the standard OCaml compiler.

Translating Expressions

OCaLustre expressions are translated directly, by erasing the downsampling operators used for clock typing:

$$
\begin{aligned}
[\![()]\!] &= \texttt{()} \\
[\![k]\!] &= \texttt{k} \\
[\![x]\!] &= \texttt{x} \\
[\![X_i]\!] &= \texttt{X}_i \\
[\![e_1 \diamond e_2]\!] &= [\![e_1]\!] \diamond [\![e_2]\!] \\
[\![\square\ e]\!] &= \square\ [\![e]\!] \\
[\![e\ \textbf{when}\ x]\!] &= [\![e]\!] \\
[\![e\ \textbf{whennot}\ x]\!] &= [\![e]\!]
\end{aligned}
$$

Lists of OCaLustre expressions become OCaml tuples of expressions:

$$
\begin{aligned}
[\![[e]]\!] &= [\![e]\!] \\
[\![e, es]\!] &= [\![e]\!], [\![es]\!]
\end{aligned}
$$

The **merge** operator is translated in the same way as the conditional operator:

$$
\begin{aligned}
[\![\textbf{if}\ e\ \textbf{then}\ ce_1\ \textbf{else}\ ce_2]\!] &= \texttt{if}\ [\![e]\!]\ \texttt{then}\ [\![ce_1]\!]\ \texttt{else}\ [\![ce_2]\!] \\
[\![\textbf{merge}\ x\ ce_1\ ce_2]\!] &= \texttt{if}\ [\![x]\!]\ \texttt{then}\ [\![ce_1]\!]\ \texttt{else}\ [\![ce_2]\!]
\end{aligned}
$$

Translating a Node and Its Equations

An OCaLustre node is translated into an OCaml function that creates a closure whose environment contains the registers required for use of the *followed-by* operator, as well as the various initialisations of the auxiliary-node instances used in the original node:

$$[\![\textbf{node } f\ (\vec{x})\ \textbf{returns}\ (\vec{y}) = \ e\vec{q}s]\!] =$$
$$\texttt{let } [\![f]\!]\ \texttt{()} = \mathit{inits}(e\vec{q}s, \texttt{fun } [\![\vec{x}]\!] \texttt{ -> } \mathit{decls}(e\vec{q}s, \mathit{updates}(e\vec{q}s, [\![\vec{y}]\!]))_0)_0$$

Initialisations: The function *inits* generates the variable declarations that make up the environment of the created closure. It is parameterised by a list of equations $e\vec{q}s$, an integer n (needed to distinguish two instances of the same node), and a continuation K representing the following compilation steps.

Each use of the $\ggg$ operator causes the definition of a register, an OCaml reference initialised with the value to the left of this operator:

$$\mathit{inits}(y \underset{ck}{=} k\ \textbf{fby}\ e; e\vec{q}s, K)_n = \texttt{let } \_[\![y]\!]\_\texttt{fby} = \texttt{ref } [\![k]\!] \texttt{ in } \mathit{inits}(e\vec{q}s, K)_n$$

Each call to an auxiliary node causes the generation of an instance of this node, by calling the function with the same name. These instances are numbered to support the case where several instances of the same node are called:

$$\mathit{inits}(\vec{y} \underset{ck}{=} g\ (\vec{e}); e\vec{q}s, K)_n = \texttt{let } [\![g]\!]\_[\![n]\!] = [\![g]\!]\ \texttt{()} \texttt{ in } \mathit{inits}(e\vec{q}s, K)_{n+1}$$

The other forms of equations produce no declaration in the environment of the created closure:

$$\mathit{inits}(eq; e\vec{q}s, K)_n = \mathit{inits}(e\vec{q}s, K)_n$$
$$\mathit{inits}(\varnothing, K)_n = K$$

A list of variables $\vec{y}$ is translated into a tuple of variables, or into the *unit* value if the list is empty:

$$[\![()]\!] = \texttt{()}$$
$$[\![y]\!] = \texttt{y}$$
$$[\![y, \vec{y}]\!] = [\![y]\!], [\![\vec{y}]\!]$$

Declarations: The body of the node is traversed by the function *decls* in order to convert each equation into a variable declaration in the closure returned by the generated function. An equation is "guarded" by the condition derived from its clock; the function *guard* is defined later in this section:

$$\mathit{decls}(y \underset{ck}{=} ce; e\vec{q}s, K)_n = \texttt{let } [\![y]\!] \ = \ \mathit{guard}(ck, ce)_y \texttt{ in } \mathit{decls}(e\vec{q}s, K)_n$$
$$\mathit{decls}(y \underset{ck}{=} k\ \textbf{fby}\ e; e\vec{q}s, K)_n = \texttt{let } [\![y]\!] \ = \ \mathit{guard}(ck, \texttt{!}\_[\![y]\!]\_\texttt{fby})_y \texttt{ in } \mathit{decls}(e\vec{q}s, K)_n$$
$$\mathit{decls}(y \underset{ck}{=} \textbf{call } \texttt{f}\ e_0\ e_1\ \ldots\ e_{m-1}); e\vec{q}s, K)_n = \texttt{let } [\![y]\!] \ = \ \mathit{guard}(ck, \texttt{f } [\![e_0]\!]\ [\![e_1]\!] \ldots [\![e_{m-1}]\!])_y \texttt{ in } \mathit{decls}(e\vec{q}s, K)_n$$
$$\mathit{decls}(\vec{y} \underset{ck}{=} g\ (\vec{e}); e\vec{q}s, K)_n = \texttt{let } [\![\vec{y}]\!] \ = \ \mathit{guard}(ck, [\![g]\!]\_[\![n]\!]\ [\![\vec{e}]\!])_{\vec{y}} \texttt{ in } \mathit{decls}(e\vec{q}s, K)_{n+1}$$
$$\mathit{decls}(\varnothing, K)_n = K$$

When the clock of a node call is false, a filler value representing absence, i.e. the $\perp$ of the language semantics, must be assigned to the declared variable. Several solutions can be envisaged for representing such a generic value. For example, all streams of the OCaLustre language could be translated into values of type `option`[3], assigning the OCaml value `None` to an absent stream. However, this solution causes a substantial increase in the size of the generated program, and leads to systematic memory allocations. Another alternative would be to use an OCaml compiler capable of handling *nullable* types [MV14], at the cost of losing portability of the produced code. More simply, the current OCaLustre compiler replaces every absent value ($\perp$) with the value `Obj.magic ()`. This special value has the property of being considered well typed whatever its context of use; for example, the expression `Obj.magic () + 42` poses no problem for the compiler. It therefore violates the assumptions that guarantee the type safety of a program. It is therefore important to be able to ensure statically that, at no point during program execution, this value is actually used, and that it is present only to satisfy the construction of an OCaml program in which an expression must always have a value, even if here it is only a default value that is never used.

The function *guard* therefore conditions certain expression evaluations and replaces them with an absence value when their clock is false, or absent. It calls a function *bottom*, which generates a representative of the absence of a value with the right number of elements:

$$\begin{aligned} guard(\bullet, e)_{\vec{y}} &= e \\ guard(ck, e)_{\vec{y}} &= \texttt{if}\ [\![ck]\!]\ \texttt{then}\ e\ \texttt{else}\ bottom(\vec{y}) \end{aligned}$$

with

$$\begin{aligned} [\![\bullet]\!] &= \texttt{true} \\ [\![ck\ \textbf{on}\ x]\!] &= (\texttt{(}[\![ck]\!]\ \texttt{\&\&}\ [\![x]\!]\texttt{)}) \\ [\![ck\ \textbf{onnot}\ x]\!] &= (\texttt{(}[\![ck]\!]\ \texttt{\&\&}\ \texttt{not}\ [\![x]\!]\texttt{)}) \end{aligned}$$

and

$$\begin{aligned} bottom(()) &= \texttt{()} \\ bottom(y) &= \texttt{Obj.magic ()} \\ bottom(y, \vec{y}) &= \texttt{Obj.magic ()},bottom(\vec{y}) \end{aligned}$$

Updates: Before returning the tuple of variables corresponding to the node's output streams, the generated closure updates each register arising from use of $\ggg$ by means of the function *updates*:

$$\begin{aligned} updates(x \underset{ck}{=} k\ \textbf{fby}\ e :: e\vec{q}s, K) &= guard(ck, [\![x]\!]\texttt{\_fby := }[\![e]\!])\ \texttt{;}\ updates(e\vec{q}s, K) \\ updates(eq :: e\vec{q}s, K) &= updates(e\vec{q}s)\ \text{(for the other forms of } eq) \\ updates(\varnothing, K) &= K \end{aligned}$$

[3] This type is defined as follows: `type 'a option = None | Some of 'a`.

Translating a Program

Finally, generating the OCaml code of an OCaLustre program amounts to generating, sequentially, the code of each node it contains:

$$[\![node :: \vec{node}]\!] = [\![node]\!]\,;\,; \\ [\![\vec{node}]\!]$$

$$[\![node :: \varnothing]\!] = [\![node]\!]$$

4.5 Example

Consider a simple OCaLustre program illustrating all the compilation steps described above. This program consists of two nodes: a first node, `count`, increments a counter at each instant and resets it if its parameter is *true*. The second node, `count_and`, counts the number of times its second and third parameters are true at the same time while its first parameter is false:

```
let%node count (reset) ~return:(cpt) =
  cpt = if reset then 0 else 0 ⋙ (cpt + 1);

let%node count_and (reset,b1,b2) ~return:(clk,cpt_sampled) =
  cpt_sampled = count (reset [@when clk]);
  clk = (b1 && b2)
```

After normalisation, the side-effect expression `(cpt + 1)` in `count` is extracted from the equation `cpt` into a separate equation, `cpt_aux`:

```
node count (reset) return (cpt) =
  cpt_aux = 0 fby (cpt + 1);
  cpt = if reset then 0 else cpt_aux

node count_and (reset,b1,b2) return (clk,cpt_sampled) =
  cpt_sampled = count (reset when clk);
  clk = (b1 && b2)
```

The scheduling step reorganises the body of `count_and` so that the stream `clk` is defined before it is used in `cpt_sampled`:

```
node count (reset) return (cpt) =
  cpt_aux = 0 fby (cpt + 1);
  cpt = if reset then 0 else cpt_aux

node count_and (reset,b1,b2) return (clk,cpt_sampled) =
  clk = (b1 && b2);
  cpt_sampled = count (reset when clk)
```

After clock inference, equations and constants are annotated with their clock types:

```
node count (reset) return (cpt) =
  cpt_aux =• 0 fby (cpt + 1•);
  cpt =• if reset then 0• else cpt_aux

node count_and (reset, b1, b2) return (clk, cpt_sampled) =
  clk =• (b1 && b2);
  cpt_sampled =•on clk count (reset when clk)
```

Finally, translation to a standard OCaml program produces the definition of two functions, each corresponding to a node of the original program:

```
let count () =
  let cpt_aux_fby = ref 0 in
  fun reset ->
    let cpt_aux = !cpt_aux_fby in
    let cpt = if reset then 0 else cpt_aux in
    cpt_aux_fby:= (cpt + 1);
    cpt

let count_and () =
  let count1 = count () in
  fun (reset,b1,b2) ->
    let clk = b1 && b2 in
    let cpt_sampled = if clk then count1 reset else Obj.magic () in
    (clk,cpt_sampled)
```

For each node of type $t_{in} \rightarrow t_{out}$, the type of the OCaml function generated at the end of this compilation process is therefore $unit \rightarrow (t_{in} \rightarrow t_{out})$, which corresponds to the type of a function that returns an *instance* of the node in the form of an OCaml function. In the previous example, the type of `count` is therefore $unit \rightarrow (bool \rightarrow int)$, and that of `count_and` is $unit \rightarrow ((bool * bool * bool) \rightarrow (bool * int))$.

Generating the Execution Machine:

The code responsible for reading inputs from the environment, executing a synchronous instant, and emitting the outputs computed during that instant is called the *execution machine* [Bou98]. The code of the main function of the OCaml program, which implements a minimal execution machine capable of running the previous synchronous program, then has the following form:

```
let () =
  (* initialisation of the main node *)
  let main = count_and () in
  while true do
   (* read inputs *)
   let (reset,b1,b2) = input_count_and () in
   (* execute one step of the main node *)
   let (clk,cpt_sampled) = main (reset,b1,b2) in
   (* write outputs *)
   output_count_and (clk,cpt_sampled)
  done
```

This code can be generated automatically by using the OCaLustre compiler option `-m`, followed by the name of the main node. This option also causes the generation of a code skeleton to be completed, containing the input and output functions of the synchronous program.

Chapter Conclusion

In this chapter, we have described the different steps that transform an OCaLustre program into a sequential OCaml program. These transformations provide several typing and schedulability guarantees that make it possible to produce programs with stronger safety properties. Several properties relating to these typing guarantees can then be checked in order to validate this increase in safety. In the following chapter, we describe the formalisation and proof of such properties.

5 OCaLustre Properties Formalised and Proved with Coq

In this chapter, we describe several properties derived from the formal specification of OCaLustre, and we formalise and prove them in Coq. These properties concern the type systems described in Section 3.2, which govern both "standard" data typing and clock typing, and make it possible to verify the consistency between these formal systems and their implementation in the prototype OCaLustre compiler. Verifying these properties therefore provides safety guarantees for certain aspects of the OCaLustre compiler, such as the correctness of OCaLustre value typing after translation to OCaml, as well as the consistency between the clock types inferred by OCaLustre and the formal system described above. In particular, the code of a *clock checker*, obtained by Coq extraction from the formal specification of a program's clocking rules, is integrated into the OCaLustre compiler, which follows the steps described in the previous chapter.

5.1 Translation and Type-Correctness

In order to leave responsibility for checking that OCaLustre nodes are well typed to the type checker included in the standard OCaml compiler, this section presents a proof of the type-correctness of OCaLustre with respect to its translation.

This proof verifies two properties ensuring that OCaLustre programs are well typed if and only if the generated OCaml code is itself well typed:

- A *preservation* property, stating that if the OCaLustre code is well typed, then its translation is also well typed[1].
- A *safety* property, according to which if the generated OCaml code is well typed, then the corresponding OCaLustre node is also well typed: no typing information is lost that could transform an ill-typed OCaLustre program into well-typed OCaml code.

The metatheoretical proof of these properties makes it possible to avoid adding an *ad hoc* type checker to OCaLustre, and thus to rely on the power of the typing mechanism of the native OCaml compiler. Indeed, if the OCaml type checker detects a typing error in the generated code, it allows us to conclude that the original OCaLustre program is ill typed.

In this section, we progressively describe the main steps of this correctness proof. Our reasoning also relies only on nodes considered to be "well formed". We consider a node *node* to be well formed, written $\vdash_{wf}$ *node*, if all the names of the streams it defines are distinct, if all its parameters are distinct, and if no input-parameter name is reused to define a new stream name in the body of the node:

[1] This property can be seen as a form of *completeness*, in the sense that no correct program is rejected: typing errors are not introduced by the translation.

$\boxed{\vdash_{wf} node}$

$$\frac{distinct(\vec{x}) \quad distinct(\vec{y}) \quad \vec{x} \cap \vec{y} = \varnothing \quad distinct(names(\vec{eqn})) \quad \vec{x} \cap names(\vec{eqn}) = \varnothing}{\vdash_{wf} \textbf{node}\, f(\vec{x}) \textbf{ returns } (\vec{y}) = \vec{eqn}}$$

with

$\boxed{distinct(\vec{x})}$

$$\frac{}{distinct(\varnothing)} \qquad \frac{y \notin \vec{y} \quad distinct(\vec{y})}{distinct(y :: \vec{y})}$$

and

$\boxed{names(\vec{eqn})}$

$$\begin{aligned} &names(\varnothing) \equiv \varnothing \\ &names(x \underset{ck}{=} ce; \vec{eqn}) \equiv x :: names(\vec{eqn}) \\ &names(x \underset{ck}{=} k \textbf{ fby } e; \vec{eqn}) \equiv x :: names(\vec{eqn}) \end{aligned}$$

The proof of type-correctness of the translation presented in this section relies mainly on an equivalence theorem, which states that a well-formed OCaLustre node is well typed if and only if its translation is also well typed:

Theorem 5.1.1 (Type-Correctness of a Node)**.**

$$\forall\, node\ t,\ \vdash_{wf} node \Rightarrow (\vdash node : t \Leftrightarrow \vdash [\![node]\!] : unit\ \rightarrow t)$$

The proof of this equivalence verifies both the preservation and safety properties. The left-to-right implication ensures, by contraposition, that if the translation of a node is ill typed in OCaml, then the node itself is ill typed in OCaLustre (*preservation*). The right-to-left implication states that if the translation of a node is well typed, then the node itself is well typed. By contraposition, if a node is ill typed, then its translation is also ill typed (*safety*).

All the theorems and auxiliary lemmas required for this proof are available online, as documentation and Coq files [⚓1]. It should be noted, however, that this proof is carried out on a simplified version of OCaLustre and of the OCaml language, which we call "*pseudo-ML*": in particular, we do not handle the parametric polymorphism adopted by OCaml; OCaLustre programs are represented as a single node, with no possibility of node calls[2]; and streams may only have type *int* or *bool*. Moreover, calls to OCaml functions via the keyword **`call`** have not been formalised. These should not compromise the correctness of the proof, since their compilation is direct: a **`call`** generates a function application, and the types coincide. Adding node calls is more complex, because it introduces new names into the generated code after compilation, since each node call creates an instance of a closure in the generated code. Integrating these constructs into the type-correctness proof of the translation is ongoing work. The version of the proof presented in this manuscript and in the associated Coq files is therefore simplified.

[2]In this sense, the OCaLustre programs considered here are close to Lustre programs, for which compilation integrates the code of the different intermediate nodes into the main node.

5.1.1 Partial Translation Using Simultaneous Declarations

To decompose the type-correctness proof, we first prove typing equivalence between the OCaLustre program and the same program translated into the *pseudo-ML* language extended with a non-standard construct. This construct, of the form "*let ... with ... in ...*", allows variables to be defined simultaneously.

For example, the following code fragment is valid in *pseudo-ML*, whereas it would not be valid in OCaml, which requires multiple variable definitions built using the keyword "*and*" to be independent of one another[3]:

```
let x = y with y = 2 in x
```

This intermediate construct is not intended to be executed. It simply makes it possible to factorise the type-correctness proofs, by postponing the introduction of the notion of well scheduling to a proof step described later; the final theorem will not contain such simultaneous declarations.

Typing Rules of *pseudo-ML*

The grammar of the pseudo-ML language, very close to that of OCaml, is given in Figure 5.1. The main difference between this grammar and that of OCaml is the addition of the "*let ... with ...*" simultaneous-declaration construct described above.

It should also be noted that, in order to simplify the proof process by ignoring name-shadowing phenomena, the namespace reserved for variables newly introduced by the translation and the namespace reserved for variables already present in the node declaration are distinct by construction: the typing environment $\mathcal{R}$ contains the set of pairs $(name, type)$ for variables introduced by compilation, whereas the environment Γ concerns the pairs $(name, type)$ for the other variables. In this case, the names contained in $\mathcal{R}$, arising from the translation of $\ggg$, all correspond to references.

[3]Mutually recursive definitions are possible in OCaml only for values of functional types.

e	::=		expressions
	|	$\bot$	magic
	|	$()$	unit
	|	k	constant
	|	x	variable
	|	x_r	register
	|	**ref** e	reference
	|	$!e$	dereferencing
	|	$e := e'$	assignment
	|	$e \diamond e'$	binary operator
	|	$\square\, e$	unary operator
	|	**if** e **then** e' **else** e''	conditional
	|	e **;** e'	sequence
	|	$(\vec{e})$	tuple
	|	**fun** $\vec{x} \rightarrow\ e$	n-ary function
	|	**let** δ **in** e'	variable declarations
	|	**let** δ_r **in** e'	variable declarations (new names)
	|	**let** $y = e$ **in** e'	variable declaration
	|	**let** $y_r = e$ **in** e'	variable declaration (new name)
δ	::=		simultaneous variable declarations
	|	$\varnothing$	
	|	$(y = e)$ **with** δ	
δ_r	::=		simultaneous variable declarations (new names)
	|	$\varnothing$	
	|	$(y_r = e)$ **with** δ_r	

Figure 5.1: Grammar of *pseudo-ML*

The typing rules for simultaneous declarations are then the following:

$\boxed{\mathcal{R}, \Gamma \vdash \delta : \vec{t}}$

$$\frac{}{\mathcal{R}, \Gamma \vdash \varnothing : \varnothing} \qquad \frac{\Gamma(x) = t \quad \mathcal{R}, \Gamma \vdash e : t \quad \mathcal{R}, \Gamma \vdash \delta : \vec{t}}{\mathcal{R}, \Gamma \vdash (x = e) \textbf{ with } \delta : t * \vec{t}}$$

$\boxed{\mathcal{R}, \Gamma \vdash \delta_r : \vec{t}}$

$$\frac{}{\mathcal{R}, \Gamma \vdash \varnothing : \varnothing} \qquad \frac{\mathcal{R}(x) = t \quad \mathcal{R}, \Gamma \vdash e : t \quad \mathcal{R}, \Gamma \vdash \delta_r : \vec{t}}{\mathcal{R}, \Gamma \vdash (x_r = e) \textbf{ with } \delta_r : t * \vec{t}}$$

They state that for a set of simultaneous variable declarations to be well typed, each of its elements must be well typed in the *same* typing environment.

All the other typing rules of *pseudo-ML*, which do not particularly differ from those of a classical ML language, are shown in Figure 5.2.

$\boxed{\Gamma \vdash \vec{x} : \vec{t}}$

$$\frac{}{\Gamma \vdash \varnothing : \varnothing} \qquad \frac{\Gamma(x) = t \quad \Gamma \vdash \vec{x} : \vec{t}}{\Gamma \vdash x :: \vec{x} : t * \vec{t}}$$

$\boxed{\mathcal{R}, \Gamma \vdash \vec{e} : \vec{t}}$

$$\frac{}{\mathcal{R}, \Gamma \vdash \varnothing : \varnothing} \qquad \frac{\mathcal{R}, \Gamma \vdash e : t \quad \mathcal{R}, \Gamma \vdash \vec{e} : \vec{t}}{\mathcal{R}, \Gamma \vdash e, \vec{e} : t * \vec{t}}$$

$\boxed{\mathcal{R}, \Gamma \vdash e : t}$

$$\frac{}{\mathcal{R}, \Gamma \vdash \bot : t} \qquad \frac{}{\mathcal{R}, \Gamma \vdash () : \mathbf{unit}} \qquad \frac{}{\mathcal{R}, \Gamma \vdash \mathit{int\_literal} : \mathbf{int}} \qquad \frac{}{\mathcal{R}, \Gamma \vdash \mathit{bool\_literal} : \mathbf{bool}}$$

$$\frac{\Gamma(x) = t}{\mathcal{R}, \Gamma \vdash x : t} \qquad \frac{\mathcal{R}(x) = t}{\mathcal{R}, \Gamma \vdash x_r : t} \qquad \frac{\mathcal{R}, \Gamma \vdash e : t}{\mathcal{R}, \Gamma \vdash \square e : t}$$

$$\frac{\mathcal{R}, \Gamma \vdash e : \mathbf{int} \quad \mathcal{R}, \Gamma \vdash e' : \mathbf{int}}{\mathcal{R}, \Gamma \vdash e \diamond_{int} e' : \mathbf{int}} \qquad \frac{\mathcal{R}, \Gamma \vdash e : \mathbf{bool} \quad \mathcal{R}, \Gamma \vdash e' : \mathbf{bool}}{\mathcal{R}, \Gamma \vdash e \diamond_{bool} e' : \mathbf{bool}}$$

$$\frac{\mathcal{R}, \Gamma \vdash e : t \quad \mathcal{R}, \Gamma \vdash e' : t}{\mathcal{R}, \Gamma \vdash e \diamond_{comp} e' : \mathbf{bool}} \qquad \frac{\mathcal{R}, \Gamma \vdash e : \mathbf{bool} \quad \mathcal{R}, \Gamma \vdash e' : t \quad \mathcal{R}, \Gamma \vdash e'' : t}{\mathcal{R}, \Gamma \vdash \mathbf{if}\, e\, \mathbf{then}\, e'\, \mathbf{else}\, e'' : t}$$

$$\frac{\mathcal{R}, \Gamma \vdash e : t}{\mathcal{R}, \Gamma \vdash \mathbf{ref}\, e : t\, \mathbf{ref}} \qquad \frac{\mathcal{R}, \Gamma \vdash e : t\, \mathbf{ref}}{\mathcal{R}, \Gamma \vdash\, !e : t} \qquad \frac{\mathcal{R}, \Gamma \vdash e : t\, \mathbf{ref} \quad \mathcal{R}, \Gamma \vdash e' : t}{\mathcal{R}, \Gamma \vdash e := e' : \mathbf{unit}}$$

$$\frac{\mathcal{R}, \Gamma \vdash e : \mathbf{unit} \quad \mathcal{R}, \Gamma \vdash e' : t}{\mathcal{R}, \Gamma \vdash e\ ;\ e' : t} \qquad \frac{\mathcal{R}, \Gamma \vdash \vec{e} : \vec{t}}{\mathcal{R}, \Gamma \vdash (\vec{e}) : \vec{t}} \qquad \frac{\Gamma' = \left(\vec{x} : \vec{t}\right) \quad \mathcal{R}, \Gamma' \cup \Gamma \vdash e : t'}{\mathcal{R}, \Gamma \vdash \mathbf{fun}\, \vec{x} \to\ e : \vec{t} \to t'}$$

$$\frac{\mathcal{R}, \Gamma \vdash e : t \quad \mathcal{R}, \{x : t\} \cup \Gamma \vdash e' : t'}{\mathcal{R}, \Gamma \vdash \mathbf{let}\, x = e\, \mathbf{in}\, e' : t'} \qquad \frac{\mathcal{R}, \Gamma \vdash e : t \quad \{x : t\} \cup \mathcal{R}, \Gamma \vdash e' : t'}{\mathcal{R}, \Gamma \vdash \mathbf{let}\, x_r = e\, \mathbf{in}\, e' : t'}$$

$$\frac{\mathit{names}(\delta) = \vec{y} \quad \Gamma' = \left(\vec{y} : \vec{t}\right) \quad \mathcal{R}, \Gamma' \cup \Gamma \vdash \delta : \vec{t} \quad \mathcal{R}, \Gamma' \cup \Gamma \vdash e' : t'}{\mathcal{R}, \Gamma \vdash \mathbf{let}\, \delta\, \mathbf{in}\, e' : t'}$$

$$\frac{\mathit{names}(\delta_r) = \vec{y} \quad \mathcal{R}' = \left(\vec{y} : \vec{t}\right) \quad \mathcal{R}' \cup \mathcal{R}, \Gamma \vdash \delta_r : \vec{t} \quad \mathcal{R}' \cup \mathcal{R}, \Gamma \vdash e' : t'}{\mathcal{R}, \Gamma \vdash \mathbf{let}\, \delta_r\, \mathbf{in}\, e' : t'}$$

FIGURE 5.2: Typing Rules of *pseudo-ML*

Algorithmic Typing Rules of OCaLustre

To prove the expected equivalence, we reason from *algorithmic* OCaLustre typing rules, similar to the *declarative* typing rules discussed in Section 3.2.2. These rules are given in Figure 5.3. The typing rule for a node refers to the function *names* introduced above, whose role is to return the names of the different streams defined by a list of equations. It should also be noted that, in order to present an easily understandable set of rules, the node typing rule here implies that the nodes of an OCaLustre program contain no locally scoped streams: all streams defined in the body of a node appear as outputs. This temporary limitation, which lets us follow the progression of the Coq proof here, will be lifted at the end of this section, where we extend our proof to nodes that return a subset of the streams defined by the equations present in the body of a node.

$\boxed{\Gamma \vdash e : t}$

$$\frac{}{\Gamma \vdash () : \mathbf{unit}} \qquad \frac{}{\Gamma \vdash \mathit{int\_literal} : \mathbf{int}} \qquad \frac{}{\Gamma \vdash \mathit{bool\_literal} : \mathbf{bool}} \qquad \frac{\Gamma \vdash e : t}{\Gamma \vdash \square\, e : t}$$

$$\frac{\Gamma \vdash e : \mathbf{int} \quad \Gamma \vdash e' : \mathbf{int}}{\Gamma \vdash e \diamond_{int} e' : \mathbf{int}} \qquad \frac{\Gamma \vdash e : \mathbf{bool} \quad \Gamma \vdash e' : \mathbf{bool}}{\Gamma \vdash e \diamond_{bool} e' : \mathbf{bool}} \qquad \frac{\Gamma \vdash e : t \quad \Gamma \vdash e' : t}{\Gamma \vdash e \diamond_{comp} e' : \mathbf{bool}}$$

$$\frac{\Gamma(x) = t}{\Gamma \vdash x : t} \qquad \frac{\Gamma \vdash e : t \quad \Gamma \vdash x : \mathbf{bool}}{\Gamma \vdash e\, \mathbf{when}\, x : t} \qquad \frac{\Gamma \vdash e : t \quad \Gamma \vdash x : \mathbf{bool}}{\Gamma \vdash e\, \mathbf{whennot}\, x : t}$$

$\boxed{\Gamma \vdash_{ce} ce : t}$

$$\frac{\Gamma \vdash e : t}{\Gamma \vdash_{ce} e : t} \qquad \frac{\Gamma \vdash x : \mathbf{bool} \quad \Gamma \vdash_{ce} ce : t \quad \Gamma \vdash_{ce} ce' : t}{\Gamma \vdash_{ce} \mathbf{merge}\, x\, ce\, ce' : t} \qquad \frac{\Gamma \vdash e : \mathbf{bool} \quad \Gamma \vdash_{ce} ce : t \quad \Gamma \vdash_{ce} ce' : t}{\Gamma \vdash_{ce} \mathbf{if}\, e\, \mathbf{then}\, ce\, \mathbf{else}\, ce' : t}$$

$\boxed{\Gamma \vdash ck}$

$$\frac{}{\Gamma \vdash \bullet} \qquad \frac{\Gamma \vdash ck \quad \Gamma(x) = \mathbf{bool}}{\Gamma \vdash ck\, \mathbf{on}\, x} \qquad \frac{\Gamma \vdash ck \quad \Gamma(x) = \mathbf{bool}}{\Gamma \vdash ck\, \mathbf{onnot}\, x}$$

$\boxed{\Gamma \vdash eqn : t}$

$$\frac{\Gamma \vdash ck \quad \Gamma(y) = t \quad \Gamma \vdash_{ce} ce : t}{\Gamma \vdash y \underset{ck}{=} ce : t} \qquad \frac{\Gamma \vdash ck \quad \Gamma(y) = t \quad \Gamma \vdash e : t \quad \Gamma \vdash k : t}{\Gamma \vdash y \underset{ck}{=} k\, \mathbf{fby}\, e : t}$$

$\boxed{\Gamma \vdash e\vec{q}n : \vec{t}}$

$$\frac{}{\Gamma \vdash \varnothing : \varnothing} \qquad \frac{\Gamma \vdash eqn : t \quad \Gamma \vdash e\vec{q}n : \vec{t}}{\Gamma \vdash eqn; e\vec{q}n : t * \vec{t}}$$

$\boxed{\vdash node : t}$

$$\frac{\Gamma = \left(\vec{y} : \vec{t'}\right) \quad \Gamma' = \left(\vec{x} : \vec{t}\right) \quad \Gamma \cup \Gamma' \vdash e\vec{q}n : \vec{t'} \quad names(e\vec{q}n) = \vec{y}}{\vdash \mathbf{node}\, f(\vec{x})\, \mathbf{returns}\, (\vec{y}) = e\vec{q}n : \vec{t} \to \vec{t'}}$$

FIGURE 5.3: *Algorithmic* Typing Rules of OCaLustre

Translation to *pseudo-ML*

The translation function $[\![\cdot]\!]$ from an OCaLustre node to *pseudo-ML* code is given in Figure 5.4. This translation is similar to the translation from OCaLustre to OCaml, except that a list of equations is not translated into a sequence of nested "*let ... in ...*" declarations, but into simultaneous "*let ... with ...*" declarations specific to *pseudo-ML*.

$\boxed{[\![e]\!]}$

$$
\begin{aligned}
[\![()]\!] &\equiv () \\
[\![k]\!] &\equiv k \\
[\![x]\!] &\equiv x \\
[\![e \diamond e']\!] &\equiv [\![e]\!] \diamond [\![e']\!] \\
[\![e \textbf{ when } x]\!] &\equiv \textbf{if } x \textbf{ then } [\![e]\!] \textbf{ else } \bot \\
[\![e \textbf{ whennot } x]\!] &\equiv \textbf{if } x \textbf{ then } \bot \textbf{ else } [\![e]\!] \\
[\![\square\, e]\!] &\equiv \square\, [\![e]\!]
\end{aligned}
$$

$\boxed{[\![ce]\!]}$

$$
\begin{aligned}
[\![\textbf{if } e \textbf{ then } ce' \textbf{ else } ce'']\!] &\equiv \textbf{if } [\![e]\!] \textbf{ then } [\![ce']\!] \textbf{ else } [\![ce'']\!] \\
[\![\textbf{merge } x\ ce\ ce']\!] &\equiv \textbf{if } [\![x]\!] \textbf{ then } [\![ce]\!] \textbf{ else } [\![ce']\!]
\end{aligned}
$$

$\boxed{[\![ck]\!]}$

$$
\begin{aligned}
[\![\bullet]\!] &\equiv \textbf{true} \\
[\![ck \textbf{ on } x]\!] &\equiv [\![ck]\!] \,\&\&\, x \\
[\![ck \textbf{ onnot } x]\!] &\equiv [\![ck]\!] \,\&\&\, (\textbf{not } x)
\end{aligned}
$$

$\boxed{[\![e\vec{q}n]\!]_{inits}}$

$$
\begin{aligned}
[\![\varnothing]\!]_{inits} &\equiv \varnothing \\
[\![y \underset{ck}{=} k \textbf{ fby } e; e\vec{q}n]\!]_{inits} &\equiv (y_r = \textbf{ref } k) \textbf{ with } [\![e\vec{q}n]\!]_{inits} \\
[\![eqn; e\vec{q}n]\!]_{inits} &\equiv [\![e\vec{q}n]\!]_{inits}
\end{aligned}
$$

$\boxed{[\![e\vec{q}n]\!]}$

$$
\begin{aligned}
[\![\varnothing]\!] &\equiv \varnothing \\
[\![y \underset{ck}{=} ce; e\vec{q}n]\!] &\equiv (y = \textbf{if } [\![ck]\!] \textbf{ then } [\![ce]\!] \textbf{ else } \bot) \textbf{ with } [\![e\vec{q}n]\!] \\
[\![y \underset{ck}{=} k \textbf{ fby } e; e\vec{q}n]\!] &\equiv (y = \textbf{if } [\![ck]\!] \textbf{ then } !\, y_r \textbf{ else } \bot) \textbf{ with } [\![e\vec{q}n]\!]
\end{aligned}
$$

$\boxed{[\![e\vec{q}n]\!]_{updates}}$

$$
\begin{aligned}
[\![\varnothing]\!]_{updates} &\equiv () \\
[\![y \underset{ck}{=} k \textbf{ fby } e; e\vec{q}n]\!]_{updates} &\equiv (\textbf{if } [\![ck]\!] \textbf{ then } (y_r := [\![e]\!]) \textbf{ else } ())\ ;\ [\![e\vec{q}n]\!]_{updates} \\
[\![eqn; e\vec{q}n]\!]_{updates} &\equiv [\![e\vec{q}n]\!]_{updates}
\end{aligned}
$$

$\boxed{[\![node]\!]}$

$$[\![\textbf{node } f(\vec{x}) \textbf{ returns } (\vec{y}) = e\vec{q}n]\!] \equiv \textbf{fun } () \rightarrow \textbf{ let } [\![e\vec{q}n]\!]_{inits} \textbf{ in } (\textbf{fun } \vec{x} \rightarrow \textbf{ let } [\![e\vec{q}n]\!] \textbf{ in } ([\![e\vec{q}n]\!]_{updates}\ ;\ ([\![\vec{y}]\!])))$$

FIGURE 5.4: Translation Function from OCaLustre to *pseudo-ML*

Type-Correctness in *pseudo-ML*

From the functions and rules defined in the preceding sections, we now present the various steps used to prove type-correctness with respect to the translation of an OCaLustre node into a pseudo-ML program.

First, we prove that every simple expression (e) preserves, from the same environment Γ, the same type after translation:

Lemma 5.1.1 (Type-Correctness of Simple Expressions)**.**

$$\forall\ \mathcal{R}\ \Gamma\ e\ t, (\Gamma \vdash e : t \Leftrightarrow \mathcal{R}, \Gamma \vdash [\![e]\!] : t)$$

From this lemma, we deduce that every conditional expression (ce) also preserves the same type after translation:

Lemma 5.1.2 (Type-Correctness of Conditional Expressions)**.**

$$\forall\ \mathcal{R}\ \Gamma\ ce\ t, (\Gamma \vdash ce : t \Leftrightarrow \mathcal{R}, \Gamma \vdash [\![ce]\!] : t)$$

From these lemmas, we prove that a list of equations preserves the same type after translation, provided that the register initialisation and update steps in the generated function, which may appear in the translated equations, are well typed in the environment $\mathcal{R}$. In addition, the names of the streams defined by these equations must all be distinct: for example, the same stream x cannot be defined twice, by two different equations, in the same node.

Lemma 5.1.3 (Type-Correctness of Equations)**.** $\forall\ \mathcal{R}\ \Gamma\ \vec{eqn}\ t$,

$$distinct(names(\vec{eqn})) \Rightarrow \mathcal{R}, \varnothing \vdash [\![\vec{eqn}]\!]_{inits} : t \Rightarrow \mathcal{R}, \Gamma \vdash [\![\vec{eqn}]\!]_{updates} : unit \Rightarrow \forall\ t', (\Gamma \vdash \vec{eqn} : t' \Leftrightarrow \mathcal{R}, \Gamma \vdash [\![\vec{eqn}]\!] : t')$$

We first reason about nodes for which all equations defined in their body are output streams. In other words, we initially handle nodes that have no streams whose scope is only local. A node *node* that defines no locally scoped stream then satisfies the predicate $nolocals(node)$:

$\boxed{nolocals(node)}$

$$\frac{\vec{y} = names(\vec{eqn})}{nolocals(\mathbf{node}\, f\, (\vec{x})\ \mathbf{returns}\ (\vec{y}) = \vec{eqn})}$$

Combining the previous lemmas proves the type-correctness lemma for a node, which states that, for any type t, if a node *node* is well formed, then it has type t if and only if its translation has type $unit \to t$ in an initially empty environment:

Lemma 5.1.4 (Type-Correctness of a Node (Without Locally Scoped Streams))**.**

$$\forall\ node\ t, \vdash_{wf} node \Rightarrow nolocals(node) \Rightarrow (\vdash node : t \Leftrightarrow \varnothing, \varnothing \vdash [\![node]\!] : unit \to t)$$

The previous lemma applies only to nodes for which no stream has local scope. In other words, the *pseudo-ML* function generated for such nodes returns a tuple containing exactly the list of all variables defined in the function. In order to extend this property to any node definition, including those that

use locally defined streams and therefore streams not contained in the list of output variables, we derive from the node typing rule presented in Figure 5.3 a new rule that in particular makes it possible to refer to the fact that the output streams are a subset of the streams computed in a node:

$\boxed{\vdash node : t}$

$$\frac{\vdash \textbf{node}\, f(\vec{x})\, \textbf{returns}\, (\vec{y}) = e\vec{q}n : \vec{t} \rightarrow \vec{t''} \qquad \vdash_{wf} \textbf{node}\, f(\vec{x})\, \textbf{returns}\, (\vec{y}) = e\vec{q}n \qquad (\vec{z} : \vec{t'}) \subseteq (\vec{y} : \vec{t''})}{\vdash \textbf{node}\, f(\vec{x})\, \textbf{returns}\, (\vec{z}) = e\vec{q}n : \vec{t} \rightarrow \vec{t'}}$$

We then finally derive the type-correctness lemma for a well-formed OCaLustre node, which may potentially contain locally scoped streams:

Lemma 5.1.5 (Type-Correctness of a Node)**.**

$$\forall\, node\ t, \vdash_{wf} node \Rightarrow (\vdash node : t \Leftrightarrow \varnothing, \varnothing \vdash [\![node]\!] : unit \rightarrow t)$$

We can therefore deduce that any OCaLustre program consistent with the typing rules of this language leads to the production of a *pseudo-ML* program that is itself correctly typed.

5.1.2 Translation Using Nested Declarations

In addition to the "*let ... with ... in ...*" construct discussed in the previous section, the *pseudo-ML* language has a "*let ... in ...*" construct that declares only a single variable, in the manner of OCaml. Thus, converting *pseudo-ML* code that uses simultaneous declarations into *pseudo-ML* code using a sequence of nested declarations would produce a program whose typing rules are similar to those of an OCaml program. Therefore, proving that the translation of OCaLustre programs to pseudo-ML preserves its type-correctness when "*let ... with ... in ...*" constructs are replaced by "*let ... in ...*" constructs verifies that the sequential OCaml program generated from an OCaLustre node, detailed earlier in Section 4.4, is itself well typed, since in this case the translation rules to *pseudo-ML* and to OCaml are very close[4].

The function $\ll \cdot \gg$, which converts a program containing simultaneous declarations into a program containing nested declarations, is described in Figure 5.5.

Nevertheless, typing equivalence between code containing simultaneous declarations and code containing nested declarations can be verified only if there exists a nesting order for the "*let*" constructs in which no variable that has not yet been declared is used by earlier variable declarations. For example, simply replacing "*let ... with ...*" with "*let ... in ...*" in the code fragment presented in the previous section would be incorrect, both in OCaml and in pseudo-ML, since the variable y would then be referenced in order to assign a value to the variable x before y had even been defined:

```
let x = y in let y = 2 in x
```

whereas changing the order of the declarations corresponds to correct code with the expected semantics:

[4]Even though some differences remain, such as the fact that certain variables are typed in a distinct environment.

$\boxed{\ll \delta_r \gg_{e'}}$

$$\ll \varnothing \gg_e \equiv e$$
$$\ll (y_r = e') \textbf{ with } \delta_r \gg_e \equiv \textbf{let } y_r = e' \textbf{ in } \ll \delta_r \gg_e$$

$\boxed{\ll \delta \gg_{e'}}$

$$\ll \varnothing \gg_e \equiv e$$
$$\ll (y = e') \textbf{ with } \delta \gg_e \equiv \textbf{let } y = e' \textbf{ in } \ll \delta \gg_e$$

$\boxed{\ll e' \gg}$

$$\ll \textbf{let } \delta \textbf{ in } e \gg \equiv \ll \delta \gg_{\ll e \gg}$$
$$\ll \textbf{let } \delta_r \textbf{ in } e \gg \equiv \ll \delta_r \gg_{\ll e \gg}$$
$$\ll \textbf{fun } \vec{x} \rightarrow\ e \gg \equiv \textbf{fun } \vec{x} \rightarrow \ll e \gg$$
$$\ll e \gg \equiv e$$

FIGURE 5.5: Function Converting Simultaneous Declarations to Nested Declarations

```
let y = 2 in let x = y in x
```

The conversion from simultaneous declarations to nested declarations is therefore valid only if there exists an order of appearance for the variables in the simultaneous declarations that avoids referring to a variable before its definition. This property is in fact exactly equivalent to the notion of "well scheduling" for an OCaLustre program. Indeed, the reordering step for OCaLustre programs is precisely intended to find a variable declaration order that makes it possible to define variables in sequential code.

Simultaneous *pseudo-ML* declarations arising from the translation of equations of an OCaLustre node can therefore be converted into nested declarations only if these equations are well scheduled:

Theorem 5.1.2 (Equivalence Between *let ... with* and *let* (Declarations))**.**

$$\forall\ \vec{x}\ \vec{t_x}\ \vec{eqn}, \delta = [\![\vec{eqn}]\!] \Rightarrow \vec{x} \cap\ vars(\delta) = \varnothing \Rightarrow distincts(vars(\delta)) \Rightarrow \vec{x} \vdash_{ws} eqns \Rightarrow$$
$$\forall\ \Gamma\ t,\ \Gamma = (\vec{x} : \vec{t_x}) \Rightarrow (\mathcal{R}, \Gamma \vdash lets\ \delta\ in\ e : t \Leftrightarrow \mathcal{R}, \Gamma \vdash \ll \delta \gg_e : t)$$

Consequently, any *pseudo-ML* program, arising from the translation of a well-scheduled OCaLustre node, with simultaneous declarations is well typed if and only if an identical program in which the simultaneous declarations are converted into nested declarations is also well typed, and has the same type:

Lemma 5.1.6 (Equivalence Between let with and let (Node))**.**

$$\forall\ node\ t,\ \vdash_{ws} node \Rightarrow (\vdash [\![node]\!] : t \Leftrightarrow \vdash \ll [\![node]\!] \gg : t)$$

In conclusion, from the lemmas and theorems stated in this section, we can deduce that any well-scheduled OCaLustre node is well typed if and only if its translation into an ML language with nested declarations, such as OCaml, is also well typed:

Theorem 5.1.3 (Type-Correctness of a Node Translated to OCaml)**.**

$$\forall\ node\ t,\ \vdash_{ws} node \Rightarrow (\vdash node : t \Leftrightarrow \vdash \ll [\![node]\!] \gg : t)$$

This theorem validates the fact that it is not really necessary to implement a type checker specific to OCaLustre, since the OCaml code generated by compiling a node contains all of that node's typing information. Thus, any node that is ill typed in OCaLustre will be detected by the OCaml compiler's type checker. To extend this correctness property to complete OCaLustre programs, however, it would be necessary to consider in particular the mechanism of node calls, which is converted into function applications in OCaml.

5.2 Clock-Type Verification

The synchronous-clock system integrated into the OCaLustre language allows the developer to associate *presence conditions* with each stream manipulated by a synchronous program. The clock-typing semantics, defined in Section 3.2.4, constrain the language operators to operate, for the most part, only on streams clocked by the same clock. Respecting this semantics therefore ensures that no absent stream value is read during program execution, avoiding erroneous and unpredictable behaviour.

To assign a clock to each stream of the language, the OCaLustre compiler implements an inference algorithm modelled on those implemented by Heptagon and Scade and derived from the work of Colaço and Pouzet [CP03], itself based on the algorithm $\mathcal{W}$ described by Milner [Mil78]. This algorithm statically associates each expression of the language with a synchronous clock while respecting the clock-typing semantics.

In order to formally ensure that the clocks inferred by the compiler do indeed respect the clocking semantics of the language, this section proposes a *clock checker* for OCaLustre. The role of this checker, integrated into the OCaLustre compiler, is to read an abstract syntax tree in which each expression is annotated with its synchronous clock, inferred by the clock-typing algorithm, and to indicate whether this AST is correct with respect to the typing semantics of synchronous clocks. Validation by the clock checker therefore contributes to the safety of the language by ensuring that the inferred clocks conform to the formal specification given in Section 3.2.4.

The resulting clock checker therefore takes its place in the compilation scheme of an OCaLustre program immediately after the clock-inference process (Fig. 5.6): if the inferred clocks are consistent with the typing rules known to the checker, then compilation of an OCaLustre program can continue. Otherwise, the compilation process is stopped and the compiler returns an error.

The solution detailed in this section therefore consists in an *a posteriori* verification of the consistency of the clocks assigned to expressions, and thus represents a compiler that *certifies* [PSS98] the safety of the inferred clock types. Such a method is an alternative to a *certified* compiler for which the correctness proof of the clock-inference algorithm implemented in OCaLustre would have been carried out.

It should be noted that checking the correct clocking of a node is compatible with the possibility that this node may have absent inputs or outputs. In this respect, our work is close to recent work that continues the effort to design a certified Lustre compiler by adding *clocked arguments* [BP19].

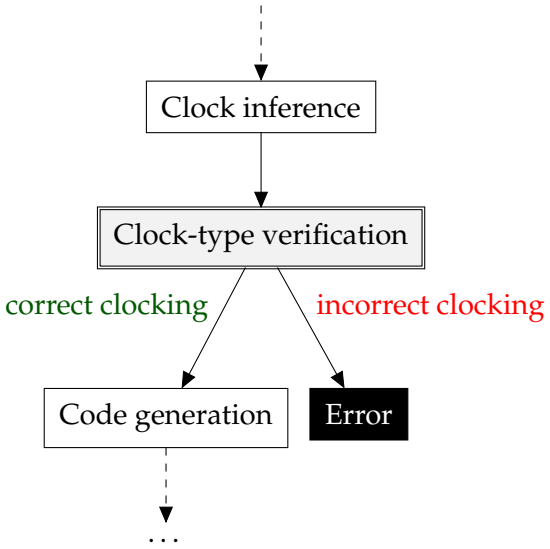


FIGURE 5.6: Insertion of the Clock Checker into the Compilation Chain of an OCaLustre Program

5.2.1 Algorithmic Rules for Correct Clocking

The OCaLustre clock checker ensures that the program, represented as the clock-annotated AST described in Section 4, respects a typing semantics representing correct stream clocking. This semantics, derived from the typing semantics presented in Section 3.2.4, takes into account the annotations added to the OCaLustre program AST during compilation in order to check its correct clocking.

The *algorithmic* clocking rules for expressions, in this version of the clock-annotated AST, are shown in Figure 5.7. They are almost identical to those for unannotated expressions, except for the clocking rules for the *unit* value, a constant, or a type constructor, which take these annotations into account. For example, whereas in its unannotated version a constant was compatible with any clock type, a constant k annotated here with a clock ck has clock type ck:

$$\frac{}{\mathcal{C} \vdash k^{ck} : ck}\ \textsc{Const}$$

The full set of clocking rules concerning the other syntactic categories is given in Figure 5.8. These clocking rules are mostly derived directly from the rules presented in Section 3.2.4; however, the rule concerning the application of a node is more complex. As described when presenting OCaLustre's clock system, it must take into account the conditional-application mechanism, as well as the various substitutions applied to variable names in clock types.

$\boxed{\mathcal{C} \vdash e : ck}$

$$\frac{}{\mathcal{C} \vdash ()^{ck} : ck}\ \textsc{Unit} \qquad \frac{}{\mathcal{C} \vdash k^{ck} : ck}\ \textsc{Const} \qquad \frac{}{\mathcal{C} \vdash X_i^{ck} : ck}\ \textsc{Constr}$$

$$\frac{\mathcal{C}(x) = ck}{\mathcal{C} \vdash x : ck}\ \textsc{Var} \qquad \frac{\mathcal{C} \vdash e : ck}{\mathcal{C} \vdash \square\, e : ck}\ \textsc{Unop} \qquad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash e' : ck}{\mathcal{C} \vdash e \diamond e' : ck}\ \textsc{Binop}$$

$$\frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash x : ck}{\mathcal{C} \vdash e\ \mathbf{when}\ x : ck\ \mathbf{on}\ x}\ \textsc{When} \qquad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash x : ck}{\mathcal{C} \vdash e\ \mathbf{whennot}\ x : ck\ \mathbf{onnot}\ x}\ \textsc{Whennot}$$

$\boxed{\mathcal{C} \vdash \vec{e} : ck}$

$$\frac{\mathcal{C} \vdash e : ck}{\mathcal{C} \vdash [e] : ck}\ \textsc{One\_E} \qquad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash \vec{e} : ck'}{\mathcal{C} \vdash e, \vec{e} : ck \times ck'}\ \textsc{Cons\_E}$$

$\boxed{\mathcal{C} \vdash_{ce} ce : ck}$

$$\frac{\mathcal{C} \vdash e : ck}{\mathcal{C} \vdash_{ce} e : ck}\ \textsc{Exp} \qquad \frac{\mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash_{ce} ce : ck \quad \mathcal{C} \vdash_{ce} ce' : ck}{\mathcal{C} \vdash_{ce} \mathbf{if}\ e\ \mathbf{then}\ ce\ \mathbf{else}\ ce' : ck}\ \textsc{If}$$

$$\frac{\mathcal{C} \vdash x : ck \quad \mathcal{C} \vdash_{ce} ce : ck\ \mathbf{on}\ x \quad \mathcal{C} \vdash_{ce} ce' : ck\ \mathbf{onnot}\ x}{\mathcal{C} \vdash_{ce} \mathbf{merge}\ x\ ce\ ce' : ck}\ \textsc{Merge}$$

Figure 5.7: Clocking Rules for Expressions in Clock-Annotated Normal Form

To represent these substitutions, we introduce the judgement $\vec{x} \underset{\mathcal{S}}{\sim} \vec{e} \triangleright \sigma$, which indicates that, for a set of carriers $\mathcal{S}$, a list of formal parameters $\vec{x}$, and a list of actual arguments $\vec{e}$, σ is the substitution from the variable names present in the list of formal parameters of the application to the names of its arguments.

The inductive rules associated with this judgement are the following:

$\boxed{\vec{y} \underset{\mathcal{S}}{\sim} \vec{e} \triangleright \sigma}$

$$\frac{x \in \mathcal{S}}{x \underset{\mathcal{S}}{\sim} [y] \triangleright \{x \mapsto y\}}\ \textsc{Sub\_E\_in} \qquad \frac{x \notin \mathcal{S}}{x \underset{\mathcal{S}}{\sim} [e] \triangleright \varnothing}\ \textsc{Sub\_E\_notin}$$

$$\frac{}{() \underset{\mathcal{S}}{\sim} [()^{ck}] \triangleright \varnothing}\ \textsc{Sub\_E\_unit} \qquad \frac{y \underset{\mathcal{S}}{\sim} [e] \triangleright \sigma \quad \vec{y} \underset{\mathcal{S}}{\sim} \vec{e} \triangleright \sigma'}{y, \vec{y} \underset{\mathcal{S}}{\sim} e, \vec{e} \triangleright \sigma \oplus \sigma'}\ \textsc{Sub\_E\_list}$$

For example, if a node f has signature $(x : \bullet) \to (y : \bullet\ \text{on}\ x)$, then the application $f\ z$ causes x to be substituted by z when instantiating the type of f in this context. Consequently, in $f\ z$, f has type $\bullet \to \bullet$ on z. We can therefore formally derive the judgement $x \underset{\{x\}}{\sim} z \triangleright \{x \mapsto z\}$.

Because the output streams of a node may be clocked by other output streams, it is also necessary to substitute the names of a node's output parameters with the actual names on the left-hand side of the equality symbol in the equation corresponding to a node application. The judgement $\vec{y} \underset{\mathcal{S}}{\sim} \vec{y'} \triangleright \sigma$ therefore means that, for a set of carriers $\mathcal{S}$, σ is the substitution from the names of the formal output parameters $\vec{y}$ to the actual names of the output streams $\vec{y'}$, corresponding to the names of the variables on the left-hand side of the equality sign in the equation concerned. The inductive rules governing this

$\boxed{\mathcal{C} \vdash \vec{y} : ck}$

$$\frac{}{\mathcal{C} \vdash () : \bullet}\ \textsc{NNil} \qquad \frac{\mathcal{C}(y) = ck}{\mathcal{C} \vdash y : ck}\ \textsc{NVariable} \qquad \frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash \vec{y} : ck'}{\mathcal{C} \vdash y, \vec{y} : ck \times ck'}\ \textsc{NList}$$

$\boxed{\mathcal{H}, \mathcal{C} \vdash eqn}$

$$\frac{\vec{x} \underset{\mathcal{S}}{\sim} \vec{e} \rhd \sigma_1 \quad ck_1 = \sigma_1(ck'_1[ck]) \quad \vec{y} \underset{\mathcal{S}}{\sim} \vec{y'} \rhd \sigma_2 \quad ck_2 = \sigma_2 \oplus \sigma_1(ck'_2[ck])}{ck_1 \to ck_2 = \underset{(\vec{e},\vec{y'},ck)}{inst}\left(\forall \bullet.(\vec{x}{:}ck'_1) \xrightarrow{\mathcal{S}} (\vec{y}{:}ck'_2)\right)}\ \textsc{Inst}$$

$$\frac{\mathcal{H}(f) = \forall \bullet .(\vec{x} : ck) \to (\vec{y} : ck') \quad \mathcal{S} = carriers(ck) +\!\!+ carriers(ck')}{\mathcal{H} \vdash f : \forall \bullet .(\vec{x} : ck) \xrightarrow{\mathcal{S}} (\vec{y} : ck')}\ \textsc{Sign}$$

$$\frac{\mathcal{H} \vdash f : \forall \bullet .(\vec{x} : ck'_1) \xrightarrow{\mathcal{S}} (\vec{y'} : ck'_2) \quad ck_1 \to ck_2 = \underset{(\vec{e},\vec{y'},ck)}{inst}\left(\forall \bullet.(\vec{x}{:}ck'_1) \xrightarrow{\mathcal{S}} (\vec{y}{:}ck'_2)\right) \quad \mathcal{C} \vdash \vec{e} : ck_1 \quad \mathcal{C} \vdash \vec{y'} : ck_2}{\mathcal{H}, \mathcal{C} \vdash \vec{y'} \underset{ck}{=} f(\vec{e})}\ \textsc{App}$$

$$\frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash_{ce} ce : ck}{\mathcal{H}, \mathcal{C} \vdash y \underset{ck}{=} ce}\ \textsc{Expr} \qquad \frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash e : ck}{\mathcal{H}, \mathcal{C} \vdash y \underset{ck}{=} k\ \textbf{fby}\ e}\ \textsc{Fby}$$

$$\frac{\mathcal{C} \vdash y : ck \quad \mathcal{C} \vdash e : ck \quad \mathcal{C} \vdash e_1 : ck \quad .. \quad \mathcal{C} \vdash e_{n-1} : ck}{\mathcal{H}, \mathcal{C} \vdash y \underset{ck}{=} \textbf{call}\ \mathtt{f}\ e\ e_1\ ..\ e_{n-1}}\ \textsc{Call}$$

$\boxed{\mathcal{H}, \mathcal{C} \vdash \vec{eqn}}$

$$\frac{\mathcal{H}, \mathcal{C} \vdash eqn}{\mathcal{H}, \mathcal{C} \vdash [eqn]}\ \textsc{OneEqn} \qquad \frac{\mathcal{H}, \mathcal{C} \vdash eqn \quad \mathcal{H}, \mathcal{C} \vdash \vec{eqn}}{\mathcal{H}, \mathcal{C} \vdash eqn;\ \vec{eqn}}\ \textsc{ConsEqns}$$

$\boxed{\mathcal{H}, \mathcal{C} \vdash node : \omega}$

$$\frac{\mathcal{H}, \mathcal{C} \vdash \vec{eqn} \quad \mathcal{C} \vdash \vec{x} : ck \quad \mathcal{C} \vdash \vec{y} : ck'}{\mathcal{H}, \mathcal{C} \vdash \textbf{node}\, f(\vec{x})\, \textbf{return}\, (\vec{y}) = \vec{eqn} : \forall \bullet .(\vec{x} : ck) \to (\vec{y} : ck')}\ \textsc{Node}$$

$\boxed{\mathcal{H} \vdash program}$

$$\frac{}{\mathcal{H} \vdash \varnothing}\ \textsc{EmptyProg} \qquad \frac{\mathcal{H}, \mathcal{C} \vdash \textbf{node}\, f(\vec{x})\, \textbf{return}\, (\vec{y}) = \vec{eqn} : \omega \quad (f : \omega) \cup \mathcal{H} \vdash \vec{nodes}}{\mathcal{H} \vdash (\mathcal{C}, \textbf{node}\, f(\vec{x})\, \textbf{return}\, (\vec{y}) = \vec{eqn});; \vec{nodes}}\ \textsc{ConsProg}$$

Figure 5.8: Clocking Rules in Clock-Annotated Normal Form (Continued)

judgement concern a different syntactic category from the previous rules: they handle lists of variable names $\vec{y}$ rather than expressions e. These rules are the following:

$$\boxed{\vec{y} \underset{\mathcal{S}}{\sim} \vec{y'} \rhd \sigma}$$

$$\frac{y \in \mathcal{S}}{y \underset{\mathcal{S}}{\sim} y' \rhd \{y \mapsto y'\}}\ \textsc{Sub\_V\_in} \qquad \frac{y \notin \mathcal{S}}{y \underset{\mathcal{S}}{\sim} y' \rhd \varnothing}\ \textsc{Sub\_V\_notin}$$

$$\frac{}{() \underset{\mathcal{S}}{\sim} () \rhd \varnothing}\ \textsc{Sub\_V\_nil} \qquad \frac{y \underset{\mathcal{S}}{\sim} y' \rhd \sigma \quad \vec{y} \underset{\mathcal{S}}{\sim} \vec{y'} \rhd \sigma'}{y, \vec{y} \underset{\mathcal{S}}{\sim} y', \vec{y'} \rhd \sigma \oplus \sigma'}\ \textsc{Sub\_V\_list}$$

For example, if the node g has signature $(x : \bullet) \to ((y : \bullet) \times (z : \bullet \text{ on } y))$, then in the equation $(a, b) = g(32)$ the type of the instance of g is $\bullet \to (\bullet \times (\bullet \text{ on } a))$, because a is the actual name used to refer to the first output value of g. This example induces the judgement $(y, z) \underset{\{y\}}{\sim} (a, b) \triangleright \{y \mapsto a\}$.

We also recall that the conditional-application mechanism of a node consists in replacing the base clock in the signature of a node with a slower clock in order to slow its execution. We write $\vec{ck}[c]$ for the substitution of all occurrences of the base clock ($\bullet$) by the clock c in $\vec{ck}$.

$\boxed{ck[ck']}$

$$
\begin{aligned}
&\bullet[ck'] \equiv ck' \\
&(ck\ \mathbf{on}\ x)[ck'] \equiv (ck[ck'])\ \mathbf{on}\ x \\
&(ck\ \mathbf{onnot}\ x)[ck'] \equiv (ck[ck'])\ \mathbf{onnot}\ x \\
&(ck_1 \to ck_2)[ck'] \equiv (ck_1[ck']) \to (ck_2[ck']) \\
&(ck_1 \times ck_2)[ck'] \equiv (ck_1[ck']) \times (ck_2[ck'])
\end{aligned}
$$

In addition, we give the definition of a function *carriers*, which computes the list of carriers contained in a clock type:

$\boxed{carriers(ck)}$

$$
\begin{aligned}
&carriers(\bullet) \equiv \varnothing \\
&carriers(ck\ \mathbf{on}\ x) \equiv x :: carriers(ck) \\
&carriers(ck\ \mathbf{onnot}\ x) \equiv x :: carriers(ck) \\
&carriers(ck_1 \to ck_2) \equiv carriers(ck_1) +\!\!+ carriers(ck_2) \\
&carriers(ck_1 \times ck_2) \equiv carriers(ck_1) +\!\!+ carriers(ck_2)
\end{aligned}
$$

To make the rule for node application easier to understand, we separate it into three distinct rules:

1. A first rule, Sign

$$\frac{\mathcal{H}(f) = \forall \bullet .(\vec{x} : ck) \to (\vec{y} : ck') \quad \mathcal{S} = carriers(ck) +\!\!+ carriers(ck')}{\mathcal{H} \vdash f : \forall \bullet .(\vec{x} : ck) \xrightarrow{\mathcal{S}} (\vec{y} : ck')}\ \textsc{Sign}$$

   retrieves the information needed for typing the application. It states that, in a global typing environment $\mathcal{H}$, if the function f has signature $(\vec{x} : ck) \to (\vec{y} : ck')$, and if the set $\mathcal{S}$ corresponds to the carriers in ck and ck', then one can deduce the judgement $f : (\vec{x} : ck) \xrightarrow{\mathcal{S}} (\vec{y} : ck')$, which associates the signature of f with its list of carriers.

2. A second rule, the instantiation rule Inst:

$$\frac{\vec{x} \underset{\mathcal{S}}{\sim} \vec{e} \rhd \sigma_1 \quad ck_1 = \sigma_1(ck'_1[ck]) \quad \vec{y} \underset{\mathcal{S}}{\sim} \vec{y'} \rhd \sigma_2 \quad ck_2 = \sigma_2 \oplus \sigma_1(ck'_2[ck])}{ck_1 \to ck_2 = \underset{(\vec{e},\vec{y'},ck)}{inst}\left(\forall \bullet .(\vec{x} : ck'_1) \xrightarrow{\mathcal{S}} (\vec{y} : ck'_2)\right)}\ \textsc{Inst}$$

   states that, for a node with signature $(\vec{x} : ck'_1) \to (\vec{y} : ck'_2)$ possessing a set $\mathcal{S}$ of carriers, if the following conditions are satisfied:

   - σ_1 is the substitution from the names of the node's formal parameters $\vec{x}$ to the names of its arguments $\vec{e}$.
   - The clock ck_1 corresponds to the result of applying the substitution σ_1 to ck'_1, as well as substituting the base clock by the clock ck of the node's conditional application.

- σ_2 is the substitution from the formal names $\vec{y}$ of the node's output streams to the names of the variables $\vec{y'}$ actually present on the left-hand side of the equality sign in the equation that performs the application.
- The clock ck_2 corresponds to the result of applying the substitutions σ_1 and σ_2 to ck'_2, as well as substituting the base clock by the clock ck of the conditional application of the node call.

then the type $ck_1 \to ck_2$ is a valid *instance* of the type of the node under consideration.

3. Finally, the third rule, App:

$$\frac{\mathcal{H} \vdash f : \forall \bullet .(\vec{x} : ck'_1) \xrightarrow{\mathcal{S}} (\vec{y'} : ck'_2) \quad ck_1 \to ck_2 = \underset{(\vec{e},\vec{y'},ck)}{inst} \left(\forall \bullet .(\vec{x}:ck'_1) \xrightarrow{\mathcal{S}} (\vec{y}:ck'_2)\right) \quad \mathcal{C} \vdash \vec{e} : ck_1 \quad \mathcal{C} \vdash \vec{y'} : ck_2}{\mathcal{H}, \mathcal{C} \vdash \vec{y'} \underset{ck}{=} f(\vec{e})} \textsc{App}$$

combines the two preceding rules: it states, for a node f, that if the type $ck_1 \to ck_2$ is an instance of the signature of f, and if the arguments of the application have type ck_1 and the returned values have type ck_2, then the application $\vec{y'} = f(\vec{e})$, whose conditional-application clock is ck, is well clocked.

As an example, Figure 5.9 represents the derivation tree for the conditional application of the node `cpt` with the expression `1 when c` as argument. In this example, no substitution between parameter names and argument names is necessary, since the signature of the node `cpt` contains no carrier. Nevertheless, for the conditional-application mechanism to be consistent with clock typing, the base clock $\bullet$ is substituted by the clock of the expression `1 when c`, that is, ($\bullet$ on c).

5.2.2 Extracting the Clock Checker to OCaml

Once the clock-typing rules for the annotated normal form have been defined, the preliminary step in producing a type checker relies on extracting the rules defined above to Coq. Using Coq extraction for the rules described with Ott makes it possible to translate them into Coq *inductives*. For example, the following excerpt is the inductive corresponding to the extraction to Coq of the clocking rule associated with applying a binary arithmetic operator.

```
Inductive clk_exp: C → exp → clock → Prop:=
  (...)
  | Binop: forall (C:C) (e:exp) (op:operator) (e':exp) (ck:clock),
     clk_exp C e ck →
     clk_exp C e' ck →
     clk_exp C (Ebinop e op e') ck.
```

From the different rules governing the correct clocking of programs, described in the previous section, Ott produces the following Coq inductives:

- *clk_exp*, where *clk_exp C e ck* corresponds to the judgement $\mathcal{C} \vdash e : ck$.

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\mathcal{H} \vdash cpt : (incr : \bullet) \to (n : \bullet) \qquad \varnothing = carriers(\bullet) +\!\!+ carriers(\bullet)}{\mathcal{H} \vdash cpt : (incr : \bullet) \xrightarrow{\varnothing} (n : \bullet)}\ \textsc{Sign} \qquad (\ \textsc{Inst}\ )}{\dfrac{\dfrac{}{\mathcal{C} \vdash 1^\bullet : \bullet}\ \textsc{Const} \qquad \dfrac{(x, \bullet \textbf{ on } c) \in \mathcal{C}}{\mathcal{C} \vdash c : \bullet}\ \textsc{Var}}{\mathcal{C} \vdash (1^\bullet \textbf{ when } c) : \bullet \textbf{ on } c}\ \textsc{When} \qquad \dfrac{(x, \bullet \textbf{ on } c) \in \mathcal{C}}{\mathcal{C} \vdash x : \bullet \textbf{ on } c}\ \textsc{Var}}{\mathcal{H}, \mathcal{C} \vdash x \underset{\bullet \textbf{ on } c}{=} cpt(1^\bullet \textbf{ when } c)}\ \textsc{App}}{\dfrac{incr \notin \varnothing}{incr \underset{\varnothing}{\sim} (1^\bullet \textbf{ when } c) \rhd \varnothing} \qquad \bullet \textbf{ on } c \ = \bullet [\bullet \textbf{ on } c] \qquad \dfrac{n \notin \varnothing}{n \underset{\varnothing}{\sim} x \rhd \varnothing} \qquad \bullet \textbf{ on } c \ = \bullet [\bullet \textbf{ on } c]}\ \textsc{Inst}}{(\bullet \textbf{ on } c) \to (\bullet \textbf{ on } c) = \underset{(1^\bullet \textbf{ when } c, x, \bullet \textbf{ on } c)}{inst}((incr : \bullet) \xrightarrow{\varnothing} (n : \bullet))}$$

Figure 5.9: Example Derivation of the Rules Inst and App for Conditional Application

- *clk_cexp*, where *clk_cexp* $\mathcal{C}$ *ce ck* corresponds to the judgement $\mathcal{C} \vdash ce : ck$.
- *Well_clocked_eq*, where *Well_clocked_eq* $\mathcal{H}$ $\mathcal{C}$ *eqn* states that an equation is well clocked in the global environment $\mathcal{H}$ and the local environment $\mathcal{C}$, i.e. $\mathcal{H}, \mathcal{C} \vdash eqn$.
- *Well_clocked_node*, where *Well_clocked_node* $\mathcal{H}$ $\mathcal{C}$ *node* states that a node *node* is well clocked in the global environment $\mathcal{H}$ and the local environment $\mathcal{C}$, i.e. $\mathcal{H}, \mathcal{C} \vdash node$.

Generating inductive relations is, however, insufficient for our use case: we want to produce a type checker, which must therefore be *executable*, so that it can form a software step in the compilation sequence of an OCaLustre program. In order to obtain a computational and executable version of the various clock-typing rules, functional versions of these rules were also implemented in Coq.

For example, the following excerpt from the recursive function `clockof_exp` is the Coq code corresponding to the computation of the clock of the application of a binary operator:

```
Fixpoint clockof_exp (C: clockenv) (e:exp):=
  match e with
    (...)
    | Ebinop e1 op e2 ⇒
      let c1:= clockof_exp C e1 in
      let c2:= clockof_exp C e2 in
      match c1,c2 with
      | Some a, Some b ⇒ if clock_eqb a b then Some a else None
      end
    end.
```

It is then fundamental, in order to guarantee the correctness of our approach, that these executable functions respect the typing semantics formally defined by the inductive rules in this section. Certification that these rules are respected relies on the Coq proof of equivalence between the inductive versions and the functional versions of these rules. We therefore proved the following equivalences in the Coq proof assistant:

- At the expression level, if, in a local environment $\mathcal{C}$, an expression e is associated with the clock ck in the inductive relation *clk_exp*, then the function *clockof_exp*, applied to e, also returns the clock ck:

$$\forall\ \mathcal{C}\ e\ ck,\ \text{clockof\_exp}\ \mathcal{C}\ e = \mathit{Some}\ ck \Leftrightarrow \mathit{clk\_exp}\ \mathcal{C}\ e\ ck$$

- This equivalence makes it possible to deduce that, at the level of control expressions, if, in a local environment $\mathcal{C}$, an expression ce is associated with the clock ck in the inductive relation *clk_exp*, then the function *clockof_cexp*, applied to ce, also returns the clock ck:

$$\forall\ \mathcal{C}\ e\ ck,\ \text{clockof\_cexp}\ \mathcal{C}\ e = \mathit{Some}\ ck \Leftrightarrow \mathit{clk\_cexp}\ \mathcal{C}\ e\ ck$$

- From these relations, we can deduce that if, for a global clock-typing environment $\mathcal{H}$ and a local environment $\mathcal{C}$, an equation *eqn* is well clocked in the inductive version of the rules, then it is also well clocked in the functional version:

$$\forall\ \mathcal{H}\ \mathcal{C}\ eqn,\ \text{well\_clocked\_eq}\ \mathcal{H}\ \mathcal{C}\ eqn \Leftrightarrow Well\_clocked\_eq\ \mathcal{H}\ \mathcal{C}\ eqn$$

- We can finally deduce that if a node *node* is well clocked in the inductive version of the rules, then it is well clocked in their functional version:

$$\forall\ \mathcal{H}\ \mathcal{C}\ node,\ \text{well\_clocked\_node}\ \mathcal{H}\ \mathcal{C}\ node \Leftrightarrow Well\_clocked\_node\ \mathcal{H}\ \mathcal{C}\ node$$

The Coq sources representing all the definitions and proofs leading to this result are available online [⚓1].

Once formally proved, these properties give us the assurance that our manually defined executable functions respect the clocking semantics represented by the inductive rules originally passed to Ott. Thus, any extraction of these functions to executable code will respect this same semantics.

The Coq functions are then extracted to ordinary OCaml functions. For example, the previous excerpt handling the clocking of a binary operator is converted into the following OCaml code excerpt:

```
let rec clockof_exp c = function
(...)
| Ebinop (e1, _, e2) ->
  let c1 = clockof_exp c e1 in
  let c2 = clockof_exp c e2 in
  (match c1 with
   | Some a ->
     (match c2 with
      | Some b -> if clock_eqb a b then Some a else None
      | None -> None)
   | None -> None)
```

The code extracted from Coq is then connected by "glue" code capable of converting certain incompatible types arising from this extraction mechanism; for example, Coq extracts character strings to lists of characters, which does not correspond to the basic `string` type of the OCaml language. Finally, the code of the clock checker is inserted into the compilation chain, and for each node definition it is checked that it respects the clocking semantics of OCaLustre programs by applying the function well_clocked_node. If this function returns the value **`false`**, the compiler generates an error and program compilation stops.

The clock checker is enabled by the OCaLustre compiler option `-check_clocks`. For example, checking the node `merger`, which simply applies its three parameters *c*, *a*, and *b* to the **`merge`** operator, produces the following output:

```
$ ocamlc -ppx "ocalustre -check_clocks" tests/merger.ml
merger:: (c:base * a:(base on c) * b:(base onnot c)) -> (d:base)
Checking of merger: OK
```

The checker thus makes it possible to ensure that the clock typer implemented in OCaLustre, which infers the clocks of each equation, deduces for each of them a clock consistent with the synchronous-clock typing system. This verification is an alternative to producing a verified typer: if OCaLustre's clock typer works correctly, then the checker systematically computes the value *true*. In the case where the checker returned the value *false*, this property could not be attested. For all the examples present in this manuscript, as well as for the tests carried out during our work, the clock checker attests that inference is correct. In the long term, this verification could make it possible to ensure that no absent value is mistakenly manipulated by the program during execution. This property would nevertheless require connecting the operational semantics of the language to its static clock-typing semantics, as discussed in Section 3.2.5.

Chapter Conclusion

The properties verified in this section provide guarantees about the programs produced. These guarantees mainly concern the typing of data manipulated by an OCaLustre program, whether this means the standard typing of values or the clocking of the program's various synchronous components. Thanks to the Coq proof of these properties, we can ensure that the typing of an OCaLustre program provides the safety expected by the language specification. Such guarantees are welcome in a critical embedded setting where user safety is at stake. Moreover, additional static analyses, taking advantage of OMicroB's portable approach, can help ensure the correct behaviour of an embedded program. In the following chapter, we present an analysis that bounds the execution time of an OCaLustre program. As in the approach adopted in this chapter, we verify the chosen method for this analysis using Coq.

6 Computing the Worst-Case Execution Time of an OCaLustre Program

The virtual-machine approach adopted in our work allows us to *factorise* several static analyses that can be performed on programs independently of the target hardware architectures. Indeed, because executable programs are represented as bytecode, common to all implementations of the virtual machine, these analyses can be performed directly on the generated bytecode files, abstracting away from the hardware on which the program runs. In this chapter, we illustrate one such analysis, which estimates the worst-case execution time (*WCET*) of an OCaLustre program. The method used takes advantage of the very simple memory model of microcontrollers, and is therefore based on the *composability* of the execution time of the program bytecode: analysing the distinct costs of each instruction of the program makes it possible to deduce its total cost. This analysis, which can easily be adapted to the execution of a program on microcontrollers of different models or architectures, is therefore compatible with the portability of the virtual-machine approach.

In what follows, we present and formalise, over an idealised bytecode, the method used to measure the worst-case execution time of an OCaLustre program, and prove its correctness using Coq. This method is then applied to the real language of OCaml bytecode instructions using a software tool called *Bytecrawler*, whose operation we describe. Finally, we discuss the limitations of the compatibility between WCET analysis and the compilation model described above, and present a *dedicated mode* for generating OCaml code that makes it possible, without changing its semantics, to estimate the maximum execution time of any OCaLustre program.

6.1 Formal Validation of the Method in Coq

In this section, we describe and formalise the method we use to compute the worst-case execution time of a bytecode program obtained by compiling an OCaLustre program. For simplicity, both the formalisation and the correctness proof of the method are carried out using an *idealised* bytecode language, reduced to a few imperative instructions. We conjecture that the results considered on this idealised bytecode language can be transposed to the concrete subset of OCaml bytecode instructions generated by compiling an OCaLustre program.

6.1.1 Definition of an Idealised Bytecode Language

The idealised bytecode language contains the following seven instructions:

$$instr ::= \mathsf{Init}\ x\ v \mid \mathsf{Assign}\ x\ y \mid \mathsf{Add}\ x\ y \mid \mathsf{Sub}\ x\ y \mid \mathsf{Branch}\ v \mid \mathsf{Branchif}\ x\ v \mid \mathsf{Stop}$$

- The instruction Init initialises a variable with a value, for example $x = 3$.
- The instruction Assign changes the value of a variable to the value of another variable ($x = y$)[1].
- The instruction Add increments the value of a variable x by that of a variable y, i.e. x += y.
- The instruction Sub decrements the value of a variable x by that of a variable y, i.e. x -= y.
- The instruction Branch performs a jump in the code.
- The instruction Branchif performs a jump if a given variable is different from 0.
- The instruction Stop stops program execution.

The values manipulated by the language are only integer values:

$$values, v, w ::= int\_literal \mid v + w \mid v - w$$

A state σ of a program corresponds to a triple containing the program code P, a structure that associates each address with the corresponding instruction of the language, a program counter pc, and a memory M, a structure associating variables with integer values:

$$\sigma \quad ::= \quad (P, pc, M)$$

The small-step operational-semantics evaluation rules of this language are defined in Figure 6.1.

Execution Traces

In what follows, we consider the sequence of all states encountered during the execution of a program written in this language. We call this sequence an *execution trace*.

Definition 6.1.1 (Execution Trace)**.** *An execution trace $\mathcal{T}$ of a program is a sequence of states in which each state is the image of the previous one by a transition in the operational semantics of the idealised language.*

Our method concerns only *finite* execution traces of a program. It would not be possible to analyse programs whose execution is infinite, since by definition their execution time could not be bounded.

Definition 6.1.2 (Finite Execution Trace)**.** *An execution trace $\mathcal{T}$ is finite, written $finite(\mathcal{T})$, if it terminates with the instruction* Stop*.*

At program execution time, all variable values are known, and the execution trace is *unique*. Moreover, when execution terminates correctly, the last instruction of the associated trace is the instruction Stop. Such a trace is called a *deterministic execution trace*.

[1] Access to a variable takes constant time.

$\boxed{\sigma \longrightarrow \sigma'}$

$$\frac{P[pc] = \mathsf{Init}\ x\ v}{(P, pc, M) \longrightarrow (P, pc + 1, M[x := v])}$$

$$\frac{P[pc] = \mathsf{Assign}\ x\ y \quad M[y] = v}{(P, pc, M) \longrightarrow (P, pc + 1, M[x := v])}$$

$$\frac{P[pc] = \mathsf{Add}\ x\ y \quad M[x] = v \quad M[y] = w}{(P, pc, M) \longrightarrow (P, pc + 1, M[x := v{+}w])}$$

$$\frac{P[pc] = \mathsf{Sub}\ x\ y \quad M[x] = v \quad M[y] = w}{(P, pc, M) \longrightarrow (P, pc + 1, M[x := v{-}w])}$$

$$\frac{P[pc] = \mathsf{Branch}\ v}{(P, pc, M) \longrightarrow (P, v, M)}$$

$$\frac{P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = 0}{(P, pc, M) \longrightarrow (P, pc + 1, M)}$$

$$\frac{P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] \neq 0}{(P, pc, M) \longrightarrow (P, v, M)}$$

FIGURE 6.1: Operational Semantics of the Idealised Language

Definition 6.1.3 (Deterministic Execution Trace)**.** *The function run, applied to a state σ, returns the deterministic execution trace whose initial state is σ:*

$\boxed{run(\sigma) = \mathcal{T}}$

$$\frac{P[pc] = \mathsf{Stop}}{run((P, pc, M)) = [(P, pc, M)]} \qquad \frac{\sigma \longrightarrow \sigma' \quad run(\sigma') = \mathcal{T}}{run(\sigma) = \sigma\ ::\ \mathcal{T}}$$

For example, consider the following program P:

$$[0 : \mathsf{Init}\ \text{"x"}\ 4\ ;\ 1 : \mathsf{Branchif}\ \text{"x"}\ 3\ ;\ 2 : \mathsf{Branch}\ 0\ ;\ 3 : \mathsf{Add}\ \text{"x"}\ \text{"x"};\ 4 : \mathsf{Stop}]$$

The sequence of states $\mathcal{T}$ therefore constitutes an execution trace of the program P, with as its initial state the state composed of P, a program counter initialised to 0, and an empty memory, i.e. $(P, 0, \varnothing)$:

$$\mathcal{T} = run((P, 0, \varnothing)) = [(P, 0, \varnothing)\ ;\ (P, 1, [x = 4])\ ;\ (P, 3, [x = 4])\ ;\ (P, 4, [x = 8])]$$

Costs

We associate a *cost* with each instruction of the language. This cost corresponds to the execution time, in number of cycles, of each instruction. Our analysis therefore depends on a cost function $cost_{instr}$, which associates each instruction with an integer value:

$$cost_{instr} : instr \rightarrow nat$$

The cost $cost_{step}$ of a transition is equal to the cost of the corresponding instruction:

$$cost_{step}(P, pc, M) = cost_{instr}(P[pc])$$

And the cost *cost* of an execution trace corresponds to the sum of the values returned by the cost function for each instruction in this trace:

$$cost(\mathcal{T}) = \sum_{\sigma \in \mathcal{T}} cost_{step}(\sigma)$$

For example, we represent the cost function for the instructions of the idealised language by the following table, which associates each instruction with an arbitrary cost:

Instruction	Cost (cycles)
Init	4
Assign	2
Add	5
Sub	7
Branch	2
Branchif	3
Stop	1

The cost of the execution trace defined in the previous example, which corresponds to the sum of the costs of its instructions, is then:

$$\begin{aligned} cost(\mathcal{T}) &= cost_{step}(P, 0, \varnothing) + cost_{step}(P, 1, [x = 4]) + cost_{step}(P, 3, [x = 4]) + cost_{step}(P, 4, [x = 8]); \\ &= cost_{instr}(P[0]) + cost_{instr}(P[1]) + cost_{instr}(P[3]) + cost_{instr}(P[4]) \\ &= cost_{instr}(\text{Init "x" 4}) + cost_{instr}(\text{Branchif "x" 3}) + cost_{instr}(\text{Add "x" "x"}) + cost_{instr}(\text{Stop}) \\ &= 13 \; cycles \end{aligned}$$

6.1.2 Variable Erasure

In a real, non-trivial program, it is common for the values of certain variables not to be known before program execution. These values may, for example, arise from the use of operators with non-deterministic semantics [CL14], or, more commonly, from the program's execution environment, such as user inputs or signals. In our application context, many values manipulated by programs indeed come from the electronic environment of physical assemblies containing a microcontroller: when it reacts, for example, to the value of a temperature sensor, that value is unknown at program compilation time. It is therefore

not always possible to evaluate the complete execution path of a given program statically and deterministically, since values from its environment may, for example, condition a branch in the program's control flow. One therefore cannot "predict" which path the program will take during its real execution when the value of a condition is statically unknown. This non-determinism, introduced by probing the program environment, makes it difficult to compute the program's total execution time: depending on which path is taken during real execution, its execution time may vary considerably.

To be able to bound the execution time of a program statically, it is therefore necessary to reason about an "abstract" representation of this program, statically representing all possible execution paths of the program. From the set produced, it is possible to deduce an execution-time value that upper-bounds the execution time of all elements of this set.

The abstract representation of a program therefore manipulates variables whose values are not known when the program is compiled. To do this, we extend below the grammar of values of the idealised language to support *unknown* variable values, represented by the symbol $\top$ ("*top*") to indicate that they may correspond to *any* value:

$$values, v, w ::= int\_literal \mid v + w \mid v - w \mid \top$$

The program memory may therefore associate certain variables with unknown values. We call *erasure* a memory in which all or some variables are associated with the value $\top$.

Definition 6.1.4 (Erasure)**.** *The following inductive rules define memory erasure:*

$\boxed{M \searrow M'}$

$$\frac{}{\varnothing \searrow \varnothing} \qquad \frac{M \searrow M'}{M[x := v] \searrow M'[x := v]} \qquad \frac{M \searrow M'}{M[x := v] \searrow M'[x := \top]}$$

A memory M' is therefore an erasure of a memory M, written $M \searrow M'$, if and only if M and M' both have the same set of variables, but some variable values that are known in M are unknown in M'.

In what follows, we use the same notation to represent a state σ' whose memory corresponds to an erasure of the memory contained in a state σ. If the memory of σ' is an erasure of the memory in σ, we write $\sigma \searrow \sigma'$.

Since the cost of a transition depends only on the program and the program counter, it does not change after erasing the memory of the associated state:

Lemma 6.1.1. $\forall\ \sigma\ \sigma', \sigma \searrow \sigma' \Rightarrow cost_{step}(\sigma) = cost_{step}(\sigma')$

By extension, the cost of an erased execution trace is identical to the cost of a non-erased trace:

Lemma 6.1.2. $\forall\ \mathcal{T}\ \mathcal{T}',\ \mathcal{T} \searrow \mathcal{T}' \Rightarrow cost(\mathcal{T}) = cost(\mathcal{T}')$

6.1.3 Non-Deterministic Evaluation

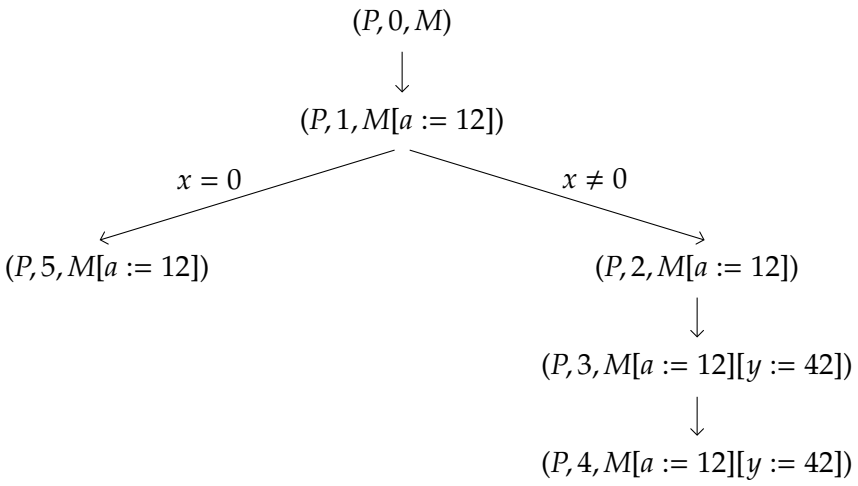
The appearance of unknown values ($\top$) introduces non-determinism into the semantics of the language. Indeed, if, for a conditional-branch instruction (Branchif x v), the value of the condition x is erased, then

it is no longer possible to know statically whether the program should take the path corresponding to the case where x is true, i.e. $x \geq 1$, or the path corresponding to the case where x is false, i.e. $x = 0$. As soon as several execution paths are possible, the total execution cost of a program may then diverge considerably depending on the path taken.

For example, consider the following program:

P = [Init "a" 12 ; 1 : Branchif "x" 5 ; 2 : Init "y" 42 ; 3 : Branch 4 ; 4 : Stop]

If the value of the variable x is not known at program compilation time, i.e. $M[x] = \top$, then it is not possible to assume its exact execution path during real execution. From the branch caused by the value of x, two different execution paths are possible:



The cost of P is therefore potentially different depending on the path actually taken: if we keep the same cost function as in the previous example, then the positive branch, the one that "jumps" directly to $pc = 5$, has a cost of 8, whereas the negative branch has a cost of 14.

To make it possible to compute an upper bound on the execution cost of a program, we introduce the notion of the *maximum cost* of an execution. This cost is computed by accumulating the costs of the various states encountered while following the transitions of the operational semantics of the idealised language. In the presence of a branch whose condition is unknown, the maximum cost is computed by following the transition towards the branch with the greatest cost.

Definition 6.1.5 (Maximum Cost). *The function $cost_{max}$ computes the maximum cost of a program from a state σ:*

$$\boxed{cost_{max}(\sigma) = c}$$

$$\frac{P[pc] = \mathsf{Stop}}{cost_{max}((P, pc, M)) = cost_{instr}(\mathsf{Stop})} \qquad \frac{\sigma \longrightarrow \sigma' \quad cost_{step}(\sigma) = c \quad cost_{max}(\sigma') = k}{cost_{max}(\sigma) = c + k}$$

$$\frac{\begin{array}{l} P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = \top \\ cost_{step}((P, pc, M)) = c \quad cost_{max}((P, pc + 1, M)) = k \quad cost_{max}((P, v, M)) = k' \end{array}}{cost_{max}((P, pc, M)) = c + max(k, k')}$$

For readability, and by abuse of notation, we use the equality symbol in the preceding rules. It should nevertheless be specified that the function $cost_{max}$ is partial, and is defined in Coq as an inductive relation. The definition of this function allows us to state the following theorem, which says that the maximum execution cost, computed for any erasure applied to the initial memory of a program, constitutes an upper bound on the execution cost of the real program.

Theorem 6.1.1 (Correctness). $\forall\ \sigma\ \mathcal{T}, run(\sigma) = \mathcal{T} \Rightarrow \forall\ \sigma',\ \sigma \searrow \sigma' \Rightarrow cost_{max}(\sigma') \geq cost(\mathcal{T})$

This theorem forms the basis for computing the WCET of our programs: if the cost function associates each instruction with its execution time, and if the initial memory associates each variable with the value $\top$, then the corresponding maximum cost is an upper bound on the worst-case execution time of the program, since a state whose memory contains only unknown values is an erasure of all possible initial states.

The rest of this section is devoted to the proof of this theorem, which establishes the correctness of the WCET-computation method.

6.1.4 Correctness Proof

The Coq sources containing the various formal representations of the notions stated in this section, as well as the proofs of the associated lemmas and theorems, are available online [⚓1].

Non-Deterministic Semantics of the Idealised Language

As illustrated above, introducing unknown values ($\top$) into the idealised language induces non-determinism in its execution: some erasures may lead to different execution traces which, although all valid, may have costs that vary greatly. To represent this non-determinism, we present a *non-deterministic semantics* for the idealised language, capable of manipulating *unknown* values. This semantics, whose evaluation rules are described in Figure 6.2, corresponds both to the direct transposition of the deterministic rules defined in Figure 6.1[2], and to the addition of two rules handling the case, for a conditional-branch instruction (Branchif), where the value of the condition is unknown. The premises of the conditional-branch rule

[2]Except that addition and subtraction operations are extended to return the value $\top$ as soon as one of the operands is unknown: for example, $x + \top = \top$. To denote this behaviour, arithmetic operators are represented in bold.

handling the case where the condition is *false* and the rule handling the case where it is *true* cannot be distinguished once the value of the branch condition is unknown. Adding the notion of values unknown at compilation time, together with these two rules, therefore gives the instruction semantics of the idealised language the expected non-deterministic character.

$\boxed{\sigma \rightsquigarrow \sigma'}$

$$\frac{P[pc] = \mathsf{Init}\ x\ v}{(P, pc, M) \rightsquigarrow (P, pc + 1, M[x := v])}$$

$$\frac{P[pc] = \mathsf{Assign}\ x\ y \quad M[y] = v}{(P, pc, M) \rightsquigarrow (P, pc + 1, M[x := v])}$$

$$\frac{P[pc] = \mathsf{Add}\ x\ y \quad M[x] = v \quad M[y] = w}{(P, pc, M) \rightsquigarrow (P, pc + 1, M[x := v + w])}$$

$$\frac{P[pc] = \mathsf{Sub}\ x\ y \quad M[x] = v \quad M[y] = w}{(P, pc, M) \rightsquigarrow (P, pc + 1, M[x := v - w])}$$

$$\frac{P[pc] = \mathsf{Branch}\ v}{(P, pc, M) \rightsquigarrow (P, v, M)}$$

$$\frac{P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = 0}{(P, pc, M) \rightsquigarrow (P, pc + 1, M)}$$

$$\frac{P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] \neq 0}{(P, pc, M) \rightsquigarrow (P, v, M)}$$

$$\frac{P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = \top}{(P, pc, M) \rightsquigarrow (P, pc + 1, M)}$$

$$\frac{P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = \top}{(P, pc, M) \rightsquigarrow (P, v, M)}$$

FIGURE 6.2: Non-Deterministic Semantics of the Idealised Language

Conservation of Transitions and Traces

Our reasoning concerns the maximum cost of the execution traces of a program whose memory contains erased variables: values from the environment, unknown before program execution. Yet during real execution of the program, the memory contains no unknown value, and the corresponding execution trace is the unique sequence of transitions in the deterministic operational semantics of the idealised language. In order for reasoning over the program's "non-deterministic" traces to be relevant, it is therefore important to ensure that the real execution trace of the program is indeed contained in the set

of non-deterministic traces considered. In other words, reasoning about "erased" versions of memory is useful only if, among all the valid erased traces computed, one corresponds to the real trace of the program; otherwise, we could conclude nothing about the program's real execution.

We begin by reasoning at the level of transitions in the semantics of the language. To this end, we first define a lemma stating that a transition in the deterministic semantics persists after "embedding" into the non-deterministic semantics of the idealised language. This property is immediate, since the non-deterministic semantics corresponds to an extension of the deterministic semantics of the idealised language:

Lemma 6.1.3. $\forall\ \sigma\ \sigma',\ (\sigma \longrightarrow \sigma') \Rightarrow (\sigma \rightsquigarrow \sigma')$

Moreover, every transition is preserved after erasure of the program memory: if there exists a transition from a state σ_1 to a state σ_2 in the non-deterministic semantics, then for any erasure σ'_1 of the original state there exists a transition leading to a state σ'_2, and this latter state is an erasure of σ_2:

Lemma 6.1.4. $\forall\ \sigma_1\ \sigma_2\ \sigma'_1, (\sigma_1 \rightsquigarrow \sigma_2) \wedge (\sigma_1 \backslash\!\searrow \sigma'_1) \Rightarrow (\exists\ \sigma'_2,\ (\sigma'_1 \rightsquigarrow \sigma'_2) \wedge (\sigma_2 \backslash\!\searrow \sigma'_2))$

By combining the two previous lemmas, we can then deduce that every transition in the deterministic semantics is preserved after embedding into the non-deterministic semantics, even after partial or complete erasure of the memory of the original state:

Lemma 6.1.5 (Conservation)**.** $\forall\ \sigma_1\ \sigma_2\ \sigma'_1, (\sigma_1 \longrightarrow \sigma_2) \wedge (\sigma_1 \backslash\!\searrow \sigma'_1) \Rightarrow (\exists\ \sigma'_2,\ (\sigma'_1 \rightsquigarrow \sigma'_2) \wedge (\sigma_2 \backslash\!\searrow \sigma'_2))$

This property is represented schematically in Figure 6.3.

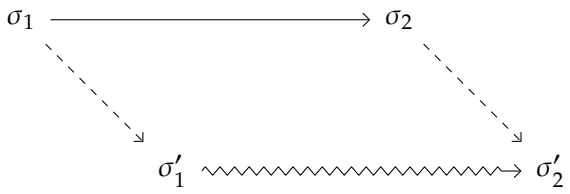


FIGURE 6.3: Transition Conservation

Finally, as illustrated by Figure 6.4, extending the preceding properties to execution traces is direct.

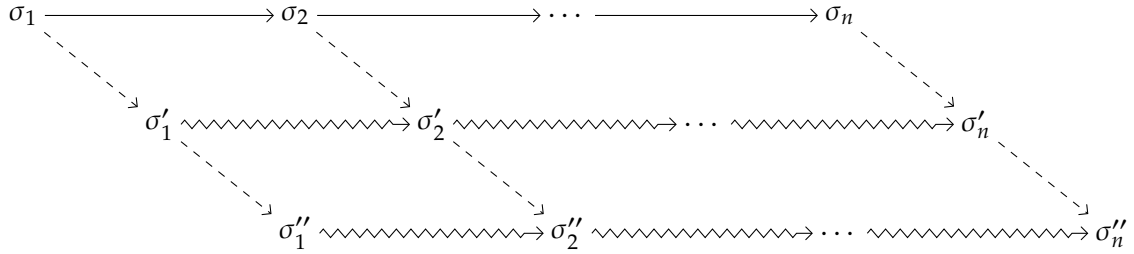


FIGURE 6.4: Trace Conservation

Upper-Bounding the Cost of the Program's Real Execution

In order to verify that the function $cost_{max}$ does indeed concern the most "costly" execution trace of the program, we formally define such a trace, called the *maximum execution trace*.

Definition 6.1.6 (Maximum Trace). *The function run_{max}, which computes the maximum execution trace, is defined inductively as follows:*

$\boxed{run_{max}\,(\sigma) = \mathcal{T}}$

$$\frac{\sigma \longrightarrow \sigma' \quad run_{max}\,(\sigma') = \mathcal{T}}{run_{max}\,(\sigma) = (\sigma \;::\; \mathcal{T})} \qquad \frac{P[pc] = \mathsf{Stop}}{run_{max}\,((P, pc, M)) = [(P, pc, M)]}$$

$$\frac{\begin{array}{l} P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = \top \\ run_{max}\,((P, pc+1, M)) = \mathcal{T} \quad run_{max}\,((P, v, M)) = \mathcal{T}' \quad cost(\mathcal{T}) > cost(\mathcal{T}') \end{array}}{run_{max}\,((P, pc, M)) = ((P, pc, M) \;::\; \mathcal{T})}$$

$$\frac{\begin{array}{l} P[pc] = \mathsf{Branchif}\ x\ v \quad M[x] = \top \\ run_{max}\,((P, pc+1, M)) = \mathcal{T} \quad run_{max}\,((P, v, M)) = \mathcal{T}' \quad cost(\mathcal{T}) \leq cost(\mathcal{T}') \end{array}}{run_{max}\,((P, pc, M)) = ((P, pc, M) \;::\; \mathcal{T}')}$$

In the same way as $cost_{max}$, the function run_{max} is a partial function defined by an inductive relation in Coq.

The maximum trace contains the same states as those considered by the function $cost_{max}$: it traverses the paths for which the total sum of costs is greatest. Its cost is therefore indeed identical to the maximum execution cost:

Lemma 6.1.6. $\forall\ \sigma,\ cost_{max}(\sigma) = cost(run_{max}(\sigma))$

By definition, the maximum trace is finite. Moreover, its cost is greater than that of any other finite trace that begins from the same state:

Lemma 6.1.7. $\forall\ \sigma\ \mathcal{T}\ k,\ finite(\sigma :: \mathcal{T}) \Rightarrow cost_{max}(\sigma) = k \Rightarrow k \geq cost(\sigma :: \mathcal{T}))$

By applying the conservation lemma to execution traces, we then derive the main property of the correctness proof of our method: the cost of the maximum execution trace is greater than the cost of any finite trace that begins from the same state, even when considering an erasure of its memory.

Lemma 6.1.8. $\forall\ \sigma\ \mathcal{T}\ k, finite(\sigma :: \mathcal{T}) \Rightarrow \forall\ \sigma',\ \sigma \searrow \sigma' \Rightarrow cost_{max}(\sigma') = k \Rightarrow k \geq cost(\sigma :: \mathcal{T})$

Since the deterministic trace is finite, combining the preceding lemmas finally allows us to conclude with the correctness theorem, which states that the maximum cost of a program, estimated from an erasure of the initial memory, is an upper bound on the real execution cost of the program:

Theorem 6.1.2 (Correctness). $\forall\ \sigma\ \mathcal{T}\ k, run(\sigma) = \mathcal{T} \Rightarrow \forall\ \sigma',\ \sigma \searrow \sigma' \Rightarrow cost_{max}(\sigma') = k \Rightarrow k \geq cost(\mathcal{T})$

6.2 Application to Computing the WCET of an OCaLustre Program: the *Bytecrawler* Tool

The method for computing worst-case execution time described in the previous section applies to an idealised language limited to a few instructions, but which nevertheless, through the branch instructions

it provides, allows a program to loop during execution. The absence of loops that are not statically bounded in the program is nevertheless an essential condition for ensuring the finiteness of the traces considered: otherwise, certain memory erasures[3] could lead to infinite execution of the static analyser, which would then be occupied indefinitely computing the maximum-cost execution trace of the program. The analysis described in the previous section was therefore suited only to *finite* traces, and thus only under the condition that the program does not loop indefinitely. Our analysis is therefore suitable only if we can ensure that no loops or recursive calls that are not statically bounded are possible in the programs under consideration.

During compilation of the OCaLustre language, each node is converted into a non-recursive OCaml function and, although the main program is executed in a main loop whose body runs at each synchronous instant, no "nested" loop is performed during an instant: the program simply computes, *instantaneously*, output values from input values. The execution traces of a synchronous instant are therefore *finite*; under these conditions, it is possible to adapt the analyses formalised in the previous section to the OCaLustre language, applying them no longer to the idealised language, but to the instruction language of the OCaml virtual machine, since this is the language to which the OCaLustre program is compiled after an initial conversion to standard OCaml code.

Although more numerous and potentially more powerful, the OCaml virtual-machine instructions generated by the OCaLustre language are comparable to those of the idealised language: analogously, some of them perform operations that change reference values (`SETFIELD`), arithmetic computations (`ADDINT`), or conditional branches (`BRANCHIF`). The analyses carried out on the idealised language are therefore transposable to the instructions of the OCaml virtual machine, and computing the worst-case execution time of a synchronous instant of an OCaLustre program then consists, as in the analysis of the idealised language, in computing the maximum execution cost of the synchronous instant. This cost then constitutes the worst-case execution time of one instant of the program.

It should be noted that the analysis presented here is valid because the microcontrollers we consider have very simple memory models. Indeed, the absence of cache systems makes the duration of each instruction predictable [BN94], without having to consider scenarios in which memory accesses would be more or less costly in time, and ensures the execution times, in number of cycles, of the different instructions of the language. In this sense, the targets considered are said to be free of timing anomalies ("*timing anomalies*" [RWT+06]) and induce the composability of execution-time analyses.

6.2.1 Computing Instruction Costs

When formalising the worst-case execution-time analysis, we discussed the existence of a specialised cost function associating each instruction of the idealised language with an integer value. In the execution context of OCaLustre, it is therefore necessary to have such a function associating each instruction of the virtual machine with a maximum execution time, in order to deduce a value that upper-bounds the execution time of the complete program. To this end, we take advantage of pre-existing static analysers that can assign a maximum number of cycles to a compiled program. For illustration, we used the software tool *Bound-T* [HS02], a memory-usage and execution-time analyser capable of computing the WCET of a program compiled for an AVR microcontroller.

[3] For example, erasing the variable that conditions execution of a loop iteration.

Our approach consisted in having Bound-T compute the worst-case execution time of each code block responsible for interpreting an instruction of the OMicroB virtual machine. In this way, Bound-T allows us to associate each virtual-machine instruction with a value corresponding to the maximum number of machine cycles for executing that instruction. It should be noted that the execution time of certain OCaml virtual-machine instructions depends in part on the value of one of their parameters; for example, the instruction `APPTERM n v` causes `n` iterations of a loop in the associated interpreter code. One way to handle these instructions is to compute their cost when their parameter is equal to the largest possible value. However, this method estimates an upper bound that may be far from reality. To associate a finer cost with each instruction, we recompute the costs of such instructions several times by providing a new hard-coded value as the parameter of these instructions. This method is slower to carry out, but it only needs to be done once: once Bound-T has been run for each virtual-machine instruction, the tool no longer needs to be reused, since the bytecode of any OCaml program will contain a subset of this instruction set whose costs we have precomputed.

6.2.2 *Bytecrawler*: an "Abstract" Bytecode Interpreter

Once the Bound-T analysis is complete, we have at our disposal a complete table representing the virtual-machine instructions and their cost, in number of cycles. An excerpt from this table, generated for the AVR ATmega2560 microcontroller for a 16-bit version of OMicroB, is reproduced in Figure 6.5. This table constitutes the cost function described in the previous section. It is then possible to replicate the maximum-cost trace computation method, considering that absent values (`⊤`) correspond to the results of calls to C primitives, via `CCALL` instructions, that communicate with the environment.

We therefore propose a static-analysis tool, named *Bytecrawler*, which executes the program by following the same operation as the function $cost_{max}$ described above, for example by traversing both branches of a conditional in order to deduce the one with maximum cost. By accumulating the values from the cost function computed by Bound-T, Bytecrawler thus deduces the cost, in number of cycles, of the maximum execution trace of an OCaLustre instant. By extending the result of the correctness theorem to the instruction language of the OCaml virtual machine, we can deduce that the cost thus found corresponds to an estimate of the worst-case execution time of the synchronous instant of the OCaLustre program. The function `wcet` in Figure 6.6 is a simplified excerpt from the function at the heart of Bytecrawler: it evaluates a program by following the standard semantics of OCaml bytecode instructions, while being able to handle *unknown* variable values arising from calls to input/output C primitives.

Moreover, in addition to associating a number of cycles with each instruction of the OCaml virtual machine, it is also necessary to compute the cost of all input/output primitives integrated into the standard library of the virtual machine. These primitives, written in C, are generally compatible with analysers such as Bound-T. Just as the instruction set does not change over time, input/output primitives, being very low level, often consisting simply in writing or reading microcontroller pin values, are not expected to change from one program to another. Whenever Bytecrawler encounters a `CCALL` instruction calling a primitive, it must therefore consult a second table that associates each C primitive name with its estimated execution time. A developer wishing to define their own C primitives must then provide their associated costs in this table.

Bytecode instruction	Cost (number of cycles)
ACC	46
PUSH	43
PUSHACC	74
POP	42
ASSIGN	52
ENVACC	50
PUSHENVACC	78
APPLY	50
RETURN	85
GETFIELD	50
SETFIELD	67
GETVECTITEM	54
SETVECTITEM	69
BRANCHIF_1B	36
BRANCHIF_2B	46
BRANCHIF_4B	67
CONST0	25
ADDINT	51
SUBINT	51
MULINT	59

FIGURE 6.5: Bytecode-Instruction Cost Table Computed by Bound-T (Excerpt)

```
let rec wcet state =
  let state' = {state with pc = state.pc + 1} in
  let instr = state.instrs.(pc) in
  cost instr + match instr with
  | CONST i -> wcet {state' with accu = Int i}
  | BRANCH ptr -> wcet {state with pc = ptr}
  | BRANCHIF ptr ->
    (match state.accu with
    | Int 0 -> wcet state'
    | Int _ -> wcet {state with pc = ptr}
    | Unknown -> max (wcet {state with pc = ptr}) (wcet state'))
  | ADDINT ->
    (match state.accu, state.stack with
    | Int x, (Int y)::s -> wcet {state' with accu = Int (x+y); stack = s}
    | _, _::s -> wcet {state' with accu = Unknown; stack = s})
  | C_CALL1 _ -> wcet {state' with accu = Unknown}
  | STOP -> 0
```

FIGURE 6.6: WCET Computation Function (Excerpt)

The approach used by Bytecrawler offers an important advantage over classical WCET-computation tools for programs compiled to native code: since the costs associated with each virtual-machine instruction are fixed, they can be reused to analyse the cost of other programs, without forcing the programmer to rerun the sometimes complex machinery of an analyser such as Bound-T after the slightest change to the program. Moreover, the WCET of a program can be computed for several different microcontroller models as soon as Bytecrawler is provided with a bytecode-instruction cost table adapted to each of these models. Such factorisation of analyses is welcome in this development context for embedded hardware, where targets, such as the circuit assembly or the type of microcontroller, may change as requirements evolve.

6.2.3 Dedicated Mode of the OCaLustre Compiler

Since the virtual machine used relies on a *garbage collector*, computing these costs could be erroneous. Indeed, when a new value is allocated on the heap, the memory-reclamation algorithm may potentially be triggered, and several machine cycles may be spent completing its execution. If not taken into account, this difficult-to-predict triggering of the garbage collector can lead to errors in computing the WCET of a program. Several methods can account for the existence of such an automatic memory-reclamation mechanism: one of them simply consists in considering that a complete memory cleanup is executed at every allocation of a value on the heap. This over-approximation correctly computes an upper bound on the execution time of a synchronous instant, but this bound lacks "precision", because it is not realistic to consider that the *garbage collector* is triggered so often.

It is therefore interesting to observe that, in the context of compiling the OCaLustre language, the values requiring allocation on the heap, typically the registers arising from use of the ⋙ operator, are declared during the node initialisation phase: these values represent the environment of the closure generated by compiling a node. For example, in the following node, only the register value used to record the previous value of the expression `cpt + 1` needs to be kept in memory:

```
let%node count () ~return:cpt =
  cpt = (0 ⋙ (cpt + 1))
```

And this particularity appears after compilation: the variable `cpt_fby` is the only one present in the environment of the closure returned by the function `count`:

```
let count () =
  let cpt_fby = ref 0 in
    fun () ->
      let cpt = !cpt_fby in
      cpt_fby:= cpt + 1;
      cpt
```

Consequently, it is possible to consider that, provided only base-type values are used, such as integers, floats[4], or Booleans, the quantity of values to allocate in memory for the execution of a node is known

[4]Thanks to our representation of values in OMicroB, floats do not in fact need to be allocated on the heap.

before program execution. Thus, if the necessary registers are allocated before execution of the OCaLustre program, no new allocation will be performed during execution of the instant. This would allow us to state that it is not useful to take into account the time taken by the garbage collector when computing the worst-case execution time of the synchronous instant, since no heap allocation is performed during that instant.

Nevertheless, the compilation method described in Section 4.4 is not entirely compatible with this assertion: the compiled code of a node may indeed contain tuples corresponding to its parameters as well as to its output streams. These tuples are consequently allocated on the heap at every instant, and the question of triggering the garbage-collection algorithm arises again whenever a node returns and/or receives more than one stream.

For example, consider the following node:

```
let%node triple (x) ~return:(a,b,c) =
  a = x;
  b = x + 1;
  c = 42
```

During compilation, this node leads to the generation of an OCaml function that produces a closure, and this closure induces the creation of a triple `(a,b,c)` whose memory representation leads to a heap allocation at each synchronous instant:

```
let triple () =
  fun x ->
    let a = x in
    let b = x + 1 in
    let c = 42 in
    (a,b,c)
```

Thus, the analysis presented in this chapter is compatible with the compilation model presented in Section 4.4 only if the nodes considered do not have several input or output parameters. This strong limitation seems too significant to make Bytecrawler genuinely usable on non-trivial programs. Consequently, in order to avoid such allocations occurring "during an instant", we present an alternative compilation scheme for OCaLustre nodes. This new compilation method preserves the semantics of the OCaLustre language, but this time allows us to ensure that no memory allocation is performed during a synchronous instant, whatever the shape of the nodes.

In this alternative compilation scheme, every node n causes the definition of an OCaml type n`_state` corresponding to the state of an instance of this node. This state contains mutable registers corresponding to the node outputs and to the internal registers required for its execution, such as those arising from use of the *followed-by* operator:

```
type ('a, 'b, 'c) triple_state = {
 mutable triple_out_a: 'a;
 mutable triple_out_b: 'b;
 mutable triple_out_c: 'c }
```

The node code is then distributed across two distinct functions. A first initialisation function corresponds to the allocation of the node state. Since at this compilation stage the value types have not yet been inferred, the value `Obj.magic ()` is used again, except for constant streams, which are initialised with their value, or equations of the form `k` $\ggg$ `e`, which are initialised with the constant `k`. The role of `Obj.magic ()` here is only to reserve on the heap the space necessary to store the mutable state of the node: it will never be evaluated during execution of the synchronous program.

```
let triple_alloc x = {
    triple_out_a = Obj.magic ();
    triple_out_b = Obj.magic ();
    triple_out_c = 42 }
```

This function generates the initial state of the node registers. Following a state-passing execution model, or *state-passing style*, the internal state of the program is then a parameter of the node's *step* function, which computes at each instant the values of the streams defined by the node:

```
let triple_step state x =
  let a = x in
  let b = x + 1 in
  let c = 42 in
  state.triple_out_a ← a;
  state.triple_out_b ← b;
  state.triple_out_c ← c
```

At each instant, this function updates, by side effects, the values of the different variables present in the node state. Since the node state is a record type whose fields are mutable, each of these fields is allocated when the state is generated, and no new memory allocation occurs for subsequent instants. Thanks to this execution model, which generates code with more "imperative" aspects, the WCET computation for the instant can thus be applied to the generated function *n*`_step`.

The code of the main function of the generated OCaml program, compatible with this compilation mode, has the following form:

```
let () =
  (* generation of the main node state *)
  let _st = triple_alloc () in
  while true do
   (* read inputs *)
   let x = input_triple_x () in
   (* execute the node *)
   triple_step _st x;
   (* write outputs *)
   output_triple _st.triple_out_a _st.triple_out_b _st.triple_out_c
  done
```

The OCaLustre compiler option -na enables the "non-allocating" compilation mode described in this section.

6.2.4 Application Example

In this section, we illustrate the operation of *Bytecrawler* on a short example. The OCaLustre program in this example is reduced to a single node named count, which represents a counter: at each synchronous instant, it computes the successor of the integer computed at the previous instant. Moreover, this counter can be reset to zero as soon as the node parameter reset is true.

The code of this node, on the left, together with the set of bytecode instructions to which this node is translated after compiling OCaLustre to an OCaml program and then standard compilation of the latter, on the right[5], are presented below:

```
let%node count reset ~return:cpt =
  cpt = (0 ⋙ if reset then 0 else (cpt + 1))
```

```
L1:  GRAB 1
     ACC 0
     GETFIELD 0
     PUSH
     ACC 2
     BRANCHIFNOT L7
     CONST 0
     BRANCH L6
L7:  ACC 0
     OFFSETINT 1
L6:  PUSH
     ACC 1
     PUSH
     ACC 3
     SETFIELD 1
     ACC 0
     PUSH
     ACC 3
     SETFIELD 0
     CONST 0
     RETURN 4
```

[5]The standard compiler option *-dinstr* displays this *labelled* bytecode.

We now declare two OCaml functions responsible for capturing the inputs and processing the outputs of this synchronous program. The first defines the input value of the node (`reset`) as the result of the test that checks whether pin number 0 of port *B* is powered, for example if a push button connected to this pin has been pressed:

```
(* input function *)
let input_count_reset () =
   read_bit PORTB PB0
```

```
L4:  CONST0
     PUSH
     CONST1
     CCALL caml_avr_read_bit, 2
     RETURN1
```

Since the value returned by reading this pin, via the call to the primitive `caml_avr_read_bit`, cannot be known at compilation time, it is represented in Bytecrawler as an unknown value (⊤).

For simplicity, we declare that the second function, which is supposed to process the value `cpt` computed by the synchronous instant, does "nothing", that is, it simply computes the "unit" value:

```
(* output function *)
let output_count cpt = ()
```

```
L3: CONST 0
    RETURN 1
```

After initialising the state of the synchronous program, according to the compilation model described in the previous section, the OCaml program is responsible, in a loop, for reading the program input, calling the step function of the synchronous instant, and calling the function that processes its outputs. So that Bytecrawler can recognise in the bytecode where each synchronous instant begins and ends, the program loop is replaced for WCET analysis by calls to the primitives `begin_loop` and `end_loop`:

```
let () =
  let _st = count_alloc () in
  begin_loop ();
    let reset = input_count_reset () in
    count_step _st reset;
    output_count _st.count_out_cpt;
  end_loop ()
```

The bytecode associated with the code framed by the calls to `begin_loop` and `end_loop` is then the following:

```
CCALL begin_loop, 1
CONST 0
PUSH
ACC 5
APPLY 1
PUSH
ACC 0
PUSH
ACC 2
PUSH
ACC 4
APPLY 2
ACC 1
GETFIELD 1
PUSH
ACC 5
APPLY 1
CONST 0
CCALL end_loop, 1
```

Bytecrawler executes the program from the very first bytecode instruction, but begins counting the costs of encountered instructions only from the call to the primitive `begin_loop`, and continues until the call to `end_loop`. This makes it possible to compute the cost, in cycles, of the synchronous instant. Thus, with an instruction-cost table computed by Bound-T for an ATmega2560, the maximum number of cycles estimated by Bytecrawler is 3080. Since such a microcontroller has a clock frequency of 16 MHz, the worst-case execution time of *one* synchronous instant of this program is 1.92×10^{-4} seconds (192 μs). Thus, if the interval between two changes in the state of pin *PB0* is greater than 192 microseconds, the synchronous hypothesis is valid.

Chapter Conclusion

In this chapter, we have taken advantage of the analysis factorisation provided by the representation of programs as bytecode in order to compute the worst-case execution time of a program. The correctness proof of the method used ensures its viability, and a software tool following this method has been implemented. Of course, one certification process for the operation of this tool would require formalising and proving the method no longer on the idealised bytecode presented in this section, but on the real subset of the OCaml bytecode instruction language produced by OCaLustre compilation. Although this would represent substantial work because of the number of OCaml bytecode instructions, this transposition nevertheless seems rather direct.

Moreover, the main interest of the method again concerns its portability: analyses concerning the bytecode and those inherent to the platform used are distinct [HK07]. The application developer therefore does not need to use special annotations to compute the WCET of an OCaLustre program. Such annotations, for example concerning the number of loop iterations, may however be used by the platform developer in order to provide a cost table for instructions and primitives. Once created, this table is then sufficient for analysing the bytecode of any OCaLustre program. The portability of the method entails the portability of the Bytecrawler tool: for the same program, simply providing a cost table measured for another microcontroller makes it possible to deduce the WCET of the program on that platform, without even recompiling it.

The method presented in this chapter could certainly benefit from additional static analyses, for example through symbolic execution [BFL18], or concolic execution [Sen07], a hybrid approach combining, in a way rather close to our method, concrete values and symbolic values, which would make it possible not to consider unreachable execution paths. However, this method is above all an illustration of an analysis that takes advantage of a common representation, bytecode, rather than a technique for estimating the maximum execution time of a program as finely as possible.

Finally, it is interesting to note that the method presented in this chapter seems easy to extend to measurements other than execution time: by changing only the definition of the cost function, and therefore the table that represents it, it would be possible to evaluate other criteria, and to estimate, for example, the maximum size of the execution stack of a synchronous program, or even the maximum quantity of values allocated on the heap during execution of a program. These considerations relating to the memory footprint of programs will be decisive in the following chapter, in which we detail different measurements used to assess the performance of the main solutions presented in this thesis.

7 Performance of OMicroB and OCaLustre

This chapter presents and analyses several measurements intended to assess the strength of the software solutions described in this document. We first present a set of measurements performed on the OMicroB virtual machine using programs designed to test its capabilities against various aspects of the OCaml language. We then consider the performance of OCaLustre, in particular through the analysis of the memory-resource consumption of a complete OCaLustre program, which we compare with an equivalent program written in the Lucid Synchrone language. Throughout the chapter, we discuss the results of these measurements in detail, and place them in perspective relative to the performance of pre-existing solutions.

7.1 OMicroB Performance Measurements

In this section, we perform several measurements to evaluate the performance of the OMicroB virtual machine. These measurements, carried out using simple test programs, concern the speed and memory consumption of the virtual machine.

7.1.1 Methodology

OMicroB is a highly configurable virtual machine. The user can specify several concrete aspects of the machinery used to execute OCaml bytecode. This configurability makes it possible to adapt the capabilities of the virtual machine precisely to the hardware on which it runs. The various aspects of the virtual machine are configured using specific compilation options, used in addition to the command for compiling an OCaml program with OMicroB:

```
omicrob <options> <file.ml>
```

In this chapter, we therefore use the following compilation options:

- The option `-arch <n>` selects the length of OCaml words in the representation used by OMicroB. This length may be 16, 32, or 64 bits. Using a 16-bit representation, sufficient in the vast majority of programs, reduces RAM usage.
- The option `-gc <algorithm>` allows the user to choose the garbage-collection algorithm used. This may be the *Stop and Copy* algorithm (`SC`) discussed in Chapter 2, or a *Mark and Compact* algorithm (`MC`).
- The option `-no-shortcut-initialization` disables the optimisation that partially evaluates the OCaml program up to its first input/output primitive.

- The option `-stack-size <n>` sets the size, in words, of the stack used by the virtual machine to execute the bytecode of the OCaml program. By default, this size is set to 64 words, but some programs may require a larger stack. The stack size is one factor in the virtual machine's RAM consumption.

- The option `-heap-size <n>` sets the number, in words, of addressable values in the heap. By default, this number is 256 words. Like the stack size, the heap size has direct consequences for the RAM consumption of the OCaml program. The chosen *garbage collection* algorithm influences the actual RAM usage of the heap: because it uses two "pools", the *Stop and Copy* algorithm requires reserving a memory section twice as large as that of a *garbage collector* implementing a *Mark and Compact* algorithm.

- Finally, the option `-no-clean-interpreter` disables the optimisation that produces a virtual machine embedding only the code for processing the bytecode instructions used by the program.

Because they are commonly used in hobbyist and professional projects, throughout this section we use a version of OMicroB intended to run on AVR-family microcontrollers. These microcontrollers, often embedded in *Arduino* boards, have fairly limited memory capacities, in particular only a few kilobytes of RAM, which makes it possible to show the importance of OMicroB's optimisations and to demonstrate the possibility of running OCaml programs on very resource-constrained hardware. Our various measurements are performed in particular on an ATmega2560 microcontroller with a computing power of 16 MIPS, 256 kilobytes of flash memory, and 8 kilobytes of RAM. Running the command `omicrob <file>.ml` then generates two notable files: a first file `<file>.avr`, corresponding to the executable program intended to be transferred to the microcontroller, and a second file `<file>.elf`, the result of compilation with the native *gcc* compiler. The latter makes it possible to simulate the execution of the programs on a PC.

The two main aspects measured in this chapter are the following:

- Size criteria: in particular, the size of the produced executable programs and their RAM consumption are important pieces of information in this application setting, where hardware memory resources are highly limited. We measure the memory consumption of the generated AVR programs using the command `avr-size`, a variant of the UNIX command `size` that displays the size of the different memory sections used by the program. In particular, the command `avr-size` provides the additional formatting option `-C`, which groups memory sections into two sections: the quantity of flash memory used by the program, and the quantity of RAM used by the program.

  To illustrate this, Figure 7.1 shows the output of the avr-size command applied to a program `prog.avr`[1]. It informs the user that this program uses 6968 bytes of flash memory (2.7% of the flash memory available on an ATmega2560) and 7104 bytes of RAM (86.7% of the RAM available on this microcontroller).

- Speed criteria that illustrate the temporal performance of the virtual machine. Speed computations are carried out both on *.elf* programs corresponding to OMicroB simulation on a PC, in order to evaluate OMicroB's capabilities relative to the standard virtual-machine implementation,

[1] The option `--mcu` specifies the microcontroller model.

```
$ avr-size -C prog.avr --mcu=atmega2560
AVR Memory Usage
----------------
Device: atmega2560

Program:    6968 bytes (2.7% Full)
(.text + .data + .bootloader)

Data:       7104 bytes (86.7% Full)
(.data + .bss + .noinit)
```

FIGURE 7.1: Example Use of the `avr-size` Command

and through physical measurements of program execution time on a microcontroller. Simulated program executions are measured using the UNIX command `time`, which measures the execution duration of a program. From these different measurements, and using the information reported by program simulation about the total number of executed instructions, it is then possible to deduce an average OMicroB speed in number of instructions per second. It should be noted that this speed may vary according to the complexity of the bytecode instructions executed by the program considered: processing a rich instruction, such as the `CLOSURE` instruction for closure creation, is potentially much longer than processing another, more basic bytecode instruction, such as the `PUSH` instruction that adds an element to the top of the stack. In this sense, the computed speed can only be an imprecise indication of OMicroB's capabilities, which may vary greatly from one program to another.

These two evaluation criteria are strongly dependent on the compiler options mentioned above. Indeed, the RAM size of programs varies according to the chosen stack and heap sizes, while the execution speed of a program may depend on the chosen architecture: for example, a microcontroller may take longer to load and process data represented on 32 bits than if the same data were represented on 16 bits. In addition, the more frequent activation of the *garbage collection* algorithm when using a small memory space increases the execution times of OCaml programs, part of whose execution is devoted to cleaning unused heap memory.

7.1.2 Test Programs

The different measurements presented in this section are performed on simple OCaml programs, each of which tests fundamental features of the language and its virtual machine. The source code of these programs is available online [⚓1], and we briefly detail the operation of each of them below:

1. The program `apply.ml` performs one thousand successive applications of a function ($\lambda f\ x.f\ (f\ x)$) representing the encoding of Church numerals. This program tests the mechanism for applying higher-order functions.

2. The program `fibo.ml` computes the first ten terms of the Fibonacci sequence one hundred thousand times. This program tests recursive function application, as well as the use of basic arithmetic operations.

3. The program `takc.ml` computes the result of the Takeuchi function one thousand times, with parameters 18, 12, and 6. This program manipulates triples, and therefore tests the garbage collector as well as recursive function application.

4. The program `oddeven.ml` computes ten thousand times the parity of the integers between 0 and 100. This program makes it possible to test the application of mutually recursive functions.

5. The program `floats.ml` applies trigonometric functions (`sin` and `cos`) to floating-point numbers ten million times. It tests the representation of floating-point numbers, as well as the performance of operations on them.

6. The program `integr.ml` computes the integral $\int_0^{10}(x^2 + 2x + 10)\,dx$ ten thousand times, with a step size of 0.01. This program tests the representation of floating-point numbers, as well as the dynamic generation of functional values.

7. The program `eval.ml` is an evaluator for Boolean logic formulae, and computes the value of ten Boolean formulae ten thousand times. It tests the definition of algebraic types, pattern matching, and the use of the `Stack` module, which represents a stack data structure.

8. The program `sieve.ml` computes the prime numbers below 50 ten thousand times using the sieve of Eratosthenes algorithm. This program creates lists, and thus tests the capabilities of the *garbage collection* algorithm.

9. The program `objet.ml` tests the representation of objects in OCaml: it defines a `point` class, generates one hundred thousand objects of this class, and for each object computes its reflection with respect to the origin ten times by calling a `sym` method.

10. The program `functor.ml` tests OCaml's parameterised-module mechanism (or *functors*). It generates a module representing a set of integers by applying the `Set.Make` functor from the standard library, then ten thousand times adds to a set of integers the values contained in a list made up of 50 consecutive integers.

11. The program `bubble.ml` is an array-based implementation of the bubble-sort algorithm. It generates an array of 60 elements populated with decreasing integer values and applies bubble sort to this array ten thousand times. This program tests the implementation of arrays, access to their values, and their update.

12. The program `jdlv.ml` implements Conway's Game of Life on a 10×10 matrix. It computes the first ten thousand configurations of the Game of Life from a state representing an oscillator. This program, which uses records with mutable fields and matrices, tests their implementation.

13. The program `share.ml` defines a list-filtering function that uses OCaml's exception mechanism to avoid unnecessarily copying elements from the original list into the resulting list. It filters the list of natural integers from 0 to 50 one hundred thousand times using the function $(\lambda x.x > 25)$. This program stacks many exception handlers and therefore tests OCaml's exception-catching mechanism.

14. The program `abrsort.ml` performs binary-search-tree sorting ten thousand times on a set of natural integers between 0 and 100. It tests OMicroB's performance on a recursive data structure.

15. Finally, the program `queens.ml` solves the n queens problem ten thousand times, with a number n of queens ranging from 1 to 6. This program tests the computing capabilities of the virtual machine on a non-trivial algorithmic example.

The measurements on these programs were carried out in two stages: the programs were first tested on a PC through a compilation of OMicroB with the *gcc* compiler. In a second stage, the same tests were run on an AVR ATmega2560 microcontroller.

7.1.3 Execution on a Computer: Results and Interpretation

The programs discussed in this section were compiled with OMicroB's `-no-shortcut-initialization` option, in order to avoid triggering OMicroB's partial-evaluation mechanism and having all the computations performed during compilation, since these tests do not call any input/output primitive.

All the measurements carried out use a 16-bit representation of OCaml values, the *Stop and Copy* garbage-collection algorithm, a heap size of 2500 words, and a stack size of 500 words. The processor of the computer used is a quad-core Intel Core i5-8259U, with a 64-bit architecture and a clock speed of 2.3 GHz.

Execution Speed: The tests performed make it possible to compare the execution speed of the OMicroB virtual machine (in number of bytecode instructions per second) with that of the bytecode interpreter of the standard ZAM implementation, called *ocamlrun*. The latter is used in a standard configuration: with values encoded on 64 bits, a stack of 256,000 words, and a minor heap of 2 megabytes. The results of the measurements obtained with the configuration described above are presented in Table 7.1. In particular, the column named "Ratio" represents the ratio between a program's execution time with OMicroB and its execution time with ocamlrun: a small value is therefore preferable. Other measurements obtained with the *Mark and Compact* GC, as well as with 32-bit and 64-bit data representations, are available in Appendices C.1, C.2, and C.3.

From the various measurements carried out on a PC, it appears that OMicroB has an execution speed that can in some cases be roughly 2 to 3 times slower than *ocamlrun*, the standard bytecode interpreter. These results are very reasonable, insofar as OMicroB's capabilities remain within an expected order of magnitude, and our optimisations do not primarily target speed performance, but rather improvements in memory consumption. This difference in speed can be explained by many factors. In particular, representing the program byte by byte in an array of C values introduces an obvious overhead, compared with the fact that *ocamlrun* loads the program into RAM and directly reads values whose length corresponds to the processor architecture of the computer (32 or 64 bits). Moreover, the dynamic-memory constraints differ between OMicroB (here: 2500 words for the heap, only half of which is available because of the use of the *Stop and Copy* garbage collector) and *ocamlrun* (by default, OCaml's minor heap is 2 megabytes on a 64-bit platform). This causes OMicroB's memory-cleaning algorithm to be triggered more frequently, slowing down program execution. For illustration, the graphs in Figure 7.2 show the execution times of the programs `takc.ml` and `abrsort.ml` as a function of different heap sizes and of the selected GC algorithm (for a 16-bit representation of values): the general tendency is for this time to decrease as the available heap size increases, until it stabilises once the program consumes less memory than is available to it.

Name	Execution Time with ocamlrun (seconds)	Execution Time with OMicroB (seconds)	Ratio	Execution Speed with OMicroB (millions of bytecode instructions/second)	Number of GC Invocations
`apply`	1.20	2.14	1.78	306.33	58
`fibo`	0.55	1.14	2.07	428.82	0
`takc`	1.05	3.07	2.92	383.31	226000
`oddeven`	0.28	0.60	2.14	614.68	0
`floats`	0.56	1.05	1.87	219.46	0
`integr`	0.04	0.12	3.00	222.16	42
`eval`	0.04	0.07	1.75	431.42	2352
`sieve`	0.04	0.07	1.75	486.00	3333
`objet`	0.10	0.17	1.70	389.77	11765
`functor`	0.24	0.60	2.50	392.77	30003
`bubble`	0.93	1.55	1.66	461.49	35
`jdlv`	0.36	0.75	2.08	457.64	6666
`share`	0.11	0.25	2.27	480.24	699
`abrsort`	0.47	1.22	2.59	321.27	119924
`queens`	1.35	3.35	2.48	413.54	149999

TABLE 7.1: Program Performance Measurements for OMicroB (on PC)
OMicroB options: -arch 16 -gc SC -stack-size 500 -heap-size 2500

Finally, the `floats` test illustrates the value of our immediate representation of floating-point numbers: on this test, OMicroB in 64-bit mode is almost as fast as *ocamlrun*, and in 32-bit mode even slightly faster (see Tables C.2 and C.3 in the appendices). Indeed, the absence of floating-point allocation both limits GC invocations and reduces the number of indirections introduced by the classic representation of floating-point numbers, in the form of boxed values allocated on the heap.

7.1.4 Execution on a Microcontroller: Results and Interpretation

In what follows, we present and interpret the performance measurements obtained when running the test programs on a microcontroller. This is an AVR ATmega2560 microcontroller, with 256 KB of flash memory, 8 KB of RAM, and a clock speed of 16 MIPS. Here too, the programs are compiled with the `no-shortcut-initialization` option, using a 16-bit representation of OCaml values, with a maximum stack of 500 words, a heap of 2500 words, and the *Stop and Copy* garbage collector. It should be noted that, because a microcontroller is relatively slow compared with a recent computer, the number of computations performed in these tests was divided by one thousand compared with the computer tests, in order to reduce the duration of the measurements. For example, the program `queens.ml` now computes the *n* queens problem only ten times (with *n* from 1 to 6).

Execution Speed: Table 7.2 shows the results of the measurements concerning program execution time for a 16-bit version of the virtual machine. The timing measurements were performed using an OCaml function `Avr.millis: unit -> int`, which returns the number of milliseconds elapsed since the program was launched on the microcontroller. The difference between the value returned by this

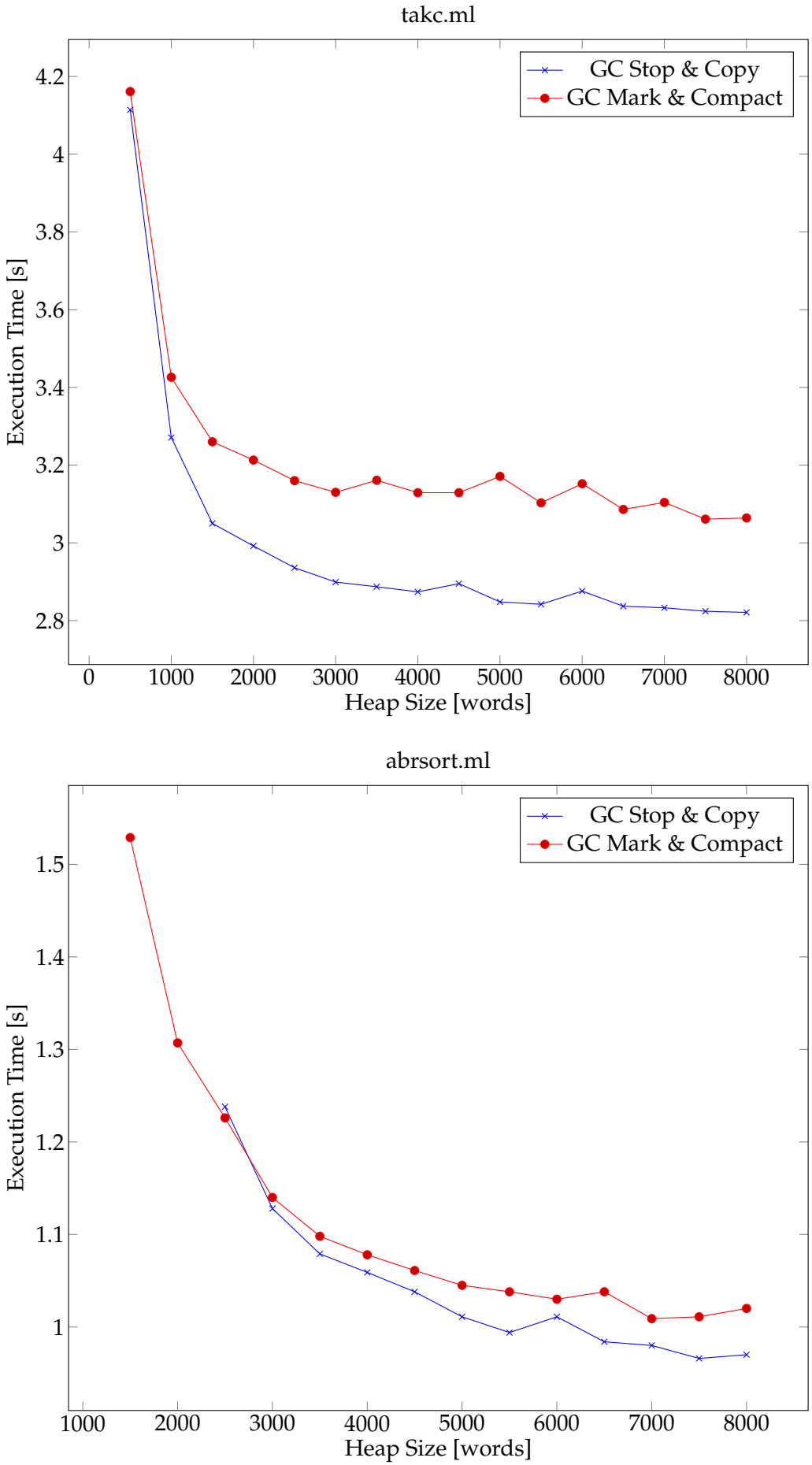


FIGURE 7.2: Influence of Heap Size on Execution Time (on PC)

Name	Execution Time (seconds)	Speed (thousands of instr./second)	Number of GC Invocations
`apply`	3.579	183.191	0
`fibo`	1.984	246.473	0
`takc`	5.475	214.951	228
`oddeven`	1.334	276.536	0
`floats`	7.482	30.812	0
`integr`	0.745	35.910	0
`eval`	0.143	212.048	2
`sieve`	0.195	174.953	3
`objet`	0.420	162.138	12
`functor`	1.351	192.904	33
`bubble`	3.762	190.169	0
`jdlv`	2.006	171.273	6
`share`	0.572	211.891	0
`abrsort`	2.677	147.965	124
`queens`	6.598	209.987	154

Table 7.2: Program Speed Measurements for OMicroB (on ATmega2560)
OMicroB options: -arch 16 -gc SC -stack-size 500 -heap-size 2500

function at the end of the program and the value returned at the beginning of the program makes it possible to estimate its execution time. This value is then transmitted, through a serial transmission protocol, to the personal computer to which the microcontroller is connected.

The measurements obtained allow us to estimate OMicroB's execution speed on AVR at approximately 200,000 bytecode instructions per second. These results depend on many factors, such as the capabilities and optimisation level of the compiler used (here, the option applied to `avr-gcc` is `-O2`), the nature of the instructions executed (whose execution time varies according to their degree of "complexity"), and hardware-specific features. In this respect, the `floats.ml` and `integr.ml` tests, which manipulate floating-point numbers, stand out because of their markedly lower execution speed. This slowdown is explained by the specific characteristics of the hardware used: on an AVR microcontroller, operations on floating-point numbers are performed in software, and are therefore slower than on a PC, where these operations are executed by the processor's arithmetic unit. In addition, our representation of floating-point numbers in the 16-bit version of the virtual machine is non-standard, and therefore imposes conversions during program execution.

Table C.4, in the appendix, shows the measurements on the same programs in a 32-bit version of the virtual machine. As expected, the measured speed is slower by a factor close to 2.

Comparison with OCaPIC: Here we compare the preceding results by running part of the test-program suite with OCaPIC, the OCaml virtual machine intended for microcontrollers in the PIC 18 family. In particular, we run these programs on a PIC 18f4620 microcontroller, with 64 KB of flash memory, 4 KB of RAM, and a computing speed of 10 MIPS. Table 7.3 shows the execution-time measurements for these programs[2]. Since OMicroB has not yet been fully ported to PIC 18 microcontrollers, these times are

[2]The other programs, which are too memory-hungry, could not be used on this hardware.

compared with those computed for the ATmega2560, extrapolating this microcontroller's computing speed so that it matches the 10 MIPS of the PIC 18f4620. In this sense, the performance comparison between OCaPIC and OMicroB can only be approximate here, and is intended solely to give a general idea of the order of magnitude of the relative performance of these two virtual machines. These measurements show that OMicroB is approximately 3.6 times slower (at equal computing frequencies) than the OCaPIC virtual machine. It is important to recall that the latter benefits from a bytecode interpreter and a garbage collector written in assembly, taking advantage of PIC-specific optimisations, which may explain its performance. In addition, certain hardware-specific aspects have consequences for program execution speed: for example, a PIC requires only 2 clock cycles to read one byte from flash memory, compared with 12 cycles on an AVR microcontroller.

Moreover, preliminary experiments aimed at porting OMicroB to PIC 18 highlighted the influence of the quality of compiler-generated code on the speed of executed OCaml programs: using the manufacturer-provided `xc8` compiler leads to execution speeds that are approximately 7 times slower on OMicroB than on OCaPIC, for the same program and on the same microcontroller. This relative slowness can partly be explained by the fact that the machine code generated by this compiler is rather poorly optimised. For example, the "*switch*" in the bytecode interpreter, whose role is to distinguish instructions according to their opcode so that they can be executed, was transformed by `xc8` into a cascade of nested conditional-branch instructions: program execution speed is plainly affected by this.

Name	**Execution Time with OMicroB on ATmega2560 at 10 MIPS** (in seconds)	**Execution Time with OCaPIC on PIC 18F4620 at 10 MIPS** (in seconds)	**Ratio OMicroB/OCaPIC**
`apply`	5.726	1.632	3.51
`fibo`	3.174	0.860	3.69
`takc`	8.76	2.535	3.46
`oddeven`	2.134	0.657	3.25
`integr`	1.192	0.252	4.73
`eval`	0.229	0.064	3.58
`sieve`	0.312	0.084	3.71
`bubble`	6.030	1.793	3.36
`share`	0.915	0.251	3.65

Table 7.3: Comparison of Execution Times with OCaPIC
OMicroB options: -arch 16 -gc SC -stack-size 500 -heap-size 2500

Nevertheless, OMicroB's temporal performance remains entirely acceptable overall, and the portability of the OMicroB virtual machine is an advantage that must be taken into account. Thus, given the portability of the solution, OMicroB's performance demonstrates the viability of using a generic virtual machine. In Chapter 8, we will present a set of applications that demonstrate the responsiveness of the virtual machine, as well as its suitability for executing embedded programs.

Comparison Elements with MicroPython: To give an indication of OMicroB's performance relative to pre-existing and fairly widely used solutions, we carry out a few comparative measurements between some example OCaml programs and their equivalents written in Python. The latter are executed by the

MicroPython implementation on a *pyboard* (version 1.1) equipped with an STM32F405RG microcontroller with 1024 kilobytes of flash memory, 192 kilobytes of RAM, and a processor with a computing speed of 168 MHz. As with the comparison with OCaPIC, we extrapolate the measurement results obtained on the pyboard in order to compare them with the OMicroB measurements obtained on an ATmega2560 at 16 MHz.

The comparison between MicroPython and OMicroB can only provide a broad indication of the relative performance of the two solutions, since the difference between programs written in OCaml and programs written in Python is substantial. Indeed, programming in an "*OCaml-like*" style makes extensive use of recursive functions and structures, whereas recursion in Python is quite strongly limited: for example, Python has no tail-call optimisation, which would make it possible to "de-recursify" certain functions. As a result, many of the test programs presented in this section are incompatible with execution under MicroPython, because they quickly consume the entire depth of Python's call stack. Moreover, this stack has a fairly small depth of only 63 levels on a pyboard.

In Table 7.4, we therefore present the measurement results obtained on a subset of our test programs, corresponding to those compatible with MicroPython without major changes to their structure. It should be noted that the `bubble` test, which performs bubble sort, uses Python *lists* in our Python version[3], since arrays from the Numpy library are unavailable in MicroPython. The `jdlv` test, for its part, was implemented using objects.

Name	**Execution Time with OMicroB on ATmega2560 at 16 MHz** (in seconds)	**Execution Time with MicroPython on pyboard (extrapolated to 16 MHz)** (in seconds)	**Ratio MicroPython/OMicroB**
`fibo`	1.984	5.481	2.76
`takc`	5.475	155.620	28.42
`bubble`	3.762	4.420	1.17
`jdlv`	2.006	3.024	1.5
`objet`	0.420	0.630	1.5

Table 7.4: Comparison of Execution Times with MicroPython
OMicroB options: -arch 16 -gc SC -stack-size 500 -heap-size 2500

The results for these few measurements are very promising: OMicroB is systematically faster at equal clock frequencies. In particular, we can note the higher speed of the `fibo` test (probably because of the large number of function calls) and OMicroB's much better performance on the `takc` test, which dynamically allocates many tuples during execution (and triggers many garbage-collection cycles), but OMicroB's performance is also good for the imperative and object-oriented tests. It should also be noted that the better results for the `fibo` and `takc` tests do not come from the OCaml compiler's tail-recursion optimisations, since they are not performed in `fibo` and only the outer recursive call of the Takeuchi function is optimised.

Size of the Programs and the Interpreter: Table 7.5 shows the footprint of each program in the microcontroller's flash memory and RAM. The almost constant RAM footprint of the programs comes from the

[3] Accessing and modifying an element in such lists nevertheless has complexity $\mathcal{O}(1)$, like OCaml arrays.

fact that stack and heap sizes are chosen at compilation time: as a result, constant arrays are generated statically to represent these memory sections, whose size cannot change during execution. It should be noted that the flash-memory size of the `objet` and `functor` programs is fairly large. For the first program, this is due to the import, into the program's *bytecode*, of the entire mechanism for creating and manipulating objects. This mechanism is fairly costly in terms of flash-memory occupation, although it should be noted that this overhead is stable: the memory footprint of a program manipulating ten times more objects and classes is not ten times larger. For the program that manipulates functors, the memory overhead is linked to the fact that *ocamlclean* is unable to detect statically the dead code present in a module resulting from a functor application. Thus, in our test, the generated bytecode contains all of the execution code of the entire module resulting from the application of the `Set.Make` functor, even though we only use a limited subset of that module's functions.

Name	Memory Footprint in Flash (bytes)	Memory Footprint in RAM (bytes)
`apply`	7626	6116
`fibo`	8614	6116
`takc`	8568	6114
`oddeven`	8230	6112
`floats`	10536	6128
`integr`	10870	6122
`eval`	13420	6239
`sieve`	9550	6119
`objet`	24120	6327
`functor`	20162	6241
`bubble`	11926	6217
`jdlv`	11772	6149
`share`	11122	6135
`abrsort`	13600	6219
`queens`	10704	6139

TABLE 7.5: Program Size Measurements for OMicroB (on ATmega2560)
OMicroB options: -arch 16 -gc SC -stack-size 500 -heap-size 2500

The virtual-machine interpreter, as well as its runtime library, also has a notable weight in these measurements. It is possible to estimate the size of the interpreter and of the *garbage collector* by measuring the size (in flash memory) of a program that only computes the value *unit*. The bytecode associated with this program is reduced to the single bytecode instruction `STOP`. Since the optimisation that removes, from the interpreter, the code for processing bytecode instructions unused by the program is enabled by default, the memory footprint of the resulting executable program is very small: in a 16-bit version of OMicroB, the `avr-size` command reports a size of 1240 bytes in flash memory. This value therefore corresponds to the size of the smallest possible OCaml program compiled with OMicroB. Conversely, when this same program is compiled with OMicroB's `no-clean-intepreter` option, the executable program size in flash memory is 21340 bytes. Consequently, the size of the bytecode interpreter and of the garbage collector (which is not loaded when the processing code for all bytecode instructions capable of allocating memory is removed) is on the order of 20 kilobytes. A program reduced to the single

`CLOSURE` instruction, which allocates a closure (and therefore includes the garbage collector), has a size of 3110 bytes when interpreter "cleaning" is enabled. Thus, the memory footprint of the *Stop and Copy* GC is on the order of 2000 bytes. The same measurements on the *Mark and Compact* garbage collector indicate a memory footprint on the order of 3000 bytes for that collector. In a 32-bit configuration of the virtual machine, the memory footprint of the interpreter and garbage collector is on the order of 28 kilobytes, while in 64-bit mode it is approximately 40 kilobytes.

These measurements therefore show that, without the optimisation provided by compiling a "custom-made" interpreter, programs using this virtual machine would quickly reach the flash-memory occupation limits of the microcontrollers targeted by our solution (for example, an ATmega32u4 has only 32 KB of flash memory). Thanks to this optimisation, the test programs consume on average only around ten kilobytes of flash memory, corresponding to the size of the interpreter specific to each program, its runtime library, and the program bytecode, which also resides in flash memory. Thus, this optimisation produces significant results and allows OMicroB to run on hardware into which the complete interpreter could not have been embedded. It should be noted that a program using all the bytecode instructions of the virtual machine would, of course, also import the entire bytecode interpreter. However, this case is very rare, and the programs presented in this manuscript illustrate the fact that most OCaml programs use only a fairly restricted subset (though one that varies from program to program) of the language's bytecode instructions.

7.2 OCaLustre Performance Measurements

In this section, we evaluate the performance of the OCaml code produced by OCaLustre, and its suitability for the hardware limitations of the microcontrollers considered in this thesis. To do so, we construct an example OCaLustre program that performs several computations, and measure the speed and memory occupation of the program executed in OMicroB. This example represents a *ripple-carry parallel adder*, which computes the binary sum of 8-bit words. This adder, shown in Figure 7.3, is made up of eight independent elements, each of which adds two values represented by a single bit. These 1-bit adders compute the sum of two bits, as well as any carry, and are said to be "*complete*" (in English: *full adders*) because they also take as input the value of the possible carry produced by the adder dedicated to computing the lower-order bit. The sum of two bytes ($a_7a_6a_5a_4a_3a_2a_1a_0$ and $b_7b_6b_5b_4b_3b_2b_1b_0$) is computed by adding each pair of bits (a_i, b_i) while propagating the carry computed by each 1-bit adder to the next adder.

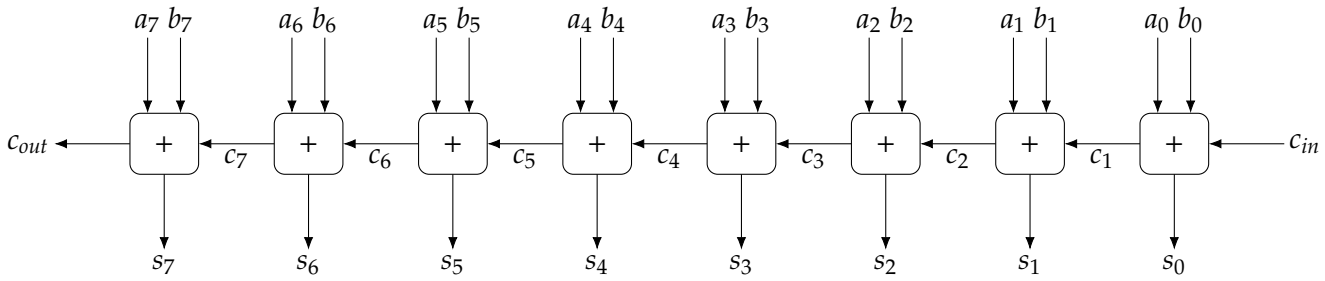


FIGURE 7.3: 8-Bit Adder

7.2.1 A Binary Adder in OCaLustre

A 1-bit adder is realised in OCaLustre by a node named `fulladder`, whose definition precisely follows the standard representation of an adder as a diagram made up of logic gates, as illustrated in Figure 7.4.

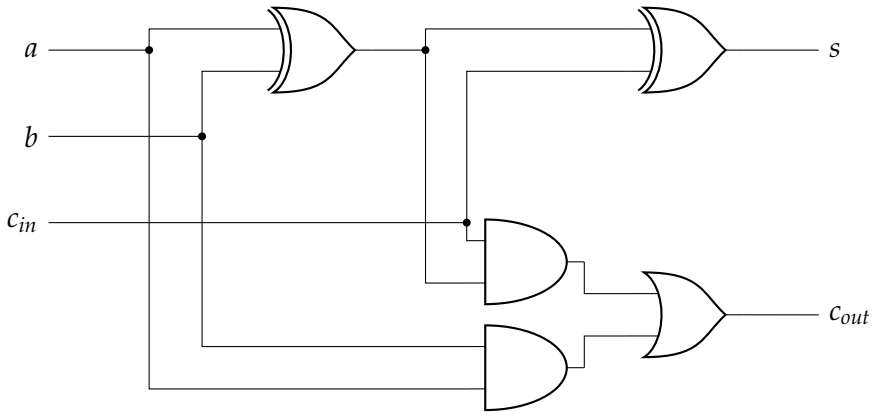


FIGURE 7.4: Full-Adder Circuit

The code of the `fulladder` node, together with the OCaml function `xor` used to compute the "exclusive or" of two Booleans, is then as follows:

```
let xor a b = if a then not b else b

let%node fulladder (a,b,cin) ~return:(s,cout) =
  x = call xor a b;
  s = call xor x cin;
  and1 = (x && cin);
  and2 = (a && b);
  cout = (and1 || and2)
```

This node computes the sum `s` of two bits `a` and `b` and an incoming carry `cin`, as well as the possible outgoing carry `cout`. Bit values are represented here by Booleans.

From a `fulladder` node, it is then easy to define an adder able to compute the sum of two 2-bit values, in the form of a `twobits_adder` node whose role is simply to chain the computations of two adders:

```
let%node twobits_adder (c0,a0,a1,b0,b1) ~return:(s0,s1,c2) =
  (s0,c1) = fulladder (a0,b0,c0);
  (s1,c2) = fulladder (a1,b1,c1)
```

This node therefore computes three Booleans corresponding to the sum $s_0 s_1$ of two values $a_0 a_1$ and $b_0 b_1$, and to the value of its possible carry c_2.

Following the same principle, we can then define an adder for four-bit values, and then an adder for values represented on 8 bits:

```
let%node fourbits_adder (c0,a0,a1,a2,a3,b0,b1,b2,b3) ~return:(s0,s1,s2,s3,c4) =
  (s0,s1,c2) = twobits_adder (c0,a0,a1,b0,b1);
```

```
  (s2,s3,c4) = twobits_adder (c2,a2,a3,b2,b3)

let%node eightbits_adder (c0,a0,a1,a2,a3,a4,a5,a6,a7,b0,b1,b2,b3,b4,b5,b6,b7)
                         ~return:(s0,s1,s2,s3,s4,s5,s6,s7,c8) =
  (s0,s1,s2,s3,c4) = fourbits_adder (c0,a0,a1,a2,a3,b0,b1,b2,b3);
  (s4,s5,s6,s7,c8) = fourbits_adder (c4,a4,a5,a6,a7,b4,b5,b6,b7)
```

This example perfectly illustrates the composability inherent in OCaLustre's synchronous approach: a node definition can be reused and combined by another node, which can itself be combined with another in order to build increasingly complex applications easily, without the developer having to worry about causality relations between the different components of the program.

7.2.2 Results

From the `eightbits_adder` node defined above, we build an OCaLustre program that computes the sum $0b11111111 + 0b11111111$ one hundred thousand times. This example is then executed, in the same way as the previous tests, using the OMicroB virtual machine on a PC.

Table 7.6 shows the performance measurements obtained on this program. In particular, OCaLustre's memory footprint is small: a stack of 80 words and a heap of 400 OCaml values are sufficient to execute such a program, for a total RAM consumption of only 1069 bytes and a flash consumption of 7668 bytes.

Name	Execution Time (seconds)	Speed (millions of instr./second)	Number of GC
`adders.ml`	0.54	138.01	550054

TABLE 7.6: Adder Speed Measurements with OMicroB (on PC)
OMicroB options: -arch 16 -gc SC -stack-size 80 -heap-size 400

From the point of view of execution speed, the performance of OCaLustre programs in OMicroB is good, although it is nevertheless slowed by the large number of GC invocations. As illustrated by the graph in Figure 7.5, a larger heap drastically reduces this number, and therefore speeds up program execution.

OCaLustre's `-na` compilation option generates "imperative" code optimised for critical embedded systems, in the sense that it does not allocate new values during a synchronous instant, and therefore avoids triggering the garbage collector while the synchronous program executes. As a result, the adder's execution speed is greatly increased when this option is used (Table 7.7).

Name	Execution Time (seconds)	Speed (millions of instr./second)	Number of GC Invocations
`adder-noalloc.ml`	0.21	383.14	0

TABLE 7.7: Adder Speed without Heap Allocations during an Instant
OMicroB options: -arch 16 -gc SC -stack-size 80 -heap-size 400

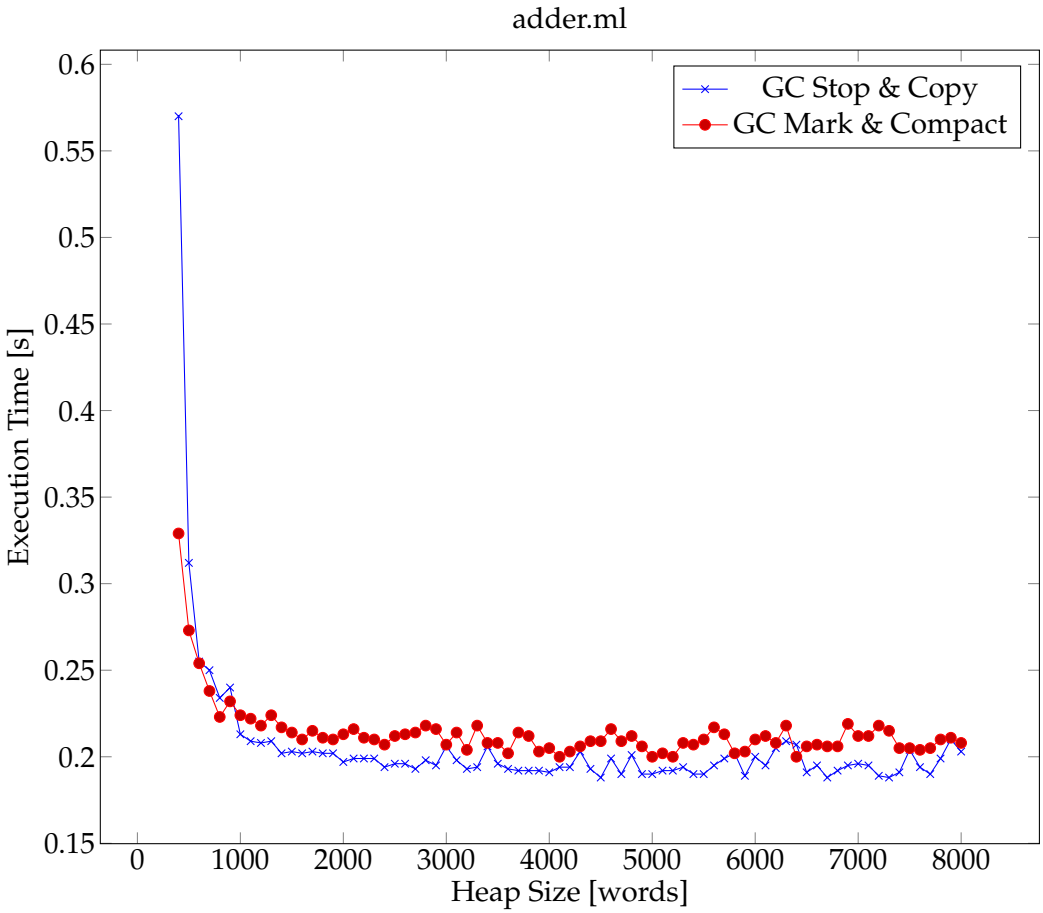


FIGURE 7.5: Adder Execution Time as a Function of Heap Size (on PC)

Comparison Elements with Lucid Synchrone: We now propose to compare the performance of this program with that of an equivalent program written in the Lucid Synchrone language. Although development of this language is now abandoned, its compiler (version 3, dating from April 2006) remains available on the project website [⚓19]. This compiler produces OCaml code, making comparison with OCaLustre possible. The main purpose of this comparison is to verify the relevance of using OCaLustre for programming resource-constrained hardware, by ensuring that the synchronous programs produced could not have been written in Lucid Synchrone while preserving equivalent performance. Of course, Lucid Synchrone is a rich extension of Lustre, whose expressiveness is greater than that of OCaLustre, and this richness is also reflected in the complexity of the generated code. The comparison proposed here is therefore not intended as a comparison of the raw performance of the two languages, but as an illustration of the space savings brought by using our syntax extension on the programs we target, owing to its relative simplicity. Figure 7.8 shows the measurements obtained on the Lucid Synchrone program `adder.ls` (whose code, very close to OCaLustre syntax, is given in Appendix B), as well as on the OCaLustre program in its standard version (`adder.ml`) and its version optimised for critical embedded systems (`adder-na.ml`). Since the memory requirements for executing the Lucid Synchrone program are higher, the measurements were performed with a stack of 150 words and a heap of 600 words.

These measurements show that, on this small program and with equal virtual-machine configurations, Lucid Synchrone's performance on this example is three to four times lower than OCaLustre's on a PC. On an ATmega2560, Lucid Synchrone's speed performance is 4 to 6 times lower than that of the OCaLustre

Name	Execution Time (seconds)	Speed (millions of instr./second)	Number of GC Invocations
`adder.ls`	0.79	222.87	440042
`adder.ml`	0.25	298.11	110010
`adder-na.ml`	0.19	423.47	0

TABLE 7.8: Speeds of Lucid Synchrone and OCaLustre Programs (PC)
OMicroB options: -arch 16 -gc SC -stack-size 150 -heap-size 600

programs[4] (Table 7.9). Indeed, because of the richness of the language, the OCaml code generated by Lucid Synchrone is much larger and more resource-intensive than that generated by OCaLustre: Figure 7.6 shows, for the `twobits_adder` node, the OCaml code generated by Lucid Synchrone and the code generated by OCaLustre compilation. In particular, the Lucid Synchrone version uses polymorphic variants, which are fairly costly, to represent certain values. It also generates several references to tuples at each instant, which quickly fills the heap of the virtual machine. Thus, the fact that the Lucid Synchrone program requires increasing the heap size and doubling the stack size is also quite penalising for use on microcontrollers whose memory resources are limited to a few kilobytes.

Name	Execution Time (seconds)	Speed (thousands of instr./second)	Number of GC	Flash Size (bytes)	RAM Size (bytes)
`adder.ls`	2.181	88.827	482	9250	1627
`adder.ml`	0.554	148.185	120	7552	1609
`adder-na.ml`	0.366	221.453	0	8472	1613

TABLE 7.9: Speeds and Sizes of Lucid Synchrone and OCaLustre Programs (ATmega2560)
OMicroB options: -arch 16 -gc SC -stack-size 150 -heap-size 600

Measurements on the Manuscript Examples: Table 7.10 shows the execution of OCaLustre programs taken from most of the examples discussed in this manuscript. The name of each program corresponds to the name of its main node, which is executed one million times. The measurements obtained validate the speed performance measured on the adder, and OCaLustre's low memory consumption. The OCaLustre extension consumes few hardware resources: its flash-memory footprint is comparable to that of other OCaml programs, and its RAM footprint is also low. This parsimonious use of resources makes OCaLustre compatible with many microcontroller models with memory capacities on the order of 2 kilobytes. Thus, OCaLustre provides a high-level programming model that makes it possible to build synchronous programs offering numerous guarantees (whether in terms of typing, clocks, manipulated data, or the detection of causal inconsistencies within programs). These advantages are all the more important because they do not increase resource consumption, and thus allow the development of embedded systems based on hardware with limited capabilities.

[4]The number of computed instants was once again divided by one thousand for our measurements on a microcontroller.

```
let twobits_adder _cl147 (_148__c0, _149__a0, _150__a1, _151__b0, _152__b1) _325
 _self_364 =
  let _self_364 = match!_self_364 with
                | ‘St_369(_self_364) -> _self_364
                | _ -> (let _368 = {_366 = ref ‘Snil_;
                                    _365 = ref ‘Snil_;
                                    _init329 = true} in
                        _self_364:= ‘St_369(_368);
                        _368) in
  let _cl339__ = ref false in
  let _338 = ref (false, false) in
  let _cl337__ = ref false in
  let _328 = ref false in
  let _340 = ref (false, false) in
   (if _cl147 then
     (_328:= (or) _325 _self_364._init329;
      _cl337__ := true;
      _338:= fulladder!_cl337__ (_149__a0, _151__b0, _148__c0)!_328 _self_364._365));
   (let (_153__s0, _154__c1) = !_338 in
    (if _cl147 then
      (_cl339__ := true;
       _340:= fulladder!_cl339__ (_150__a1, _152__b1, _154__c1)!_328 _self_364._366));
    (let (_155__s1, _156__c2) = !_340 in
     _self_364._init329 ← (&)!_328 (not _cl147);
     (_153__s0, _155__s1, _156__c2)))
```

```
let twobits_adder () =
  let fulladder1_app = fulladder () in
  let fulladder2_app = fulladder () in
  fun (c0, a0, a1, b0, b1) ->
    let (s0, c1) = fulladder1_app (a0, b0, c0) in
    let (s1, c2) = fulladder2_app (a1, b1, c1) in
    (s0, s1, c2)
```

FIGURE 7.6: OCaml Code for the `twobits_adder` Node Generated by Lucid Synchrone (top) and by OCaLustre (bottom)

Name	Execution Time with ocamlrun (seconds)	Execution Time with OMicroB (seconds)	Ratio	Execution Speed with OMicroB (millions of bytecode instructions/second)	Number of GC with OMicroB
arith	0.85	1.54	1.81	337.76	357142
blinker	0.02	0.04	2.00	521.67	0
call_cpt	0.06	0.11	1.83	488.14	19058
cpt	0.01	0.03	3.00	645.06	0
ex_const	0.04	0.09	2.25	518.05	17721
ex_norm	0.03	0.06	2.00	575.05	0
ex_tuples	0.06	0.11	1.83	396.31	37411
fibonacci	0.02	0.04	2.00	635.31	0
fibonacci2	0.02	0.04	2.00	660.56	0
merge	0.03	0.06	2.00	558.22	0
ordo	0.06	0.13	2.16	397.50	37411
two_cpt	0.03	0.08	2.66	481.79	18705
watch	0.06	0.13	2.16	428.58	40404
when	0.06	0.10	1.66	395.53	18365
whennot	0.03	0.05	1.66	548.65	0

TABLE 7.10: Performance Measurements of OCaLustre Programs (on PC)
OMicroB options: -arch 16 -gc SC -stack-size 50 -heap-size 500

Chapter Conclusion

The various measurements carried out in this chapter validate the fact that our approach, based on the use of a virtual machine, is viable and suitable for programming microcontrollers in a high-level language. The mere fact that OCaml programs can be executed, using a generic interpreter, on hardware with such limited resources is already a success. In addition, the performance of the OMicroB virtual machine and of the OCaLustre synchronous extension is entirely satisfactory both in terms of execution speed and in terms of the memory consumption of the generated programs: non-trivial applications can be envisaged for execution on hardware with very severe memory constraints. In the following chapter, we therefore present a few concrete applications that benefit from OMicroB and OCaLustre.

8 Applications for Electronic Circuits

We present a set of practical or playful applications built with the software solutions described in this manuscript. These applications, which can be executed on microcontrollers with small amounts of memory and computational resources, demonstrate the value of the high-level approach adopted in this thesis. It is indeed easy to build small embedded programs that benefit from the guarantees provided by the OCaml language and the OCaLustre synchronous layer. We present three example programs, whose purpose is to demonstrate the expressiveness of our solution while validating the fact that it is compatible with low-resource hardware. In particular, the programs produced can be executed on hardware with less than 8 kilobytes of RAM. Each program described below is linked to a concrete electronic circuit. The first example is a program controlling a simple punched-card reader, in which each card contains the binary representation of one byte. This example illustrates in particular the correspondence between the notion of a synchronous *clock* and that of "physical" clocks, which govern the arrival rate of electrical signals at the program input. The second program controls the operation of a *chocolate tempering machine*: a kitchen appliance used to melt chocolate at a precise temperature. The final example is the implementation of a "Snake" video game for an *Arduboy*, a small handheld device dedicated to running simple video games, conventionally written in C.

The full source code of these programs is available in Appendix D.

8.1 A Punched-Card Reader

For our first example, we build a microcontroller program that will be integrated into a circuit representing a punched-card reader. Each card contains the representation of a byte by means of two horizontal rows containing *holes*. The first row represents the *clock* signal: if the circuit recognises a hole in this row, it means that a datum must also be read. This *datum* is represented on the second row of the card: a hole represents the binary value 1, whereas the absence of a hole represents the value 0. Figure 8.1 is the schematic of a punched card: this card corresponds to the byte represented in binary by the value $0b10001010$[1].

8.1.1 Circuit

A fairly simple electronic circuit, represented by the schematic in Figure 8.2, can implement a reader for such punched cards. It is sufficient to connect two digital input pins (labelled I_0 and I_1 in the figure) of a microcontroller to two metal rods, each of which exerts slight pressure at the level of each row of holes in the punched card. The card itself rests on a metal plate supplied with an electric current. Once the punched card is slid between the plate and the metal rods, the values represented by holes (or their absence) can be read. Indeed, if a rod passes over a hole in the card, electrical contact is made

[1] Reading one byte from a card is done from left to right, using a *big-endian* representation.

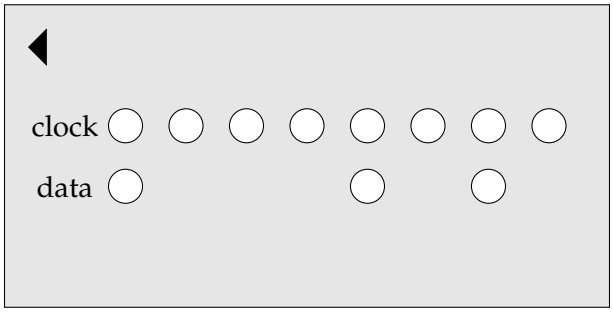


FIGURE 8.1: Schematic Example of a Punched Card

between this rod and the metal plate, and the associated microcontroller pin is therefore connected to the electric-current source: the value 1 (*HIGH*) is recognised by the device. Conversely, when a metal rod rests on the card, contact is not made, since the card is not conductive, and the microcontroller pin is then connected to the circuit ground (through a *pull-down* resistor): the value 0 (*LOW*) is measured. Finally, the binary value measured on a card is emitted by the microcontroller on a set of eight LEDs (connected to the pins labelled O_0 to O_7), each of which represents one of the bits of the byte present on the card: if the bit is 1, the corresponding LED is lit; otherwise it is off.

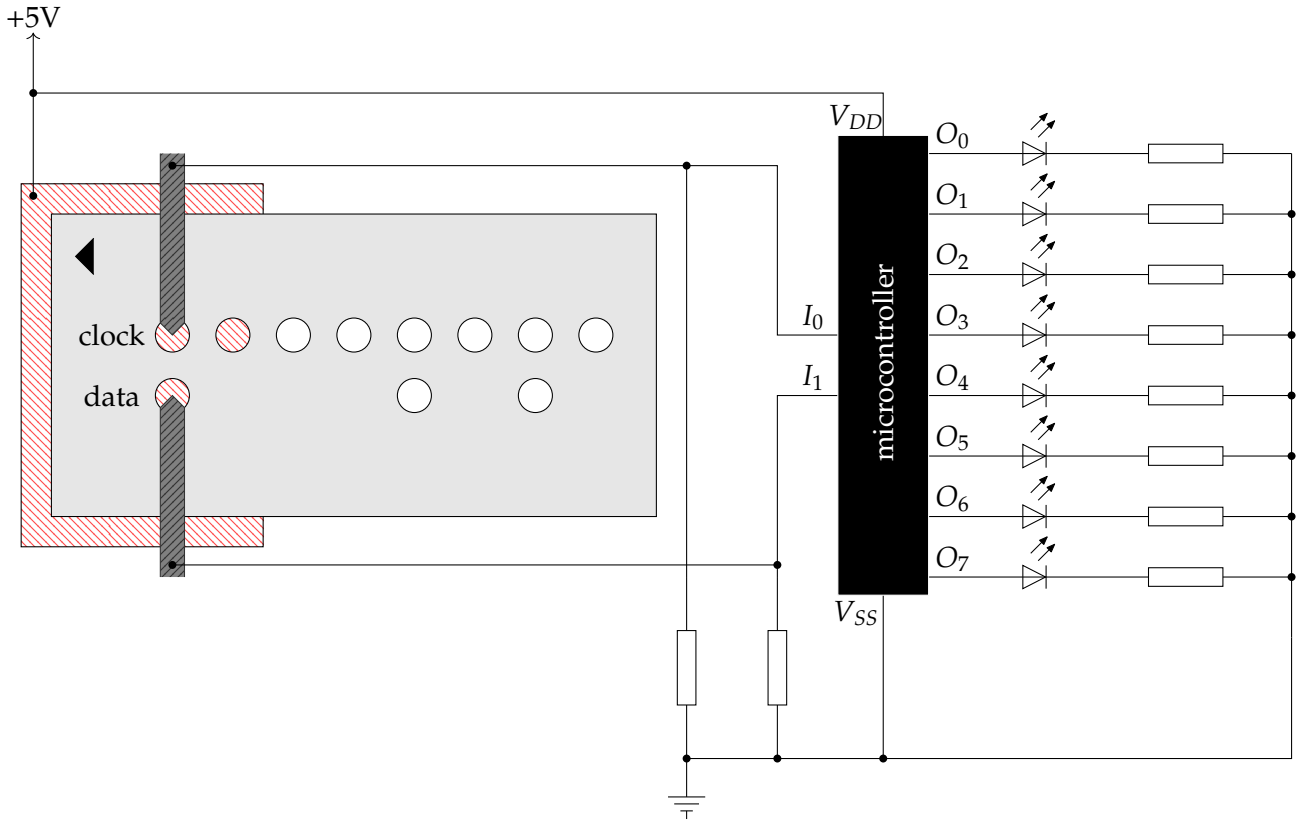


FIGURE 8.2: Punched-Card Reader Circuit

8.1.2 Program

Using a clock signal represented physically by a sequence of holes on the punched card illustrates the notion of a clock in a synchronous program. Thus, an OCaLustre program can easily be written to control the circuit described above. In this program, a clock signal must be *true* as soon as the metal rod positioned on the row representing the clock signal enters a hole. However, as long as the rod remains in the same hole, a sequence of `true` values must not be produced continuously: in reality, it is the *entry* into a hole that must indicate a *tick* of the clock signal. To do this, we define an `edge` node that detects such a rising edge:

```
let%node edge x ~return:e =
  e = (x && (not (true ⋙ x)))
```

Thanks to this node, we then define the `read_bit` node, which reads one bit on the card whenever the clock signal is true:

```
let%node read_bit (top,bot) ~return:(clk,data) =
  clk = edge top;
  data = bot [@when clk]
```

The stream `top` corresponds to the state of the first row of the punched card, and the stream `bot` to the state of the second row. It is interesting to note here that the "physical" representation of the clock by holes in the punched card translates directly into the definition of a synchronous clock signal `clk`, which represents the sampling frequency of the data signal `data`.

The program must detect that eight bits have been read. To do this, we declare a `count` node that counts the number of synchronous instants, from 0 to the integer preceding a reset value named `reset`:

```
let%node count (reset) ~return:(cpt) =
  cpt = (0 ⋙(cpt + 1)) mod reset
```

Each time the clock signal is true, the value of the data signal is stored in an array of 8 cells. Such an array `t` is then declared in OCaml:

```
let t = Array.make 8 false
```

In this program, once eight bits have been read, the corresponding byte is displayed by eight LEDs connected to the microcontroller. We then define a main node `read_card`, which represents the complete reading of the punched card: once 8 bits have been read, the stream `send` is set to true, in order to signal to the OCaml program that it can update the state of the LEDs:

```
let%node read_card (top,bot) ~return:(i,data,clk,send) =
  (clk,data) = read_bit (top,bot);
  i = count(8 [@when clk]);
  send = merge clk (i = 7) false
```

The index of the array cell corresponding to the bit currently being read is represented by the stream `i`. This stream is present (and its value is incremented) only when the clock `clk` is true.

On an AVR ATmega2560 microcontroller for which the metal rods are connected to pins 13 and 12, and the LEDs are connected to pins 42 to 49, the OCaml code responsible for the program's input/output is then as follows:

```
open Avr

let t = Array.make 8 false
let clk = PIN13
let data = PIN12
let leds = [| PIN42; PIN43; PIN44; PIN45; PIN46; PIN47; PIN48; PIN49 |]

let init () =
  (* configure the pins as inputs *)
  pin_mode clk INPUT;
  pin_mode data INPUT;
  (* configure the pins as outputs *)
  Array.iter (fun x -> pin_mode x OUTPUT) leds

let input_clk () = bool_of_level (digital_read clk)
let input_data () = bool_of_level (digital_read data)

(* function that switches one LED on/off *)
let update_led i b = digital_write leds.(i) (level_of_bool b)

let output i data clk send =
  if clk then t.(i) ← data;
  if send then Array.iteri update_led t
```

Finally, the code of the program's main loop, which interfaces between these functions and the synchronous OCaLustre program, is presented below:

```
let () =
 (* hardware initialisation *)
 init ();
 (* creation of the main node state *)
 let st = read_card_alloc () in
 while true do
   (* read inputs *)
   let c = input_clk () in
   let d = input_data () in
   (* update the main node state *)
   read_card_step st c d;
   (* emit outputs *)
   let i = st.read_card_out_i in
   let data = st.read_card_out_data in
   let clk = st.read_card_out_clk in
   let send = st.read_card_out_send in
   output i data clk send
 done
```

This code is compatible with OCaLustre's compilation model for real-time embedded systems. Thus, during execution of the program's main loop, no new OCaml value is allocated on the heap: the *garbage collector* is therefore never activated during execution of the synchronous program, and the execution time of a synchronous instant can be measured using *Bytecrawler*. The option `-m` followed by the name of the main node automatically generates the code for the program's main loop, as well as the prototypes of the functions that allow interaction between the synchronous program and its environment. We will illustrate the use of this option in the following examples.

8.1.3 Static Program Analyses

Using the tools described earlier in this manuscript, we can perform several static analyses on the program source code, as well as on its associated bytecode.

Clock Typing: The clock types induced by the clock-inference algorithm for the nodes of the punched-card reader program are the following[2]:

```
edge:: (x:base) -> (e:base)
read_bit:: (top:base * bot:base) -> (clk:base * data:(base on clk))
count:: (reset:base) -> (cpt:base)
read_card:: (top:base * bot:base) -> (i:(base on clk) * data:(base on clk) * clk:base * send:base)
```

These types correspond exactly to what is expected: in particular, the values of the streams `i` and `data` are present only when the clock stream `clk` is true. Using the merge operator `merge` to define the stream `send` makes it possible to oversample the value `(cpt=7)` on the base clock. This clock-typing information must be taken into account when implementing the interface functions between the synchronous program and the OCaml program: for example, it would be incorrect for these functions to access the value of the stream `i` when the stream `clk` is false.

Worst-Case Execution Time: On an AVR ATmega2560 microcontroller, the worst-case execution time of a synchronous instant of this program, including calls to input/output primitives, is estimated by the Bytecrawler tool at 57162 cycles.

It should nevertheless be noted that using arrays in the program involves calls to specific C primitives for reading and writing array cells. By default, these primitives check that the indices of the cells read or written are indeed smaller than the array size, and raise an exception otherwise. However, the Bound-T tool (which we also use to estimate the execution time of primitives) cannot bound the execution time of a function that may raise an exception. Consequently, we compile the program with the `-unsafe` option of the *ocamlc* compiler, in order to remove these array-bounds checks and thereby make it possible to estimate the maximum execution times of the primitives that read from or write to an array.

Since the microcontroller used has a computing frequency of 16 MHz, our punched-card reader is therefore able to process a (*clock*, *data*) pair in approximately 3.57 milliseconds. This short duration ensures that the punched-card user plainly cannot slide a card through the device too quickly and distort the reading.

[2]Clock-typing information for nodes is displayed using OCaLustre's `-i` option.

8.1.4 Memory Consumption

The minimum stack size for this program to run smoothly is 36 values, while the heap size is 318 values. Thus, in a 16-bit configuration of the virtual machine, 879 bytes of RAM are sufficient for this program to run. Its flash-memory footprint is approximately 12.1 kilobytes, which makes it compatible with microcontrollers whose memory resources are even more limited than those of the ATmega2560, such as the ATTiny1614, with 16 kilobytes of flash memory and 2048 bytes of RAM, and whose purchase cost is on the order of 60 euro cents.

Enabling anticipated evaluation, which consists in precomputing the values produced by initialising OCaml modules as soon as the program is compiled, further reduces its memory consumption: it then requires only a stack of 35 values and a heap of 160 values, for a flash-memory footprint of 10.8 kilobytes and a consumption of only 533 bytes of RAM.

8.2 A Chocolate Tempering Machine

For our second example application, we describe the circuit and program used to build a chocolate tempering machine. This device is a kitchen appliance that heats chocolate to a temperature specified by a user. Our tempering machine is built from a fairly inexpensive electronic circuit. In this circuit, an ATmega2560 microcontroller is connected to the following electronic components:

- An electric heating resistor that converts an electrical signal into heat.
- A temperature probe that converts the temperature of its environment into an analogue value.
- A two-line LCD screen of type LCD1602A, often supplied in microcontroller development kits.
- Two push buttons.

Figure 8.3 is a schematic representation of this electronic circuit.

The components of this circuit are used as follows: the heating resistor is placed in a container filled with water, and the chocolate is then heated in a bain-marie by the tempering machine, which controls the water temperature. As soon as the current temperature of the preparation is below the temperature desired by the user, the heating resistor is activated, increasing that temperature. The interface made up of the LCD screen and the two push buttons makes it possible to increase or decrease the desired temperature value.

The role of the program associated with this circuit is therefore to send an electrical signal on the pin connected to the resistor according to the value returned by the temperature sensor. The desired temperature is computed from successive presses of the push buttons. In parallel, the program displays at all times the current temperature of the preparation and the desired temperature on the LCD screen.

8.2.1 OCaLustre Program

The microcontroller at the heart of this circuit is responsible for executing the program by *synchronising* the different electronic components involved in the operation of the tempering machine. The synchronous programming model is therefore particularly well suited to such an embedded program: each node of

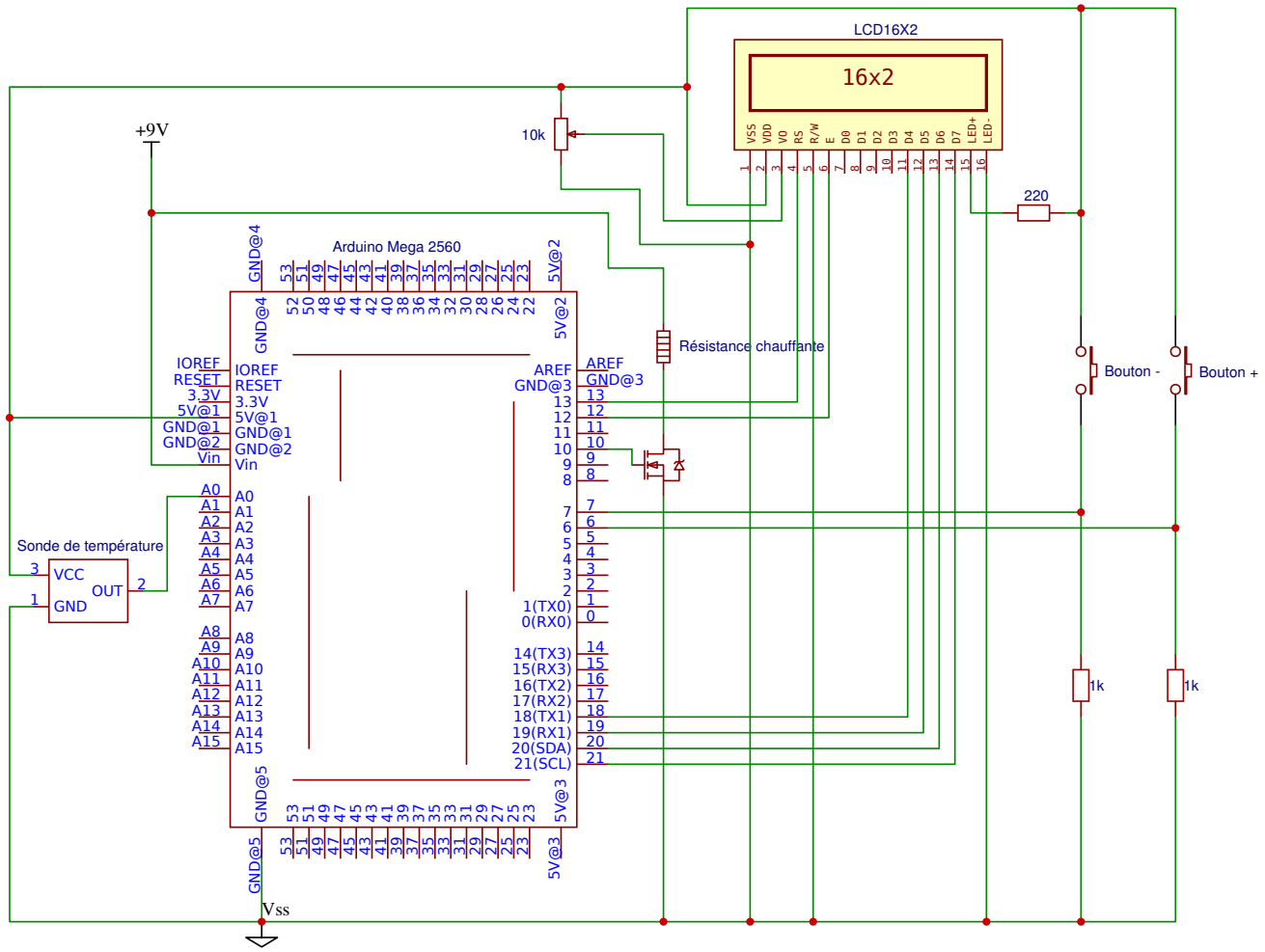


FIGURE 8.3: Schematic Representation of the Chocolate Tempering Machine

the synchronous program represents a component of the application, which must execute at the *same time* as the others, and their interaction governs the operation of the complete device.

The synchronous program associated with this circuit is then very simple: only three OCaLustre nodes are needed to define the program's behaviour:

```
(** switch on/off if + and - are pressed at the same time **)
let%node thermo_on (p,m) ~return:(b) =
  b = (true ⋙ if p && m then not b else b)

(** modify the desired temperature according to the pressed button **)
let%node set_wanted_temp (p,m) ~return:(w) =
  w = (325 ⋙ if p then w+5 else if m then w-5 else w)

(** main node: compute the desired temperature and resistor state **)
(** Temperatures are in tenths of degrees Celsius **)
let%node thermo (plus,minus,real_temp) ~return:(on,wanted,real,resistor) =
  on = thermo_on (plus,minus);
  wanted = set_wanted_temp (plus[@when on], minus[@when on]);
  real = real_temp [@when on];
  heat = (real < wanted);
  resistor = merge on heat false
```

The program manipulates temperature values as integers representing tenths of degrees Celsius. In particular, the main node receives the value of the two push buttons and the value measured by the temperature probe, and is responsible for producing five distinct streams:

- The stream `on` distinguishes the active state of the device from the standby state: the tempering machine switches from one to the other as soon as the two push buttons are pressed at the same time. This stream is the clock that governs the presence of most of the streams manipulated by the program.
- The stream `wanted` is defined as the result of calling the node `set_wanted_temp`, which increases or decreases the desired temperature (initialised to 32.5 °C), with a granularity of 0.5 °C, according to the buttons pressed by the user of the tempering machine. This stream is defined only if the stream `on` is true.
- The stream `real` is the result of subsampling the stream `real_temp`, which represents the value measured by the temperature probe, by the clock `on`.
- The stream `heat` defines the condition under which the resistor must start heating: if the real temperature is below the desired temperature, then this stream is *true*. It should be noted that, because we want to simplify the examples presented in this chapter, this version of the tempering machine is extremely naive: the inertia of the resistor's heating can cause the temperature to rise beyond the desired temperature. A "real" device would take into account the different physical aspects of the circuit environment in order to control temperature variations more finely, for example by stopping heating as soon as the preparation reaches a temperature close to the desired temperature, or by

computing the duty cycle needed to maintain a stable temperature. Nevertheless, these modifications go beyond the purpose of this chapter, whose aim is to illustrate, through simple examples, the possible applications of synchronous programming with OCaLustre. An OCaLustre program that measures the heating duty cycle is nevertheless given in Appendix D.2.3.

- The stream `resistor` results from merging the preceding stream with a constant stream computing the value `false`. This stream indicates whether the heating resistor should be powered. When the device is switched off, this resistor must be off (`false`); when the tempering machine is switched on, the value of the stream `heat` defines the state of the heating resistor.

Clock Typing and Node Signatures: The clock types of the nodes automatically inferred by OCaLustre during their compilation are the following:

```
thermo_on:: (p:base * m:base) -> (b:base)
set_wanted_temp:: (p:base * m:base) -> (w:base)
thermo:: (plus:base * minus:base * real_temp:base)
              -> (on:base * wanted:(base on on) * real:(base on on) * resistor:base)
```

In accordance with the description of the different streams defined by the program, the streams `wanted` and `real` returned by the node `thermo` are indeed clocked by the stream `on`: their values are therefore available only when the tempering machine is active.

8.2.2 Interaction Loop

OCaLustre's `-m <node>` option generates the code of the program execution machine, responsible for repeatedly calling the node `<node>`. This node then acts as the main node. This option generates a file `<node>_io.ml` pre-filled with the signatures of the functions `init_<node>`, `input_<node>`, and `output_<node>`, which correspond to the interaction functions between the program's synchronous kernel and the rest of the OCaml code. In addition, the option `-d <ms>` clocks the main loop, so that one execution of a synchronous instant of the program is performed every `<ms>` milliseconds. Since the chocolate tempering machine program does not involve significant speed constraints, an execution frequency of 1/500 ms seems sufficient: the program is therefore compiled with the options `-m thermo -d 500`.

A file `thermo_io.ml` containing the prototypes of the functions allowing interaction between the synchronous part of the program and its environment is then generated by OCaLustre. Once completed, the code of these functions is as follows:

```
(*** Interface functions for the synchronous instant ***)

(** initialisation function **)
let init_thermo () =
  (* analogue-read initialisation *)
  Avr.adc_init ();
  (* screen initialisation *)
  LiquidCrystal.lcdBegin lcd 16 2;
  (* pin initialisation *)
  pin_mode sensor INPUT;
  pin_mode resistor OUTPUT;
  pin_mode plus INPUT;
  pin_mode minus INPUT

(** input function **)
let input_thermo () =
  let plus = digital_read plus in
  let minus = digital_read minus in
  let plus = bool_of_level plus in
  let minus = bool_of_level minus in
  let real_temp = read_temp () in
  (plus,minus,real_temp)

(** output function **)
let output_thermo (on,wanted,real,res) =
  if on then
    begin
      print_temp wanted real;
      digital_write resistor (if res then HIGH else LOW)
    end
  else idle ()
```

These functions use auxiliary OCaml functions mainly responsible for converting the temperature measured by the electronic probe into degrees Celsius, as well as displaying the temperatures on the LCD screen. The code of these functions, together with the declarations of the different variables used by the program, is as follows:

```
open Avr

(* screen declaration *)
let lcd = LiquidCrystal.create4bitmode PIN13 PIN12 PIN18 PIN19 PIN20 PIN21

(* pin declarations *)
let plus = PIN7
let minus = PIN6
let resistor = PIN10
let sensor = PINA0

(* temperature conversion *)
let convert_temp t =
  let f = (float_of_int (1033 - t) /. 11.67) in
  int_of_float (f*.100.)

(* temperature reading *)
let read_temp () =
  let t = analog_read sensor in
  convert_temp t

(* temperature display on the LCD screen *)
let print_temp wanted real =
  let split_temp t =
    let u = t/10 in
    let dec = t mod 10 in
    (u,dec) in
  LiquidCrystal.home lcd;
  let (wu,wd) = split_temp wanted in
  let (ru,rd) = split_temp real in
  LiquidCrystal.print lcd "Wanted T:";
  LiquidCrystal.print lcd ((string_of_int wu)^"."^(string_of_int wd));
  LiquidCrystal.setCursor lcd 0 1;
  LiquidCrystal.print lcd "Actual T:";
  LiquidCrystal.print lcd ((string_of_int ru)^"."^(string_of_int rd))

let idle () =
  LiquidCrystal.home lcd; LiquidCrystal.clear lcd; LiquidCrystal.print lcd "..."
```

8.2.3 Memory Consumption

The tempering-machine program requires at minimum a stack of 61 values and a heap of 448 values, which, in a 16-bit representation of OCaml values, amounts to a total memory footprint of 17.3 kilobytes of flash memory and only 1277 bytes of RAM (with the *Stop and Copy* GC[3]). Nevertheless, because

[3]It should be noted that, because this GC algorithm is used, the number of OCaml values that can actually be allocated corresponds to half the heap size.

frequent activation of the GC algorithm slows down program execution (as we illustrated in the previous chapter), it would be regrettable not to benefit from the full extent of the RAM of the microcontroller used. A heap of 3800 words thus makes it possible to fill almost all of the 8 kilobytes of RAM of the ATmega2560 microcontroller, reducing the number of GC-algorithm invocations during the first one hundred synchronous instants from 699 to only 4.

8.2.4 Simulation

OMicroB's integrated simulator makes it possible to execute this program directly on a computer, mainly to make debugging easier. Figure 8.4 shows the simulator interface for this program: the state of the heating resistor is represented here by an LED located between the two + and - buttons used to adjust the temperature. The horizontal bar below these elements makes it possible to choose the value measured on the analogue input pin to which the temperature probe is actually connected.

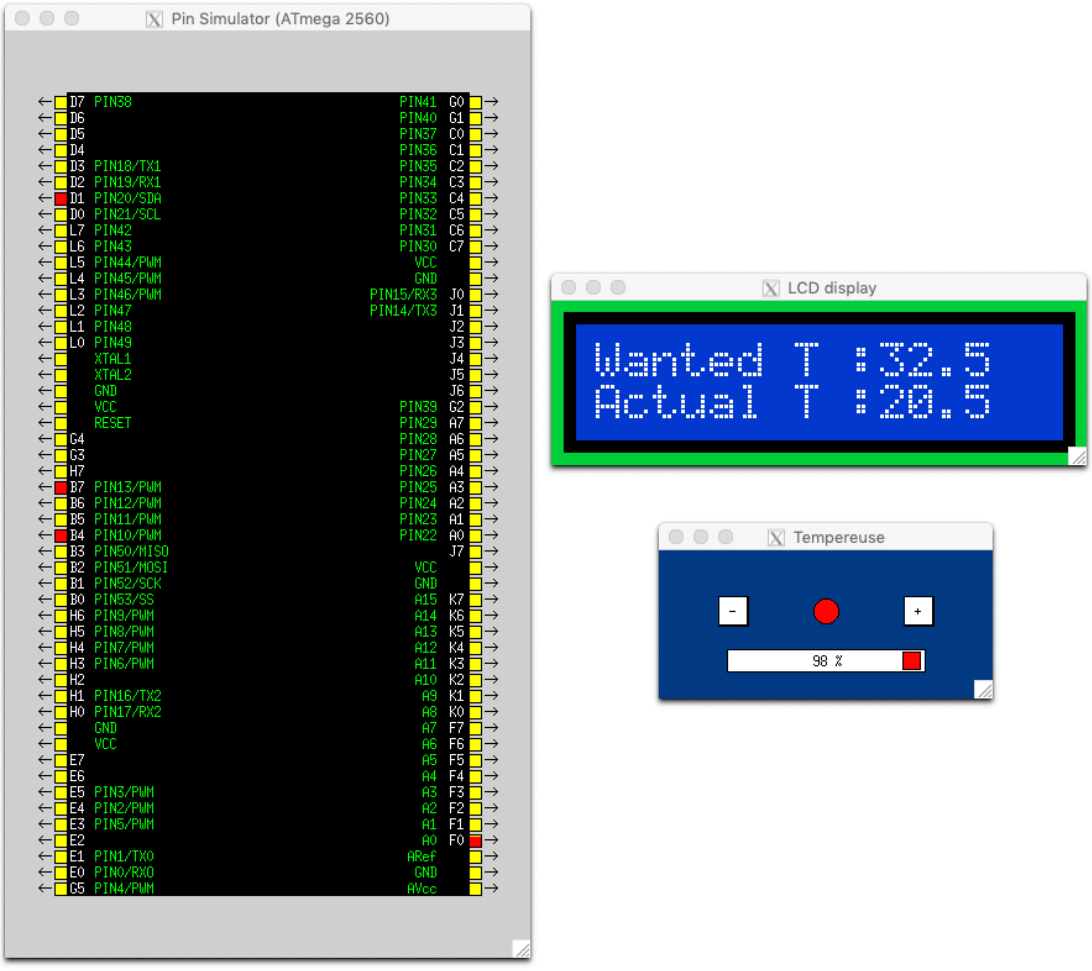


FIGURE 8.4: Execution of the Tempering-Machine Program in the Simulator

8.3 A Snake Game

The final application presented in this manuscript is a small "Snake" video game, in which a snake, represented by a sequence of contiguous cells, moves in a two-dimensional world in order to eat apples,

which appear as the game evolves. Whenever the snake eats an apple, it grows by one cell, and a new apple appears on the screen. The aim of the game is therefore for the snake to eat as many apples as possible in order to reach a certain size, while avoiding collision with itself, which kills it (and causes the game to be lost).

This video game was very popular in the late 1990s and early 2000s, during the spread of the mobile phone: phones of that period, whose computing speeds were on the order of those of the microcontrollers studied in this thesis[4], could indeed run a few simple video games adapted to the limited performance of mobile processors at the time. We therefore detail in this section an implementation of this game adapted to a small device whose hardware resources are likewise very limited. This device, called the *Arduboy*, is a small credit-card-sized handheld game console intended for the development of small games often associated with the *retrogaming* movement. For performance reasons and precise use of the device's resources, programs developed by the Arduboy community are generally written in C. Through this example, however, we demonstrate that it is entirely conceivable to develop an application for it with the OMicroB/OCaLustre pair. The software solutions presented in this thesis therefore make it possible to use high-level abstractions on hardware whose memory resources are very constrained. Figure 8.5 shows an Arduboy running the Snake game.

FIGURE 8.5: Un Arduboy

The full source code of the modules presented in this section is available in Appendix D.3.

8.3.1 Anatomy of an Arduboy

The main component of an Arduboy is a microcontroller from the AVR family: the ATmega32u4. This microcontroller is part of what we consider to be strongly constrained hardware: its computing power could theoretically reach 16 MHz, but since the Arduboy battery has a fairly low voltage (between 3.4 V and 3.7 V), the actual performance of this microcontroller is more on the order of 11 MHz to 12 MHz. In addition, the ATmega32u4 has 32 KiB of flash memory and only 2.5 KiB of RAM, which requires the programs produced to have a very small memory footprint.

[4]For example, the Nokia 3310 marketed in 2000 had a 13 MHz Texas Instruments MAD2WD1 processor.

An Arduboy also contains several electronic components mainly intended for user interaction: it has a set of 6 push buttons (four representing direction arrows, up, down, left, and right, and two action buttons, A and B), a monochrome OLED (*Organic Light-Emitting Diode*) matrix screen with a resolution of 128 × 64 pixels, three LEDs (one red, one green, and one blue) positioned next to the screen, and a piezoelectric speaker able to play low-definition sounds. For our application, we use the screen to display the snake and the apple to be eaten, the left and right direction buttons to move the snake, and the LEDs to indicate to the player whether the game is won (green) or lost (red).

The electronic schematic representing the Arduboy components used by our program is given in Figure 8.6.

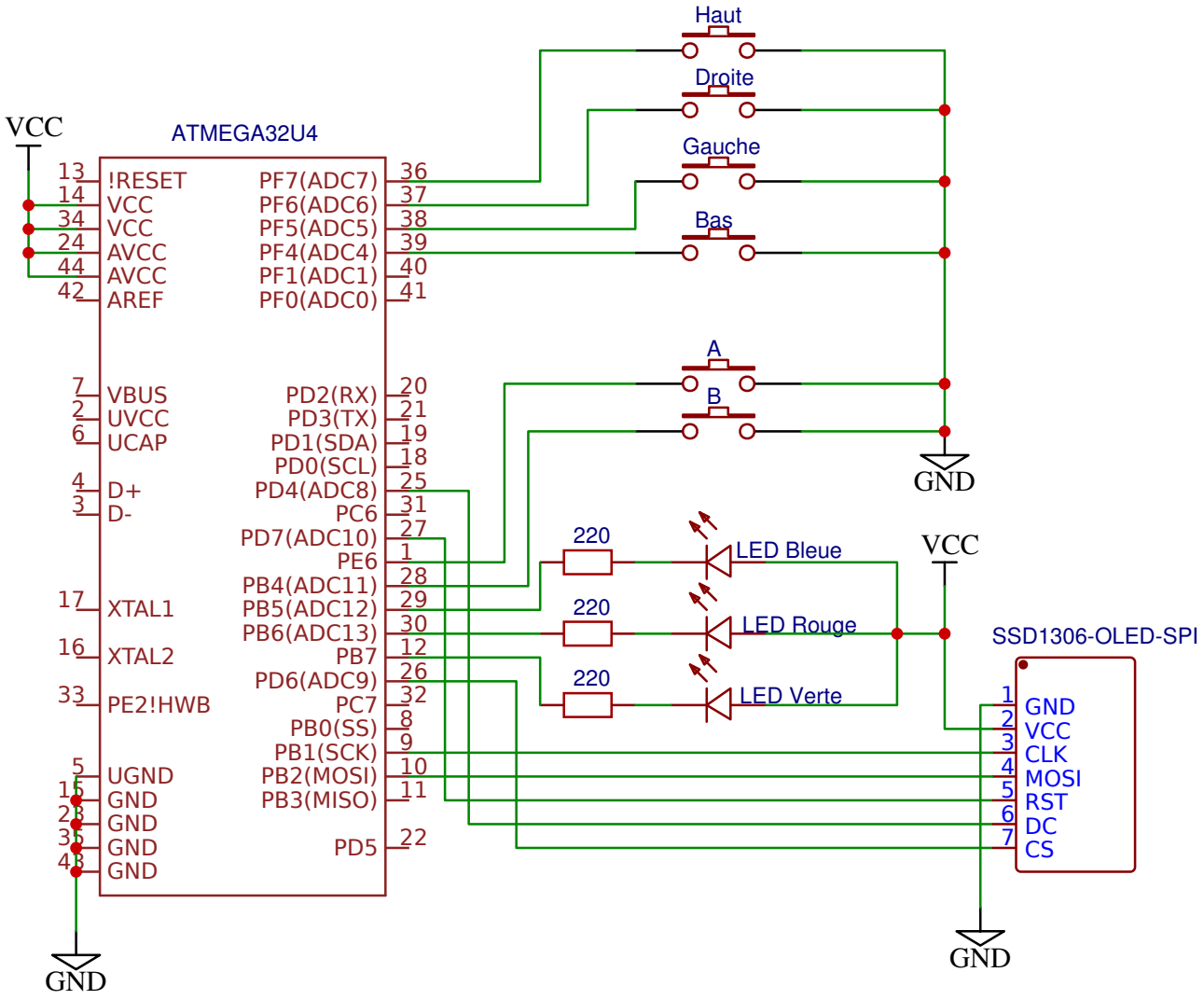


FIGURE 8.6: Arduboy Components Used by the Snake Game

8.3.2 Program for the Electrical Circuit

The program produced takes into account the technical specificities of the Arduboy: we therefore first describe the code of the OCaml modules that allow interaction between the microcontroller and the input/output components required for the game to operate correctly.

SPI Link and Connection to the OLED Screen: In an Arduboy, data transfer between the microcontroller and the OLED screen (of type SSD 1309) is performed through an SPI (*Serial Peripheral Interface*) interface using two wires:

- One wire is connected to the SCK (*Serial Clock*) pin of the microcontroller and represents the clock signal used to pace communication.
- One wire is connected to the MOSI (*Master Output, Slave Input*) pin of the microcontroller and represents the command or data signal sent to the screen.

The SPI link is implemented in hardware by the microcontroller: it only needs to be initialised correctly[5] for any byte placed in the `SPDR` register to be emitted by the microcontroller. The OCaml code used to configure and interact with the SPI link is represented by an *Spi* module, whose code is shown in Figure 8.7. This module uses the *Avr* module provided in OMicroB's standard library, which defines the microcontroller registers and pins, as well as functions for changing their state.

```
(*** SPI (Serial Peripheral Interface) connection management module ***)

open Avr

(** Initialise the SPI connection **)
let begin_spi ~sck ~mosi =
  set_bit SPCR MSTR;
  set_bit SPCR SPE;
  set_bit SPSR SPI2x;
  pin_mode sck OUTPUT;
  pin_mode mosi OUTPUT

(** Stop the SPI connection **)
let end_spi () = clear_bit SPCR SPE

(*** Emit data through the SPI connection ***)
let transfer data = write_register SPDR data
```

FIGURE 8.7: The `spi.ml` Module Allowing Interaction with a Serial Peripheral

In addition, the signals used to control the screen are carried by three distinct wires:

- A wire connected to the D/C (*Data/Command*) pin of the screen makes it possible to place it in "command" mode or in "data" mode.
- A wire connected to the CS (*Chip Select*) pin of the screen allows communication between the screen and the microcontroller only if it is at the low level (*LOW*).
- A wire connected to the RST (*Reset*) pin of the screen makes it possible to reset the screen: if this pin goes from the high level (*HIGH*) to the low level (*LOW*), the screen resets.

[5]The technical details involved in configuring the SPI link by changing the value of certain bits in several registers are outside the scope of this manuscript.

Furthermore, in order to illustrate interoperability between OCaml and C, the screen data will be represented in a C array of bytes (in which each byte represents 8 points on the screen) acting as a buffer. It should be noted that, in order to reduce the program's memory footprint and make the game easier to read, the screen resolution actually used by Snake will be 64×32 points. The display will nevertheless occupy the full width and height of the screen: the game's "points" will therefore have a size of 2×2 pixels on the screen.

The OCaml module *Oled*, an excerpt of which is shown in Figure 8.8, contains the functions needed to manipulate the screen and the buffer. In particular, a function `Oled.flush` transfers to the screen a new image representing the state of the game. Details of the screen characteristics and configuration are available in the SSD 1306 screen datasheet [⚓4].

```
(*** Write a point (true=black/false=white) ***)
let draw x y color = write_buffer x y color

(*** Clear the screen ***)
let clear () =
 for _i = 0 to 1023 do
   Spi.transfer(0x00)
 done

(*** Send the buffer to the screen ***)
let flush () =
  for _i = 0 to 1023 do
    Spi.transfer(get_byte_buffer())
  done

(*** Screen initialisation ***)
let boot ~cs ~dc ~rst =
  digital_write rst HIGH;
  digital_write rst LOW;
  digital_write rst HIGH;
  command_mode cs dc;
  transfer_program boot_program;
  data_mode cs dc;
  clear()
```

FIGURE 8.8: The `oled.ml` Module (Excerpt) Used to Configure and Control the SSD 1306 Screen

Configuring the Arduboy: Finally, an Arduboy module (Figure 8.9) configures its components through functions from the standard library. The pin numbers used correspond to the naming convention of the Arduino library (in this case, that of the Arduino Leonardo, which also contains an ATmega32u4 microcontroller).

- The function `pin_mode` defines a pin as an input (`INPUT`), an output (`OUTPUT`), or an input connected through a *pull-up* resistor. AVR pins have an internal *pull-up* resistor, which means that the value measured on a pin is at the high level (*HIGH*) when it is not connected to any component. Consequently, when the Arduboy push buttons are open, the value read is *HIGH*, and when they are closed, the value read is *LOW*, because they are connected to the circuit ground.

```
open Avr

(** The pins used **)
let cs = PIN12
let dc = PIN4
let rst = PIN6
let button_left = PINA2
let button_right = PINA1
let button_down = PINA3
let button_up = PINA0
let button_a = PIN7
let button_b = PIN8
let blue = PIN9
let red = PIN10
let green = PIN11

(** initialisation of the common-anode RGB LEDs (HIGH = off) **)
let init_led l =
  digital_write l HIGH

(** switch on one common-anode RGB LED (LOW = on) **)
let light_led l =
  digital_write l LOW

(** pin initialisation **)
let boot_pins () =
  List.iter (fun x -> pin_mode x INPUT_PULLUP) [button_left; button_right; button_up;
                                                button_down];
  pin_mode button_a INPUT_PULLUP;
  pin_mode button_b INPUT_PULLUP;
  List.iter (fun x -> pin_mode x OUTPUT) [red;green;blue];
  List.iter (fun x -> pin_mode x OUTPUT) [cs;dc;rst];
  List.iter init_led [red;green;blue]

(** initialisation of the pins, SPI link, and screen **)
let init () =
  boot_pins ();
  Spi.begin_spi ~sck:SCK ~mosi:MOSI;
  Oled.boot ~cs:cs ~dc:dc ~rst:rst
```

FIGURE 8.9: The `Arduboy.ml` Module: Hardware Configuration and Initialisation Code

- The function `digital_write` writes a binary value (high level or low level) to a pin.

This module, whose implementation follows the electronic schematic representing the connections between the microcontroller and the external components, configures all the hardware needed for this application: in particular, the function `init` configures the pins used by the program, activates the SPI link, and initialises the screen.

The different modules presented in this subsection in fact implement a *library* for controlling the Arduboy's internal circuit. This library can be used for the implementation of our Snake game, but it can also be reused for many other projects targeting this hardware. Indeed, all the functionality presented in this part simply reflects the characteristics of the electronic circuit and the relations between its different components, which do not change from one program to another. This library illustrates the fact that using a high-level language such as OCaml is also suitable for defining low-level interactions, as is the case here for the electronic interactions between the ATmega32u4 microcontroller and the components to which it is connected.

8.3.3 Game Program

Using the library described in the previous section, we now turn to the details of the implementation of the Snake game. This game has the structure of a synchronous program that uses both an OCaLustre kernel, representing reactions to the player's actions, and OCaml functions whose role is mainly to handle communication between this kernel and external peripherals such as the screen or the directional arrows.

In this program, the snake is an array `snake`, each cell of which represents the positions of the different sections of the snake's body:

```
let max_size = 15
let snake = Array.make max_size (0,0)
```

Two values `head` and `tail` represent the index of the array cell corresponding respectively to the head and to the tail of the snake. To represent the snake's movement, at each instant of the program these two pointers each advance to the cell with the immediately higher index, and the position of the new snake head is written into the new cell pointed to by `head`. As illustrated in Figure 8.10, the array has a circular structure: when either of these pointers leaves the array after moving, it is repositioned at index 0.

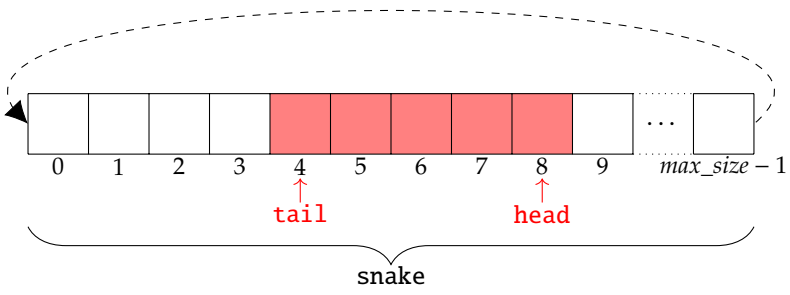


FIGURE 8.10: Snake Structure

Synchronous Kernel: The game loop is responsible for computing the snake's direction according to whether the right or left button is pressed, computing the new position of its head, and taking into account the fact that the snake grows as soon as its head reaches the same position as the apple (in other words, as soon as it eats the apple). The code for this loop is an OCaLustre node that receives as input the maximum size of the snake, the state of the left and right buttons, and the dimensions of the world. It computes the new value of the `head` and `tail` pointers, the new position of the snake's head `(new_x,new_y)`, the position of the apple `(apple_x,apple_y)`, and indicates whether the player has won (`win`):

```
let%node game_loop (max_size,left,right,width,height)
     ~return:(head,tail,new_x,new_y,apple_x,apple_y,win) =
  (new_x,new_y) = new_head (dir,width,height);
  dir = direction(left,right);
  head = (1 ⋙((head+1) mod max_size));
  eats = (apple_x = new_x && apple_y = new_y);
  (a_x,a_y) = new_apple (width [@when eats], height [@when eats]);
  apple_x = (10 ⋙merge eats a_x (apple_x [@whennot eats]));
  apple_y = (10 ⋙merge eats a_y (apple_y [@whennot eats]));
  tail = merge eats ((0⋙tail)[@when eats]) (0 ⋙((tail+1) mod max_size)[@whennot eats]);
  size = (1 ⋙merge eats ((size + 1) [@when eats]) (size [@whennot eats]));
  win = (size = max_size - 1)
```

This node defines a clock `eats`, which is true as soon as the snake eats the apple. This clock governs the execution of several computations:

- When `eats` is true, a new position `(a_x,a_y)` for the apple is computed by a `new_apple` node:

```
(** Random draw of an integer **)
let new_position n = Random.int n

(** New position of the apple **)
let%node new_apple (width,height) ~return:(a_x,a_y) =
  (a_x,a_y) = (call new_position width, call new_position height)
```

- When `eats` is true, the size `size` of the snake increases.
- If `eats` is true, the pointer to the snake's tail (`tail`) is not shifted, which simulates the snake's growth.

The game loop also calls a `direction` node to compute the snake's direction:

```
let left_of = function South -> East | North -> West | East -> North | West -> South
let right_of = function South -> West | North -> East | East -> South | West -> North

(** Turn left **)
```

```
let%node left (dir) ~return:ndir =
  ndir = call left_of dir

(** Turn right **)
let%node right (dir) ~return:ndir =
  ndir = call right_of dir

(** Snake direction from its previous direction **)
let%node direction (l,r) ~return:dir =
  pre_dir = (South ⋙ dir);
  dir = if l then
          (merge l (left (pre_dir [@when l])) (pre_dir [@whennot l]))
        else
          (merge r (right (pre_dir [@when r])) (pre_dir [@whennot r]))
```

In particular, using the ⋙ operator defines the stream `dir` according to its previous value: thus, at the beginning of the program, the snake moves south, then, when the left (respectively right) button is pressed, it turns left (respectively right) relative to its previous position. The snake keeps the same direction if no button is pressed.

The `game_loop` node also calls a `new_head` node, which computes the new position of the head:

```
(** Modulo **)
let nmod x y = (x + y) mod y

(** New coordinate **)
let%node new_coord (dir,max,v,dir1,dir2) ~return:n =
  n = if dir = dir1 then call nmod (v-1) max
      else if dir = dir2 then call nmod (v+1) max
      else v

(** New position of the snake's head **)
let%node new_head (dir,w,h) ~return:(x,y) =
  x = new_coord (dir,w,0 ⋙ x,West,East);
  y = new_coord (dir,h,0 ⋙ y,North,South)
```

Finally, the main node `main` of the OCaLustre program detects a rising edge on the two buttons used to turn the snake left or right, and calls the node implementing the game loop:

```
(** Rising edge (for the buttons) **)
let%node rising_edge i ~return:o =
  o = (i && (not (false ⋙ i)))

(** Main node **)
  let%node main (max_size,button1,button2,width,height)
    ~return:(head,tail,nx,ny,apple_x,apple_y,win) =
  left = rising_edge (button1);
```

```
right = rising_edge (button2);
(head,tail,nx,ny,apple_x,apple_y,win) = game_loop(max_size,left,right,width,height)
```

Main Loop and Interaction Functions: Just as in the previous example, OCaLustre's `-m` option is used to generate the code for the program's execution machine, responsible for repeatedly executing its main node.

Once the `-m main` option has been provided, the file `main_io.ml` is generated by OCaLustre with the prototypes of the interaction functions. After completion, this code essentially contains the functions needed to display the game and to read the values of the Arduboy buttons. Nevertheless, since the current version of OCaLustre does not handle arrays, the function that checks whether the snake collides with itself (named `eats_itself`) is also defined in this file, directly in OCaml.

8.3.4 Memory Consumption

In a 16-bit configuration of the virtual machine, and with the *Stop and Copy* garbage collector, the Snake game program can run with a minimum stack size of 60 values and a heap of at least 744 values, for a flash-memory footprint of 17.1 kilobytes and 2204 bytes of RAM, which comes very close to the 2.5 kilobytes of RAM available on the microcontroller. Using anticipated evaluation nevertheless reduces the stack size to 58 values, and using the *Mark and Compact* garbage collector doubles the memory-allocation area (and therefore reduces the frequency of GC calls) without increasing the RAM size. Over the first one hundred synchronous instants of the program, the number of invocations of the memory-cleaning algorithm thus falls from 49 with the *Stop and Copy* GC to 18 with the *Mark and Compact* GC.

Chapter Conclusion

The examples presented in this chapter have demonstrated the possibility of executing simple, yet complete, programs on devices with very limited resources. These examples also highlight the advantages of using high-level programming paradigms. The programs produced benefit from the guarantees provided by high-level languages (worst-case execution-time estimation, typing, causality-loop detection) while remaining executable on hardware whose resources are highly constrained. Thus, a microcontroller with less than 2.5 kilobytes of RAM is able to execute all the programs described in this chapter. This very small footprint of OCaLustre programs embedding the OMicroB virtual machine makes it reasonable to envisage execution on hardware with greater memory resources (such as microcontrollers based on ARM Cortex-M0 processors, with memory resources that can reach several hundred kilobytes of flash memory and several tens of kilobytes of RAM), for the production of even richer and more complex programs, benefiting from advanced components such as Bluetooth modules, accelerometers, or multicolour touch screens.

Conclusion and Perspectives

The aim of this thesis was to propose a set of solutions for programming microcontrollers in high-level languages, in order to demonstrate that it is entirely possible to benefit from richer programming models while respecting the constraints of such hardware. To this end, we have described in this manuscript a series of software solutions and formal approaches that provide a progressive rise in abstraction, bringing new guarantees about the programs produced at each level of this succession of abstractions. In this conclusion, we recall the various contributions made by this thesis, in light of the rise in abstraction that they illustrate and of the new guarantees they provide. We then discuss the various perspectives and future work that would be appropriate for continuing our approach of improving the safety and expressiveness of microcontroller programming.

Contributions

Figure 9.1 schematically summarises the positioning of each chapter of this manuscript with respect to the abstraction gains conferred by the different approaches presented in this thesis.

The first contribution of this thesis was based on a *virtual-machine approach* enabling the execution of multi-paradigm languages on varied hardware. We therefore proposed *OMicroB*, a virtual machine for the OCaml language, designed with the aim of being ported to many microcontroller models while retaining a limited memory footprint. Configurable and optimised, this virtual machine has been executed on microcontrollers whose memory resources do not exceed 8 kilobytes of RAM and a few tens of kilobytes of flash memory. Moreover, OMicroB's speed performance is quite promising, since it is relatively close to that of the standard OCaml virtual machine, which does not benefit from the same advantages in reducing memory occupation. OMicroB also offers a better approach for program debugging, first because its complete compatibility with the bytecode generated by the standard *ocamlc* compiler makes it possible to benefit from classic OCaml analysis and debugging tools (such as the *ocamldebug* debugger), but also because its simulator can represent the components of an electronic circuit to which the microcontroller to be programmed will be connected, and test their interactions directly from the computer on which the application is developed. In addition, the safety of typing statically verified by the OCaml compiler greatly reduces the number of potential program-execution errors. Through the portability of its bytecode, the expressiveness of the language, and the use of automatic memory-management mechanisms, using OMicroB thus provides an abstraction of the hardware on which the program will execute, making it possible to calmly consider deploying the same program on several different devices. Our work on the virtual-machine approach and on the implementation of OMicroB was presented at the ERTS^2 conference (*Embedded Real Time Softwares and Systems*) in 2016 [VVC16] and in 2018 [VVC18a].

Hardware abstraction formed a first foundation for the following abstractions. We then addressed the fact that a very large number of embedded systems, in which microcontrollers play a central role, expose *concurrent* aspects. This concurrency, inherent in the multiple interactions between such a system and

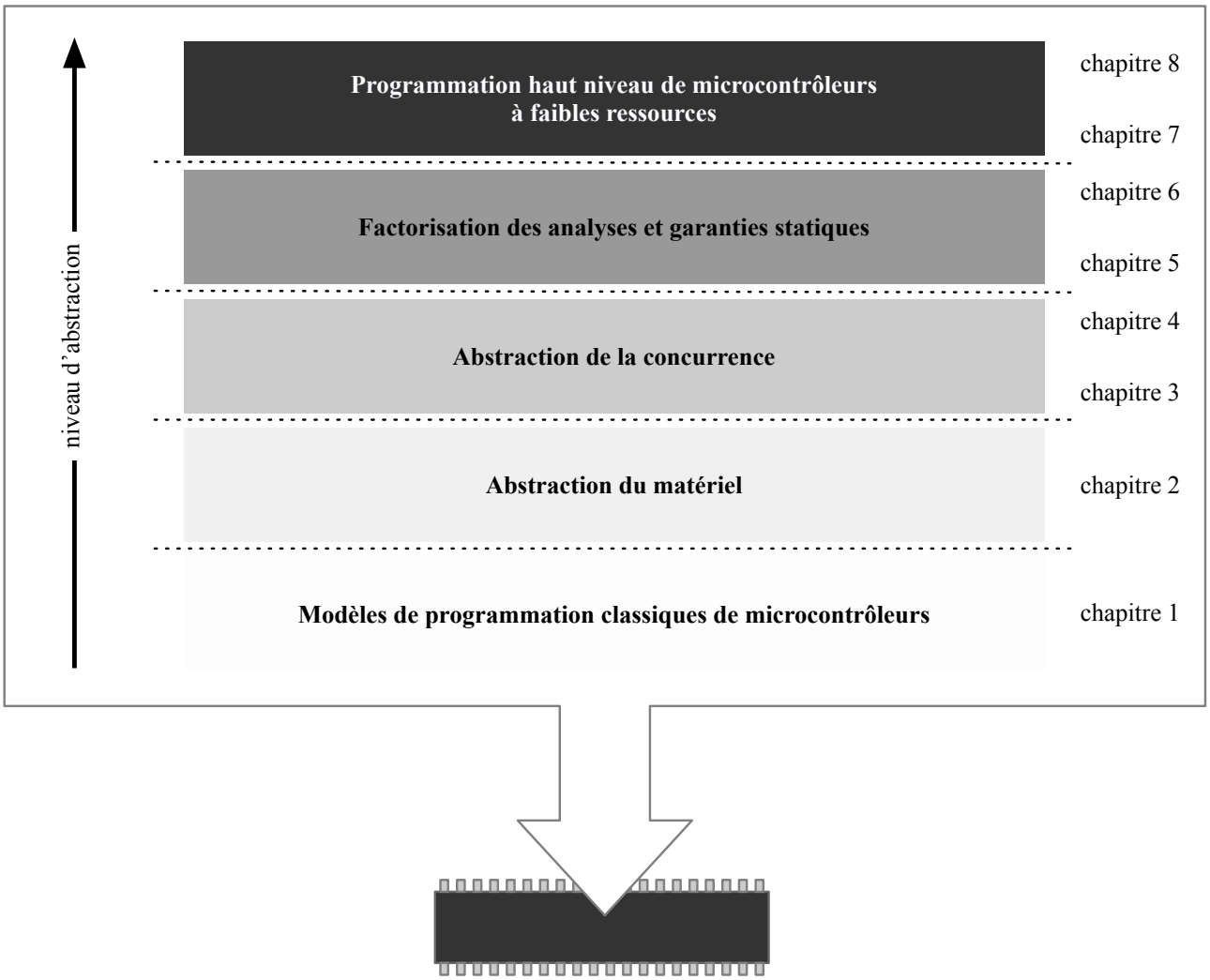


FIGURE 9.1: Rise in Abstraction for High-Level Programming of Low-Resource Microcontrollers

its environment, justifies adding a programming model suited to the development of such systems, in addition to the more *algorithmic* aspects provided by general-purpose languages. We therefore proposed *OCaLustre*, an extension of the OCaml language that makes it possible to use synchronous-programming features for building embedded programs, while benefiting from the advantages of the host language and of the OMicroB virtual machine. This language extension has a compilation model that gives it a small memory footprint, and is therefore suited to the hardware aspects of the microcontrollers considered. The OCaLustre synchronous extension therefore offers a rise in abstraction *over concurrency*: using a synchronous programming model makes the construction and interaction of the different concurrent elements in a program implicit. Furthermore, choosing a *dataflow* model makes it easy to represent interactions between the program and its physical environment, composed of electronic components that communicate by emitting electrical values whose "value" changes over time. This synchronous extension in turn offers new guarantees about the programs produced, for example by verifying at compilation time the *causal coherence* of the different software components involved in a program, thus avoiding deadlock situations during program execution. In addition, OCaLustre benefits from a formal specification, several aspects of which have been mechanically verified using formalisation and proof tools. For example, the system of synchronous clocks, which governs the absence or presence of values during program execution, was formally defined. A tool for verifying the coherence between this system and the clock types inferred by the OCaLustre compiler was then formalised and proved correct in Coq, before being extracted to OCaml code for integration into the compiler. This formal specification of the language and the verification of some of its properties therefore give programs written in OCaLustre an increased level of safety. The description of the OCaLustre language and its compilation was published in the proceedings of JFLA (*Journées Francophones et Langages Applicatifs*) in 2017 [VVC17].

The abstractions induced by our work do not come at the expense of verifying certain guarantees essential to the development of embedded, sometimes critical, programs, and they even make it possible to *factorise* certain analyses. Indeed, we have illustrated the advantage of using bytecode common to all implementations of the OMicroB virtual machine in order to perform worst-case execution-time analysis of a synchronous program. Estimating this execution time is often necessary for the correct behaviour of a critical embedded system, in order to ensure that the reaction time of the synchronous program is lower than the frequency at which its inputs appear. We therefore proposed a method for computing an upper bound on the real execution time of an OCaLustre program through analysis of its bytecode instructions. This method was then proved correct on a language similar to a subset of the OCaml bytecode instructions and implemented in a tool, named *Bytecrawler*, which computes the worst-case execution time of an OCaLustre program without subjecting users of our solutions to complex considerations relating to the low-level code actually executed by the virtual-machine interpreter. Using bytecode thus constitutes, in a sense, a third level of abstraction provided by our solution: this abstraction makes analyses more direct, and less dependent on hardware specificities. The description of our worst-case execution-time analysis, together with its formalisation and correctness proof, was presented at the WCET workshop (*International Workshop on Worst-Case Execution Time Analysis*), a satellite of the ECRTS conference (*Euromicro Conference on Real-Time Systems*), in 2019 [VC19].

The various tools developed during our work, together with the formal specification of OCaLustre and the partial verification of its associated properties, form a complete chain enabling the development of safer and richer applications on microcontrollers whose hardware resources are highly limited. The performance measurements used to estimate the resource consumption of the OMicroB/OCaLustre pair

validate the viability of our solution, whose memory consumption is low. In this respect, three examples of complete applications, each based on a different electronic circuit, have been described, and can be executed on microcontrollers with very limited resources. The example of programming a video game for an *Arduboy* also served as a practical application for the development of an invited tutorial at JFLA 2018 [VVC18b]. All the examples presented also demonstrate the relevance of our different contributions, from the expressive power of the OCaml language to the safety of the guarantees provided by the synchronous extension. The simplicity of deploying such applications is made possible by the portability of the proposed solutions. Indeed, all the software solutions described in this thesis are portable: OCaLustre programs are compiled to standard bytecode, executable on a generic virtual machine, and analysable with tools that adapt easily to different microcontrollers. This portability is at the heart of the ambitions of this thesis, aiming to enable the development of safe applications freed from considerations specific to the hardware used. The implementation work carried out already makes the different prototypes of the software solutions presented in this thesis fully usable, and industrial applications are envisaged in the framework of the LCHIP project (*Low Cost Integrity Platform*) [⚓11], in partnership with the company CLEARSY. This project consists of a safe, low-cost execution platform based on PIC32 microcontrollers. Within this platform, our virtual-machine approach and our synchronous programming model could constitute an alternative execution mode intended to introduce redundancy in program execution in order to verify its correctness.

Perspectives

The various works undertaken in this thesis constitute a first complete approach for programming microcontrollers using high-level languages and programming models, which provide additional guarantees about programs compared with traditional microcontroller-programming techniques. To conclude this manuscript, we propose a set of envisaged directions intended to further increase the expressive power and guarantees provided by our solutions, as well as their compatibility with hardware having (very) limited resources.

First, the OMicroB virtual machine could benefit from additional optimisations aimed at enabling, on resource-constrained hardware, the execution of programs whose memory consumption is currently incompatible with our solution. One of the main causes of excessive RAM consumption by an OCaml program comes from the persistence in the heap of immutable values that are never freed, such as strings that may correspond to the names of exceptions used explicitly or not in a program. For example, the exceptions `Out_of_memory` and `Stack_overflow` are systematically declared by the runtime library, and their names remain in the heap throughout execution. Ongoing work aims to move such immutable values from the microcontroller RAM to its flash memory, whose size is generally larger, but which is not intended to be modified during program execution. The heap would thus be distributed between RAM for processing mutable values and flash memory for processing immutable values. This optimisation would free a non-negligible amount of space in the heap for handling genuinely dynamic values. Moreover, we are continuing our work on porting OMicroB to more varied hardware, in order to further validate the portability of our approach. For example, we are targeting ESP32 boards with 520 kilobytes of RAM and 4 megabytes of flash memory, as well as STM32 microcontrollers present in Nucleo development boards, the most resource-constrained of which have 2 kilobytes of RAM and 16 kilobytes of flash memory.

The first efforts to formalise and verify certain properties of the OCaLustre language could, in the long term, lead to complete certification of the compiler. In particular, proving that compiled programs respect the semantics of the language would make it possible to verify that no absent stream value is ever read during program execution, and thus to strengthen the safety of synchronous-clock typing. Work is also envisaged to extract directly from Coq the code implementing most of the compilation steps of an OCaLustre program, in order to ensure that this compilation process respects the adopted formal specification. In this respect, we could move closer to the work of Bourke *et al.* on Vélus [BBD+17], a Lustre compiler certified in Coq.

In addition, new guarantees about the coherence of the programs produced could be verified through *synchronous contracts*, following the model of the Lesar [Rat92] and Kind2 [CMST16] *model-checking* tools. Initial attempts have indeed been made to define contracts in OCaLustre with extraction to the WhyML language (compatible with the Why3 program-verification platform [FP13]) or to Isabelle/HOL. However, their verification quickly becomes difficult as soon as these contracts refer to previous stream values (for example, to describe that a stream has values increasing over time), because proving such properties involves a principle of *k-induction* that general-purpose verification tools struggle to exploit automatically.

Furthermore, initial work has been undertaken to represent OCaLustre clock typing within OCaml's own type system, through the use of generalised algebraic data types (GADTs). This promising work could make it possible to leave to the standard OCaml compiler the task of verifying the full typing safety of an OCaLustre program, both with respect to standard data typing (which the correctness proof of typing with respect to translation into OCaml verifies) and with respect to clock typing. Such use of OCaml's rich type system represents an additional advantage of using a high-level language.

The specification of the OCaLustre language, as well as its prototype, currently concerns only streams whose values have simple types, such as Booleans or integers. This restriction was chosen in order to respect the same semantics as Lustre, and to be able to propose a "non-allocating" compilation mode compatible with all valid OCaLustre programs. However, it would seem advantageous to benefit from the richness of the host language by allowing the manipulation of values of more complex types. Such an extension would make it possible to create more expressive OCaLustre programs. For example, an OCaLustre program could manipulate streams of tuples, streams of lists, or even streams of functions or nodes, allowing the introduction of a higher-order synchronous programming model, following the example of Lucid Synchrone. Extending OCaLustre to more complex data types seems direct with respect to the "standard" compilation model discussed in Chapter 4, but would make WCET verification difficult because of the dynamic allocation of values that it would introduce.

Finally, our solution could benefit from more advanced static analyses on OCaml programs, making it possible, for example, to bound the amount of memory allocated by a program in order to ensure that no heap-overflow phenomenon can occur during its execution. We envisage drawing inspiration from and extending work close to our own, which targets the programming of embedded systems in an ML-like language. This work integrates into the language an explicit region system in order to estimate statically an upper bound on the maximum number of live values in the heap during program execution [SC16a]. These analyses could also make it possible to integrate GC execution time into WCET computation (even if that means triggering it deliberately, for example at the beginning of a synchronous instant).

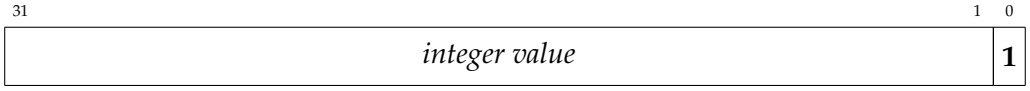
All these perspectives would ultimately continue the approach presented in this thesis, by increasing the safety of embedded-systems programming and by offering even higher-level programming models on microcontrollers which, for their part, retain the same limited resources.

A Representation of OCaml Values

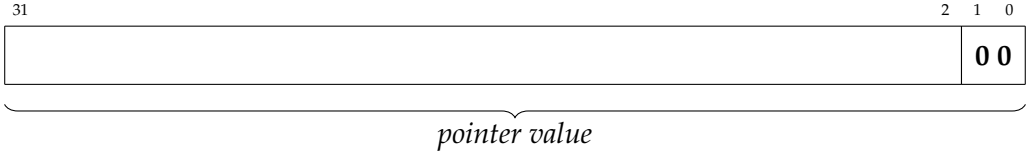
A.1 Representation of Values in the ZAM

A.1.1 32-Bit Representation

Integer:



Pointer:

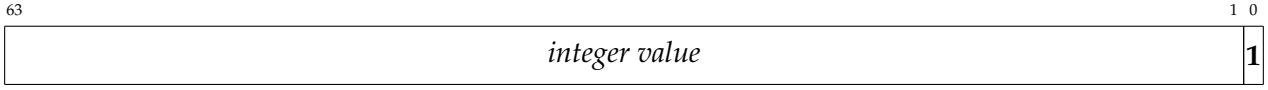


Block Header:

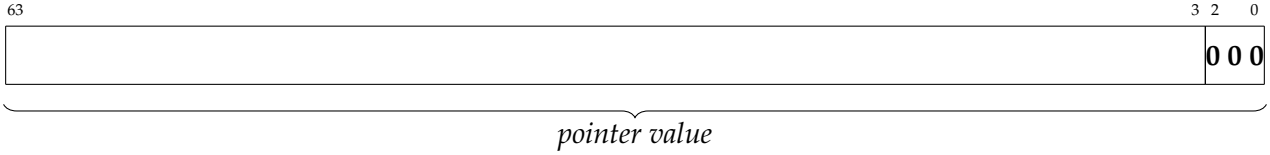
Floating-point numbers are allocated on the heap in blocks containing two OCaml values (they are double-precision floating-point numbers on $32 \times 2 = 64$ bits).

A.1.2 64-Bit Representation

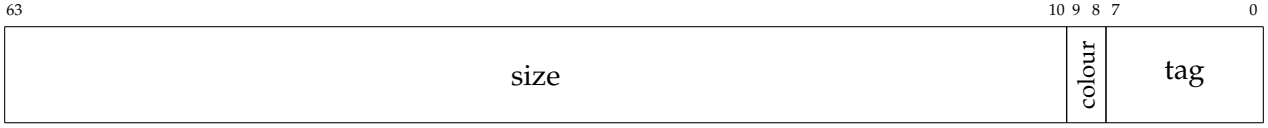
Integer:



Pointer:



Block Header:

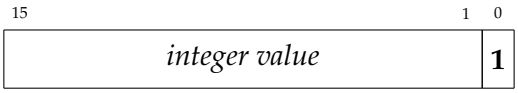


Floating-point numbers are allocated on the heap in blocks containing one OCaml value (they are therefore also double-precision 64-bit floating-point numbers).

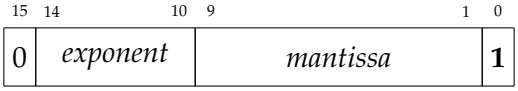
A.2 Representation of Values in OMicroB

A.2.1 16-Bit Representation

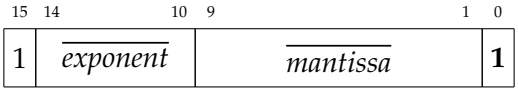
Integer:



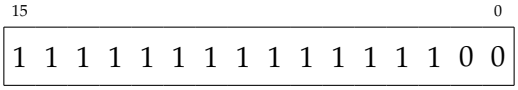
Floating-Point Number (Positive):



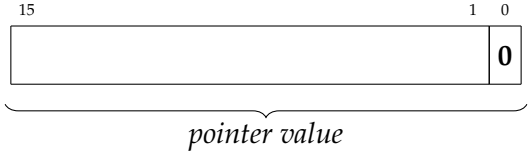
Floating-Point Number (Negative):



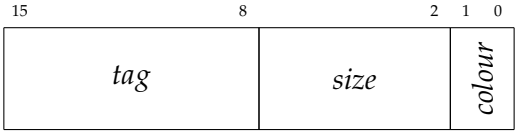
NaN:



Heap Pointer:

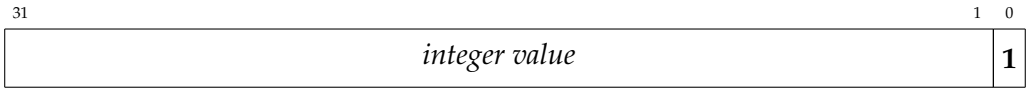


Block Header:

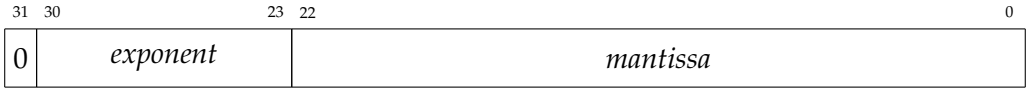


A.2.2 32-Bit Representation

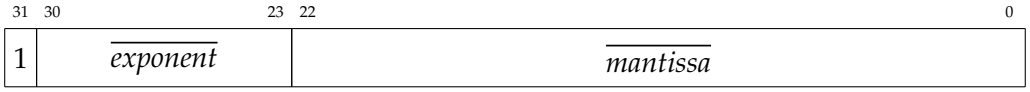
Integer:



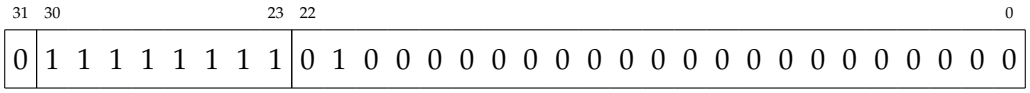
Floating-Point Number (Positive):



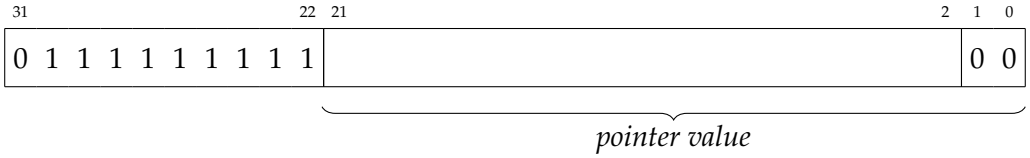
Floating-Point Number (Negative):



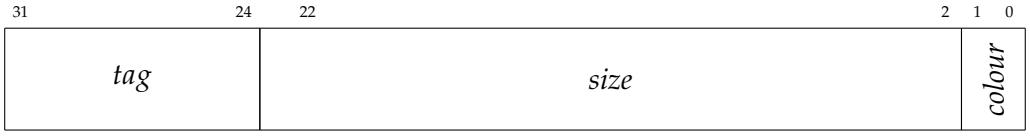
NaN:



Heap Pointer:

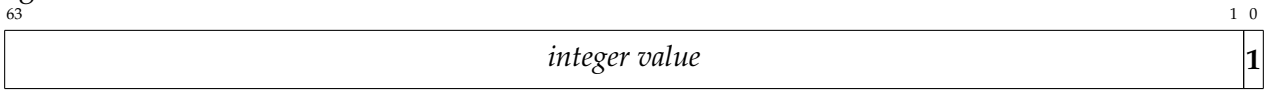


Block Header:

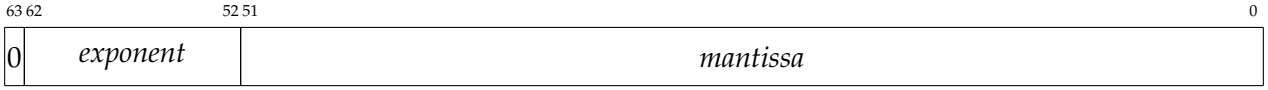


A.2.3 64-Bit Representation

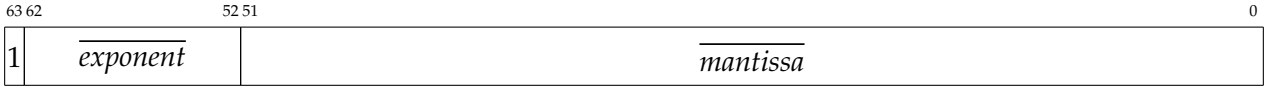
Integer:



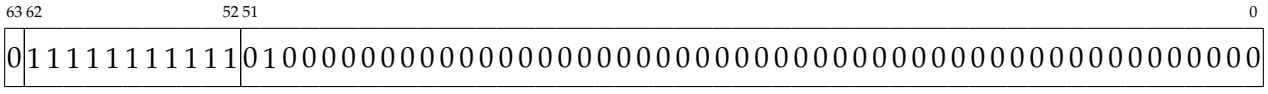
Floating-Point Number (Positive):



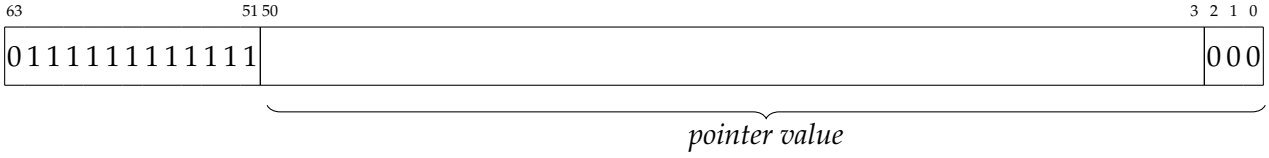
Floating-Point Number (Negative):



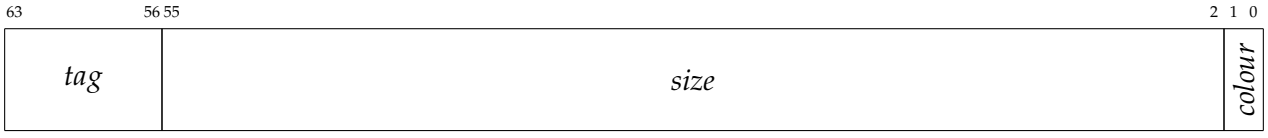
NaN:



Heap Pointer:



Block Header:



B Lucid Synchrone Code for the Adder

```
let xor a b = if a then not b else b

let node fulladder (a,b,cin) =
  let x = (xor a b) in
  let s = (xor x cin) in
  let and1 = (x && cin) in
  let and2 = (a && b) in
  let cout = (and1 || and2) in
  (s,cout)

let node twobits_adder (c0,a0,a1,b0,b1) =
  let (s0,c1) = fulladder (a0,b0,c0) in
  let (s1,c2) = fulladder (a1,b1,c1) in
  (s0,s1,c2)

let node fourbits_adder (c0,a0,a1,a2,a3,b0,b1,b2,b3) =
  let (s0,s1,c2) = twobits_adder (c0,a0,a1,b0,b1) in
  let (s2,s3,c4) = twobits_adder (c2,a2,a3,b2,b3) in
  (s0,s1,s2,s3,c4)

let node heightbits_adder (c0,a0,a1,a2,a3,a4,a5,a6,a7,b0,b1,b2,b3,b4,b5,b6,b7) =
  let (s0,s1,s2,s3,c4) = fourbits_adder (c0,a0,a1,a2,a3,b0,b1,b2,b3) in
  let (s4,s5,s6,s7,c8) = fourbits_adder (c4,a4,a5,a6,a7,b4,b5,b6,b7) in
  (s0,s1,s2,s3,s4,s5,s6,s7,c8)
```

C Performance Measurement Results

Name	Execution Time with ocamlrun (seconds)	Execution Time with OMicroB (seconds)	Ratio	Execution Speed with OMicroB (millions of bytecode instructions/second)	Number of GC Invocations
apply	1.22	2.14	1.75	306.33	29
fibo	0.55	1.30	2.36	376.04	0
takc	1.05	3.11	2.96	378.38	110500
oddeven	0.28	0.61	2.17	604.60	0
floats	0.56	1.05	1.87	219.46	0
integr	0.04	0.12	3.00	222.16	21
eval	0.04	0.08	2.00	377.50	1153
sieve	0.04	0.07	1.75	486.00	1666
objet	0.08	0.17	2.12	389.77	3424
functor	0.26	0.58	2.23	406.31	10001
bubble	0.93	1.51	1.62	473.71	17
jdlv	0.36	0.74	2.05	463.82	2999
share	0.11	0.27	2.45	444.67	321
abrsort	0.47	1.17	2.48	335.00	38084
queens	1.37	3.47	2.53	399.24	56666

TABLE C.1: Performance Measurements with the Mark and Compact GC (on PC)
OMicroB options: -arch 16 -gc MC -stack-size 500 -heap-size 2500

Name	Execution Time with ocamlrun (seconds)	Execution Time with OMicroB (seconds)	Ratio	Execution Speed with OMicroB (millions of bytecode instructions/second)	Number of GC Invocations
apply	1.21	2.27	1.87	288.79	58
fibo	0.54	1.16	2.14	421.43	0
takc	1.04	2.94	2.82	400.26	221999
oddeven	0.28	0.62	2.21	594.85	0
floats	0.59	0.57	0.96	404.28	0
integr	0.04	0.06	1.50	440.00	41
eval	0.04	0.08	2.00	377.50	2307
sieve	0.04	0.07	1.75	486.00	3333
objet	0.08	0.16	2.00	414.13	10989
functor	0.24	0.60	2.50	392.77	30002
bubble	0.93	1.43	1.53	500.21	35
jdlv	0.37	0.72	1.94	476.70	6666
share	0.11	0.26	2.36	461.77	684
abrsort	0.47	1.14	2.42	343.81	115198
queens	1.37	3.22	2.35	430.23	149999

Table C.2: Performance Measurements with a 32-Bit Value Representation (on PC)
OMicroB options: -arch 32 -gc SC -stack-size 500 -heap-size 2500

Name	Execution Time with ocamlrun (seconds)	Execution Time with OMicroB (seconds)	Ratio	Execution Speed with OMicroB (millions of bytecode instructions/second)	Number of GC Invocations
apply	1.26	2.36	1.87	277.77	58
fibo	0.58	1.21	2.08	404.01	0
takc	1.06	2.97	2.80	396.21	219666
oddeven	0.29	0.60	2.06	614.68	0
floats	0.58	0.75	1.29	307.25	0
integr	0.05	0.06	1.20	444.33	41
eval	0.04	0.08	2.00	377.50	2272
sieve	0.05	0.07	1.40	486.00	3333
objet	0.08	0.18	2.25	368.12	10666
functor	0.24	0.60	2.50	392.77	30002
bubble	1.00	1.55	1.55	461.49	34
jdlv	0.37	0.83	2.24	413.53	6666
share	0.11	0.25	2.27	480.24	675
abrsort	0.52	1.20	2.30	326.62	112604
queens	1.35	3.29	2.43	421.08	144999

Table C.3: Performance Measurements with a 64-Bit Value Representation (on PC)
OMicroB options: -arch 64 -gc SC -stack-size 500 -heap-size 2500

Name	**Execution Time** (seconds)	**Speed** (thousands of instr./second)	**Number of GC Invocations**
`apply`	7.860	83.432	0
`fibo`	2.999	163.065	0
`takc`	13.730	85.714	434
`oddeven`	2.262	163.085	0
`floats`	4.780	48.229	0
`integr`	0.379	69.902	0
`eval`	0.355	85.416	4
`sieve`	0.390	87.477	9
`bubble`	7.469	94.519	0
`jdlv`	3.737	91.938	16
`share`	1.373	88.275	1

Table C.4: Program Speed Measurements for OMicroB in 32-Bit Mode (on ATmega2560)
OMicroB options: -arch 32 -gc SC -stack-size 500 -heap-size 1400

Programs not shown in this table are those that cannot run with this reduced heap size, which compensates for the size of the values.

D Application Code

D.1 Punched-Card Reader Program

```
open Avr

let t = Array.make 8 false
let clk = PIN13
let data = PIN12
let leds = [| PIN42 ; PIN43; PIN44; PIN45; PIN46; PIN47; PIN48; PIN49 |]

let init () =
  (* réglage des broches en entrée *)
  pin_mode clk INPUT;
  pin_mode data INPUT;
  (* réglage des broches en sortie *)
  Array.iter (fun x -> pin_mode x OUTPUT) leds

let input_clk () = bool_of_level (digital_read clk)
let input_data () = bool_of_level (digital_read data)

(* fonction qui allume/éteint une des LED *)
let update_led i b = digital_write leds.(i) (level_of_bool b)

let output i data clk send =
  if clk then t.(i) ← data;
  if send then Array.iteri update_led t

let%node edge x ~return:e =
  e = (x && (not (true ⋙ x)))

let%node read_bit (top,bot) ~return:(clk,data) =
  clk = edge top;
  data = bot [@when clk]

let%node count (reset) ~return:(cpt) =
  cpt = (0 ⋙ (cpt + 1)) mod reset

let%node read_card (top,bot) ~return:(i,data,clk,send) =
  (clk,data) = read_bit (top,bot);
  i = count(8 [@when clk]);
```

```
38   send = merge clk (i = 7) false
39
40 let () =
41   (* initialisation du matériel *)
42   init ();
43   (* création de l'état du noeud principal *)
44   let st = read_card_alloc () in
45   while true do
46     (* lecture des entrées *)
47     let c = input_clk () in
48     let d = input_data () in
49     (* mise à jour de l'état du noeud principal *)
50     read_card_step st c d;
51     (* émission des sorties *)
52     let i = st.read_card_out_i in
53     let data = st.read_card_out_data in
54     let clk = st.read_card_out_clk in
55     let send = st.read_card_out_send in
56     output i data clk send
57   done
```

D.2 Tempering-Machine Program

D.2.1 tempereuse.ml

```
1  (** on allume/éteint si on appuie en même temps sur + et - **)
2  let%node thermo_on (p,m) ~return:(b) =
3    b = (true ⋙ if p && m then not b else b)
4
5  (** modification de la température désirée selon le bouton appuyé **)
6  let%node set_wanted_temp (p,m) ~return:(w) =
7    w = (325 ⋙ if p then w+5 else if m then w-5 else w)
8
9  (** noeud principal : calcul de la température désirée et de l'état de la résistance **)
10 (** Les températures sont en dixièmes de degrés C **)
11 let%node thermo (plus,minus,real_temp) ~return:(on,wanted,real,resistor) =
12   on = thermo_on (plus,minus);
13   wanted = set_wanted_temp (plus[@when on], minus[@when on]);
14   real = real_temp [@when on];
15   heat = (real < wanted);
16   resistor = merge on heat false
```

D.2.2 thermo_io.ml

```
1  open Avr
2
3  (* déclaration de l'écran *)
4  let lcd = LiquidCrystal.create4bitmode PIN13 PIN12 PIN18 PIN19 PIN20 PIN21
```

```
5
6  (* déclaration des pins *)
7  let plus = PIN7
8  let minus = PIN6
9  let resistor = PIN10
10 let sensor = PINA0
11
12 (* conversion de température *)
13 let convert_temp t =
14   let f = (float_of_int (1033 - t) /. 11.67) in
15   int_of_float (f*.100.)
16
17 (* lecture de la température *)
18 let read_temp () =
19   let t = analog_read sensor in
20   Serial.write_string "an=";
21   Serial.write_int t;
22   convert_temp t
23
24 (* Affichage des températures sur l'écran LCD *)
25 let print_temp wanted real =
26   let split_temp t =
27     let u = t/10 in
28     let dec = t mod 10 in
29     (u,dec) in
30   LiquidCrystal.clear lcd;
31   LiquidCrystal.home lcd;
32   let (wu,wd) = split_temp wanted in
33   let (ru,rd) = split_temp real in
34   LiquidCrystal.print lcd "Wanted T :";
35   LiquidCrystal.print lcd ((string_of_int wu)^"."^(string_of_int wd));
36   LiquidCrystal.setCursor lcd 0 1;
37   LiquidCrystal.print lcd "Actual T :";
38   LiquidCrystal.print lcd ((string_of_int ru)^"."^(string_of_int rd))
39
40 (*** Fonctions d'entrées/sorties de l'instant synchrone ***)
41
42 (** fonction d'initialisation **)
43 let init_thermo () =
44   (* initialisation de la lecture analogique *)
45   Avr.adc_init ();
46   (* initialisation de l'écran *)
47   LiquidCrystal.lcdBegin lcd 16 2;
48   (* initialisation des broches *)
49   pin_mode sensor INPUT;
50   pin_mode resistor OUTPUT;
51   pin_mode plus INPUT;
```

```
52   pin_mode minus INPUT
53
54 (** fonction d'entrée **)
55 let input_thermo () =
56   let plus = digital_read plus in
57   let minus = digital_read minus in
58   let plus = bool_of_level plus in
59   let minus = bool_of_level minus in
60   let real_temp = read_temp () in
61   (plus,minus,real_temp)
62
63 (** fonction de sortie **)
64 let output_thermo (on,wanted,real,res) =
65   if on then
66     begin
67       print_temp wanted real;
68       digital_write resistor (if res then HIGH else LOW)
69     end
70   else
71     begin
72       LiquidCrystal.home lcd;
73       LiquidCrystal.clear lcd;
74       LiquidCrystal.print lcd "..."
75     end;
```

D.2.3 Taking the Heating Duty Cycle into Account

The following OCaLustre program takes into account the inertia of the preparation's temperature increase: it measures the proportion for which the heating resistor must be activated.

```
1
2  let%node min(a,b) ~return:c =
3    c = if a < b then a else b
4
5  let%node max(a,b) ~return:c =
6    c = if a > b then a else b
7
8  (** calcul de la proportion de chauffe (en %) **)
9  let%node update_prop (wtemp,ctemp) ~return:(prop) =
10   delta = min (10,max (-10,wtemp-ctemp));
11   delta2 = if delta < 0 then (-delta * delta) else (delta*delta);
12   offset = min (10,delta2);
13   pre_prop = (0 ⋙ prop);
14   prop = min (100,max (0, (pre_prop+offset)))
15
16 let%node timer (number) ~return:(alarm) =
17   time = (0 ⋙ (time + 10)) mod 100;
18   alarm = if (time < number) then true else false
19
```

```
let%node heat (w,c) ~return:(h) =
  count = (0 ⋙ count + 1) mod 10;
  update = (count = 0);
  (* le rapport cyclique (prop) est mis à jour tous les 10 instants *)
  prop = merge update
           (update_prop (w [@when update],c [@when update]))
           ((0 ⋙ prop) [@whennot update]);
  h = timer (prop)

(** on allume/éteint si on appuie en même temps sur + et - **)
let%node thermo_on(p,m) ~return:(b) =
  b = (true ⋙ if p && m then (not b) else b)

(** modification de la température désirée selon le bouton appuyé **)
let%node set_wanted_temp (p,m) ~return:(w) =
  w = (325 ⋙ if p then w+5 else if m then w-5 else w)

(** noeud principal : calcul de la température désirée et de l'état de la résistance **)
(** Les températures sont en dixièmes de degrés celsius **)
let%node thermo (plus,minus,real_temp) ~return:(on,wanted,real,resistor) =
  on = thermo_on (plus,minus);
  wanted = set_wanted_temp (plus[@when on], minus[@when on]);
  real = real_temp [@when on];
  heat = heat (wanted,real);
  resistor = merge on heat false
```

D.3 Snake Program

D.3.1 spi.ml

```
(*** Module de gestion de la connexion SPI (Serial Peripheral Interface) ***)

open Avr

(** Initialiser la connexion SPI **)
let begin_spi ~sck ~mosi =
  set_bit SPCR MSTR;
  set_bit SPCR SPE;
  set_bit SPSR SPI2x;
  pin_mode sck OUTPUT;
  pin_mode mosi OUTPUT

(** Arrêter la connexion SPI **)
let end_spi () = clear_bit SPCR SPE

(*** Emettre des données via la connexion SPI **)
let transfer data = write_register SPDR data
```

D.3.2 oled.ml

```
open Avr

(*** Fonctions d'écriture/lecture dans le buffer d'affichage ***)
external write_buffer : int -> int -> bool -> unit = "caml_buffer_write"
external read_buffer : int -> int -> bool = "caml_buffer_read"
external get_byte_buffer : unit -> int = "caml_buffer_get_byte"

(*** Programme de boot de l'écran ***)
let boot_program =
  [|
    0xD5; 0xF0; (* Set display clock divisor = 0xF0 *)
    0x8D; 0x14; (* Enable charge Pump *)
    0xA1;       (* Set segment re-map *)
    0xC8;       (* Set COM Output scan direction *)
    0x81; 0xCF; (* Set contrast = 0xCF *)
    0xD9; 0xF1; (* Set precharge = 0xF1 *)
    0xAF;       (* Display ON *)
    0x20; 0x00; (* Set display mode = horizontal addressing mode *)
  |]

(*** Envoi du programme sur l'écran ***)
let transfer_program prog =
  Array.iter Spi.transfer prog

(*** Passer l'écran en mode "commande" ***)
let command_mode cs dc =
  digital_write cs HIGH;
  digital_write dc LOW;
  digital_write cs LOW

(*** Passer l'écran en mode "données" ***)
let data_mode cs dc =
  digital_write dc HIGH;
  digital_write cs LOW

(*** Envoyer une commande à l'écran ***)
let send_lcd_command cs dc com =
  command_mode cs dc;
  Spi.transfer com;
  data_mode cs dc

(*** Écrire un pixel (true=noir/false=blanc) ***)
let draw x y color =
  write_buffer x y color

(*** Effacer l'écran ***)
```

```
47 let clear() =
48  for _i = 0 to 1023 do
49    Spi.transfer(0x00)
50  done
51
52 (*** Envoyer le buffer sur l'écran ***)
53 let flush () =
54   for _i = 0 to 1023 do
55     Spi.transfer(get_byte_buffer())
56   done
57
58 (*** Initialisation de l'écran ***)
59 let boot ~cs ~dc ~rst =
60   digital_write rst HIGH;
61   digital_write rst LOW;
62   digital_write rst HIGH;
63   command_mode cs dc;
64   transfer_program boot_program;
65   data_mode cs dc;
66   clear()
```

D.3.3 arduboy.ml

```
1  open Avr
2
3  (** Les broches utilisées **)
4  let cs = PIN12
5  let dc = PIN4
6  let rst = PIN6
7  let button_left = PINA2
8  let button_right = PINA1
9  let button_down = PINA3
10 let button_up = PINA0
11 let button_a = PIN7
12 let button_b = PIN8
13 let blue = PIN9
14 let red = PIN10
15 let green = PIN11
16
17 (** initialisation des LED rgb à anode commune (HIGH = éteinte) **)
18 let init_led l =
19   digital_write l HIGH
20
21 (** allumer une des LED rgb à anode commune (LOW = allumé) **)
22 let light_led l =
23   digital_write l LOW
24
25 (** initialisation des broches **)
```

```
26 let boot_pins () =
27   List.iter (fun x -> pin_mode x INPUT_PULLUP) [button_left; button_right; button_up;
28                                                 button_down];
29   pin_mode button_a INPUT_PULLUP;
30   pin_mode button_b INPUT_PULLUP;
31   List.iter (fun x -> pin_mode x OUTPUT) [red;green;blue];
32   List.iter (fun x -> pin_mode x OUTPUT) [cs;dc;rst];
33   List.iter init_led [red;green;blue]
34
35 (** initialisation des broches, de la liaison SPI, et de l'écran **)
36 let init () =
37   boot_pins ();
38   Spi.begin_spi ~sck:SCK ~mosi:MOSI;
39   Oled.boot ~cs:cs ~dc:dc ~rst:rst
```

D.3.4 snake.ml

```
1
2 (** Direction du serpent **)
3 type direction = South | North | East | West
4
5 (** Fonctions d'orientation : le serpent tourne à gauche et à droite **
6  ** par rapport à sa direction courante **)
7 let left_of = function
8   | South -> East
9   | North -> West
10  | East -> North
11  | West -> South
12
13 let right_of = function
14  | South -> West
15  | North -> East
16  | East -> South
17  | West -> North
18
19 (** Modulo **)
20 let nmod x y =
21   (x + y) mod y
22
23 (** Tirage aléatoire d'un entier **)
24 let new_position n = Random.int n
25
26 (** Nouvelle cordonnée en x ou en y **)
27 let%node new_coord (dir,max,v,dir1,dir2) ~return:n =
28   n = if dir = dir1 then call nmod (v-1) max
29       else if dir = dir2 then call nmod (v+1) max
30       else v
31
```

```
32 (** Nouvelle position de la tête du serpent **)
33 let%node new_head (dir,w,h) ~return:(x,y) =
34   x = new_coord (dir,w,0 ⋙ x,West,East);
35   y = new_coord (dir,h,0 ⋙ y,North,South)
36
37 (** Tourner à gauche **)
38 let%node left (dir) ~return:ndir =
39   ndir = call left_of dir
40
41 (** Tourner à droite **)
42 let%node right (dir) ~return:ndir =
43   ndir = call right_of dir
44
45 (** Direction du serpent à partir de sa direction précédente **)
46 let%node direction (l,r) ~return:dir =
47   pre_dir = (South ⋙ dir);
48   dir = if l then
49           (merge l (left (pre_dir [@when l])) (pre_dir [@whennot l]))
50         else
51           (merge r (right (pre_dir [@when r])) (pre_dir [@whennot r]))
52
53 (** Nouvelle position de la pomme **)
54 let%node new_apple (width,height) ~return:(a_x,a_y) =
55   (a_x,a_y) = (call new_position width, call new_position height)
56
57 (** Boucle de jeu **)
58 let%node game_loop (max_size,left,right,width,height)
59       ~return:(head,tail,new_x,new_y,apple_x,apple_y,win) =
60   (new_x,new_y) = new_head (dir,width,height);
61   dir = direction(left,right);
62   head = (1 ⋙ ((head+1) mod max_size));
63   eats = (apple_x = new_x && apple_y = new_y);
64   (a_x,a_y) = new_apple (width [@when eats], height [@when eats]);
65   apple_x = (10 ⋙ merge eats a_x (apple_x [@whennot eats]));
66   apple_y = (10 ⋙ merge eats a_y (apple_y [@whennot eats]));
67   tail = merge eats ((0⋙ tail)[@when eats]) (0 ⋙ ((tail+1) mod max_size) [@whennot eats]);
68   size = (1 ⋙ merge eats ((size + 1) [@when eats]) (size [@whennot eats]));
69   win = (size = max_size -1)
70
71 (** Front montant (pour les boutons) **)
72 let%node rising_edge i ~return:o =
73   o = (i && (not (false ⋙ i)))
74
75 (** Noeud principal **)
76 let%node main (max_size,button1,button2,width,height) ~return:(head,tail,nx,ny,apple_x,apple_y,win) =
77   left = rising_edge (button1);
78   right = rising_edge (button2);
```

```
79   (head,tail,nx,ny,apple_x,apple_y,win) = game_loop(max_size,left,right,width,height)
```

D.3.5 main_io.ml

```
1  open Avr
2
3  (*** Fonctions du jeu ***)
4
5  (** Le tableau qui contient les positions du corps du serpent **)
6  let max_size = 15
7  let snake = Array.make max_size (0,0)
8
9  (** Teste si le serpent entre en collision avec lui-même *)
10 exception Lose
11 let eats_itself head tail =
12   let f c = if c = snake.(head) then (raise Lose) in
13   try
14     if tail < head then
15       begin
16         for i = tail to head - 1 do
17           f snake.(i);
18         done
19       end
20     else
21       begin
22         for i = tail to max_size - 1 do
23           f snake.(i);
24         done;
25         for i = 0 to head - 1 do
26           f snake.(i);
27         done;
28       end;
29     false
30   with Lose -> true
31
32 (*** Fonctions d'affichage ***)
33
34 let draw_snake head tail : unit =
35   let (x,y) = snake.(head) in
36   let (x',y') = snake.(tail) in
37   (* on affiche une nouvelle tête *)
38   Oled.draw x y true;
39   (* on efface la queue pour donner l'illusion de déplacement *)
40   Oled.draw x' y' false
41
42 let draw_apple x y : unit =
43   Oled.draw x y true
44
```

```
45 (*** Fonctions de la machine d'exécution synchrone ***)
46
47 (** Initialisation du programme synchrone **)
48 let init_main () = Arduboy.init ()
49
50 (** Fonction d'entrée de chaque instant synchrone **)
51 let input_main () =
52   let l = Avr.digital_read Arduboy.button_left in
53   let r = Avr.digital_read Arduboy.button_right in
54   (max_size,bool_of_level l,bool_of_level r,64,32)
55
56 (** Fonction de sortie de chaque instant synchrone **)
57 let output_main (head,tail,nx,ny,apple_x,apple_y,win) =
58   snake.(head) ← (nx,ny);
59   draw_snake head tail;
60   draw_apple apple_x apple_y;
61   Oled.flush ();
62   if win then
63     begin
64       (* Gagné *)
65       Arduboy.light_led Arduboy.green;
66       Avr.delay 2000;
67       raise Exit
68     end
69   else if eats_itself head tail then
70     begin
71       (* Perdu *)
72       Arduboy.light_led Arduboy.red;
73       Avr.delay 2000;
74       raise Exit
75     end
```

Web References

[⚓1] Annexes électroniques du manuscrit.
https://stevenvar.github.io/these.

[⚓2] Certifications de KCG.
http://www.esterel-technologies.com/products/scade-suite/automatic-code-generation/scade-suite-kcg-certification-kits/.

[⚓3] Description de PPX.
http://ocamllabs.io/doc/ppx.html.

[⚓4] Fiche technique de l'écran SSD1306.
https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf.

[⚓5] Global MCU Market 2018 by Manufacturers, Regions, Type and Application, Forecast to 2023.
https://www.orianresearch.com/request-sample/553889.

[⚓6] Instructions de la machine virtuelle OCaml (site du projet Cadmium).
http://cadmium.x9c.fr/distrib/caml-instructions.pdf.

[⚓7] Page de l'IDE Arduino (site de la plateforme Arduino).
https://www.arduino.cc/en/Main/Software.

[⚓8] Page de l'IDE Atmel Studio (site de Microchip).
https://www.microchip.com/mplab/avr-support/atmel-studio-7.

[⚓9] Page de l'outil ocamlclean (site du projet OCaPIC).
http://www.algo-prog.info/ocapic/web/index.php?id=ocamlclean.

[⚓10] Page du compilateur MPLAB (site de Microchip).
https://www.microchip.com/mplab.

[⚓11] Page du projet LCHIP (site de Clearsy).
http://www.clearsy.com/2016/10/4260/.

[⚓12] Page du projet python-on-a-chip (archive).
https://code.google.com/archive/p/python-on-a-chip/.

[⚓13] Site de la machine virtuelle Java HaikuVM.
http://haiku-vm.sourceforge.net.

[⚓14] Site de la machine virtuelle NanoVM.
http://www.harbaum.org/till/nanovm.

[⚓15] Site de la plateforme Arduino.
https://www.arduino.cc.

[⚓16] Site de la plateforme microEJ.
https://www.microej.com.

[⚓17] Site de la suite logicielle Proteus.
https://www.labcenter.com.

[⚓18] Site de l'implémentation Bigloo.
https://www-sop.inria.fr/mimosa/fp/Bigloo.

[⚓19] Site de Lucid Synchrone.
https://www.di.ens.fr/~pouzet/lucid-synchrone/index.html.

[⚓20] Site de micro:bit.
https://microbit.org/.

[⚓21] Site de ReactiveML.
http://rml.lri.fr.

[⚓22] Site du langage Guile.
https://www.gnu.org/software/guile.

[⚓23] Site du programmateur usbpicprog.
http://usbpicprog.org.

[⚓24] Site du projet AVR-Ada.
https://sourceforge.net/projects/avr-ada.

[⚓25] Site du projet AVR Libc.
https://www.nongnu.org/avr-libc.

[⚓26] Site du projet AVR-Rust.
https://github.com/avr-rust.

[⚓27] Site du projet SDCC.
http://sdcc.sourceforge.net.

[⚓28] Site du projet TinyGO.
https://tinygo.org.

[⚓29] Tutoriel OCaml - Objets (site officiel du langage OCaml) .
https://ocaml.org/learn/tutorials/objects.html.

Bibliography

[ABC+17] Andrew W. Appel, Lennart Beringer, Adam Chlipala, Benjamin C. Pierce, Zhong Shao, Stephanie Weirich, and Steve Zdancewic: *Position paper: the science of deep specification*. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 375(2104):20160331, September 2017. https://doi.org/10.1098/rsta.2016.0331. *[cited on page 37]*

[Abr96] Jean-Raymond Abrial: *The B-Book: Assigning Programs to Meanings*. Cambridge University Press, New York, NY, USA, 1996, ISBN 0-521-49619-5. *[cited on page 37]*

[ACR+08] Periklis Akritidis, Cristian Cadar, Costin Raiciu, Manuel Costa, and Miguel Castro: *Preventing memory error exploits with WIT*. In *Proceedings of the 2008 IEEE Symposium on Security and Privacy (S&P 2008)*, pages 263–277, Oakland, CA, USA, May 2008. IEEE Computer Society. https://doi.org/10.1109/SP.2008.30. *[cited on page 34]*

[AGG07] Elvira Albert, Samir Genaim, and Miguel Gómez-Zamalloa: *Heap Space Analysis for Java Bytecode*. In *Proceedings of the 6th International Symposium on Memory Management (ISMM 2007)*, pages 105–116, Montréal, Québec, Canada, October 2007. ACM. https://doi.org/10.1145/1296907.1296922. *[cited on page 20]*

[AS87] Bowen Alpern and Fred B. Schneider: *Recognizing safety and liveness*. Distributed Computing, 2(3):117–126, September 1987, ISSN 1432-0452. https://doi.org/10.1007/BF01782772. *[cited on page 34]*

[Aug13] Cédric Auger: *Compilation certifiée de SCADE/LUSTRE*. PhD thesis, Université Paris-Sud, Orsay, France, 2013. https://tel.archives-ouvertes.fr/tel-00818169. *[2 citations pages 37 and 103]*

[AW77] Edward A. Ashcroft and William W. Wadge: *Lucid, a Nonprocedural Language with Iteration*. Communications of the ACM, 20(7):519–526, 1977. https://doi.org/10.1145/359636.359715. *[2 citations pages 30 and 75]*

[Bad14] Yusuf Abdullahi Badamasi: *The working principle of an arduino*. In *Proceedings of the 11th International Conference on Electronics, Computer and Computation (ICECCO 2014)*, pages 1–4, Abuja, Nigeria., September 2014. https://doi.org/10.1109/ICECCO.2014.6997578. *[cited on page 14]*

[BB91] Albert Benveniste and Gérard Berry: *The synchronous approach to reactive and real-time systems*. Proceedings of the IEEE, 79(9):1270–1282, September 1991, ISSN 0018-9219. https://doi.org/10.1109/5.97297. *[cited on page 27]*

[BBD+17] Timothy Bourke, Lélio Brun, Pierre-Évariste Dagand, Xavier Leroy, Marc Pouzet, and Lionel Rieg: *A formally verified compiler for lustre*. In *Proceedings of the 38th ACM SIGPLAN*

Conference on Programming Language Design and Implementation (PLDI 2017), pages 586–601, Barcelone, Espagne, June 2017. ACM. https://doi.org/10.1145/3062341.3062358. *[3 citations pages 37, 103, and 209]*

[BBP18] Timothy Bourke, Lélio Brun, and Marc Pouzet: *Towards a verified lustre compiler with modular reset*. In *Proceedings of the 21st International Workshop on Software and Compilers for Embedded Systems, SCOPES 2018*, pages 14–17, Sankt Goar, Allemagne, May 2018. ACM. https://doi.org/10.1145/3207719.3207732. *[cited on page 37]*

[BC84] Gérard Berry and Laurent Cosserat: *The ESTEREL synchronous programming language and its mathematical semantics*. In *Seminar on Concurrency*, volume 197 of *Lecture Notes in Computer Science*, pages 389–448, Carnegie-Mellon University, Pittsburg, PA, USA, July 1984. Springer. https://doi.org/10.1007/3-540-15670-4_19. *[cited on page 28]*

[BCE+03] Albert Benveniste, Paul Caspi, Stephen A. Edwards, Nicolas Halbwachs, Paul Le Guernic, and Robert de Simone: *The synchronous languages 12 years later*. Proceedings of the IEEE, 91(1):64–83, 2003. https://doi.org/10.1109/JPROC.2002.805826. *[cited on page 35]*

[BCF+14] Martin Bodin, Arthur Charguéraud, Daniele Filaretti, Philippa Gardner, Sergio Maffeis, Daiva Naudziuniene, Alan Schmitt, and Gareth Smith: *A trusted mechanised javascript specification*. In *POPL*, pages 87–100. ACM, 2014. *[cited on page 37]*

[BCHP08] Dariusz Biernacki, Jean-Louis Colaço, Grégoire Hamon, and Marc Pouzet: *Clock-directed modular code generation for synchronous data-flow languages*. In *Proceedings of the 2008 ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES'08)*, pages 121–130, Tucson, AZ, USA, June 2008. ACM. https://doi.org/10.1145/1375657.1375674. *[2 citations pages 105 and 116]*

[BCL08] Niels Brouwers, Peter Corke, and Koen Langendoen: *Darjeeling, a java compatible virtual machine for microcontrollers*. In *Proceedings of the 9th ACM/IFIP/USENIX International Middleware Conference (Middleware 2008)*, pages 18–23, Leuven, Belgique, December 2008. ACM. https://doi.org/10.1145/1462735.1462740. *[cited on page 22]*

[BCRS10] Clément Ballabriga, Hugues Cassé, Christine Rochange, and Pascal Sainrat: *OTAWA: an open toolbox for adaptive WCET analysis*. In *Proceedings of the 8th IFIP Workshop on Software Technologies for Future Embedded and Ubiquitous Computing Systems (SEUS 2010)*, pages 35–46, Weidhofen/Ybbs, Autriche, 2010. Springer. https://doi.org/10.1007/978-3-642-16256-5_6. *[cited on page 36]*

[BDPR17] Timothy Bourke, Pierre Evariste Dagand, Marc Pouzet, and Lionel Rieg: *Vérification de la génération modulaire du code impératif pour Lustre*. In *Actes des 28èmes Journées Francophones des Langages Applicatifs (JFLA 2017)*, Gourette, France, January 2017. https://hal.inria.fr/hal-01403830. *[cited on page 89]*

[Bel17] Charles Bell: *Introducing MicroPython*, pages 27–57. Apress, Berkeley, CA, 2017, ISBN 978-1-4842-3123-4. https://doi.org/10.1007/978-1-4842-3123-4_2. *[cited on page 22]*

[Ber86] Jean-Louis Bergerand: *LUSTRE : un langage déclaratif pour le temps réel*. PhD thesis, Institut National Polytechnique de Grenoble, Grenoble, France, 1986. *[cited on page 73]*

[BFL18] Clément Ballabriga, Julien Forget, and Giuseppe Lipari: *Symbolic WCET computation*. ACM Transactions on Embedded Computing Systems (TECS), 17(2):39, 2018. *[cited on page 164]*

[BN94] Swagato Basumallick and Kelvin Nilsen: *Cache issues in real-time systems*. In *In Proceedings of the 1994 ACM SIGPLAN Workshop on Language, Compiler and Tool Support for Real-Time System*, Orlando, FL, USA, June 1994. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.31.3755. *[cited on page 155]*

[Bou98] Hédi Boufaied: *Machines d'exécution pour langages synchrones*. PhD thesis, Université de Nice-Sophia Antipolis, Nice, France, 1998. *[cited on page 121]*

[BP13] Timothy Bourke and Marc Pouzet: *Zélus: a synchronous language with odes*. In *Proceedings of the 16th international conference on Hybrid systems: computation and control (HSCC 2013)*, pages 113–118, Philadelphia, PA, USA, April 2013. ACM. https://doi.org/10.1145/2461328.2461348. *[cited on page 34]*

[BP19] Timothy Bourke and Marc Pouzet: *Clocked arguments in a verified Lustre compiler*. In *Actes des 30èmes Journées Francophones des Langages Applicatifs (JFLA 2019)*, page 16, Les Rousses, France, January 2019. https://hal.inria.fr/hal-02005639. *[cited on page 133]*

[BRS18] Frédéric Bour, Thomas Refis, and Gabriel Scherer: *Merlin: a language server for ocaml (experience report)*. Proceedings of the ACM on Programming Languages, 2(ICFP):103:1–103:15, September 2018. https://doi.org/10.1145/3236798. *[cited on page 97]*

[BSR12] Thomas W. Barr, Rebecca Smith, and Scott Rixner: *Design and implementation of an embedded python run-time system*. In *Proceedings of the 2012 USENIX Annual Technical Conference*, pages 297–308, Boston, MA, USA, June 2012. USENIX Association. https://www.usenix.org/conference/atc12. *[cited on page 23]*

[BV97] Ross Bannatyne and Greg Viot: *Introduction to microcontrollers*. In *Proceedings of the 1997 Western Electronics Show and Convention (WESCON/97)*, pages 564–574, Santa Clara, CA, USA, November 1997. https://doi.org/10.1109/WESCON.1997.632384. *[cited on page 13]*

[Cam16] Giulio Camilleri: *An LLVM Bitcode Interpreter for 16-bit MSP430 Microcontrollers*. Mémoire de licence, Faculty of ICT - University of Malta, May 2016. https://www.um.edu.mt/library/oar/handle/123456789/13777. *[cited on page 21]*

[CH86] Paul Caspi and Nicolas Halbwachs: *A functional model for describing and reasoning about time behaviour of computing systems*. Acta Informatica, 22(6):595–627, 1986. https://doi.org/10.1007/BF00263648. *[cited on page 30]*

[Cha92] Emmanuel Chailloux: *An Efficient Way of Compiling ML to C*. In *Proceedings of the 1992 ACM Workshop on ML and its Applications*, pages 37–51, San Francisco, CA, USA, June 1992. ACM. *[cited on page 69]*

[CHP06] Paul Caspi, Grégoire Hamon, and Mark Pouzet: *Lucid synchrone, un langage de programmation des systèmes réactifs*. Systèmes Temps-réel : Techniques de Description et de Vérification Théorie et Outils, 1:217260, 2006. *[cited on page 33]*

[CL14] Raphaëlle Crubillé and Ugo Dal Lago: *On probabilistic applicative bisimulation and call-by-value λ-calculi*. In *Proceedings of the 23rd European Symposium on Programming (ESOP 2014)*, volume 8410 of *Lecture Notes in Computer Science*, pages 209–228, Grenoble, France, April 2014. Springer. https://doi.org/10.1007/978-3-642-54833-8_12. *[cited on page 148]*

[CMST16] Adrien Champion, Alain Mebsout, Christoph Sticksel, and Cesare Tinelli: *The kind 2 model checker*. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV 2016)*, pages 510–517, Toronto, Canada, July 2016. Springer International Publishing. https://doi.org/10.1007/978-3-319-41540-6_29. *[cited on page 209]*

[CP99] Paul Caspi and Marc Pouzet: *Lucid Synchrone: une extension fonctionnelle de Lustre*. In *Actes des 10èmes Journées Francophones des Langages Applicatifs (JFLA 99)*, Avoriaz, France, February 1999. INRIA. https://hal.archives-ouvertes.fr/hal-01574464. *[2 citations pages 33 and 75]*

[CP01] Antoine Colin and Isabelle Puaut: *A modular & retargetable framework for tree-based WCET analysis*. In *Proceedings of the 13th Euromicro Conference on Real-Time Systems (ECRTS 2001)*, pages 37–44, Delft, Pays-Bas, June 2001. IEEE Computer Society. https://doi.org/10.1109/EMRTS.2001.933995. *[cited on page 36]*

[CP03] Jean-Louis Colaço and Marc Pouzet: *Clocks as first class abstract types*. In *Proceedings of the 3rd International Conference on Embedded Software (EMSOFT 2003)*, volume 2855 of *Lecture Notes in Computer Science*, pages 134–155, Philadelphia, PA, USA, October 2003. Springer. https://doi.org/10.1007/978-3-540-45212-6_10. *[4 citations pages 85, 97, 114, and 133]*

[CP04] Jean-Louis Colaço and Marc Pouzet: *Type-based initialization analysis of a synchronous dataflow language*. International Journal on Software Tools for Technology Transfer (STTT), 6(3):245–255, 2004. https://doi.org/10.1007/s10009-004-0160-y. *[cited on page 77]*

[CPHP87] Paul Caspi, Daniel Pilaud, Nicolas Halbwachs, and John Plaice: *Lustre: A declarative language for programming synchronous systems*. In *Conference Record of the Fourteenth Annual ACM Symposium on Principles of Programming Languages (POPL 87)*, pages 178–188, Munich, Germany, January 1987. ACM Press. https://doi.org/10.1145/41625.41641. *[3 citations pages 30, 73, and 100]*

[CPP17] Jean-Louis Colaço, Bruno Pagano, and Marc Pouzet: *SCADE 6: A formal language for embedded critical software development (invited paper)*. In *Proceedings of the 11th International Symposium on Theoretical Aspects of Software Engineering (TASE 2017)*, pages 1–11, Sophia Antipolis, France, September 2017. IEEE Computer Society. https://doi.org/10.1109/TASE.2017.8285623. *[3 citations pages 10, 33, and 116]*

[Cri92] Régis Cridlig: *An optimizing ML to C compiler*. In *Proceedings of the 1992 ACM Workshop on ML and its Applications*, pages 28–36, San Francisco, CA, USA, June 1992. ACM. *[cited on page 69]*

[DF05] Danny Dubé and Marc Feeley: *BIT: A very compact scheme system for microcontrollers*. Higher-Order and Symbolic Computation, 18(3-4):271–298, 2005. https://doi.org/10.1007/s10990-005-4877-4. *[cited on page 23]*

[DM82] Luis Damas and Robin Milner: *Principal type-schemes for functional programs*. In *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '82)*, pages 207–212, Albuquerque, New Mexico, January 1982. ACM, ISBN 0-89791-065-6. http://doi.acm.org/10.1145/582153.582176. *[cited on page 85]*

[Dor08] Francois Xavier Dormoy: *Scade 6: a model based solution for safety critical software development*. In *Proceedings of the 4th European Congress on Embedded Real Time Software (ERTS 2008)*, pages 1–9, Toulouse, France, January 2008. *[cited on page 83]*

[DRM11] Gwenaël Delaval, Eric Rutten, and Hervé Marchand: *Intégration de la synthèse de contrôleurs discrets dans un langage de programmation*. In *Actes du 8ème colloque francophone sur la modélisation des systèmes réactifs (MSR'11)*, Lille, France, November 2011. https://hal.inria.fr/inria-00629104. *[cited on page 34]*

[DS00] Stephan Diehl and Peter Sestoft: *Abstract machines for programming language implementation*. Future Generation Computer Systems, 16(7):739–751, May 2000, ISSN 0167-739X. http://doi.org/10.1016/S0167-739X(99)00088-6. *[cited on page 20]*

[FD03] Marc Feeley and Danny Dubé: *Picbit: A scheme system for the pic microcontroller*. In *Proceedings of the 4th Workshop on Scheme and Functional Programming*, pages 7–15, Boston, MA, USA, November 2003. http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.142.5903. *[cited on page 23]*

[FP13] Jean-Christophe Filliâtre and Andrei Paskevich: *Why3 - Where Programs Meet Provers*. In *Proceedings of the 22nd European Symposium on Programming (ESOP 2013)*, volume 7792, pages 125–128, Rome, Italie, March 2013. Springer. https://doi.org/10.1007/978-3-642-37036-6_8. *[cited on page 209]*

[Gér13] Léonard Gérard: *Programmer le parallélisme avec des futures en Heptagon un langage synchrone flot de données et étude des réseaux de Kahn en vue dune compilation synchrone*. PhD thesis, Université Paris Sud - Paris XI, September 2013. https://tel.archives-ouvertes.fr/tel-00929932. *[2 citations pages 75 and 85]*

[GG87] Thierry Gautier and Paul Le Guernic: *SIGNAL: A declarative language for synchronous programming of real-time systems*. In *Proceedings of Functional Programming Languages and Computer Architecture*, volume 274 of *Lecture Notes in Computer Science*, pages 257–277, Portland, Oregon, USA, September 1987. Springer. https://doi.org/10.1007/3-540-18317-5_15. *[cited on page 32]*

[GGPP12] Léonard Gérard, Adrien Guatto, Cédric Pasteur, and Marc Pouzet: *A modular memory optimization for synchronous data-flow languages: application to arrays in a lustre compiler*. In *Proceedings of the 2012 SIGPLAN/SIGBED Conference on Languages, Compilers and Tools for Embedded Systems (LCTES '12)*, pages 51–60, Pékin, Chine, June 2012. ACM. https://doi.org/10.1145/2248418.2248426. *[cited on page 34]*

[GHKT14] Pierre-Loïc Garoche, Falk Howar, Temesghen Kahsai, and Xavier Thirioux: *Testing-based compiler validation for synchronous languages*. In *Proceedings of the 6th International Symposium on NASA Formal Methods (NFM 2014), Houston*, volume 8430 of *Lecture Notes in Computer Science*, pages 246–251, Houston, TX, USA, April 2014. Springer. https://doi.org/10.1007/978-3-319-06200-6_19. *[cited on page 116]*

[GPPT16] Fei Guan, Long Peng, Luc Perneel, and Martin Timmerman: *Open source freertos as a case study in real-time operating system evolution*. Journal of Systems and Software, 118:19–35, 2016. https://doi.org/10.1016/j.jss.2016.04.063. *[cited on page 26]*

[Gud93] David Gudeman: *Representing Type Information in Dynamically Typed Languages*. Technical Report 93-27, University of Arizona, Phoenix, AZ, USA, October 1993. *[cited on page 63]*

[Gut97] Scott B. Guthery: *Java card (industry report)*. IEEE Internet Computing, 1(1):57–59, 1997. https://doi.org/10.1109/4236.585173. *[cited on page 22]*

[Hal03] Dean W. Hall: *Pymite: A flyweight python interpreter for 8-bit architectures*. In *Proceedings of the First Python Community Conference (PyCon 2003)*, pages 26–28, Washington, DC, USA, March 2003. http://ftp.ntua.gr/mirror/python/pycon/papers/pymite/index.html. *[cited on page 23]*

[Hal05] Nicolas Halbwachs: *A synchronous language at work: the story of lustre*. In *Proceedings of the 3rd ACM & IEEE International Conference on Formal Methods and Models for Co-Design (MEMOCODE 2005)*, pages 3–11, Verona, Italie, July 2005. IEEE Computer Society. https://doi.org/10.1109/MEMCOD.2005.1487884. *[cited on page 30]*

[HCRP91] Nicolas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud: *The synchronous data flow programming language lustre*. Proceedings of the IEEE, 79(9):1305–1320, Sep. 1991. https://doi.org/10.1109/5.97300. *[cited on page 73]*

[HG16] Caleb Helbling and Samuel Z. Guyer: *Juniper: a functional reactive programming language for the arduino*. In *Proceedings of the 4th International Workshop on Functional Art, Music, Modelling, and Design (FARM 2016)*, pages 8–16, Nara, Japon, September 2016. ACM. https://doi.org/10.1145/2975980.2975982. *[cited on page 9]*

[HK07] Trevor Harmon and Raymond Klefstad: *A Survey of Worst-Case Execution Time Analysis for Real-Time Java*. In *Proceedings of the 21th International Parallel and Distributed Processing Symposium (IPDPS 2007)*, pages 1–8, Long Beach, CA, USA, March 2007. IEEE Computer Society. *[cited on page 164]*

[HPK+09] Kirak Hong, Jiin Park, Taekhoon Kim, Sungho Kim, Hwangho Kim, Yousun Ko, Jongtae Park, Bernd Burgstaller, and Bernhard Scholz: *Tinyvm, an efficient virtual machine infrastructure for sensor networks*. In *Proceedings of the 7th International Conference on Embedded Networked Sensor Systems (SenSys 2009)*, pages 399–400, Berkeley, CA, USA, November 2009. ACM. https://doi.org/10.1145/1644038.1644121. *[cited on page 22]*

[HR01] Nicolas Halbwachs and Pascal Raymond: *A tutorial of lustre*. 2001. https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.25.872. *[cited on page 105]*

[HS02] Niklas Holsti and Sam Saarinen: *Status of the Bound-T WCET tool*. In *Proceedings of the 2nd International Workshop on Worst-Case Execution Time Analysis (WCET 2002)*, Vienne, Autriche, June 2002. http://www.cs.york.ac.uk/rts/wcet2002. *[cited on page 155]*

[JGS93] Neil D. Jones, Carsten K. Gomard, and Peter Sestoft: *Partial evaluation and automatic program generation*. Prentice Hall international series in computer science. Prentice Hall, 1993, ISBN 0-13-020249-5. *[cited on page 68]*

[Jon97] Mike Jones: *What really happened on Mars Rover Pathfinder*. ACM Forum on Risks to the Public in Computers and Related Systems, 19(49), 1997. http://www.cs.cornell.edu/courses/cs614/1999sp/papers/pathfinder.html. *[cited on page 27]*

[JRH19] Erwan Jahier, Pascal Raymond, and Nicolas Halbwachs: *The lustre v6 reference manual*. 2019. http://www-verimag.imag.fr/DIST-TOOLS/SYNCHRONE/lustre-v6/doc/lv6-ref-man.pdf. *[cited on page 75]*

[JWC08] Jeong Hoon Ji, Gyun Woo, and Hwan Gue Cho: *A plagiarism detection technique for java program using bytecode analysis*. In *Proceedings of the 3rd International Conference on Convergence and Hybrid Information Technology*, volume 1, pages 1092–1098. IEEE Computer Society, November 2008. https://doi.org/10.1109/ICCIT.2008.267. *[cited on page 20]*

[Kah74] Gilles Kahn: *The semantics of simple language for parallel programming*. In *Proceedings of IFIP Congress 1974*, pages 471–475, Stockholm, Suède, August 1974. *[cited on page 30]*

[Kat10] Yoshiharu Kato: *Splish: A visual programming environment for arduino to accelerate physical computing experiences*. In *Proceedings of the 8th International Conference on Creating, Connecting and Collaborating through Computing (C^5 2010)*, pages 3–10, La Jolla, CA, USA, January 2010. IEEE Computer Society. https://doi.org/10.1109/C5.2010.20. *[cited on page 9]*

[KEH+09] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood: *sel4: formal verification of an OS kernel*. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles 2009 (SOSP '09)*, pages 207–220, Big Sky, MT, USA, 2009. ACM. http://doi.acm.org/10.1145/1629575.1629596. *[cited on page 37]*

[KLW14] Robbert Krebbers, Xavier Leroy, and Freek Wiedijk: *Formal C semantics: Compcert and the C standard*. In *Proceedings of the 5th International Conference on Interactive Theorem Proving (ITP 2014)*, volume 8558 of *Lecture Notes in Computer Science*, pages 543–548, Vienne, Autriche, July 2014. Springer. https://doi.org/10.1007/978-3-319-08970-6_36. *[cited on page 37]*

[KMNO14] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens: *Cakeml: A verified implementation of ml*. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '14)*, pages 179–191, San Diego, CA, USA, 2014. ACM, ISBN 978-1-4503-2544-8. http://doi.acm.org/10.1145/2535838.2535841. *[cited on page 37]*

[Kri07] Jean-Louis Krivine: *A call-by-name lambda-calculus machine*. Higher-Order and Symbolic Computation, 20(3):199–207, 2007. https://doi.org/10.1007/s10990-007-9018-9. *[cited on page 47]*

[Lam77] Leslie Lamport: *Proving the correctness of multiprocess programs*. IEEE Trans. Software Eng., 3(2):125–143, 1977. https://doi.org/10.1109/TSE.1977.229904. *[cited on page 34]*

[Ler90] Xavier Leroy: *The ZINC experiment: an economical implementation of the ML language*. Technical Report 117, INRIA, Rocquencourt, France, February 1990. https://hal.inria.fr/inria-00070049/file/RT-0117.pdf. *[2 citations pages 23 and 47]*

[LF08] Francesco Logozzo and Manuel Fähndrich: *On the relative completeness of bytecode analysis versus source code analysis*. In *Proceedings of the 17th International Conference on Compiler Construction (CC 2008)*, volume 4959 of *Lecture Notes in Computer Science*, pages 197–212, Budapest, Hongrie, March 2008. Springer. https://doi.org/10.1007/978-3-540-78791-4_14. *[cited on page 20]*

[LHS10] Michael W. Lew, Thomas B. Horton, and Mark Sherriff: *Using LEGO MINDSTORMS NXT and LEJOS in an advanced software engineering course*. In *Proceedings of the 23rd IEEE Conference on Software Engineering Education and Training (CSEE&T 2010)*, pages 121–128, Pittsburgh, Pennsylvania, USA, March 2010. https://doi.org/10.1109/CSEET.2010.31. *[cited on page 22]*

[LL05] Benjamin Livshits and Monica S. Lam: *Finding security vulnerabilities in java applications with static analysis*. In *Proceedings of the 14th USENIX Security Symposium*, Baltimore, MD, USA, 2005. USENIX Association. https://www.usenix.org/conference/14th-usenix-security-symposium/. *[cited on page 20]*

[LM97] Yau-Tsun Steven Li and Sharad Malik: *Performance analysis of embedded software using implicit path enumeration*. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 16(12):1477–1487, 1997. https://doi.org/10.1109/43.664229. *[cited on page 36]*

[Lop09] Bruno Cardoso Lopes: *Understanding and writing an llvm compiler back-end (tutorial)*. In *ELC'09: Embedded Linux Conference*, San Francisco, CA, USA, April 2009. *[cited on page 21]*

[LYBB14] Tim Lindholm, Frank Yellin, Gilad Bracha, and Alex Buckley: *The Java Virtual Machine Specification, Java SE 8 Edition*. Addison-Wesley Professional, 1st edition, 2014, ISBN 013390590X, 9780133905908. *[cited on page 22]*

[MDLM18] Darius Mercadier, Pierre-Évariste Dagand, Lionel Lacassagne, and Gilles Muller: *Usuba: Optimizing & trustworthy bitslicing compiler*. In *Proceedings of the 4th Workshop on Programming Models for SIMD/Vector Processing (WPMVP@PPoPP 2018)*, pages 4:1–4:8, Vienne, Autriche, February 2018. ACM. https://doi.org/10.1145/3178433.3178437. *[cited on page 116]*

[Mil78] Robin Milner: *A theory of type polymorphism in programming*. Journal of Computer and System Sciences, 17(3):348 – 375, 1978, ISSN 0022-0000. https://doi.org/10.1016/0022-0000(78)90014-4. *[2 citations pages 114 and 133]*

[MP05] Louis Mandel and Marc Pouzet: *Reactiveml: a reactive extension to ML*. In *Proceedings of the 7th International ACM SIGPLAN Conference on Principles and Practice of Declarative Programming*, pages 82–93, Lisbonne, Portugal, July 2005. ACM. https://doi.org/10.1145/1069774.1069782. *[cited on page 29]*

[MPP10] Louis Mandel, Florence Plateau, and Marc Pouzet: *Lucy-n: a n-synchronous extension of lustre*. In *Proceedings of the 10th International Conference on Mathematics of Program Construction (MPC 2010)*, volume 6120 of *Lecture Notes in Computer Science*, pages 288–309, Québec city, Québec, Canada, June 2010. Springer. https://doi.org/10.1007/978-3-642-13321-3_17. *[cited on page 30]*

[MS17] Mahesh M and Sivraj P: *Drawcode: Visual tool for programming microcontrollers*. In *Proceedings of the 3rd International Conference on Advances in Computing,Communication Automation (ICACCA 2017)*, pages 1–6, Dehradun, Inde, September 2017. https://doi.org/10.1109/ICACCAF.2017.8344708. *[cited on page 9]*

[MV13] Michel Mauny and Benoît Vaugon: *OCamlCC - Traduire OCaml en C en passant par le bytecode*. In *Actes des 24èmes Journées Francophones des Langages Applicatifs (JFLA 2013)*, Aussois, France, February 2013. https://hal.inria.fr/hal-00779721. *[cited on page 69]*

[MV14] Michel Mauny and Benoît Vaugon: *Nullable Type Inference*. In *Proceedings of the 2014 OCaml Users and Developers Workshop (OCaml 2014)*, Gothenbourg, Suède, September 2014. https://hal.inria.fr/hal-01413294. *[cited on page 118]*

[NPW02] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel: *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002, ISBN 978-3-540-45949-1. https://doi.org/10.1007/3-540-45949-9. *[cited on page 37]*

[PAM+09] Bruno Pagano, Olivier Andrieu, Thomas Moniot, Benjamin Canou, Emmanuel Chailloux, Philippe Wang, Pascal Manoury, and Jean-Louis Colaço: *Experience report: Using Objective Caml to develop safety-critical embedded tools in a certification framework*. In *Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming (ICFP 2009)*, pages 215–220, Édimbourg, Royaume-Uni, August 2009. ACM. https://doi.org/10.1145/1596550.1596582. *[cited on page 33]*

[PB19] Basile Pesin and Julien Bissey: *Programmation de haut niveau pour applications sur micro-contrôleurs*. Rapport de travaux encadrés de recherche, Sorbonne Université, June 2019. *[cited on page 70]*

[Pes18] Basile Pesin: *Paradigmes de programmation alternatifs sur microcontrôleurs via l'OMicroB*. Rapport de stage, Sorbonne Université, June 2018. *[cited on page 64]*

[PFB+11] Claire Pagetti, Julien Forget, Frédéric Boniol, Mikel Cordovilla, and David Lesens: *Multi-task implementation of multi-periodic synchronous programs*. Discrete Event Dynamic Systems, 21(3):307–338, Sep 2011, ISSN 1573-7594. https://doi.org/10.1007/s10626-011-0107-x. *[cited on page 27]*

[Pie02] Benjamin C. Pierce: *Types and programming languages*. MIT Press, 2002, ISBN 978-0-262-16209-8. *[cited on page 35]*

[Pla88] John Plaice: *Sémantique et compilation de LUSTRE, un langage déclaratif synchrone*. PhD thesis, Institut National Polytechnique de Grenoble, Grenoble, France, 1988. *[cited on page 105]*

[Pla89] John Plaice: *Nested clocks: The lustre synchronous dataflow language*. In *Proceedings of the 1989 International Symposium on Lucid and Intensional Programming*, pages 1–17, 1989. *[cited on page 83]*

[Pou06] Marc Pouzet: *Lucid synchrone - tutorial and reference manual*. 2006. https://www.di.ens.fr/~pouzet/lucid-synchrone/lucid-synchrone-3.0-manual.pdf. *[cited on page 33]*

[PSS98] Amir Pnueli, Michael Siegel, and Eli Singerman: *Translation Validation*. In *Proceedings of the 4th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1998)*, pages 151–166, Lisbonne, Portugal, March 1998. Springer. https://doi.org/10.1007/BFb0054170. *[cited on page 133]*

[Rat92] Christophe Ratel: *Définition et réalisation d'un outil de vérification formelle de programmes LUSTRE*. PhD thesis, Université Joseph Fourier, Grenoble, France, 1992. https://tel.archives-ouvertes.fr/tel-00341223. *[cited on page 209]*

[Reg12] Pablo Vieira Rego: *Integrating 8-bit avr micro-controllers in ada*. The Ada User Journal, 33(4):301, 2012. *[cited on page 21]*

[Rey98] John C. Reynolds: *Definitional interpreters for higher-order programming languages*. Higher-Order and Symbolic Computation, 11(4):363–397, 1998. *[cited on page 40]*

[RP19] Mario Aldea Rivas and Héctor Pérez: *Leveraging real-time and multitasking ada capabilities to small microcontrollers*. Journal of Systems Architecture - Embedded Systems Design, 94:32–41, 2019. https://doi.org/10.1016/j.sysarc.2019.02.015. *[cited on page 21]*

[RWT+06] Jan Reineke, Björn Wachter, Stefan Thesing, Reinhard Wilhelm, Ilia Polian, Jochen Eisinger, and Bernd Becker: *A Definition and Classification of Timing Anomalies*. In *Proceedings of the 6th International Workshop on Worst-Case Execution Time Analysis (WCET '06)*, volume 4 of *OpenAccess Series in Informatics (OASIcs)*, Dresden, Allemagne, July 2006. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik. https://doi.org/10.4230/OASIcs.WCET.2006.671. *[cited on page 155]*

[SC16a] Jérémie Salvucci and Emmanuel Chailloux: *Memory Consumption Analysis for a Functional and Imperative Language*. In *Proceedings of the 1st international workshop on Resource Aware Computing (RAC 2016)*, volume 330 of *Electronic Notes in Theoretical Computer Science*, pages 27 – 46, Eindhoven, Pays-Bas, April 2016. https://hal.sorbonne-universite.fr/hal-01420298. *[cited on page 209]*

[SC16b] Rémy El Sibaïe and Emmanuel Chailloux: *Synchronous-reactive web programming*. In *Proceedings of the 3rd International Workshop on Reactive and Event-Based Languages and Systems (REBLS 2016)*, pages 9–16, Amsterdam, Pays-Bas, November 2016. ACM. https://doi.org/10.1145/3001929.3001931. *[cited on page 29]*

[Sen07] Koushik Sen: *Concolic testing*. In *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2007)*, pages 571–572, Atlanta, Georgia, USA, November 2007. ACM. https://doi.org/10.1145/1321631.1321746. *[cited on page 164]*

[SF09] Vincent St-Amour and Marc Feeley: *PICOBIT: A compact scheme system for microcontrollers*. In *Revised Selected Papers of the 21st Internation Symposium on Implementation and Application of Functional Languages (IFL 2009)*, volume 6041 of *Lecture Notes in Computer Science*, pages 1–17, South Orange, NJ, USA, September 2009. Springer. https://doi.org/10.1007/978-3-642-16478-1_1. *[cited on page 23]*

[SLNM04] Frank Singhoff, Jérôme Legrand, Laurent Nana, and Lionel Marcé: *Cheddar: a flexible real time scheduling framework*. In *Proceedings of the 2004 Annual ACM SIGAda International Conference on Ada*, pages 1–8, Atlanta, GA, USA, November 2004. ACM. https://doi.org/10.1145/1032297.1032298. *[cited on page 26]*

[SN08] Konrad Slind and Michael Norrish: *A brief overview of HOL4*. In *Proceedings of the 21st International Conference on Theorem Proving in Higher Order Logics (TPHOLs 2008)*, volume 5170 of *Lecture Notes in Computer Science*, pages 28–32, Montréal, Québec, Canada, August 2008. Springer. https://doi.org/10.1007/978-3-540-71067-7_6. *[cited on page 37]*

[SNO+10] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Thomas Ridge, Susmit Sarkar, and Rok Strnisa: *Ott: Effective tool support for the working semanticist*. Journal of Functional Programming, 20(1):71–122, 2010. https://doi.org/10.1017/S0956796809990293. *[cited on page 89]*

[SP06] Martin Schoeberl and Rasmus Pedersen: *Wcet analysis for a java processor*. In *Proceedings of the 4th International Workshop on Java Technologies for Real-time and Embedded Systems (JTRES '06)*, pages 202–211, Paris, France, 2006. ACM, ISBN 1-59593-544-4. http://doi.acm.org/10.1145/1167999.1168033. *[cited on page 20]*

[Spa17] Philip Sparks: *The route to a trillion devices*. Technical report, ARM, June 2017. *[cited on page 8]*

[Spi89] Michael Spivey: *An introduction to Z and formal specifications*. Software Engineering Journal, 4(1):40–50, 1989. *[cited on page 37]*

[SSD+17] Anatoliy Shamraev, Elena Shamraeva, Anatoly Dovbnya, Andriy Kovalenko, and Oleg Ilyunin: *Green Microcontrollers in Control Systems for Magnetic Elements of Linear Electron Accelerators*, pages 283–305. Springer International Publishing, Cham, 2017, ISBN 978-3-319-44162-7. https://doi.org/10.1007/978-3-319-44162-7_15. *[cited on page 10]*

[STD85] *Ieee standard for binary floating-point arithmetic*. ANSI/IEEE Std 754-1985, pages 1–20, October 1985. http://doi.org/10.1109/IEEESTD.1985.82928. *[cited on page 62]*

[Tea19] The Coq Development Team: *The Coq Proof Assistant, version 8.9.0*, January 2019. https://doi.org/10.5281/zenodo.2554024. *[3 citations pages 11, 37, and 89]*

[VB14] Jérôme Vouillon and Vincent Balat: *From Bytecode to JavaScript: the Js_of_ocaml Compiler*. Software: Practice and Experience, 44(8):951–972, 2014. https://doi.org/10.1002/spe.2187. *[cited on page 29]*

[VC19] Steven Varoumas and Tristan Crolard: *WCET of ocaml bytecode on microcontrollers: An automated method and its formalisation*. In *Proceedings of the 19th International Workshop on Worst-Case Execution Time Analysis (WCET 2019)*, volume 72 of *OpenAccess Series in Informatics (OASICs)*,

pages 5:1–5:12. Schloss Dagstuhl, July 2019. https://doi.org/10.4230/OASIcs.WCET.2019.5. *[cited on page 207]*

[VVC16] Steven Varoumas, Benoît Vaugon, and Emmanuel Chailloux: *Concurrent Programming of Microcontrollers, a Virtual Machine Approach*. In *Proceedings of the 8th European Congress on Embedded Real Time Software and Systems (ERTS^2 2016)*, Toulouse, France, 2016. https://hal.archives-ouvertes.fr/ERTS2016. *[2 citations pages 27 and 205]*

[VVC17] Steven Varoumas, Benoît Vaugon, and Emmanuel Chailloux: *OCaLustre : une extension synchrone d'OCaml pour la programmation de microcontrôleurs*. In *Vingt-huitièmes Journées Francophones des Langages Applicatifs (JFLA 2017)*, Gourette, France, 2017. https://hal.archives-ouvertes.fr/JFLA2017. *[cited on page 207]*

[VVC18a] Steven Varoumas, Benoît Vaugon, and Emmanuel Chailloux: *A Generic Virtual Machine Approach for Programming Microcontrollers: the OMicroB Project*. In *Proceedings of the 9th European Congress on Embedded Real Time Software and Systems (ERTS 2018)*, Toulouse, France, 2018. https://hal.archives-ouvertes.fr/ERTS2018. *[cited on page 205]*

[VVC18b] Steven Varoumas, Benoît Vaugon, and Emmanuel Chailloux: *La programmation de microcontrôleurs dans des langages de haut niveau - Cours invité*. In *Actes des 29èmes Journées Francophones des Langages Applicatifs (JFLA 2018)*, BANYULS, France, January 2018. https://hal.sorbonne-universite.fr/hal-01762414. *[cited on page 208]*

[VVF18] Nicolas Valot, Pierre Vidal, and Louis Fabre: *Increase avionics software development productivity using Micropython and Jupyter notebooks*. In *Proceedings of the 9th European Congress on Embedded Real Time Software and Systems (ERTS^2 2018)*, Toulouse, France, January 2018. https://hal.archives-ouvertes.fr/ERTS2018. *[cited on page 23]*

[VWC15] Benoît Vaugon, Philippe Wang, and Emmanuel Chailloux: *Programming microcontrollers in ocaml: The ocapic project*. In *Proceedings of the 17th International Symposium on Practical Aspects of Declarative Languages (PADL 2015)*, volume 9131 of *Lecture Notes in Computer Science*, pages 132–148, Portland, OR, USA, June 2015. Springer. https://doi.org/10.1007/978-3-319-19686-2_10. *[cited on page 23]*

[Wal00] David Walker: *A type system for expressive security policies*. In *Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2000)*, pages 254–267, Boston, Massachusetts, USA, January 2000. ACM. https://doi.org/10.1145/325694.325728. *[cited on page 91]*

[WDS+10] W. E. Wong, V. Debroy, A. Surampudi, H. Kim, and M. F. Siok: *Recent catastrophic accidents: Investigating how software was responsible*. In *Proceedings of the 4th International Conference on Secure Software Integration and Reliability Improvement (SSIRI)*, pages 14–22, June 2010. https://doi.org/10.1109/SSIRI.2010.38. *[cited on page 34]*

[WEE+08] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström: *The worst-case execution-time problem - overview of methods and survey of tools*. ACM Transactions on Embedded

Computing Systems (TECS), 7(3):36:1–36:53, May 2008, ISSN 1539-9087. https://doi.org/10.1145/1347375.1347389. *[cited on page 36]*

Abstract

Microcontrollers are programmable integrated circuits found in many everyday objects. Because their resources are limited, they are often programmed in low-level languages such as C or assembly language. These languages do not provide the same abstractions and guarantees as high-level programming languages such as OCaml. This thesis proposes a set of techniques for bringing high-level programming paradigms to microcontroller programming. These techniques provide several layers of abstraction and, in particular, make it possible to develop portable programs that are independent of the underlying hardware. We first introduce a hardware-abstraction layer based on an OCaml virtual machine, which preserves many of the benefits of the language while maintaining a small memory footprint. We then extend OCaml with a synchronous programming model inspired by the Lustre dataflow language, providing an abstraction for the concurrent aspects of a program. The language is specified formally, and several typing properties are proved. The abstractions introduced in this work also make certain static analyses portable across platforms, since they can be performed on program bytecode. We propose one such analysis, which estimates the worst-case execution time (WCET) of a synchronous program. Together, the contributions of this thesis form a complete development toolchain, illustrated by several practical examples that demonstrate the benefits of the proposed approach.